

Sorbonne Université

École Doctorale Informatique, Télécommunications et Électronique (ED130)

Laboratoire d'Informatique de Paris 6

Protocoles pair à pair pour un apprentissage décentralisé efficace et résilient

Théorie, conception et évaluation

Par Mohamed Amine LEGHERABA

Doctorat en Informatique

Dirigée par
Pr Maria POTOP-BUTUCARU
Pr Sébastien TIXEUIL

Présentée et soutenue publiquement le 01/01/1970

Devant un jury composé de :

Maria	POTOP-BUTUCARU	Directrice de thèse
Sébastien	TIXEUIL	Directeur de thèse
Prénom	Nom	Rapporteur·euse
Prénom	Nom	Rapporteur·euse
Prénom	Nom	Examinateur·rice
Prénom	Nom	Examinateur·rice
Prénom	Nom	Invité·e

Except where otherwise noted, this work is licensed under
https://creativecommons.org/licenses/by-nc-nd/4.0/

Abstract

Decentralized peer-to-peer systems offer a compelling alternative to centralized architectures for large-scale distributed computing: they are inherently resilient to failures, scalable by design, and respectful of data locality. Yet coordinating a large number of autonomous nodes without any central authority remains a fundamental challenge, particularly when the application layer demands collaborative machine learning over private, heterogeneous data.

This thesis makes two original contributions. The first is Elevator, a fully decentralized overlay protocol that organizes a peer-to-peer network into a two-tier topology through an emergent hub election mechanism. At each cycle, every node connects preferentially to the h most connected nodes in its two-hop neighborhood while retaining $c - h$ random connections. This preferential attachment causes h hub nodes to emerge spontaneously within a small number of cycles, forming a dense interconnected core to which all other nodes are directly attached. Elevator is complemented by Lift, a Byzantine-resilient extension that prevents adversarial nodes from manipulating the election outcome through a shared pseudo-random number generator seeded by the current hub identifiers. Elevator is analyzed theoretically and evaluated through large-scale simulations and on a real TCP/IP network, while Lift is evaluated through simulation.

The second contribution is HEAL (Hub Enhanced Adaptive Learning), a decentralized federated learning framework built on Elevator. HEAL organizes model aggregation into two successive phases per cycle: an intra-hub phase, in which each hub aggregates the models of its attached nodes, and an inter-hub phase, in which hubs exchange and merge their respective aggregates. This hierarchical structure accelerates convergence compared to flat gossip and epidemic learning baselines. HEAL is evaluated on image and text classification tasks under fault-free operation, static node failures, and churn. A complementary protocol, FLAIR, adapts hub-based aggregation to physical wireless networks by electing cluster heads based on node capabilities following the LEACH framework, and is evaluated on the ns-3 simulator.

Contents

List of Figures

List of Tables

Remerciements

Je tiens tout d'abord à remercier les rapporteurs d'avoir accepté de consacrer leur temps et leur expertise à la relecture de ce manuscrit, ainsi que l'ensemble des membres du jury pour leur participation à ma soutenance. C'est un honneur de soumettre mes travaux à leur jugement.

Mes remerciements les plus sincères vont à mes directeurs de thèse, Maria Potop-Butucaru et Sébastien Tixeuil, pour leur accompagnement tout au long de ces années. Leur guidance, leurs conseils précieux, leur patience et leur pédagogie ont été déterminants dans l'aboutissement de ces travaux. Ils ont su, à chaque étape, trouver les mots justes pour me guider, et me pousser à donner le meilleur de moi-même. Je n'aurais pas pu espérer meilleurs encadrants.

Je remercie les membres de mon comité de suivi individuel, Luciana Arantes et Fabien Mathieu, pour leur accompagnement lors des réunions annuelles et leurs conseils avisés sur la conduite de la thèse.

Je remercie chaleureusement Bruno Baynat pour son accompagnement durant près d'un an autour de la preuve de convergence et de stabilité d'Elevator. Les nombreuses pistes que nous avons explorées ensemble autour des chaînes de Markov, les allers-retours, les reformulations, ont été une expérience intellectuelle aussi exigeante qu'enrichissante.

Je remercie également Megumi Kaneko pour son accueil au National Institute of Informatics de Tokyo durant mes trois mois de stage. Sa patience, son aide précieuse dans mes travaux de recherche, et son attention à mon intégration au sein de l'équipe ont rendu cette expérience inoubliable, tant sur le plan scientifique qu'humain.

Merci à toute l'équipe du NPA, et en particulier à Bilal, Alexandre, Guillaume et Stefan, pour les échanges, les discussions de couloir, et cette atmosphère de travail à la fois sérieuse et bienveillante. J'ai également une pensée toute particulière pour les stagiaires avec qui j'ai eu la chance de collaborer : Alicia, Nour, Victor et Ihssan. Leur implication, leur curiosité et leur sérieux ont été d'une aide considérable dans mes travaux, et les résultats de cette thèse leur doivent beaucoup.

Merci à tous les doctorants et membres du LIP6 que j'ai croisés au fil de ces années, pour les discussions scientifiques, les pauses café, et tout ce qui fait la richesse d'une vie de laboratoire. Merci également à l'ensemble des chercheurs et à l'organisation du LINCS, institution si singulière, dont j'ai eu le privilège de côtoyer le monde durant ces années de thèse.

Je remercie l'administration de Sorbonne Université, du LIP6 et de l'école doctorale EDITE pour leur accompagnement tout au long de ce parcours. Je suis profondément fier et honoré d'avoir réalisé mon doctorat au sein d'une institution aussi prestigieuse.

Ma pensée la plus profonde va bien sûr à ma famille. À ma mère et mon père. À mes frères, et en particulier à toi, Iskander, qui as toi-même soutenu ta thèse l'année dernière après cinq ans de doctorat dans des conditions difficiles.

À toi, Shehrazade, ma très chère fiancée, que j'ai eu la chance de rencontrer durant cette thèse : merci pour ton soutien sans faille durant la rédaction de ce manuscrit, pour ta patience, et pour la lumière que tu apportes chaque jour.

Je terminerai par celui qui, le premier, m'a donné l'envie et le courage de me lancer dans l'aventure du doctorat : le Dr Mounir El Kadiri. Merci pour vos conseils, votre confiance et votre soutien depuis le début.

À toi, lecteur, merci d'accorder ton temps précieux à la lecture de ces travaux. J'espère sincèrement que tu y trouveras quelque chose d'utile.

Résumé en français

Les systèmes distribués à grande échelle sont aujourd'hui omniprésents, qu'il s'agisse de réseaux de diffusion de contenu, de systèmes de stockage décentralisés, ou plus récemment de plateformes d'apprentissage automatique collaboratif. Dans ce contexte, les architectures pair à pair occupent une place particulière : elles permettent de coordonner un grand nombre de nœuds sans recourir à une autorité centrale, ce qui les rend intrinsèquement résistantes aux pannes, passables à l'échelle, et respectueuses de la confidentialité des données. Cependant, concevoir des protocoles pair à pair qui soient à la fois efficaces, robustes aux défaillances, et capables de supporter des charges applicatives complexes comme l'apprentissage automatique reste un défi ouvert.

Cette thèse s'inscrit dans ce contexte et propose deux contributions principales. La première, Elevator, est un protocole de gestion de topologie pair à pair basé sur l'élection dynamique de nœuds hubs, formant une structure d'overlay hiérarchique à deux niveaux. La seconde, HEAL (Hub Enhanced Adaptive Learning), est un cadre d'apprentissage fédéré décentralisé construit sur Elevator, qui tire parti de la structure hub pour accélérer la convergence des modèles d'apprentissage tout en préservant les propriétés de robustesse de l'overlay sous-jacent. Un protocole complémentaire, FLAIR (Federated Learning with Adaptive Integrity-preserving Randomness), est également introduit comme alternative adaptée aux contraintes des réseaux sans fil physiques.

La thèse débute par l'introduction d'un modèle formel du système qui sert de socle à l'ensemble des contributions. Ce modèle définit les abstractions fondamentales utilisées tout au long du manuscrit : la notion de nœud, de voisinage, de vue partielle, et de cycle de communication. Le réseau est modélisé comme un graphe dynamique dont la topologie évolue au fil du temps sous l'effet des entrées et sorties de nœuds, des pannes franches, et des décisions protocolaires.

Le modèle de pannes adopté couvre trois scénarios distincts. Les pannes franches correspondent à l'arrêt définitif d'un nœud sans préavis. Le churn désigne la dynamique continue d'arrivées et de départs de nœuds, caractéristique des réseaux pair à pair ouverts. Les nœuds byzantins, enfin, représentent des participants malveillants capables d'envoyer des messages arbitraires ou incorrects dans le but de perturber le protocole. Ce dernier modèle de faute est le plus général et le plus difficile à tolérer.

Le chapitre suivant dresse un panorama de l'état de l'art sur les réseaux overlay pair à pair, en se concentrant sur les aspects pertinents pour les contributions de la thèse. Les réseaux overlay sont des réseaux logiques construits par-dessus un réseau physique sous-jacent, dans lesquels chaque nœud maintient une vue partielle de ses voisins et communique exclusivement avec eux. Deux grandes familles de protocoles sont distinguées : les overlays structurés, dans lesquels la topologie obéit à des invariants précis (comme dans Chord ou Kademlia), et les overlays non structurés, dans lesquels les connexions sont établies de manière aléatoire ou épidémique.

Les protocoles de peer sampling, qui permettent à chaque nœud d'obtenir un sous-ensemble représentatif et frais de l'ensemble du réseau, occupent une place centrale dans ce panorama. Des protocoles comme Cyclon, Newscast, ou HyParView sont présentés et analysés selon leur résistance au churn, la qualité de la vue partielle qu'ils maintiennent, et leur capacité à s'auto-réparer après une partition. Ces propriétés constituent les critères d'évaluation qui seront repris pour Elevator.

L'état de l'art sur la gestion de topologies hiérarchiques et l'élection de super-pairs est également couvert. Les approches existantes, qu'elles soient basées sur la capacité des nœuds, sur leur degré de connectivité, ou sur des mécanismes d'élection distribués, sont passées en revue. Cette revue met en évidence les limites des approches actuelles en matière de dynamicité et de tolérance aux fautes byzantines, et motive la conception d'Elevator comme une alternative plus robuste et plus flexible.

Elevator est ensuite introduit, la première contribution originale de la thèse. Elevator est un protocole d'overlay pair à pair qui organise le réseau en une structure à deux niveaux : un ensemble de nœuds hubs, élus dynamiquement, et un ensemble de nœuds génériques qui se connectent à ces hubs. L'élection des hubs est au cœur du protocole. Elevator repose sur un mécanisme d'élection entièrement décentralisé et émergent, qui ne requiert ni coordination explicite ni connaissance globale du réseau. À chaque cycle, chaque nœud observe ses voisins à distance deux et établit ses connexions sortantes de manière préférentielle : parmi ses c connexions totales, il en consacre h aux nœuds qui présentent le plus grand nombre de connexions entrantes dans sa vue locale, et maintient $c - h$ connexions choisies aléatoirement. Ce mécanisme de connexion préférentielle, inspiré des processus d'attachement préférentiel étudiés dans la littérature sur les réseaux complexes, crée une dynamique de renforcement : les nœuds les plus connectés attirent davantage de connexions, ce qui accroît encore leur degré entrant. En quelques cycles seulement, cette dynamique fait émerger spontanément h nœuds hubs auxquels la quasi-totalité des nœuds du réseau est directement connectée. Le statut de hub n'est pas attribué explicitement : il est la conséquence observable d'un processus collectif décentralisé, piloté uniquement par les paramètres globaux c et h.

La tolérance aux fautes byzantines est adressée par un protocole complémentaire, Lift, conçu pour s'exécuter par-dessus Elevator. Dans Elevator seul, le mécanisme de connexion préférentielle est aveugle à la nature des nœuds : un nœud byzantin qui parvient à accumuler un grand nombre de connexions entrantes peut s'imposer comme hub et ainsi occuper une position privilégiée dans le réseau. Lift modifie le processus d'élection en s'appuyant sur un générateur de nombres pseudo-aléatoires partagé. Les identifiants des h hubs courants sont utilisés comme graine de ce générateur ; comme tous les nœuds du réseau disposent de la même graine, ils peuvent tous dériver indépendamment et de manière identique un nouvel ensemble de h identifiants, qui désignent les prochains hubs. Ce mécanisme retire aux nœuds byzantins toute capacité d'influence sur l'élection : un attaquant ne peut pas manipuler le résultat du générateur sans contrôler les identifiants qui servent de graine, et ces identifiants sont déterminés par le processus Elevator sous-jacent. Lift conserve ainsi l'ensemble des propriétés topologiques d'Elevator tout en offrant des garanties de robustesse renforcées en présence d'attaquants actifs.

L'évaluation d'Elevator est conduite selon trois axes complémentaires. Une analyse théorique établit d'abord les propriétés formelles du protocole : convergence vers une topologie hub stable, conditions de stabilité en fonction des paramètres c et h, et borne sur le temps de convergence en nombre de cycles. Ces résultats analytiques fournissent des garanties sur le comportement asymptotique du protocole indépendamment de toute hypothèse sur l'implémentation. Des simulations à grande échelle réalisées sur PeerSim permettent ensuite d'évaluer empiriquement les propriétés topologiques de l'overlay — diamètre, connectivité, stabilité du hub set — sous différentes conditions de churn et de taille de réseau allant jusqu'à 1 000 nœuds, et de confronter les prédictions théoriques au comportement observé. Enfin, une évaluation sur un réseau TCP/IP réel est conduite pour mesurer les performances du protocole dans des conditions de déploiement effectives, en termes de latence de reconfiguration et de volume de messages échangés.

Le cinquième chapitre introduit les fondements théoriques de l'apprentissage automatique et de l'apprentissage fédéré décentralisé, et constitue le pont entre la partie réseau et la partie applicative de la thèse. L'apprentissage fédéré est présenté dans sa formulation canonique centralisée, telle qu'elle a été introduite par FedAvg. Cette approche, bien qu'efficace, repose sur un serveur central qui agrège les mises à jour des clients à chaque ronde, ce qui en limite la passage à l'échelle et la résistance aux pannes.

L'apprentissage fédéré décentralisé est ensuite introduit comme alternative naturelle. Dans ce paradigme, il n'existe plus de serveur central : chaque nœud agrège les modèles de ses voisins directs, et la

convergence vers un modèle global émerge des échanges épidémiques entre pairs. L'apprentissage par bavardage et l'apprentissage épidémique sont présentés comme les représentants les plus aboutis de cette approche. Leurs propriétés de convergence, leur comportement sous hétérogénéité des données, et leurs limites en termes de vitesse de convergence sont analysés en détail.

La revue de la littérature met en évidence un manque important : les protocoles d'apprentissage décentralisé existants n'exploitent pas la structure du réseau superposé pour accélérer la propagation des modèles. C'est précisément cette lacune que HEAL vient combler.

Le sixième chapitre présente HEAL et FLAIR, les contributions applicatives de la thèse. HEAL est un cadre d'apprentissage fédéré décentralisé qui s'appuie sur la structure hub d'Elevator pour organiser l'agrégation des modèles en plusieurs phases successives. L'architecture de HEAL est organisée en trois couches. La couche overlay, fournie par Elevator, détermine la topologie du réseau et l'identité des hubs à chaque cycle. La couche d'agrégation définit le protocole d'échange et de fusion des modèles entre nœuds. La couche applicative interface HEAL avec la tâche d'apprentissage concrète — classification d'images, classification de texte — via une abstraction de modèle local.

Le protocole HEAL se déroule en cinq phases à chaque cycle. Premièrement, chaque nœud effectue une mise à jour locale de son modèle sur ses données privées. Deuxièmement, les nœuds génériques envoient leur modèle à leur hub de rattachement. Troisièmement, chaque hub agrège les modèles reçus de ses nœuds génériques. Quatrièmement, les hubs échangent leurs modèles agrégés entre eux et réalisent une seconde agrégation au niveau inter-hub. Cinquièmement, chaque hub rediffuse le modèle global agrégé vers ses nœuds génériques. Cette structure en deux niveaux d'agrégation — intra-hub puis inter-hub — est la propriété fondamentale qui distingue HEAL des approches de gossip learning à plat, et qui lui permet d'atteindre une convergence plus rapide en réduisant le nombre de cycles nécessaires à la propagation d'une mise à jour à travers l'ensemble du réseau.

L'évaluation expérimentale de HEAL est conduite sur plusieurs tâches d'apprentissage — MNIST, Spambase — et sous trois scénarios de fautes : absence de pannes, pannes franches statiques, et churn. Les résultats montrent que HEAL converge systématiquement plus vite que les baselines de gossip learning et d'epidemic learning, et qu'il maintient une précision élevée même en présence de churn significatif.

FLAIR est introduit en fin de chapitre comme un protocole complémentaire à HEAL, conçu pour les contraintes des réseaux sans fil physiques. FLAIR s'appuie sur LEACH, un protocole d'élection de têtes de grappe (cluster heads) initialement développé pour les réseaux de capteurs. Dans FLAIR, l'élection des têtes de grappe repose sur un score calculé à partir des capacités physiques de chaque nœud (bande passante disponible, puissance de calcul, mémoire, …). Les nœuds présentant les meilleures capacités sont élus têtes de grappe et assument le rôle d'agrégateurs de modèles pour leur grappe respective. Cette approche permet d'adapter naturellement la charge d'agrégation aux nœuds les plus capables du réseau, tout en limitant le volume de communications nécessaires à la convergence. FLAIR est évalué sur le simulateur ns-3, qui modélise fidèlement les conditions de propagation et d'interférence d'un réseau sans fil réel.

Un chapitre en annexe documente l'ensemble de l'infrastructure de simulation développée au cours de la thèse. Cette infrastructure, largement invisible dans les chapitres principaux, a nécessité un investissement considérable et constitue une contribution d'ingénierie significative en elle-même. Le simulateur PeerSim a été profondément modifié pour les besoins de la thèse : migration du contrôle de version de SVN vers Git, introduction de Gradle pour la gestion des dépendances, conteneurisation via Docker, pipeline CI/CD sur GitLab, parallélisation du moteur de simulation, implémentation des modèles de fautes et des attaques byzantines, et ajout de plusieurs protocoles d'overlay. Pour la simulation de l'apprentissage décentralisé, une approche hybride combinant PeerSim pour la topologie

et Gossipy pour la couche d'apprentissage a été adoptée après une évaluation approfondie des alternatives disponibles — Flower, OMNeT++, NS-3, decentralizepy — dont les limitations respectives sont documentées. L'ensemble du workflow expérimental, de la génération des topologies à la production des figures, est entièrement automatisé et reproductible.

Les travaux présentés dans cette thèse montrent qu'il est possible de concevoir un protocole d'overlay pair à pair qui soit à la fois dynamique, robuste aux fautes byzantines, et exploitable comme substrat pour l'apprentissage fédéré décentralisé. Elevator et HEAL forment un système cohérent dans lequel les propriétés topologiques de l'overlay se traduisent directement en propriétés de convergence de l'apprentissage.

Plusieurs directions de recherche restent ouvertes. Sur le plan protocolaire, l'extension de HEAL à des scénarios d'apprentissage personnalisé, dans lesquels chaque nœud cherche à optimiser un modèle adapté à sa distribution locale plutôt qu'un modèle global partagé, constitue une piste naturelle. La gestion de l'hétérogénéité des données — scénario non-IID — reste un défi pour les protocoles de gossip learning en général, et HEAL n'échappe pas à cette limitation. Des mécanismes d'agrégation robuste à l'hétérogénéité, intégrés dans la couche hub d'Elevator, pourraient atténuer ce problème. Sur le plan de l'infrastructure, le simulateur unifié en cours de développement en fin de thèse, qui intègre topologie dynamique, apprentissage décentralisé, et modèles de fautes dans un cadre unique entièrement en Python, ouvre la voie à des expérimentations plus systématiques et à une meilleure reproductibilité des résultats dans la communauté.

Chapter 1

Introduction

This thesis is concerned with peer-to-peer protocols for efficient and resilient decentralised learning. Each of these terms carries precise technical meaning, and each reflects a deliberate design choice. Before unpacking them, it is worth stepping back to understand the broader context from which this research emerges. Peer-to-peer networks did not appear in a vacuum — they are a product of a longer history of communication infrastructure, and understanding what makes them distinctive requires understanding the landscape they emerged from. Similarly, decentralised learning cannot be appreciated without first understanding what machine learning is, how it developed, and why the conditions under which it is practised today raise questions that go well beyond the technical. This introduction therefore proceeds in two steps: it first traces the evolution of communication networks, from their earliest forms to the Internet and the peer-to-peer systems it made possible; it then turns to artificial intelligence and machine learning, from their theoretical foundations to their current concentration in the hands of a small number of actors. From the convergence of these two trajectories, the research question at the heart of this thesis naturally emerges.

Communication networks have long been central to the development of human societies. From the telegraph to the telephone, from postal routes to radio broadcasting, each successive generation of infrastructure has reduced the cost of exchanging information across distance and, in doing so, has reshaped the economic, political, and cultural fabric of the societies it connected. The Internet represents the culmination of this trajectory — and a qualitative leap beyond it. Born out of a desire to build a communication infrastructure resilient to partial failure, it was designed from the outset so that no single node, no single institution, and no single government could control or interrupt the flow of information. By the turn of the millennium, this global network had become the substrate on which an ever-growing share of human activity took place, enabling near-instantaneous exchange of information across the planet at negligible cost.

Yet the open and decentralised architecture of the Internet did not prevent the emergence of centralisation at the layer of services built on top of it. The protocols that gave rise to the World Wide Web were open and accessible to all, but the services that came to dominate the Web were not. Over the course of two decades, a small number of technology companies — today grouped under the informal label of Big Tech — came to occupy a position of structural dominance over the digital lives of billions of people. Their success rested on a straightforward logic: a centralised service, operated from a single infrastructure, can serve the entire planet while continuously improving through the aggregation of its users' data. The simplicity they offered was genuine, and the economies of scale they achieved were real. But these advantages came at a price that was not always made explicit: the progressive transfer of personal data, behavioural traces, and, ultimately, a degree of individual autonomy to private entities whose decisions remain largely opaque.

In response to this concentration, communities of developers and researchers began, from the early days of the Web, to imagine and build alternatives. Peer-to-peer protocols — systems in which participants interact directly with one another rather than through a central intermediary — offered a different vision of what networked services could look like: distributed, egalitarian, and resilient by design. Some of these efforts achieved remarkable reach; others remained confined to niche communities. But together, they demonstrated that decentralised alternatives were technically feasible, and they raised questions of sufficient depth and generality to attract the attention of the academic

research community. How does a system coordinate without a centre? How does it remain reliable when its participants are unreliable? How does it scale when no single entity oversees its growth? These questions sit at the intersection of computer science, mathematics, and the theory of complex systems, and they remain active areas of inquiry today.

Beyond technical architecture, the question of whether digital infrastructure should be centralised or decentralised is, at its core, a question about power: who controls the systems that mediate communication, commerce, and knowledge; who has access to the data these systems generate; and who bears the risks when they fail or are misused. There is no neutral answer. Centralised services offer convenience and efficiency, but they concentrate power and create dependencies that individuals and institutions may not fully perceive until they are already deeply embedded. Decentralised alternatives demand more from their participants — a degree of technical literacy, a willingness to share responsibility, and a collective commitment to maintaining the commons — but they offer in return a form of autonomy and resilience that centralised architectures cannot provide by design. Whether this trade-off is worth making is not a question this thesis attempts to answer. It is an open societal debate, and one that will not be resolved by technical means alone. What this thesis does attempt is to advance the technical foundations that would make decentralised alternatives more viable, so that this choice can be made on informed and practical grounds.

On a separate but parallel trajectory, artificial intelligence has been a driving intellectual ambition since the earliest days of computing. From the foundational theoretical work of Turing and McCarthy to the modern era of machine learning and deep learning, the field has undergone successive waves of progress, each expanding the range of tasks that machines can perform and the scale at which they can operate. Today, machine learning models underpin an ever-growing share of the systems that structure daily life, from search engines and recommendation platforms to medical diagnosis and language understanding.

What distinguishes modern machine learning systems from classical software is not merely their capability, but their fundamental nature. A traditional algorithm is a set of explicit instructions written by a programmer: its behaviour is fully determined by its code, which can in principle be read, audited, and understood. A machine learning model, by contrast, does not follow instructions — it learns from data. Before it can be used, it must be trained on a large corpus of examples, a process through which it develops internal representations that no human explicitly designed. The result is a system whose behaviour emerges from the interaction of its architecture, its training data, and the precise conditions of the training process — none of which are recoverable from the model alone. This is what researchers mean when they speak of the black box problem: not that the system is mysterious in principle, but that its decisions cannot be traced back to interpretable rules. This opacity is already a concern when the model is developed in an academic or open setting. It becomes considerably more acute in the context of commercial AI systems, where users interact with a model through an interface or an API without any access to the underlying architecture, the training data, the source code, or even the random seed that determined how training unfolded. In some cases, the weights of a model are made publicly available — but weights alone, without the data and the process that produced them, offer only a partial and often insufficient basis for understanding or auditing the system's behaviour. The practical consequence is that the systems which today exert the most influence over decisions affecting millions of people are also the least transparent, and the least amenable to independent scrutiny.

These two trajectories — the progressive concentration of digital services in the hands of a small number of platform operators, and the rise of machine learning as the dominant paradigm in artificial intelligence — are not independent phenomena. They share a common root: the training of machine learning models is, by its very nature, a resource-intensive operation that favours centralisation. It requires access to large volumes of data, which platforms accumulate as a byproduct of their scale,

and to significant computational infrastructure, which only a handful of organisations can afford to operate. Centralisation is not an incidental feature of modern AI development — it is, under current conditions, a structural tendency. Yet this tendency is not inevitable. One may legitimately ask whether it is possible to train machine learning models differently — without aggregating raw data in a single location, without delegating control to a central authority, and without requiring any participant to expose information they would prefer to keep private. Is there, in other words, a viable alternative to the centralised paradigm that dominates the field today?

One candidate answer has existed for over two decades, though it was developed in an entirely different context. Peer-to-peer networks, which emerged in the late 1990s and achieved widespread use in the early 2000s, were designed precisely to enable large-scale coordination without central control. Though their mainstream adoption has remained confined to specific niches — file sharing, cryptocurrency, distributed storage — the architectural principles they embody are still actively studied and deployed. The question this thesis asks is whether these principles can be transplanted into the domain of machine learning: whether a peer-to-peer network can serve not merely as a medium for transferring files or reaching consensus, but as the substrate on which a machine learning model is collectively trained, iteratively refined, and made available to its participants — without any of them ever surrendering their data to a central party.

Pursuing this question requires bridging two research communities that have, until recently, evolved largely independently of one another. The networking and distributed systems community has developed a rich body of work on peer-to-peer protocols, graph theory, and self-stabilising systems, and is accustomed to reasoning about well-defined, often deterministic environments: nodes, links, message complexity, consensus protocols, and fault models with precise formal guarantees. The machine learning community operates in a fundamentally different register — one of statistics, optimisation, and empirical validation, where the objects of study are loss functions, gradient estimates, and generalisation bounds, and where the notion of a network, if it appears at all, refers to the architecture of a model rather than the topology of a communication system. These two communities do not share the same vocabulary, the same proof techniques, or the same experimental culture. The consequence is that the space where they overlap — decentralised machine learning over peer-to-peer networks — has remained comparatively underexplored. Addressing it seriously requires fluency in both domains simultaneously: any theoretical analysis must account for the stochastic dynamics of learning and the adversarial dynamics of distributed coordination at the same time, and any experimental evaluation must be credible from both perspectives.

The motivations for pursuing this research direction extend well beyond technical curiosity. The ethical dimensions are concrete and pressing. At stake are three interrelated concerns: privacy, since centralised training requires raw data to leave the hands of those who generated it and be aggregated under the control of a third party; sovereignty, since the growing dependence of public institutions on models developed and controlled by a small number of private actors creates dependencies that individuals, nations, and democratic systems may find increasingly difficult to accept; and accountability, since a model trained on undisclosed data, by an unreproducible process, and deployed through an opaque interface offers little purchase to the conventional instruments of democratic oversight — transparency, contestability, and liability. These concerns crystallise around a common pattern: there exist many situations in which no single participant holds enough data to train a useful model on their own, yet every participant has strong reasons — legal, competitive, or ethical — not to share that data with a third party. Decentralised learning addresses precisely this tension.

Consider the medical domain. A hospital treating patients with a rare disease may have access to only a handful of relevant cases — far too few to train a reliable diagnostic model. Neighbouring hospitals face the same constraint. Yet patient data is among the most sensitive information that exists, and

sharing it with an external party raises serious concerns about confidentiality, consent, and regulatory compliance. A decentralised training framework would allow these institutions to collaboratively build a model of a quality that none of them could achieve alone, without any patient record ever leaving the institution that collected it.

A second case arises among competing actors who share a common interest. Consider the automotive industry, where multiple manufacturers are independently developing autonomous driving systems. Each company guards its data jealously, as it represents a significant competitive asset. Yet all of them share an interest in reducing the rate of accidents — a goal that improves with the volume and diversity of training data. Decentralised learning makes it possible to aggregate the benefits of a shared training process without requiring any participant to expose proprietary data to its competitors. The result is a model that is safer for everyone, produced by a process that compromises no one.

A third case concerns the construction of large language models — the systems that today underpin conversational AI, automated writing, and an ever-growing range of decision-support tools. These models are currently trained by a small number of private actors on datasets whose composition is rarely disclosed. This concentration creates a risk that is not merely technical: a sufficiently influential actor could, intentionally or not, shape the values, assumptions, and blind spots of a model that billions of people interact with daily. A language model trained collectively, by a distributed set of participants with heterogeneous data and divergent perspectives, would be structurally more resistant to this kind of influence — whether commercial, ideological, or political.

What these cases share is something deeper than a technical property. Decentralised learning is, at its core, an exercise in building a common good. Each participant contributes what they have, retains what they wish to protect, and benefits from the collective result. No single actor controls the outcome, and no single actor can capture it. This vision of collaborative intelligence — distributed, egalitarian, and resistant to capture — is what gives the research direction developed in this thesis its broader significance.

Decentralised learning is not the only possible response to these concerns, and it would be misleading to present it as a perfect solution. Several alternative approaches exist, each with its own merits and limitations. One option is to encrypt data before sharing it, using techniques such as homomorphic encryption, which allows computations to be performed directly on encrypted data without ever decrypting it. This approach offers strong theoretical guarantees, but it comes at a computational cost that remains, in practice, prohibitive for training large models. Another option is to add carefully calibrated statistical noise to the data or to the model updates before they are shared — a technique known as differential privacy — which makes it mathematically impossible to recover individual records from the shared information. This is a valuable complement to decentralised learning, but it does not by itself address the question of who controls the training process. A third option is to rely on trusted execution environments: specialised hardware enclaves that guarantee, at the chip level, that even the operator of the infrastructure cannot access the data being processed. This provides a technical guarantee without requiring decentralisation, but it transfers the question of trust from the service provider to the hardware manufacturer. A fourth option is to place a centralised actor under strict legal obligations, relying on regulation rather than architecture to enforce privacy and accountability. This approach has real value, and the European Union's efforts in this direction — through the GDPR, the AI Act, and related instruments — represent a serious attempt to make it work. But a well-intentioned and legally compliant centralised actor remains a single point of failure: it can be compromised by a cyberattack, subject to unconscious biases in how it curates training data, or placed under political or economic pressure that no regulatory framework can fully anticipate. Finally, one might ask whether the problem can be sidestepped altogether — either by generating synthetic data locally to augment small datasets, or by developing model architectures that require less data

to achieve useful performance. These are active and promising research directions, but they do not generalise: for many applications, and in particular for rare or highly specific phenomena, neither synthetic generation nor data-efficient architectures can substitute for the real, distributed data that participants hold.

The appeal of decentralised learning, in this context, lies in the fact that it addresses the problem at the architectural level. Privacy and the absence of central control are not properties that depend on the goodwill of an operator, the robustness of a legal framework, or the security of a hardware supply chain — they are structural consequences of how the system is designed. This does not mean that decentralised systems are without weaknesses. They introduce their own trade-offs: coordinating training across many independent participants without a central authority is harder, slower, and more vulnerable to certain classes of malicious behaviour than centralised training.

This raises the central question of this thesis: is it possible to build a decentralised learning system that matches the efficiency of its centralised counterparts, while being fully decentralised and resilient to failures and malicious participants? This question is not merely rhetorical — it is technically non-trivial, and the answer is far from obvious. This thesis attempts to address it through two complementary lenses: a structured review of the existing literature at the intersection of peer-to-peer systems and machine learning, which maps the landscape of prior work and identifies the open problems that motivate the contributions that follow; and the design and evaluation of original protocols for decentralised federated learning, which make deliberate design choices to advance what is achievable in terms of efficiency, decentralisation, and resilience against malicious behaviour.

1.1 Contributions

This thesis makes four original contributions at the intersection of peer-to-peer networking and decentralised machine learning.

The first contribution is **Elevator**, a decentralised peer sampling protocol that organises nodes into a hub-and-spoke overlay through a lightweight, self-organising random election mechanism. Elevator requires no pre-assigned coordinator, makes no assumption about the application running on top of it, and is designed to tolerate node crashes and churn. By elevating a dynamic subset of nodes to the role of hubs, it introduces a structured aggregation topology into an otherwise flat peer-to-peer network, without sacrificing the decentralised nature of the system.

The second contribution is **Lift**, an extension of Elevator that addresses Byzantine fault tolerance. Where Elevator assumes that participants follow the protocol honestly, Lift introduces an analysis of the protocol's behaviour under Byzantine attacks and equips it with a defence mechanism that limits the influence of malicious participants on the hub election process. Lift strengthens the security guarantees of the entire protocol stack built on top of Elevator.

The third contribution is **HEAL**, a decentralised federated learning protocol built directly on top of the Elevator overlay. HEAL exploits the two-level structure of the overlay — hubs aggregating local model updates, and inter-hub communication reconciling aggregates across the network — to recover the convergence efficiency of classical federated learning while preserving full decentralisation. HEAL is evaluated under fault-free conditions as well as under crash failures and churn, across multiple model architectures and learning tasks.

The fourth contribution is **FLAIR** , an adaptation of HEAL to the constraints of physical wireless networks. In a WiFi environment, the overlay topology cannot be chosen freely: it is shaped by radio range, interference, and the physical proximity of devices. FLAIR draws inspiration from LEACH-style clustering to align the logical aggregation structure with the physical network layer, making decentralised federated learning deployable in realistic wireless settings.

1.2 Organisation of the manuscript

The manuscript is organised into six chapters, reflecting the progressive construction of the framework from its theoretical foundations to its applied extensions.

Chapter 2 introduces the theoretical model that underpins the entire thesis. It formalises the system assumptions, the network model, and the abstractions used throughout the subsequent chapters.

Chapter 3 surveys the state of the art on overlay networks in peer-to-peer systems. It reviews peer sampling protocols, structured and unstructured overlays, and existing approaches to hub election and topology management, situating the Elevator contribution within the existing landscape.

Chapter 4 presents the Elevator and Lift protocols. It covers the design of the hub election mechanism, the theoretical analysis of the resulting overlay properties, and the experimental evaluation conducted on PeerSim and over a real TCP/IP network.

Chapter 5 surveys the state of the art on decentralised learning. It introduces the necessary background on federated learning, and gossip learning, and reviews the main approaches to decentralized aggregation in the learning literature.

Chapter 6 presents the HEAL and FLAIR protocols. It describes the architecture of HEAL, its aggregation scheme, and its experimental evaluation across multiple learning tasks and fault scenarios. The chapter then introduces FLAIR and its adaptation to physical wireless network constraints, evaluated on the ns-3 simulator.

Chapter 2

Model

The peer-to-peer literature encompasses a wide variety of systems, protocols, and architectures, ranging from structured overlays and to highly dynamic gossip-based protocols. These systems differ in their objectives, their communication patterns, and their assumptions, but despite this diversity, most peer-to-peer systems rely on a common set of fundamental principles.

In order to reason about such systems in a systematic way, it is necessary to go beyond individual protocol descriptions and introduce a unified formal model. The purpose of this chapter is to define such a model, capturing the essential components and dynamics shared by a broad class of peer-to-peer systems, while abstracting away implementation-specific details.

This formalization serves two main objectives. First, it provides a common framework for comparing different peer-to-peer protocols on a principled basis, independently of their concrete realization. Second, it enables the rigorous analysis of system properties such as performance, efficiency, robustness, and convergence.

The model introduced in this chapter forms the foundation for the rest of the manuscript. It will be used to describe execution assumptions, network dynamics, evaluation metrics, and to support both analytical arguments and experimental results presented in the following chapters.

We adopt a bottom-up approach, starting from the node as the fundamental building block of the system, then modeling the peer-to-peer network formed by their interactions, and finally analyzing the emergent phenomena that arise at the network level. The definitions and abstractions introduced in this chapter follow the general principles of distributed system modeling as presented in *Introduction to Reliable and Secure Distributed Programming* [2].

2.1 Node

Nodes constitute the fundamental components of the system. Each node acts as an autonomous entity that executes local computations, maintains an internal state, and interacts with other nodes through the peer-to-peer network. In the literature on distributed systems and peer-to-peer networks, nodes are commonly referred to using different terms such as *processors*, *peers*, or *agents*, depending on the modeling perspective and application domain. In this work, we use the term node to emphasize its generality and to remain independent of any specific implementation or execution environment.

Definition 2.1.1 (Node)

A node (also referred to as a participant, agent, or peer) is an abstract computational entity.

A node is characterized by:

(i) a memory, representing its local state, assumed to be arbitrarily large for modeling purposes;

(ii) computational capabilities, abstracted from physical limitations and assumed to be unbounded.

♣

Example. A machine running a BitTorrent client constitutes a node in the BitTorrent system.

We deliberately abstract away any form of node heterogeneity, as our primary focus is on the interactions induced by the protocol rather than on resource disparities between nodes.

Assumption 2.1.2

All nodes are identical in terms of memory or computational capabilities

A node is executed according to an abstract execution model that captures its behavior independently of any implementation details. In this model, a node is viewed as a state machine that evolves over time by executing local computations.

Starting from an initial state, the node repeatedly executes a local computation based solely on its current state (i.e., its local memory), and transitions to a new state. This execution model abstracts away timing, concurrency, and hardware constraints, and focuses exclusively on how local states evolve as a result of computation.

Definition 2.1.3 (Node Execution Model)

A node is modeled as a state machine defined by the tuple (S, s_0, δ), where:

- S is the set of all possible local states of the node;
- $s_0 \in S$ is the initial state;
- $\delta : S \to S$ is a state transition function.

At each execution step, the node applies the transition function δ to its current state $s \in S$, producing a new state $s' = \delta(s)$.

2.2 Network

In a distributed system, nodes do not operate in isolation but interact with each other through a network. The network provides the structural substrate that enables communication, coordination, and information exchange between nodes.

In our model, the network captures how nodes are interconnected and how interactions between them are made possible, independently of the specific communication mechanisms or protocols. By introducing the network abstraction, we move from the behavior of an individual node to the collective behavior of a set of interacting nodes, which is a fundamental step toward understanding the dynamics of peer-to-peer systems.

2.2.1 Network Assumptions

A peer-to-peer network is typically implemented as a virtual network, also called an overlay network, on top of a physical network. A clear distinction must therefore be made between the physical network, such as the Internet, and the overlay network. Each node in the overlay network is hosted on a node of the physical network, but the reverse is not necessarily true. Moreover, two neighbouring nodes in the overlay network are not necessarily neighbours in the physical network. An overlay network can itself be implemented on top of another overlay network. For example, the Lightning Network [3] operates as an overlay on top of the Bitcoin network, which itself relies on the Internet protocol stack.

We abstract the underlying physical network, as peer-to-peer algorithms do not directly operate on physical networking mechanisms. We assume that the underlying network provides basic communication primitives required by the overlay network.

In particular, we assume that:
(i) the underlying network is connected, i.e., any node can eventually reach any other node,
(ii) nodes can send messages to other nodes, and messages are routed to their intended destination.

We abstract away message transmission by assuming that the underlying network is **reliable**. In particular, message delivery is assumed to be instantaneous, and messages are neither lost nor corrupted. Under this abstraction, peer-to-peer algorithms do not need to explicitly account for network-level delays or failures.

These assumptions allow us to focus on the design and analysis of the overlay network and the associated peer-to-peer protocols, independently of the underlying physical infrastructure.

2.2.2 Graph Terminology

To model the overlay network, we rely on graph theory, which provides a natural and well-established framework for representing interactions between nodes in a peer-to-peer system. Graph-based models allow us to reason about network structure, connectivity, and information propagation without explicitly accounting for low-level networking or hardware-specific characteristics.

By abstracting the overlay as a graph, nodes are represented as vertices and communication relationships as edges. This abstraction enables a clear and generic analysis of network properties and dynamics, independently of the underlying physical infrastructure.

In the following, we introduce the standard definitions and notations from graph theory that will be used throughout this manuscript. These definitions are classical and widely used in the literature on distributed systems [4], [5].

Definition 2.2.1 (Undirected Graph)

An undirected graph is an ordered pair $G = (V, E)$:

- V, a set of vertices (also called nodes or points);
- $E \subseteq \{\{x, y\} \mid x, y \in V, x \neq y\}$, a set of edges (also called links or lines), which are unordered pairs of vertices (that is, an edge is associated with two distinct vertices).

♣

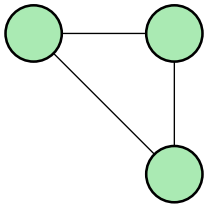

Figure 1: Example of an undirected graph with three vertices and three edges.

Definition 2.2.2 (Directed Graph)

A directed graph or digraph is a graph in which edges have orientations. A directed graph is an ordered pair $G = (V, E)$:

- V, a set of vertices (also called nodes or points);
- $E \subseteq \{(x, y) \mid (x, y) \in V^2, x \neq y\}$, a set of edges (also called directed edges, directed links, directed lines, arrows or arcs) which are ordered pairs of vertices (that is, an edge is associated with two distinct vertices).

♣

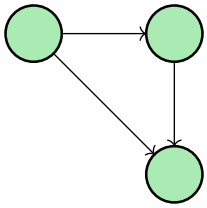

Figure 2: Example of a directed graph with three vertices and three directed edges.

Definition 2.2.3 (Path)

Let $G = (V, E)$ be a graph. A **path** from a vertex $u \in V$ to a vertex $v \in V$ is a finite sequence of vertices $(v_0, v_1, ..., v_k)$ such that:

- $v_0 = u$ and $v_k = v$,
- for all $i \in \{0, ..., k-1\}$, $(v_i, v_{i+1}) \in E$.

The **length** of a path is defined as the number of edges it contains, i.e., k.

♣

Definition 2.2.4 (Connected Components)

Let $G = (V, E)$ be an undirected graph.

- A **connected component** is a maximal subset of nodes $C \subseteq V$ such that for every pair of nodes $u, v \in C$, there exists a path connecting u and v.

The set of connected components induces a partition of the vertex set V. The number of connected components characterizes the fragmentation of the graph.

♣

Definition 2.2.5 (Strongly and Weakly Connected Components)

Let $G = (V, E)$ be a directed graph.

- A **strongly connected component (SCC)** is a maximal subset of nodes $C \subseteq V$ such that for every pair of nodes $u, v \in C$, there exists a directed path from u to v and from v to u.
- A **weakly connected component (WCC)** is a maximal subset of nodes $C \subseteq V$ such that the underlying undirected graph obtained by ignoring edge directions is connected.

♣

Definition 2.2.6 (Distance)

Let $G = (V, E)$ be a graph. For any two vertices $u, v \in V$, the distance between u and v, denoted by $\text{dist}_G(u, v)$, is defined as the length of a shortest path between u and v in G.

If no path exists between u and v, the distance is defined as $\text{dist}_G(u, v) = +\infty$.

This definition naturally extends to sets of vertices. For two subsets $U, W \subseteq V$, the distance between U and W is defined as

$$\text{dist}_G(U, W) = \min\{\text{dist}_G(u, w) \mid u \in U, w \in W\}$$

♣

Definition 2.2.7 (Distance (in a directed graph))

Let $G = (V, E)$ be a directed graph. A **directed path** from a vertex $u \in V$ to a vertex $v \in V$ is a sequence of vertices $(u = v_0, v_1, ..., v_k = v)$ such that $(v_i, v_{i+1}) \in E$ for all $0 \leq i < k$.

The distance from u to v, denoted by $\text{dist}_G(u, v)$, is defined as the minimum length (number of edges) of any directed path from u to v that **respects the orientation of the edges**. If no such directed path exists, we set $\text{dist}_G(u, v) = +\infty$.

In general, the distance in a directed graph is **not symmetric**: $\text{dist}_G(u, v)$ may differ from $\text{dist}_G(v, u)$, and one of them may be finite while the other is infinite.

♣

Definition 2.2.8 (Average Path Length)

Let $G = (V, E)$ be a graph.

The **average path length** of the network, denoted by $a(G)$, is defined as:

$$a(G) = \sum_{u,v \in V, u \neq v} \frac{\text{dist}_G(u, v)}{|V| \ (|V| - 1)}$$

♣

Definition 2.2.9 (Diameter)

Let $G = (V, E)$ be a graph. The **diameter** of G, denoted by $\text{diam}(G)$, is defined as the maximum distance between any pair of vertices in V:

$$\text{diam}(G) = \max_{u,v \in V} \text{dist}_G(u, v)$$

♣

Definition 2.2.10 (Neighborhood)

Let $G = (V, E)$ be an undirected graph and let $v \in V$ be a vertex.

The **neighborhood** of v, denoted $\text{neigh}_G(v)$, is the set of vertices that are adjacent to v, that is, vertices connected to v by an edge.

Equivalently, the neighborhood of v can be defined as the set of vertices at distance exactly one from v:

$$\text{neigh}_G(v) = \{u \in V \mid (u, v) \in E\} = \left\{u \in V \mid \text{dist}_{G(u,v)} = 1\right\}.$$

♣

Definition 2.2.11 (Successors and Predecessors (Directed Graphs))

Let $G = (V, E)$ be a directed graph and let $v \in V$ be a vertex.

- The **successors** of v, denoted $\text{succ}_G(v)$, is the set of vertices that

can be reached from v by a single directed edge:

$$\text{succ}_G(v) = \{u \in V \mid (v, u) \in E\}.$$

- The **predecessors** of v, denoted $\text{pred}_G(v)$, is the set of vertices that

have a directed edge toward v:

$$\text{pred}_G(v) = \{u \in V \mid (u, v) \in E\}.$$

Equivalently, these sets correspond to vertices at directed distance one from or to v, respectively.

♣

Definition 2.2.12 (Degree)

Let $G = (V, E)$ be an undirected graph and let $v \in V$ be a vertex.

The **degree** of vertex v in graph G, denoted by $\text{degree}_G(v)$, is defined as the number of vertices adjacent to v, or equivalently, the size of its neighborhood:

$$\text{degree}_G(v) = |\text{neigh}_G(v)|$$

♣

Definition 2.2.13 (In-degree and Out-degree)

Let $G = (V, E)$ be a directed graph and let $v \in V$ be a vertex.

The **out-degree** of v in G, denoted by $\text{outdegree}_G(v)$, is the number of successors of v:

$$\text{outdegree}_G(v) = |\text{succ}_G(v)|$$

The **in-degree** of v in G, denoted by $\text{indegree}_G(v)$, is the number of predecessors of v:

$$\text{indegree}_G(v) = |\text{pred}_G(v)|$$

♣

Definition 2.2.14 (k-Neighborhood)

Let $G = (V, E)$ be an undirected graph and let $v \in V$ be a vertex.

The **k-neighborhood** of v in G, denoted $\text{neigh}_G^k(v)$, is the set of vertices at distance exactly k from v:

$$\text{neigh}_G^k(v) = \{u \in V \mid \text{dist}_G(v, u) = k\}$$

♣

Definition 2.2.15 (k-Successors and k-Predecessors (Directed Graphs))

Let $G = (V, E)$ be a directed graph and let $v \in V$ be a vertex.

- The **k-successors** of v, denoted $\text{succ}_G^k(v)$, is the set of vertices reachable from v by a directed path of length exactly k:

$$\text{succ}_G^k(v) = \{u \in V \mid \text{dist}_G(v, u) = k\}$$

- The **k-predecessors** of v, denoted $\text{pred}_G^k(v)$, is the set of vertices from which v can be reached by a directed path of length exactly k:

$$\text{pred}_G^k(v) = \{u \in V \mid \text{dist}_G(u, v) = k\}$$

♣

Definition 2.2.16 (Clustering Coefficient)

Let $G = (V, E)$ be a graph. For any vertex $v \in V$ with $\text{degree}_G(v) \geq 2$, the local clustering coefficient of v, denoted $C(v)$, is defined as the fraction of pairs of neighbours of v that are themselves connected:

$$C(v) = \frac{|\{\{u, w\} \in E \mid u \in \text{neigh}_G(v), w \in \text{neigh}_G(v)\}|}{\binom{\text{degree}_G(v)}{2}}$$

For vertices with $\text{degree}_G(v) < 2$, the clustering coefficient is conventionally set to $C(v) = 0$.

The global clustering coefficient of G, denoted $C(G)$, is defined as the average local clustering coefficient over all vertices:

$$C(G) = \frac{1}{|V|} \sum_{v \in V} C(v)$$

The global clustering coefficient takes values in $[0, 1]$, where $C(G) = 0$ indicates that no two neighbours of any node are connected, and $C(G) = 1$ indicates that every neighbourhood forms a complete subgraph.

♣

2.2.3 Overlay Network Modeling

With the basic concepts of graph theory in place, we can now formalize the representation of an overlay network. In our model, the overlay network is abstracted as a graph $G = (V, E)$, where each vertex represents a participant (node) in the system, and edges represent the communication links between nodes.

Each system node is thus associated with a vertex in the graph, and its unique address serves as the identifier of that vertex. This correspondence allows us to leverage graph-theoretic notions to describe and analyze the network's structure and connectivity.

Definition 2.2.17 (Node Address)

Each node in the system is assigned a unique address, also referred to as its identifier. Node addresses are drawn from the finite set $S = \{0, \ldots, N - 1\}$, where N denotes the total number of nodes in the network. This assignment ensures that every node can be uniquely identified within the system.

♣

Assumption 2.2.18

The address of a node carries no semantic information about its capabilities, role, or properties, and has no influence on the behavior of the node.

♡

Communication Channels

In a peer-to-peer system, nodes interact by sending messages to one another. We abstract away the technical details of message transmission, such as bandwidth limits, latency, or packet loss. Instead, we introduce the notion of a **communication channel** as a conceptual link that allows a node to deliver messages to another node.

Channels can be **bidirectional**, similar to a TCP connection, where messages can flow in both directions, or **unidirectional**, similar to a UDP link, where messages flow in a single direction. In our model, a channel exists simply if a node can send a message to another node.

Definition 2.2.19 (Bidirectional Communication Channel)

A bidirectional communication channel between two nodes u and v allows both nodes to send messages to each other. In the graph representation of the network, a bidirectional channel corresponds to an undirected edge $\{u, v\} \in E$ connecting the two vertices associated with the nodes.

♣

Definition 2.2.20 (Unidirectional Communication Channel)

A unidirectional communication channel from node u to node v allows u to send messages to v, but not necessarily the other way around. In the graph representation, a unidirectional channel corresponds to a directed edge $(u, v) \in E$ from the vertex representing u to the vertex representing v.

♣

Partial View

In large-scale peer-to-peer networks, it is unrealistic for a node to maintain knowledge of all other nodes in the system. Instead, each node maintains a **partial view**, which is a subset of nodes it is aware of and can communicate with. This partial view is stored in the node's local state as a list of node addresses.

Although we consider that nodes have unbounded memory for abstraction purposes, the size of the partial view is bounded by a constant c, with $c \ll N$, where N is the total number of nodes in the network. In the graph representation of the network, the partial view of a node corresponds to its **neighborhood**, i.e., the set of nodes connected to it by edges.

Definition 2.2.21 (Partial View)

The **partial view** of a node v, denoted $P(v)$, is the set of nodes that v maintains in its local state and can send messages to, where $P(v) \subseteq V$ is a subset of all nodes in the network.

- The size of the partial view is bounded: $|P(v)| \leq c$, where c is a constant much smaller than N, the total number of nodes.
- In the network graph $G = (V, E)$ with undirected channels:

$$P(v) = \text{neigh}_{G(v)}$$

♣

Remark

For directed channels, the partial view corresponds to the set of successors of v:

$$P(v) = \text{succ}_{G(v)}$$

i.e., nodes to which v can send messages. A node does not necessarily maintain a list of its predecessors, only its successors.

Assumption 2.2.22

It is assumed that a node must know the address of another node in order to send it a message. Consequently, each node communicates only with its neighbours at distance 1 in the overlay network.

Assumption 2.2.23 (Directed Channel Reply)

In the case of a directed communication channel, a node can respond to a received message even if the sender is not in its partial view (successors). This is because each message carries the address of the sender.

Protocol

In a distributed system, each node executes a **protocol** that governs its behavior and interactions with other nodes. In this work, we do not consider protocols at the underlying physical or network layers (e.g., routing, congestion control, or transport mechanisms). Instead, we assume that nodes can exchange messages through abstract communication channels, as introduced earlier.

At the overlay level, all nodes execute the same protocol. This protocol defines how nodes process incoming messages, update their local state, and decide when and to whom messages are sent. It therefore captures the collective logic of the peer-to-peer system and determines how global network properties emerge from local interactions.

Definition 2.2.24 (Peer-to-Peer Protocol)

A peer-to-peer protocol is a distributed algorithm executed by each node in the network that specifies:

(i) the local state maintained by a node,
(ii) the set of messages that can be exchanged between nodes,
(iii) the rules governing message generation, transmission, and handling
(iv) the local state transitions performed by a node upon internal events or message reception.

The protocol is executed independently by all nodes. Each node follows the same protocol specification, but may exhibit different behaviors depending on its local state, its partial view of the network, and the messages it receives. In our abstract model, we do not consider how the protocol is concretely implemented (e.g., programming language, runtime environment, or communication framework). We focus solely on the **pseudocode** of the protocol, which specifies the rules governing state transitions and message exchanges.

This definition is consistent with the execution model introduced earlier, where a node is modeled as a state machine. In this framework, the peer-to-peer protocol corresponds to the state transition

function executed by each node. Given the current local state of a node and the occurrence of an event (e.g., message reception or internal trigger), the protocol determines the next local state and the set of messages to be emitted. Thus, we assume that for a given local state and a given event, all nodes react deterministically and in exactly the same way.

In addition to their local state, nodes share a common knowledge of a set of **global parameters** defined by the protocol. These parameters are identical for all nodes and remain constant throughout the execution of the system. They capture configuration choices of the protocol, such as bounds on local resources or structural constraints, and ensure that all nodes operate under the same assumptions. For instance, the maximum size of a node's partial view is a global parameter known to all nodes.

Definition 2.2.25 (Global Protocol Parameter)

A **global protocol parameter** is a constant value that is known to all nodes in the system and shared across the entire network.

Formally, let P denote the set of global parameters of a protocol. Each parameter $p \in P$ has a fixed value that is identical for all nodes and does not depend on the local state of any node. ♣

Remark

The pseudocode of the protocol itself can be viewed as a global parameter of the system.

We assume that each peer-to-peer protocol executed by a node is composed of two main components: an initialization function and a main protocol function.

The initialization function is executed once when a node joins the system. During this phase, the node initializes its local state, including in particular its partial view of the network, i.e., its list of neighbors.

After initialization, the node executes the main protocol logic in the form of an infinite loop. This reflects the fact that peer-to-peer protocols are typically designed to run continuously and do not have a predefined termination condition.

We assume that each iteration of the protocol loop is executed atomically: a node cannot be interrupted in the middle of a protocol cycle, and no two executions of the protocol logic overlap on the same node.

If, during its execution, a node contacts another node, the contacted node processes the incoming request using a background execution thread. The internal scheduling of protocol execution and background message handling is abstracted away. The handling of incoming requests is also assumed to be atomic, and responses are generated and returned instantaneously.

2.3 System

Having defined the behavior of individual nodes and the structure of the overlay network, we can now formalize the system as a whole.

At any given time, the state of the peer-to-peer system is entirely determined by the collection of local states of all nodes. In other words, the **global state** of the system corresponds to the concatenation of the local states maintained by each node.

Definition 2.3.1 (Global State)

Let V be the set of nodes in the system.

The **global state** of the system is defined as the tuple:

$$S_{\text{global}} = (s_v)_{v \in V}$$

where $s_v \in S_v$ is the current local state of node v, as defined in Definition 2.1.3.

♣

2.3.1 Synchronization

When modeling the evolution of a distributed system, it is necessary to specify how nodes progress from one state to another. Since the system consists of multiple autonomous agents, this raises the question of synchronization between nodes.

In our model, we abstract away from synchronization issues at the physical or network layers. We assume that message transmission is instantaneous and reliable, i.e., messages are neither delayed nor lost. As a consequence, we do not consider timing, buffering, or failures at the communication level.

At the overlay level, however, synchronization still plays a conceptual role. In real-world peer-to-peer systems, nodes operate fully asynchronously: there is no global clock, and each node evolves at its own pace based on local events and message arrivals. While this behavior accurately reflects practical systems, it makes mathematical analysis and simulation significantly more complex.

To enable a tractable and precise formalization, we introduce an abstract notion of time based on **protocol steps** and **protocol cycles**. These notions do not represent real time, but rather logical execution units that allow us to reason about the global evolution of the system in a structured manner.

Definition 2.3.2 (Protocol Step)

A **protocol step** is defined as a single execution of the protocol pseudocode by one node.

During a protocol step, a node:

(i) processes an internal event or a received message,
(ii) applies the protocol's state transition rules,
(iii) updates its local state, and
(iv) possibly emits messages to other nodes.

♣

Assumption 2.3.3

We assume that each protocol step takes the same amount of time, regardless of the node executing it or the updates performed during the step.

In this model, we abstract away from execution time and computational cost.

♡

Definition 2.3.4 (Protocol Cycle)

A **protocol cycle** is defined as a logical execution round in which every node in the network executes exactly one protocol step.

Formally, a protocol cycle consists of a sequence of protocol steps such that each node in V executes the protocol once.

♣

Regarding the execution of a protocol cycle, we distinguish between two possible execution models.

In the **synchronous cycle model**, all nodes execute their protocol step simultaneously. Each node computes its state transition using its local state and the messages produced during the previous cycle. All state updates and message emissions take effect only at the end of the cycle. This model corresponds to a fully synchronous execution, where cycles act as global logical barriers.

In the **sequential cycle model**, nodes execute their protocol steps one after another within a cycle, following a random order. Each node updates its local state immediately upon execution and may emit messages that can be observed by nodes executing later in the same cycle. As a result, the state of the system may evolve during the cycle itself.

2.3.2 Self Organization

Beyond local execution semantics, many peer-to-peer protocols are designed to achieve specific objectives at the level of the system as a whole. While each node executes the protocol independently and relies solely on local information, the collective behavior of the system may exhibit coordinated dynamics that serve a common goal. Typical examples include decentralized aggregation protocols, where nodes collaboratively compute global statistics such as the average, sum, or maximum of locally held values, as well as classical coordination tasks such as leader election or consensus.

These protocols are often characterized by a notion of convergence: starting from an arbitrary initial global state, the system is expected to evolve toward a stable or desirable global configuration that satisfies the protocol's objective. This convergence is achieved without centralized control and emerges from repeated local interactions between nodes constrained by the evolving network topology.

Definition 2.3.5 (Global Objective)

A peer-to-peer protocol is said to pursue a global objective if there exists a set of desirable global states S^* such that the protocol aims to drive the system toward this set through local interactions.

♣

Definition 2.3.6 (Convergence)

The protocol is said to converge if, for any initial global state $S(0)$, the sequence of global states $S(t)_{t\geq 0}$ produced by the protocol satisfies:

$$\exists T \geq 0 \text{ such as } \forall t \geq T, S(t) \in S^*$$

Convergence may be exact or approximate, and may hold deterministically or with high probability, depending on the assumptions made on the protocol execution and the network dynamics.

♣

2.3.3 Randomness

Our system model is fundamentally deterministic: nodes execute protocol logic based on local state and received messages, without access to external sources of entropy or true random oracles. This design choice ensures reproducibility of executions, simplifies formal reasoning about protocol correctness, and avoids reliance on potentially biased or manipulable randomness sources in adversarial environments.

However, certain protocol mechanisms benefit from behaviors that **appear** random. To support such use cases without introducing non-determinism, we allow nodes to locally instantiate a pseudo-random number generator (PRNG, see Definition 2.3.7) when the protocol specification explicitly requires randomized choices.

Definition 2.3.7 (Pseudo-Random Number Generator)

Let $\lambda \in \mathbb{N}$ be a security parameter.

A pseudo-random number generator (PRNG) is a deterministic algorithm

$$G : \{0,1\}^\lambda \to \{0,1\}^*$$

such that:

- (Determinism) For any seed $s \in \{0,1\}^\lambda$, the output $G(s)$ is uniquely determined. In particular, two executions of G on the same seed produce the same output.
- (Pseudo-randomness) When the seed s is sampled uniformly at random from $\{0,1\}^\lambda$, the output $G(s)$ is computationally indistinguishable from a truly random bitstring of the same length.

♣

2.3.4 Dynamic Network

So far, we have considered the overlay network as a static structure on which nodes execute a protocol over time. However, in many peer-to-peer systems, the network topology itself evolves as the system runs. As nodes repeatedly execute the protocol, both the local views of nodes and the global network topology may change over time. Thus a first level of dynamicity arises from the protocol execution itself. After each execution of the protocol loop, a node may update its partial view of the network, for instance by adding, removing, or replacing neighbors. As a consequence, the set of outgoing edges of a node in the overlay graph may change from one protocol cycle to another. At this level of dynamicity, the set of nodes remains constant, and only the edge set of the graph evolves over time.

Churn

A second level of dynamicity is introduced by churn [6], that is, the dynamic arrival and departure of nodes in the system. We model churn as a temporally localized phenomenon rather than a permanent one. Specifically, churn occurs during a predefined period spanning several protocol cycles.

During a protocol cycle affected by churn, a given fraction of nodes is selected uniformly at random and disconnected from the network, similarly to permanent crash failures. These nodes are assumed to leave the system definitively and never rejoin. Within the same cycle, an equal number of new nodes joins the network. Each joining node executes the initialization function and subsequently participates in the protocol execution as a regular node.

After the churn period ends, no further nodes join or leave the system. The protocol continues to execute on a fixed set of nodes, allowing the network to evolve solely under the effect of the protocol logic. This post-churn phase is used to observe whether, and under which conditions, the system is able to recover from the instability induced by churn and converge back to a stable configuration.

This modeling choice reflects the fact that continuous churn keeps the system in a permanently unstable state. By separating churn phases from stabilization phases, we can explicitly study the resilience and self-healing properties of the peer-to-peer protocol.

In our model, such dynamics are captured by allowing the network graph $G = (V, E)$ to vary across protocol cycles. Both sources of dynamicity—updates of local views driven by protocol execution, and the addition/removal of nodes due to churn—contribute to changes in the network topology over time. Consequently, the overlay network can be naturally represented as a **time-varying graph** (or temporal graph), which formalizes the evolving set of nodes and edges as a function of time, as discussed

in the literature [7]. This representation allows us to treat the structure of the overlay network as part of the system state, fully integrating network dynamics into the global evolution of the system.

Definition 2.3.8 (Time-Varying Graph with churn)

A time-varying graph is a tuple $G = (V, E, T)$ where:
- T is a time domain, which is discrete;
- $V(t)$ is the set of vertices present at time $t \in T$;
- $E(t) \subseteq \{\{x, y\} \mid x, y \in V(t), x \neq y\}$ is the set of edges present at time t.

The graph $G(t) = (V(t), E(t))$ represents the network topology at time t. The evolution of the graph over time captures the appearance and disappearance of vertices and edges.

♣

2.4 Failure Models

In the previous sections, we have formalized the behavior of nodes, the evolution of the overlay network, and the dynamics of protocol execution in terms of steps and cycles. Having established this abstract execution framework, we now consider the possibility that nodes may fail during the system evolution. Node failures are an important source of perturbation in peer-to-peer systems and can affect both local computations and global network dynamics.

In this work, we explicitly situate our analysis within standard fault models from distributed computing, in order to precisely characterize the assumptions under which the protocol operates. We adopt the terminology and notational conventions introduced by Raynal in **Fault-Tolerant Message-Passing Distributed Systems: An Algorithmic Approach** [8].

We distinguish two main classes of failures in peer-to-peer systems: crash failures and Byzantine failures.

2.4.1 Crash Failures

A **crash failure** occurs when a node permanently stops executing the protocol. This may result from hardware faults, software errors, or permanent network disconnection. From the perspective of our abstract model, the specific cause is irrelevant; what matters is the observable effect: a crashed node ceases all activity.

We assume that crash failures are modeled using the **Crash-prone Synchronous Message-Passing model**, denoted **CSMP ⟨n, t⟩ [∅]** (following [8]), where:

- **n** is the total number of nodes in the system,
- **t** is the maximum number of nodes that may crash during execution,
- [∅] indicates that no additional failure detectors or oracles are assumed.

Under this model, the system is synchronous, communication is reliable, and nodes may only fail by crashing.

When a node crashes:

(i) it no longer updates its local state,
(ii) it cannot send messages,
(iii) it cannot receive or process incoming messages.

Any attempt by another node to contact a crashed node results in the absence of a response. We assume that crash failures are permanent: once a node crashes, it never recovers and never rejoins the system. In our model, a node can only crash at the beginning of a protocol cycle, never in the middle of a cycle. This assumption simplifies the analysis by ensuring that protocol steps and cycles remain atomic.

Assumption 2.4.1

A node can detect that one of its neighbors has crashed if the neighbor does not respond to a message. Since message transmission is assumed to be instantaneous and reliable, the absence of an immediate response is interpreted as a crash.

2.4.2 Byzantine Failures

In contrast to crash failures, a Byzantine node remains active but no longer follows the prescribed protocol. Instead, it behaves according to an arbitrary (Byzantine) strategy.

Byzantine behaviors can take many forms, including sending incorrect, inconsistent, or misleading messages, selectively responding to certain nodes, or attempting to disrupt the protocol execution. The common characteristic of Byzantine nodes is that they act maliciously, with the goal of corrupting the protocol execution or degrading the overall behavior of the peer-to-peer network.

Byzantine failures are modeled using the **Byzantine Synchronous Message-Passing model**, denoted **BSMP ⟨n, t⟩ [∅]** (following [8]), where:

- **n** is the total number of nodes in the system,
- **t** is the maximum number of Byzantine nodes,
- [∅] indicates that no additional assumptions (such as authentication, signatures, or trusted components) are made beyond synchrony and reliable communication.

Unless stated otherwise, Byzantine nodes are assumed to have full control over their local state and outgoing messages, while still being subject to the constraints of the underlying communication network.

2.5 Conclusion

In this chapter, we have introduced a formal framework for modeling peer-to-peer systems. Starting from the node as the fundamental computational entity, we modeled each participant as a state machine executing a common protocol and interacting with others through abstract communication channels.

We then represented the overlay network as a graph, whose vertices correspond to nodes and whose edges capture communication relationships. By introducing partial views, protocol steps, protocol cycles, and synchronization models, we provided a structured execution model that enables precise reasoning about the global evolution of the system. This framework was further extended to dynamic settings, where the overlay topology evolves over time due to protocol-driven neighbor updates and churn, naturally leading to a representation in terms of time-varying (temporal) graphs.

Within this abstraction, global system behavior emerges from repeated local interactions, allowing us to reason about properties such as self-organization, convergence, resilience, and fault tolerance in a principled manner. Modeling failures explicitly, and in particular crash failures, further grounds the framework in realistic peer-to-peer settings while preserving analytical tractability.

Having established this unified and abstract modeling framework—where a peer-to-peer system is viewed as a temporal graph whose nodes are state machines—we now turn to the existing literature on overlay management. The following chapter surveys the state of the art, and establishes why no existing protocol fully addresses our requirements. This sets the stage for our own contribution, introduced in the subsequent chapter.

Chapter 3

Overlay Management in Peer to Peer Systems

Peer-to-peer systems have a long history, from early file-sharing networks such as Napster and BitTorrent to anonymisation overlays such as Tor, and more recently to blockchain-based infrastructures. What these systems share is a common architectural principle: nodes communicate directly with one another without relying on a central coordinator, forming a logical overlay network on top of the physical Internet. A detailed account of this history and the motivations behind peer-to-peer architectures is provided in Appendix A.2.

These applications — from file sharing to distributed computing and decentralised finance — all rely on a common foundation: a logical overlay network that governs how peers discover one another, exchange messages, and maintain connectivity. Overlay management refers to the mechanisms used to construct, maintain, and adapt this logical topology. In centralized approaches, a single entity (or a small set of entities) is responsible for managing the network structure, which naturally leads to star or multi-star topologies. While such solutions are simple and efficient, they are not desirable in peer-to-peer systems, where decentralization, fault tolerance, and the absence of a single point of failure are key design goals. Consequently, overlay management in peer-to-peer networks must be performed in a fully decentralized manner.

This chapter builds upon the system model introduced in the previous chapter, in which the peer-to-peer system is viewed as a set of autonomous nodes interacting through a logical overlay network. In this model, the overlay abstracts the underlying physical network and defines the effective communication topology on which all higher-level distributed protocols operate. An overlay management algorithm can therefore be understood as a decentralized protocol whose primary objective is to ensure the creation, maintenance, and adaptation of this logical topology. By continuously managing neighbor relationships, such protocols must cope with the dynamic conditions inherent to peer-to-peer systems, including node arrivals, departures, and failures, while preserving desired structural properties of the overlay.

Definition 3.1 (Overlay Management)

An overlay management algorithm is a fully decentralized protocol (see Definition 2.2.24) whose goal is to construct, maintain, and adapt the logical topology of a peer-to-peer system. It governs how nodes establish and update their neighbor sets (also called their partial view, see Definition 2.2.21) in order to ensure connectivity, robustness to churn and failures, and structural properties required by higher-level distributed protocols.

♣

3.1 Metrics

Before surveying the existing overlay management protocols, we first establish the quantitative criteria against which they will be evaluated. In order to assess the effectiveness of an overlay management protocol, it is necessary to define metrics that capture the structural and dynamical properties of the resulting network. In an overlay network, the state of the system at a given instant can be represented as a graph snapshot of the underlying time-varying graph (as seen in Definition 2.3.8). Various metrics can then be computed on this graph in order to characterize the structure of the network, monitor its evolution over time, and compare different protocols. Metrics provide insights into connectivity, resilience, efficiency, and overall behavior of the network. In the context of time-varying graphs, these

metrics can be computed either on a single snapshot $G(t)$ or observed as time-dependent quantities $m(t) = m(G(t))$ that evolve as the network topology changes. Commonly used metrics include the indegree and outdegree distributions, the clustering coefficient, the average path length, and the network diameter.

Degree Distribution

The **indegree** and **outdegree** distributions are fundamental metrics that describe how connections are distributed among nodes at a given time. They are derived from the notions of degree introduced in Definition 2.2.12 and Definition 2.2.13. Formally, given a directed overlay graph $G = (V, E)$ at time t, the outdegree distribution is the distribution of $\text{outdegree}_G(v)$ over all $v \in V$, while the indegree distribution is the distribution of $\text{indegree}_G(v)$ over all $v \in V$.

Degree distributions serve as a primary tool for characterising the topology produced by an overlay management protocol. As the number of nodes may reach several thousands, direct graph visualisation becomes impractical; degree distributions provide a compact summary of the structural properties of the network. Moreover, deviations from the expected distribution — such as an unexpected concentration of indegree on a small subset of nodes — may signal undesired behaviour in the overlay protocol, such as load imbalance or the unintended emergence of bottlenecks.

In most unstructured peer-to-peer overlays, the outdegree is constrained by design and remains approximately constant, as it corresponds to the fixed size of the partial view maintained by each node. Consequently, structural heterogeneity primarily appears in the indegree distribution. In overlays that approximate random graphs, the indegree distribution typically follows a binomial distribution. In contrast, in scale-free or power-law overlays, the indegree distribution follows a power-law of the form $P(k) \sim k^{\gamma}$, leading to the emergence of highly connected nodes (hubs).

Clustering Coefficient

The clustering coefficient (see Definition 2.2.16) measures the tendency of nodes to form tightly connected groups, quantifying how likely it is that the neighbours of a node are also connected to each other. In a dynamic peer-to-peer network, the clustering coefficient can be computed at each time step on the snapshot $G(t)$, yielding a time-dependent metric that reflects the local cohesiveness of the network as it evolves. This metric is particularly useful for identifying the emergence of clusters or community structures.

In the context of overlay management, monitoring the clustering coefficient provides critical insights into the trade-offs between connectivity, routing efficiency, and maintenance overhead. A low clustering coefficient typically indicates a topology approaching a random graph, which favours uniform load distribution and short average path lengths. Conversely, a high clustering coefficient suggests denser, more locally interconnected structures that can enhance neighbourhood discovery and fault tolerance through redundant alternative paths, at the cost of increased link maintenance overhead and potential routing bottlenecks.

Average Path Length

The average path length, formally defined in Definition 2.2.8, measures the mean shortest-path distance between all pairs of distinct nodes in the overlay graph. In the context of time-varying overlays, this metric is computed at each time step on the current graph snapshot $G(t)$, yielding a time-dependent quantity $a(G(t))$ that reflects the evolution of the network's structural efficiency. A smaller average path length indicates a more efficient topology, as it implies that messages can traverse the network in fewer hops on average. Consequently, overlays with low average path length enable faster information dissemination, improved responsiveness, and better scalability.

Beyond raw routing efficiency, monitoring the average path length is crucial for assessing the robustness and adaptability of overlay management protocols under churn. A sudden increase in $a(G(t))$ often signals topological degradation, such as emerging bottlenecks, inadequate neighbor-replacement strategies, or early stages of network partitioning. Tracking this metric over time thus allows protocol designers to verify convergence toward theoretical routing guarantees and detect structural anomalies.

Diameter

The diameter of a graph, formally defined in Definition 2.2.9, captures the maximum shortest-path distance between any pair of nodes in the overlay network.

Similarly to the average path length, the diameter is computed at each time step on the current snapshot $G(t)$, yielding a time-dependent quantity $\mathrm{diam}(G(t))$ that reflects the worst-case communication distance in the network.

A small diameter is highly desirable, as it guarantees that even in the worst case, any node can reach another within a bounded number of hops. Overlays exhibiting small diameters provide strict upper bounds on message dissemination delay and improve robustness against network fragmentation. In contrast, a large or rapidly increasing diameter may signal structural inefficiencies, degraded connectivity, or early-stage partitioning.

In the context of overlay management, monitoring the diameter is essential for verifying worst-case routing guarantees and assessing the convergence speed of maintenance mechanisms under churn. While the average path length characterises typical communication latency, the diameter exposes tail-latency risks and worst-case bottlenecks that can critically impact time-sensitive applications, consensus protocols, or structured lookups. Together with the average path length, the diameter provides a complete picture of the overlay's latency distribution and structural resilience.

Metrics for Byzantine Resilience

Unlike crash failures, where topological and structural metrics (e.g., diameter, average path length, clustering coefficient, degree distribution) directly quantify network degradation, Byzantine failures do not admit a universal set of graph-theoretic indicators. Byzantine nodes can behave arbitrarily—sending malformed messages, lying about their state, or selectively connecting to honest peers—which means that standard overlay metrics may remain nominally stable while the protocol's correctness is silently compromised. Consequently, assessing Byzantine resilience requires metrics that are explicitly tied to the protocol's threat model, safety guarantees, and convergence properties.

In the context of overlay management, three complementary dimensions are typically monitored:

(i) **Protocol Liveness and Convergence:** The most fundamental metric is whether the overlay protocol continues to operate and eventually reaches a stable, desired configuration despite the presence of Byzantine nodes. This can be quantified by tracking the time-to-convergence, the success rate of membership operations (joins, leaves, repairs), or the fraction of protocol cycles that complete without deadlocks or livelocks. A resilient system should maintain bounded convergence times and preserve its target topology invariants under adversarial conditions.

(ii) **Byzantine Infiltration in Partial Views:** Since Byzantine nodes aim to remain embedded in the network to influence routing, data aggregation, or consensus, monitoring their representation in honest nodes' partial views provides a direct measure of containment. Relevant indicators include the average proportion of Byzantine peers in $P(i)$ across honest nodes i, the maximum Byzantine degree observed in the overlay, or the eviction rate of suspicious neighbours by honest participants.

(iii) **Overhead of Resilience Mechanisms:** When Byzantine-tolerant countermeasures are deployed (e.g., redundant messaging, consistency checks, reputation systems, or secure neighbour selection), their impact on system efficiency must be quantified. Key metrics include the relative increase in convergence time compared to the benign baseline, the additional state or bandwidth consumed per node, and the algorithmic complexity introduced in message routing or view maintenance. An effective resilience strategy minimises this overhead while guaranteeing safety, reflecting the classic trade-off between security and performance in distributed systems.

In the remainder of this chapter, we will analyse the main overlay management protocols described in the literature, comparing them on the basis of the metrics defined above.

3.2 Structured Networks

From an overlay management perspective, peer-to-peer networks can be broadly classified into structured and unstructured networks. This classification is determined by the protocol governing the construction and maintenance of the overlay, which constrains how nodes join, connect, and interact within the system.

In structured networks, each node is assigned a logical identifier, often derived from a hash function, and connections are established according to this identifier space. As a result, the network forms a well-defined topology that enables deterministic and efficient routing, typically with logarithmic complexity in the number of nodes.

When a node joins the network, it must follow the protocol rules to establish links only with a specific subset of authorized neighbors. For instance, in ring-based topologies, each node maintains connections with its immediate predecessor and successor, forming a logical ring. This structure guarantees that any node can be reached by traversing the ring in a finite number of hops.

Node departures, whether voluntary or due to failures, require the remaining nodes to reconfigure their connections in order to preserve the global topology. This maintenance process is essential to ensure the correctness of routing and data lookup operations, and is a defining characteristic of structured peer-to-peer systems.

Structured peer-to-peer networks are most commonly implemented through Distributed Hash Tables (DHTs), which provide a scalable and fully decentralized mechanism for data storage and retrieval. In a DHT, both nodes and data items are mapped to a shared logical identifier space, typically using consistent hashing[1]. Each data item is assigned to a node responsible for a specific region of this space, enabling an even distribution of storage and lookup responsibilities across the network. A DHT can be viewed as a decentralized database. When a node searches for data, or more generally for a key associated with a value, that it does not store locally, it issues a request to the network. This request is forwarded from node to node according to the routing protocol until it reaches the node whose identifier is closest to the searched key in the logical identifier space. This node is responsible for the key and returns the requested data to the requester.

A defining property of structured peer-to-peer networks is their predictable routing complexity. DHT-based systems typically guarantee that lookup operations are completed in $O(\log N)$ hops, where N denotes the number of participating nodes. This logarithmic bound is achieved through carefully designed routing tables that allow each node to forward requests to peers that are progressively closer to the target identifier in the logical space. Such guarantees sharply contrast with unstructured networks, where resource discovery often relies on flooding or random walks, resulting in higher and less predictable communication overhead.

[1] https://en.wikipedia.org/wiki/Consistent_hashing

Among the seminal works on Distributed Hash Tables is Chord [9]. Chord relies on consistent hashing to assign both nodes and keys to a shared identifier space, typically generated using a cryptographic hash function such as SHA-1. Nodes are logically organized in a ring topology, where each node is responsible for the keys whose identifiers fall between its predecessor and itself. In its simplest form, a ring-based organization would require up to N hops to locate a key in a network of N nodes. To address this limitation, Chord introduces a routing structure called the *finger table*. Each node maintains a finger table with m entries, where the i-th entry points to the successor of $\left(n + 2^{\{i\}}\right)$ in the identifier space, with n denoting the identifier of the current node. These additional links provide shortcuts across the ring and allow lookup operations to be completed in $O(\log N)$ hops with high probability.

Another well-known protocol for building structured peer-to-peer networks is Pastry [10]. In Pastry, each node is assigned a random identifier, called a *nodeId*, from a large identifier space. Each node maintains several data structures to support efficient and robust routing, including a *routing table* organized by shared identifier prefixes, a *leaf set* containing nodes with numerically closest nodeIds, and a *neighborhood set* composed of nodes that are close in terms of network proximity. When a node issues a request for a given key, the message is forwarded to nodes whose nodeIds share progressively longer prefixes with the key, until it reaches the node responsible for that key. This prefix-based routing strategy enables efficient lookup operations while exploiting network locality. Pastry is resilient to node failures, as it dynamically repairs its routing structures in response to node joins and departures.

Kademlia [11] is a widely used DHT protocol, notably employed by BitTorrent. As in Pastry, each node in Kademlia is assigned a random identifier (nodeId) from a large identifier space. Kademlia defines a distance between identifiers using the XOR metric, which induces a logical tree-like structure over the identifier space. Each node maintains a routing table composed of multiple *k-buckets*, where each bucket stores up to k node contacts whose identifiers fall within a specific range of XOR distances from the local node. When a node performs a lookup for a given key, it computes the XOR distance between the key and known node identifiers, and iteratively queries the nodes that are closest to the target key. This process is repeated until the node responsible for the key is reached. Kademlia supports parallel and iterative lookups, which improves robustness and resilience to node failures.

Not all structured networks are based on DHTs. For example, Tiara [12] is a structured overlay network that maintains a deterministic topology based on skip lists and skip graphs. Unlike DHTs, Tiara does not rely on consistent hashing to map keys to nodes or objects; instead, nodes are totally ordered according to their identifiers and connected following strict structural rules. A skip list is a layered linked structure where each node maintains forward pointers at multiple levels, allowing searches to be performed in logarithmic time by progressively skipping over nodes. A skip graph generalizes this idea to a distributed setting by organizing nodes into multiple overlapping ordered lists, which improves fault tolerance and connectivity. Tiara combines these concepts with self-stabilization mechanisms, ensuring that the overlay converges deterministically to the desired structured topology even in the presence of transient faults or churn.

We now assess structured overlay protocols against the criteria that matter for our objective: efficient and resilient decentralized learning.

Do structured overlays achieve small diameter and fast information propagation? Yes — this is precisely their design objective. Protocols such as Chord, Kademlia, and Pastry guarantee logarithmic diameter in the number of nodes, ensuring that any message reaches any destination in $O(\log N)$ hops. Information propagation is therefore efficient by construction, and theoretical bounds are well established.

Are they resilient to crash failures? Not robustly. Structured overlays maintain correctness only as long as the fraction of failed nodes remains below a protocol-specific threshold. Beyond this threshold,

the structure itself breaks: routing tables become inconsistent, lookups fail, and the overlay may partition. Even below the threshold, recovery requires time to detect the failure and repair the affected connections, during which the protocol's guarantees do not hold.

Are they resilient to churn? Only under moderate churn rates. High churn — frequent and unpredictable node arrivals and departures — prevents the overlay from ever reaching a stable configuration. Maintenance traffic grows rapidly as the protocol attempts to continuously repair a structure that is constantly being disrupted, and in extreme cases the overhead of maintenance can dominate the available bandwidth.

Are they resilient to Byzantine failures? This is a fundamental weakness of structured overlays. Because the rules governing node placement and connection are public and deterministic, Byzantine nodes can exploit this knowledge to position themselves strategically within the topology — occupying critical routing positions, corrupting lookup results, or systematically isolating honest nodes. The very structure that makes these overlays efficient also makes them predictable and therefore exploitable by adversaries.

Is the maintenance overhead acceptable? It is significant. Building the initial structure requires a coordinated bootstrapping phase. Every node join or departure triggers a sequence of connection updates to preserve the structural invariants, generating overhead that grows with the rate of topological change. In large and dynamic networks, this overhead can become a limiting factor.

Are structured overlays flexible and application-agnostic? No — this is perhaps their most fundamental limitation for our purposes. Structured overlays are typically co-designed with a specific application, such as distributed hash tables or key-value stores. The overlay structure reflects the data model of the application, making it difficult to repurpose for a different use case without redesigning the protocol. Furthermore, to the best of our knowledge, no structured overlay protocol natively supports the notion of hub nodes — nodes with elevated connectivity and aggregation responsibility — which is the core mechanism we seek to introduce.

Conclusion. Structured overlays offer strong guarantees on diameter and routing efficiency, but these come at the cost of fragility under failures and churn, vulnerability to Byzantine adversaries, significant maintenance overhead, and tight coupling to specific application semantics. For the objective of this thesis — a resilient, efficient, and application-agnostic decentralized learning framework — structured overlays are not the right foundation. This motivates the study of unstructured overlay protocols, which trade deterministic routing guarantees for greater robustness, flexibility, and adaptability to dynamic environments.

3.3 Unstructured Networks & Peer Sampling

Unlike structured systems, unstructured networks do not impose a predefined topology or DHT-based organization; instead, nodes establish and maintain connections in an ad-hoc fashion or rely on peer-sampling services to dynamically discover peers, favoring resilience and adaptability over deterministic lookup guarantees.

Because of the absence of a global structure, unstructured P2P networks rely on a set of fundamental services to ensure connectivity, information dissemination, and resource discovery. A core component of such systems is the *peer sampling service*, whose role is to provide each node with a continuously refreshed, quasi-random subset of peers. This service is essential to preserve network connectivity, avoid topological bias, and prevent partitioning, especially in the presence of churn.

In addition to peer sampling, unstructured P2P networks typically require several complementary services. *Membership management* mechanisms are used to handle node arrivals and departures, ensuring that local neighbor sets remain up-to-date. *Neighbor selection and topology management* strategies

may be employed to shape the overlay according to specific objectives, such as latency reduction or load balancing. Furthermore, *information dissemination services*, often based on gossip or epidemic protocols, enable efficient broadcast, aggregation, and synchronization of state across the network. Finally, *resource discovery* in unstructured networks generally relies on probabilistic techniques such as flooding, random walks, or gossip-based search, trading deterministic guarantees for scalability and resilience. Together, these services allow unstructured peer-to-peer networks to operate efficiently in highly dynamic and decentralized environments, making them particularly suitable for large-scale systems where strict structural maintenance would be costly or impractical. As part of our work, **we focus on the peer sampling service**, which constitutes a fundamental building block of unstructured peer-to-peer systems.

A peer sampling service provides each node with addresses of other nodes in the network, thereby enabling communication and maintaining connectivity. Indeed, it is often impractical or impossible for a node to store the addresses of all other nodes in the network in memory, particularly in peer-to-peer networks, which can often contain several hundred thousand nodes. Furthermore, these nodes do not necessarily remain connected all the time, and the number of nodes in the network fluctuates constantly. Each node will therefore only maintain a limited number of addresses of other nodes in the network, known as the partial view or neighbor list of each node. There is therefore a need for a service that allows us to connect to other nodes in the network, particularly given the risk that all our neighbors may be disconnected and that we may thus find ourselves disconnected from the network due to a lack of neighbors with whom to communicate. Typically, the objective of a peer sampling service is to supply node addresses that are as random and uniformly distributed as possible, so as to avoid structural bias and network partitioning.

Definition 3.3.1 (Peer Sampling Oracle)

An ideal peer sampling oracle is a random variable S taking values in V, distributed according to a probability distribution π over V.

Each call to the oracle returns an independent sample drawn according to π.

♣

Conceptually, a peer sampling service exposes a minimal interface composed of two main functions. An initialization function, *init()*, initializes the service at the node level, while a function *getPeer()* returns the address of another node in the network.

Peer sampling services can be implemented in a centralized manner, where a central entity maintains knowledge of all participating nodes and responds to sampling requests. In this case, the central entity acts as a sampling oracle, having global knowledge of the network state and being able to return node identifiers according to a prescribed probability distribution, typically uniform. While such an approach is simple, it suffers from scalability limitations and introduces a single point of failure.

Alternatively, peer sampling can be implemented in a fully decentralized way, where nodes continuously exchange and update peer information using only local interactions. Decentralized peer sampling services are more scalable and allow the construction of fully decentralized peer-to-peer systems that don't need to rely on a centralized service.

Since the goal of a peer sampling service is to return a uniformly random node, the overlay graph induced by such a service naturally converges toward a random graph (see Appendix A.1.0.7).

Definition 3.3.2 (Decentralized Peer Sampling Service)

A decentralized peer sampling service is a randomized local function

$$S_u : \mathrm{L}_u \mapsto V$$

executed by each node $u \in V$, where L_u denotes the local state of u (typically including its partial view).

The output of S_u induces a probability distribution $\hat{\pi}_u$ over V that approximates a target distribution π.

Formally, for all $v \in V$,

$$\hat{\pi}_{u(v)} \approx \pi(v).$$

♣

Among the earliest decentralized protocols addressing membership management and peer sampling in large-scale distributed systems is Lightweight Probabilistic Broadcast (lpbcast), proposed by P. T. Eugster, R. Guerraoui, S. B. Handurukande, P. Kouznetsov, and A.-M. Kermarrec [13]. Lpbcast is a gossip-based publish/subscribe protocol designed to disseminate membership information and application events in highly dynamic and unstructured peer-to-peer environments. In this protocol, each node periodically exchanges gossip messages with a subset of peers selected from its local partial view. These messages encapsulate subscriptions, unsubscriptions, and published events, allowing nodes to progressively build and maintain a probabilistic view of the system. To ensure bounded resource consumption, lpbcast relies on fixed-size buffers to store received messages. When these buffers become full, messages are discarded using a probabilistic eviction strategy, favoring the removal of stale or redundant information while preserving dissemination efficiency. Node joins are handled through subscription messages initially sent to a single node and subsequently propagated through gossip; with high probability, these messages eventually reach all active nodes. Unsubscriptions follow a similar dissemination process but include timestamps to prevent outdated leave information from persisting indefinitely in the network. A key aspect of lpbcast is its peer weighting mechanism, which assigns weights to known peers based on the frequency of received notifications. Peers with higher weights are more likely to be removed from local views, a strategy that promotes peer diversity and prevents topological convergence, thereby reducing the risk of network partitioning under churn. The protocol offers probabilistic delivery guarantees rather than deterministic reliability, trading strict consistency for improved scalability, fault tolerance, and adaptability to dynamic membership. The authors provide a theoretical analysis of message dissemination time and partition probability, demonstrating that lpbcast achieves rapid and reliable propagation with limited overhead. Experimental evaluation combines simulations with real-world deployments conducted on two local-area networks composed of 60 and 65 SUN Ultra 10 workstations, respectively, interconnected via Fast Ethernet. Experiments involving up to 125 concurrent processes, each publishing 40 events per gossip round, confirm the protocol's ability to scale while maintaining acceptable latency and network load.

A. Montresor, H. Meling, and Ö. Babaoğlu [14] proposes an early conceptual framework for building distributed systems that are self-organizing, self-repairing, and resilient, drawing strong inspiration from complex adaptive systems and biological metaphors such as ant colonies. Rather than focusing on a single protocol, the authors advocate a system-level approach in which global behavior emerges from simple local interactions between autonomous agents. This vision is materialized through the Anthill project, whose objective is to provide a generic framework for the design and deployment of peer-to-peer applications built as swarms of agents. In Anthill, agents interact with and modify a shared environment, enabling adaptive behaviors such as dynamic reconfiguration, fault tolerance,

and recovery from failures without centralized control. The framework also serves as an experimental platform for studying the fundamental properties of CAS-based peer-to-peer systems, allowing researchers to evaluate scalability, robustness, and performance under dynamic conditions such as churn. This work laid important conceptual foundations for later gossip- and epidemic-based protocols by emphasizing emergence, decentralization, and adaptability as first-class design principles in large-scale distributed systems.

Astrolabe, introduced by R. Van Renesse, K. P. Birman, and W. Vogels [15], is a distributed information management system designed to support scalable monitoring, management, and data mining in large-scale distributed environments. Unlike fully decentralized peer-to-peer systems, Astrolabe relies on a hierarchical zoning architecture in which nodes are organized into nested zones forming a tree rooted at a global root zone. Each zone elects one or more designated routers responsible for aggregating and propagating information between hierarchical levels. While Astrolabe employs epidemic gossip protocols for information dissemination within and across zones, the presence of explicitly defined zones and routing roles makes the system fundamentally hierarchical rather than purely peer-to-peer. Within each zone, nodes periodically exchange state information using gossip, while inter-zone communication follows the hierarchy, with updates being propagated toward the least common ancestor zone and aggregated along the way. Each node maintains a peer list of logarithmic size relative to the system, ensuring scalability. Security is enforced through the use of public-key certificates, allowing authenticated communication and administrative control. Although Astrolabe avoids centralized servers, it is not fully decentralized: the hierarchical topology is manually configured, and system administrators are able to maintain a global view of the system state. The protocol was evaluated through simulations and experimentally deployed on the Emulab testbed with up to 126 agents running on 63 hosts, demonstrating the scalability and robustness of hierarchical gossip-based aggregation.

In contrast to hierarchical approaches, A. J. Ganesh, A.-M. Kermarrec, and L. Massoulié introduced SCAMP [16], one of the first fully decentralized peer-to-peer membership protocols designed for large-scale gossip-based systems. SCAMP abandons the assumption that nodes have access to a global membership list or even to the total number of participants, assumptions that are unrealistic in the presence of churn. Instead, each node maintains a partial view of the network, whose size remains logarithmic in the number of nodes, ensuring scalability and robustness. SCAMP relies on epidemic dissemination to propagate membership information and guarantees, with high probability, the strong atomicity property, meaning that a broadcast message eventually reaches all nodes. More precisely, if each node gossips to $\log(n) + k$ peers on average, the probability of complete dissemination converges asymptotically to $e^{-e^{-k}}$. Membership dynamics are handled through explicit join and leave procedures. When a node joins the system, its identifier is disseminated through controlled random walks, and recipient nodes probabilistically decide whether to include it in their partial view, allowing the overlay to self-balance without any global coordination. Each node maintains both a partial view, representing its outgoing neighbor references, and an in-view list, corresponding to incoming references from other nodes. This distinction is exploited during graceful departures: when a node leaves, it informs its neighbors, which replace its identifier using the entries contained in its in-view list, preserving the desired average degree of the overlay. To cope with failures and churn, SCAMP incorporates heartbeat-based failure detection, lease mechanisms that periodically expire neighbor relationships, and re-subscription procedures for isolated nodes. An indirection mechanism with decreasing tokens further prevents overload and contributes to maintaining balanced partial views.

Araneola [17] is a multicast system designed to provide scalable and reliable message dissemination in dynamic network environments. Unlike fully connected overlays, Araneola maintains a sparse undirected network in which each node has either K or K+1 neighbors, forming a K-connected graph

that ensures resilience to node failures. Membership information is exchanged infrequently, primarily to integrate new nodes or perform maintenance, which reduces overhead and improves scalability. By carefully structuring the network and limiting the number of connections per node, Araneola achieves reliable multicast delivery while remaining efficient in highly dynamic conditions.

A. Stavrou, D. Rubenstein, and S. Sahu [18] introduced PROOFS (P2P Randomized Overlays to Obviate Flash-crowd Symptoms), a lightweight protocol designed to cope with Internet flash crowds and sudden surges in demand for data in a network. PROOFS relies on the construction and continuous maintenance of a random overlay network through a shuffling mechanism, combined with a random-walk-based object lookup protocol. The authors provide theoretical guarantees showing that the resulting directed overlay exhibits strong resilience properties, including the ability to self-heal to avoid network partitions, ensuring eventual connectivity despite churn and failures. The shuffle operation consists of a periodic exchange of partial neighbor views between pairs of randomly selected peers: Each node stores a local peer list, called a **cache**, of fixed size c. Periodically, every Δ time units, a node initiates a **shuffle operation** to refresh its view. The node first selects a random subset of l peers from its cache and randomly chooses one peer q from this subset as the shuffle partner. The chosen peer is removed from the subset, and the initiating node adds its own address to the subset. This subset is then sent to q. Upon receiving the shuffle request, q replies with a randomly selected subset of its own cache. The initiating node removes its own address and any entries already in its cache from the received subset, preventing duplicates. Finally, the cache is updated by incorporating the remaining entries, maintaining the cache size c. Repeated execution of this exchange continuously reshapes the overlay into a random directed graph, which underlies the robustness and self-healing properties of PROOFS. In Figure 3 we give the pseudocode of the protocol and in Figure 4 we give an example of a shuffling operation. In this example, the Node 1 sends addresses {itself, 2, 3} to Node 4. Node 4 sends back {5,6,8}. After this operation, Node 1 removes from its cache the addresses of nodes 2,3 and 4, thus removing its connection to these nodes, and replaces them with the addresses received from Node 4 (addresses 5,6 and 8). Node 4 does the same, removing the connections of nodes 5, 6, 8 and replacing them with the addresses 1, 2 and 3. After this operation, the overlay remains connected, but the neighbor relationships have been updated according to the shuffle: the connection between Node 1 and Node 4 is now directed from Node 4 to Node 1.

```
   initial peer list: cache
   cache size: c
   shuffle length: l
   node address: p
1  loop
2      wait(Δ)
3      subset ← selectRandomSubset(cache, l)
4      cache.remove(subset)
5      q ← selectRandom(subset)
6      subset.remove(q)
7      subset.add(p)
8      send(q, subset)
9      subset_q ← receive(q)
10     subset_q.remove(p)
11     subset_q.removeAll(cache)
12     cache.add(subset_q)
```

Figure 3: PROOFS algorithm.

Figure 4: Before and after a shuffling operation in the PROOFS protocol.

M. Jelasity, R. Guerraoui, A.-M. Kermarrec, and M. Van Steen [19] proposed a generic framework for implementing gossip-based peer sampling services. The protocol relies on only two core functions: init() to initialize the service and getPeer() to retrieve a peer address. The authors analyze peer sampling protocols along three dimensions: (i) peer selection — random, head (most recently added), or tail (oldest node), (ii) view propagation — push, pull, or push-pull, and (iii) view selection — random, head, or tail. These dimensions yield 27 possible combinations, including lpbcast [13] (rand, rand, push) and Newscast [20] (rand, head, push-pull). A good peer sampling service should satisfy several properties: a uniform degree distribution to avoid hubs, a short average path length for scalability, and a moderate clustering coefficient to reduce message redundancy. Experimental results show that some combinations have limitations: (head, any, any) tends to create a highly clustered network, (any, tail, any) is poorly resilient under dynamic node joins and leaves, and (any, any, pull) converges to a star topology. In contrast, protocols (any, rand, pushpull) provide the closest approximation to a random topology in terms of average path length and clustering coefficient, while (rand, head, any) produces a degree distribution close to uniform. Overall, the resulting topologies exhibit small-world properties, ensuring both connectivity and resilience in unstructured networks.

In their extended work, M. Jelasity, S. Voulgaris, R. Guerraoui, A.-M. Kermarrec, and M. Van Steen [21] proposed a refined gossip-based peer sampling protocol (based on Newscast) along with a framework for systematic evaluation. During each exchange, the active thread selects a peer from the local peer list, sends a buffer containing half of its freshest entries (and optionally its own address), receives a buffer from the remote peer if pull is enabled, and updates its peer list by combining entries from both buffers while removing duplicates and old items, maintaining a fixed cache size. In Figure 5 and

Figure 6 we provide the pseudocodes for the Newscast protocol. The protocol introduces two tunable parameters: H, the self-healing parameter controlling the removal of stale links, and S, the swap parameter prioritizing entries received from the remote peer. These parameters define a triangular design space with three archetypes: blind (H=0, S=0), maintaining the same subset; healer (H=c/2), keeping the freshest entries; and swapper (H=0, S=c/2), maximizing swaps. Experimental results show that a push-pull approach outperforms push-only or pull-only strategies, which are prone to partitioning or star-like topologies. Furthermore, robustness against churn is enhanced by dropping old entries in the peer list, although a tradeoff exists between load balancing and churn resilience.

```
   initial peer list: cache
   cache size: c
   node address: p
   healing parameter: H
   swapped parameter: S
1  loop
2      wait(Δ)
3      n ← selectRandom(cache)
4      if push
5          buffer ← (address=p, age=0)
6          cache.permute()
7          buffer.append(cache.head(c/2-1))
8          send(buffer, n)
9      else
10         send(null, n)
11     if pull
12         buffer_p ← receive(n)
13         cache ← cache.select(c, H, S, buffer_p)
14         cache.increaseAge()
```

Figure 5: Newscast algorithm (active thread).

```
1  loop
2      buffer_p ← receive(n)
3      if pull
4          buffer ← (address=p, age=0)
5          cache.permute()
6          buffer.append(cache.head(c/2-1))
7          send(buffer, n)
8      cache ← cache.select(c, H, S, buffer_p)
9      cache.increaseAge()
```

Figure 6: Newscast algorithm (passive thread).

The paper **Correctness of a Gossip-Based Membership Protocol** [22] proposes a peer-sampling protocol in which each node executes the algorithm periodically and independently in an asynchronous manner. At each round, a node i first builds a list L1 by collecting the local views of f peers selected uniformly at random from its current peer list. In parallel, it constructs a second list L2 containing the identities of all nodes that have contacted i during the current round to request its peer list. The node then updates its peer list by randomly selecting k nodes from the union of L1 and L2, where a weighting parameter ω biases the selection toward entries from L2 relative to L1. New nodes join the system by initially contacting a bootstrap server. The authors provide formal correctness proofs showing that the protocol has a very low probability of network partitioning and that, despite churn due to node joins and leaves, the overlay converges with high probability to a random graph topology.

Among the protocols that have contributed to the state of the art, we also have Cyclon [23]. The paper that presents the Cyclon protocol defines a lightweight gossip-based membership protocol designed to maintain random unstructured overlays at low cost. The authors first recall the PROOFS protocol [18], where peers periodically exchange subsets of their neighbor lists. CYCLON improves upon this approach by associating an age value with each neighbor entry and by always initiating the shuffle with the oldest known peer, which accelerates the removal of stale links and improves robustness

under churn. Experimental results show that the resulting overlay converges to the properties of a random graph rather than a small-world topology: the average path length remains very small, while the clustering coefficient decreases exponentially as the network size grows. To support efficient node joins while preserving randomness, a new peer contacts an introducer node and sends c shuffle messages that perform random walks in the overlay. Each of the c nodes reached by these messages replies with one of its neighbors and replaces that neighbor with the address of the joining peer. This join procedure allows newcomers to rapidly acquire a random peer list without disrupting the overall randomness of the network.

Another interesting approach is L. Massoulié, E. Le Merrer, A.-M. Kermarrec, and A. Ganesh [24]. The main contribution of the work is to show that random walk mechanisms can be used both to estimate global properties of the network and to obtain unbiased peer samples, without requiring global knowledge or structured overlays. The authors first propose the Random Tour method to estimate the size of a peer-to-peer network. In this approach, a counter is forwarded along a random walk and incremented at each visited node by a value proportional to $\frac{\varphi(i)}{d(i)}$, where $d(i)$ is the node degree and $\varphi(i)$ is a function of interest (the identity function when estimating network size). When the counter returns to the initiator, it is scaled by the initiator's degree to obtain an unbiased estimate of the network size. The accuracy of this estimate is shown to depend on the spectral gap of the overlay graph, highlighting the importance of good expansion properties. The paper then introduces Sample and Collide, a second technique based on Continuous Time Random Walks (CTRW), whose key novelty is to provide unbiased peer sampling. In this method, a value T is propagated through the network and decremented at each hop by a random amount derived from an exponential distribution parameterized by the node degree. When T reaches zero, the current node reports its identifier to the initiator. By collecting multiple such samples and stopping when repeated samples occur, the initiator can estimate the network size using maximum likelihood estimation. Experimental results demonstrate that Sample and Collide is robust under churn, reinforcing the paper's central message that random walk–based methods can be used for decentralized measurement and sampling in dynamic overlay networks.

In H. Terelius and K. H. Johansson [25], the authors address the problem of efficiently disseminating a data stream from a small set of seed nodes to all other participants in a peer-to-peer network subject to churn. The authors propose a distributed protocol that incrementally constructs a so-called gradient topology, starting from an initially random overlay. In this topology, nodes are organized according to a utility function that captures their performance characteristics, such as bandwidth or forwarding capacity, with higher-utility nodes positioned closer to the seed nodes that act as data sources. Each node locally adapts its set of neighbors in order to maximize its own utility, which indirectly improves the global efficiency of data dissemination by favoring high-capacity paths near the sources. The paper provides a formal analysis of the protocol using Markov chain techniques, proving convergence toward the desired gradient graph, characterizing the expected convergence time, and showing that the topology can re-converge efficiently even in the presence of churn.

HyParView [26] is a membership protocol designed to support reliable gossip-based broadcast in large and dynamic networks. Each node maintains two views: an active view consisting of established TCP connections used for message dissemination, and a passive view that contains additional nodes that can replace failed members of the active view. The passive view is maintained using a cyclic strategy, where each node periodically performs a shuffle operation with a randomly selected node to update and refresh its list. This dual-view structure allows HyParView to provide both reliability and resilience to node failures while keeping the network overhead low. HyParView is used as the membership protocol of some frameworks to build distributed networks, like Partisan [27].

X-BOT [28] is an evolution of the HyParView protocol, developed by the same authors, designed to optimize unstructured overlay networks while preserving resilience. Like HyParView, each node

maintains an active and a passive view to support reliable gossip-based communication. The main innovation in X-BOT is the introduction of a local "oracle" that evaluates the quality of the node's connections according to a given criterion, such as network latency or other performance metrics. Each node iteratively adjusts its connections to maximize the oracle's score, using only local information, until the overlay topology is optimized. This approach allows X-BOT to adapt the network structure to improve efficiency and performance while retaining the fault tolerance and low overhead properties of the original HyParView protocol.

Readers who wish to explore gossip networks in more depth can refer to this survey [29], which not only explains the mechanics of peer sampling in gossip systems but also addresses broader topics such as information dissemination, epidemic modeling, and aggregation techniques.

We now apply the same evaluative lens to unstructured overlay protocols, assessing them against the criteria that matter for our objective.

A first group of protocols — including lpbcast [13] and Astrolabe [15] — must be set aside immediately: despite operating in large-scale networks, they rely on hierarchical structures or centralized oracles to coordinate membership and dissemination. This reintroduces exactly the kind of control point we seek to eliminate, and they are therefore incompatible with our decentralization objective.

The remaining protocols — SCAMP [16], PROOFS [18], Cyclon [23], Newscast [21], HyParView [26], and their variants — form the core of the gossip-based and random-walk-based peer sampling literature. We now assess them on the same criteria as structured overlays.

Do gossip-based overlays achieve small diameter and fast information propagation? The diameter is reasonable — gossip protocols typically produce overlays with small-world properties, where the average path length grows logarithmically with the number of nodes. However, information propagation is inherently slow: without any global aggregation phase, information spreads through successive pairwise exchanges, and convergence requires many rounds. More fundamentally, the absence of structure makes global aggregation structurally impossible — no node has sufficient visibility over the network to orchestrate it, which means that the slow dissemination is not merely a performance issue but a fundamental limitation of the paradigm.

Are they resilient to crash failures and churn? Yes — this is the primary strength of gossip-based overlays. Random k-out graph topologies remain connected with high probability even when a large fraction of nodes fail or leave the system. Protocols such as Cyclon and Newscast are specifically designed to handle churn gracefully, continuously refreshing partial views through periodic shuffles. The absence of structural invariants means there is nothing to break when nodes depart unexpectedly.

Are they resilient to Byzantine failures? No. In a gossip-based overlay, a Byzantine node can freely manipulate the partial views of its neighbors by injecting false peer addresses, withholding entries, or systematically biasing the sampling distribution. Because there is no mechanism to verify the integrity of exchanged peer lists, Byzantine nodes can corrupt the overlay topology silently and persistently, without any honest node being able to detect the manipulation.

Is the maintenance overhead acceptable? Yes — this is another advantage of gossip-based protocols. Maintenance consists of periodic lightweight shuffle operations between pairs of nodes, with no global coordination required. The overhead per node is bounded and does not grow with network size, making these protocols highly scalable.

Are gossip-based overlays flexible and application-agnostic? Yes. Unlike structured overlays, gossip-based peer sampling services expose a minimal interface — typically just getPeer() — and impose no constraints on the application layer. The same protocol can serve as the substrate for learning, aggregation, broadcast, or any other distributed algorithm, without modification.

Do they support hub election? No. Gossip-based protocols are explicitly designed to produce uniform degree distributions, preventing the emergence of highly connected nodes. The absence of degree heterogeneity is a deliberate design choice motivated by load balancing and resilience, but it makes native hub election impossible within this framework. Furthermore, the additional randomness introduced by the absence of structure adds significant uncertainty to convergence speed, making it difficult to provide meaningful bounds on dissemination time in practice.

Conclusion on gossip-based overlays. These protocols offer strong resilience, low overhead, and good flexibility, but they fundamentally cannot support global aggregation or hub election. The very properties that make them robust — uniformity, randomness, absence of structure — are precisely what prevents them from achieving the aggregation efficiency we seek.

All the previously discussed protocols primarily aim at constructing random k-out graph topologies, in which each node maintains approximately k outgoing links and the distribution of incoming degrees follows a binomial law centered around k. Such topologies are known for their strong resilience to node crashes and churn: even when a large fraction of nodes fail or leave the system, the overlay graph remains connected with high probability. However, this robustness comes at a cost in terms of performance. In particular, information dissemination may be suboptimal, and some nodes can experience relatively high incoming degrees, leading to increased load. This naturally leads us to examine a distinct family of unstructured overlays: **power-law and scale-free networks** (see Appendix A.1.0.9). These topologies deliberately introduce degree heterogeneity, allowing some nodes to accumulate significantly more connections than others. This heterogeneity is precisely the property that underlies the notion of hubs — nodes with elevated connectivity that can serve as natural aggregation points. We therefore examine this family in more detail, as it represents the closest existing approximation to the hub-based overlay structure we aim to construct.

3.4 Peer sampling in Power-law networks

Several peer sampling and overlay management protocols are explicitly designed to construct or maintain power-law topologies. Unlike the protocols discussed in the previous section, which produce random graphs as a natural byproduct of uniform peer sampling, these protocols actively engineer degree heterogeneity. This is typically achieved by biasing neighbor selection or rewiring mechanisms toward high-degree nodes, thereby shaping the overlay toward a target power-law degree distribution. The goal is to exploit the presence of hubs — nodes that concentrate a disproportionate share of connections — to improve efficiency, scalability, and information dissemination. Power-law and scale-free overlays represent the closest existing family of protocols to our objective, and we therefore examined this literature in depth.

Gia [30] is an unstructured peer-to-peer overlay designed to address the scalability limitations of early Gnutella-like systems by explicitly accounting for heterogeneity in node capacities. Unlike structured overlays or DHT-based systems, Gia preserves the flexibility and robustness of unstructured networks while introducing mechanisms that significantly improve query efficiency and load management. The system is built around the notion of supernodes, although this role is not statically assigned: instead, nodes with higher capacity naturally emerge as better connected and more central in the overlay. A key observation underlying Gia is that the effective capacity of a peer is multidimensional and depends on several factors, including processing power, disk access latency, and, most importantly, available network bandwidth. Traditional unstructured overlays treat all nodes uniformly, which leads to severe bottlenecks when low-capacity nodes become transit points for large volumes of queries. Gia addresses this issue through a combination of dynamic topology adaptation and explicit flow control. The topology adaptation protocol continuously reshapes the overlay so that most nodes remain within a small number of hops from high-capacity peers. Each node independently evaluates its local connectivity using a satisfaction metric S, defined as a value in the interval [0,1]. This metric captures

how well a node's current neighbors meet its performance expectations in terms of query handling and forwarding capacity. When a node's satisfaction level is below one, it actively seeks better neighbors, typically favoring peers with higher advertised capacity. This process continues until the node reaches a stable configuration in which its satisfaction is maximized, leading to a topology where high-degree nodes are also those most capable of sustaining heavy query loads. To prevent overload and ensure fair utilization of resources, Gia introduces a token-based flow control mechanism. Nodes periodically distribute flow-control tokens to their neighbors, where each token authorizes the transmission of a single query. A node may only forward a query to a neighbor if it holds a valid token from that neighbor, effectively bounding the rate at which any node can receive queries. Token allocation is aligned with the node's processing capacity: nodes issue tokens at a rate proportional to the number of queries they can handle. If a node experiences congestion, either due to excessive incoming queries or insufficient tokens from its neighbors, it temporarily queues excess queries and adapts by reducing its token distribution rate, thereby throttling incoming traffic. An important aspect of Gia's design is the incentive mechanism for truthful capacity advertisement. Rather than distributing tokens uniformly among neighbors, nodes allocate tokens in proportion to their neighbors' advertised capacities. As a result, nodes that declare higher capacity receive more tokens for issuing their own queries, directly benefiting from honest reporting. This feedback loop encourages high-capacity nodes to expose their true capabilities, reinforcing the formation of a topology where connectivity and load are aligned with available resources.

A. Montresor [31] introduces SG-1, a gossip-based protocol designed to construct and maintain superpeer overlay networks in a fully decentralized and adaptive manner. The core idea is to let nodes periodically exchange local state information with randomly selected peers, including their role (client or superpeer) and their current load. Based solely on this local knowledge, nodes can autonomously change roles or reassign clients in order to balance load and reduce the overall number of superpeers. As a result, the system converges toward a configuration in which each client is attached to exactly one superpeer, superpeers are interconnected through an approximately random overlay, and the set of superpeers is close to minimal with respect to the aggregate capacity required to serve all clients. A key design choice of SG-1 is to build the superpeer topology as an additional overlay extracted from an existing connected network, rather than replacing the underlying topology. Any protocol capable of maintaining connectivity can be used for this base layer; in the paper, a gossip-based peer sampling protocol is employed to provide an approximately random connected graph. This layered approach significantly improves robustness, as it allows the system to recover even if a large fraction of superpeers fail simultaneously. In such cases, affected clients can temporarily promote themselves to superpeers, after which the gossip process gradually selects a new, balanced set of superpeers among the remaining nodes. The protocol is shown to be highly efficient, with a total message overhead that scales linearly with network size and without concentrating excessive load on any single node. Moreover, the time needed to reach a near-optimal superpeer configuration is constant with respect to the number of nodes, while convergence to an optimal configuration grows only logarithmically. Experimental results indicate fast convergence in practice and show that the resulting superpeer topology exhibits heterogeneous connectivity patterns resembling power-law networks, combining scalability with robustness under churn and failures.

Phenix [32] is a peer sampling protocol designed to construct and maintain resilient, low-diameter peer-to-peer topologies while preserving scalability under churn and adversarial conditions. The protocol is motivated by the observation that low-diameter networks exhibit an average shortest-path length of $O(\log n)$, enabling efficient information dissemination at scale. While unstructured peer-to-peer networks are generally resilient to churn and crashes, they often suffer from limited performance, whereas structured overlays provide better performance at the cost of reduced robustness. Phenix aims to reconcile these trade-offs by introducing heterogeneous connectivity patterns inspired by

real-world networks. Unlike approaches based on homogeneous random graph topologies, Phenix explicitly constructs power-law degree distributions, where the probability that a node has degree K follows $p(K) \sim K^{-\gamma}$. The parameter γ controls the level of heterogeneity and is empirically close to 2.2 for the Internet topology. This results in the natural emergence of a small number of highly connected nodes, or hubs, which significantly reduce the network diameter and improve information dissemination. Importantly, Phenix is among the first peer-to-peer topology construction protocols to explicitly consider resilience against targeted attacks that aim to remove highly connected nodes in order to fragment the network. Phenix builds upon the principle of **preferential attachment**, according to which new nodes tend to establish links with nodes that are already highly connected. This mechanism, originally introduced in the seminal work of Barabási and Albert, provides a simple generative explanation for the emergence of power-law degree distributions in large-scale networks. By biasing attachment toward high-degree nodes, preferential attachment naturally amplifies degree heterogeneity and leads to the formation of hubs. In the context of overlay management, preferential attachment can be implemented in a decentralized manner by probabilistically favoring neighbors with higher degree during connection establishment. This local rule is sufficient to drive the global topology toward a power-law structure while preserving scalability.

When a node i joins the network, it first obtains a list of peer addresses either by contacting a host cache server or by using a locally stored cache from a previous session. This list is then divided into two subsets: *random_nodes* and *friends_nodes*. Node i sends a ping message with a time-to-live (TTL) of 1 to all nodes in the *friends_nodes* set. Each of these nodes replies by sending its own neighbor list to node i and forwards the ping message to its neighbors while decrementing the TTL and incrementing a hop counter. All nodes that receive this message, corresponding to friends-of-friends, temporarily store node i in a list called γ for a duration τ, which serves as a protection mechanism against crawling attacks. Node i aggregates all received neighbor lists into a set of *candidate_nodes* and ranks them according to their frequency of occurrence. Nodes that appear most frequently are selected as *preferred_nodes*, reflecting their higher structural importance in the local topology. Node i then establishes connections with nodes in both the *random_nodes* and *preferred_nodes* sets. When a node m receives a connection request from node i, it creates a backward connection to node i only if node i appears in its γ list and if the backward connection counter remains below a threshold that bounds the maximum number of backward connections as a function of the node's incoming degree. This constraint prevents excessive hub formation while preserving the desired power-law structure. If node i receives a backward connection from a node m, it removes m from the *preferred_nodes* list and adds it to a *highly_preferred_nodes* list. The final peer view maintained by node i thus consists of *random_nodes*, *preferred_nodes*, *highly_preferred_nodes*, and backward connections, collectively enforcing a heterogeneous topology with low diameter. In Figure 7 and Figure 8, we give the pseudocode of the algorithm used when a node enters the network, constructing its cache with **random** and **preferred** nodes, and the pseudocode of the background thread. To protect the network from malicious behavior, Phenix incorporates several defensive mechanisms. Nodes attempting to crawl the network are detected and blacklisted. Backward connection lists are never shared, protecting highly connected nodes from being explicitly identified. Additionally, when a node detects that its number of connections has dropped below a predefined threshold, as may occur after a targeted attack, it enters a maintenance mode in which it temporarily favors the selection of preferred nodes over random nodes to rapidly restore connectivity. The protocol considers multiple adversarial strategies, including attackers that form tightly connected subgraphs to artificially increase their likelihood of becoming preferred nodes, as well as attackers that behave honestly before simultaneously disconnecting to induce network fragmentation. Simulation results show that Phenix significantly outperforms random topologies in terms of resilience: even with 30% malicious nodes, approximately 70% of the network remains connected after an attack. Phenix was implemented and evaluated on a real PlanetLab testbed

composed of 81 nodes distributed across 43 sites in eight countries. Experimental results confirm the emergence of a heterogeneous degree distribution, with most nodes maintaining between three and four connections and a small number of nodes acting as hubs with significantly higher degrees, reaching up to 18 connections. When these highly connected nodes were deliberately shut down, the network recovered to a stable state in less than one second, demonstrating strong resilience to targeted failures.

```
   cache size: c
   initial peer list: cache
   initial backward list: backward_peers (empty)
   number of preferential connections: s
 1 G_random, G_friend ← split(cache)
 2 cache ← {}
 3 cache.append(G_random)
 4 G_candidates ← {}
 5 for peer in G_friend
 6 |   neighbor_list ← send(peer, CACHE_REQUEST)
 7 |   G_candidates ← G_candidates ∪ neighbor_list
 8 sort(G_candidates)
 9 G_preferred ← G_candidates[0..(s-1)]
10 for peer in G_preferred
11 |   send(peer, CONNECTION_REQUEST)
12 cache.append(G_preferred)
```

Figure 7: Phenix algorithm (initialisation).

```
cache size: c
peer list: cache
gamma list: Γ
initial backward list: backward_peers (empty)
number of preferential connections: s (fixed at c/2)
backward connection counter: c_m (initially 0)
backward connections constant: γ (fixed at 20)
loop
    Γ.removeOldItems()
    request, peer ← receive()
    if request = CACHE_REQUEST
        send(cache, peer)
        for node in cache
            send(node, PING_REQUEST)
    if request == PING_REQUEST
        Γ.add(peer)
    if request == CONNECTION_REQUEST
        c_m ← c_m + 1
        if c_m >= γ
            backward_peers.add(peer)
            c_m ← c_m - γ
```

Figure 8: Phenix algorithm (background thread).

LLR [33] is a peer-to-peer network construction scheme designed to achieve low diameter, location awareness, and resilience. Nodes join the network individually, obtaining an initial peer list from a bootstrap server. Each node measures physical proximity to its peers using hop counts on the underlying Internet topology and selects the closest peers to form its neighbor set. A preferential attachment mechanism is then applied among these nearby nodes to strengthen connectivity. The protocol also includes a rewiring procedure, allowing nodes to replace distant connections with closer ones while maintaining the preferential connectivity. LLR is explicitly designed to handle churn and potential Byzantine behaviors, and its evaluation relies on topological data from real-world networks such as Abilene and Sprint. No simulation results are reported, but the design emphasizes physical proximity and robustness in dynamic network conditions.

V. Vishnumurthy and P. Francis [34] addresses the construction of heterogeneous unstructured overlay networks and the efficient selection of random nodes within them. The authors propose practical algorithms that adapt the number of outgoing links a node establishes based on its capacity, allowing high-capacity nodes to maintain more connections and thus achieve heterogeneity in the network. New nodes joining the network require knowledge of at least one existing member, which can be facilitated by a well-known rendezvous node. A key contribution is the SwapLinks mechanism, designed to counteract the self-reinforcing effect where early or high-degree nodes accumulate disproportionately many incoming links. SwapLinks actively redistributes incoming links from high-degree nodes to lower-degree nodes, maintaining balance in the network. Additionally, the approach leverages biased random walks during the graph construction and node selection processes, ensuring that connectivity remains efficient while preserving resilience to churn. Simulations indicate that this methodology produces scalable and robust overlay networks suitable for dynamic environments.

A. Brocco, F. Frapolli, and B. Hirsbrunner [35] proposes a self-organized overlay construction algorithm that aims at achieving a bounded network diameter without relying on hubs. Inspired by biological systems, the approach uses lightweight agents, referred to as "ants", which are periodically sent by nodes to explore the network and collect topological information. Based on the feedback brought by these ants, nodes locally adapt their connections in order to reduce the overall diameter while avoiding highly connected central nodes. Although the construction process is distributed in spirit, the algorithm assumes global knowledge of the network topology to guide optimization decisions, and it relies on a master entity to enforce certain disconnection operations. As a result, the system achieves low-diameter overlays with balanced degrees, but at the cost of stronger assumptions that limit its applicability in fully decentralized and purely peer-to-peer environments.

H. Guclu and M. Yuksel [36] investigates how to construct scale-free overlay networks for unstructured peer-to-peer systems while explicitly limiting the emergence of hubs. The network is assumed to grow incrementally, with nodes joining one at a time, and nodes are assumed to know the total network size. The key idea is to preserve the benefits of scale-free topologies while enforcing a hard cut-off on node degree, thereby preventing any node from accumulating an excessive number of connections. This degree cap applies to peer connections and ensures a bounded level of heterogeneity. As a reference, the paper also considers the Configuration Model to generate random networks with a prescribed degree distribution, although this approach requires global knowledge and is therefore not directly applicable in decentralized settings. To overcome this limitation, the authors propose two distributed algorithms that rely only on local information. The first, Hop-and-Attempt Preferential Attachment, builds connections by iteratively selecting neighbors of neighbors: a joining node first connects to a random node, then probabilistically attempts to connect to one of its peers, and repeats this process until its peer list is filled or the degree constraints are met. The second approach, Discover-and-Attempt Preferential Attachment, leverages information from the underlying physical network to discover candidate peers and apply a similar preferential attachment mechanism. Both algorithms approximate scale-free degree distributions under bounded degree constraints.

S. Eum, S. Arakawa, and M. Murata [37] proposed a self-organizing mechanism for constructing scale-free topologies in peer-to-peer networks, with the explicit goal of reconciling desirable structural properties of power-law graphs with the practical constraints of decentralized systems. Nodes are assumed to join the network incrementally, one after another, reflecting a realistic P2P setting. To enable the initial contact with the network, the authors assume the existence of a bootstrapping server whose sole role is to return identifiers of randomly selected peers already present in the overlay. This assumption is kept minimal and does not require the server to maintain or expose any global view of the topology. The proposed model is governed by two parameters that directly shape the resulting degree distribution. The first parameter, m, represents the number of links that a newly joining peer establishes upon arrival. The second parameter, α, controls the balance between random attachment and preferential attachment. More precisely, when a new peer seeks to establish a connection, the endpoint is chosen according to a mixed strategy: with probability α, the attachment is purely random, while with probability $(1 - \alpha)$, the attachment follows a preferential rule favoring already well-connected peers. By tuning α, the algorithm allows a fine-grained control over the resulting power-law exponent, enabling the construction of a wide range of scale-free topologies. A key contribution of this work lies in the fully decentralized realization of this mixed attachment process. Rather than requiring global knowledge of node degrees or network size, all decisions are delegated to existing peers. In practice, the joining peer N first contacts the bootstrapping server to obtain the identifiers of m randomly selected peers in the current overlay. For each such peer D, the joining node asks D to suggest an attachment target. Peer D then returns either its own identifier with probability α, or the identifier of one of its neighbors with probability $(1 - \alpha)$. The new peer finally connects to the peer whose identifier is returned. As a result, the preferential attachment effect emerges implicitly

through local neighbor selection, without ever exposing degree information or global topology data to the joining node. This design choice has important robustness and security implications. Since the new peer does not observe the structure of the overlay and only follows connection decisions made by existing peers, the algorithm naturally limits the information available to a potentially malicious node. The authors argue that this property improves resilience against targeted attacks, as attackers cannot easily infer or exploit high-degree nodes. Moreover, the simplicity of the local rules makes the construction robust under churn: although node arrivals and departures slightly perturb the topology, the scale-free characteristics are largely preserved over time. Beyond topology construction, the paper evaluates the functional benefits of the resulting overlays. In particular, the authors demonstrate that the constructed scale-free networks improve search efficiency when using common P2P search mechanisms such as flooding and random walks. The presence of highly connected nodes accelerates query dissemination, while the adjustable attachment parameter α mitigates the well-known drawback of classical scale-free networks, namely the excessive load placed on a very small number of hubs.

T-MAN [38] is a gossip-based protocol designed for fast and fully decentralized construction of overlay network topologies that approximate a desired target structure. The core idea of the protocol is to view topology management as a distributed ranking problem: each node maintains a preference ordering over other nodes according to some application-defined distance or ranking function, and attempts to connect to those that rank highest with respect to this function. Unlike static overlays, T-MAN continuously refines the topology through gossip exchanges, allowing it to adapt to dynamic environments. In its most naive form, such a ranking problem could be solved by having each node broadcast its identifier to the entire network, collect the full list of nodes, and then locally sort them according to the ranking criterion. While this approach is conceptually simple, it is clearly not scalable. T-MAN replaces this global dissemination with an epidemic exchange of node descriptors, where each node periodically communicates with a small number of peers and incrementally improves its local view of the network. A key contribution of T-MAN is that it generalizes the notion of distance beyond simple metrics such as identifier proximity. The ranking function can encode arbitrary criteria, including physical proximity, node capacity, or application-specific attributes. Each node locally ranks the descriptors it knows and retains those corresponding to the most preferred peers. Through repeated gossip exchanges, high-quality descriptors propagate quickly through the system, leading to rapid convergence toward the target topology. The paper highlights an important design trade-off related to the choice of the ranking method. If the ranking is independent of the base node, meaning that all nodes use the same global ranking criterion, the protocol naturally induces a star-like or hub-centered topology. In this case, many nodes are attracted to the same high-ranking peers, which accelerates convergence because these central nodes are contacted frequently and can rapidly collect and redistribute high-quality descriptors.

VICINITY [39] builds directly upon the principles introduced by T-MAN and can be seen as an explicit improvement of that approach, aimed at increasing robustness and convergence quality in dynamic environments. Like T-MAN, VICINITY addresses overlay topology construction as a distributed ranking problem, where each node seeks to connect to peers that are closest according to an application-defined distance function. The target topology thus emerges from local decisions driven by ranking and gossip-based exchanges. The main limitation identified in T-MAN is its strong determinism: nodes always try to optimize their neighborhood strictly according to the ranking function. While this leads to fast convergence, it can also cause premature convergence to suboptimal local structures, sensitivity to churn, and excessive load on highly ranked nodes. VICINITY mitigates these issues by explicitly introducing controlled randomness into the neighbor selection process, striking a balance between deterministic optimization and random exploration. To achieve this, VICINITY combines T-MAN-style ranking with mechanisms inspired by peer sampling protocols such as Cyclon. Each node maintains a neighborhood that is periodically refreshed using both structured exchanges

and random peer samples. The use of Cyclon ensures that the overlay remains well mixed and that nodes continue to discover new peers, preventing the topology from becoming rigid or trapped in poor configurations. In addition, neighbor selection is performed in a round-robin fashion, which avoids repeatedly contacting the same peers and improves fairness in communication. A key insight of the paper is that optimal performance is obtained by carefully balancing determinism and randomness. Experimental results show that allocating roughly half of the neighbor selection to ranking-based choices and half to random choices yields the best trade-off between convergence speed, stability, and robustness. Too much determinism reproduces the weaknesses of T-MAN, while too much randomness slows convergence and degrades the quality of the final topology.

UMM [40] is a self-organizing group communication overlay that combines a random base overlay with dynamically constructed, source-specific multicast dissemination trees. The base overlay is initialized as a random graph and is continuously refined using a local "short–long" heuristic that distinguishes latency-optimized short tunnels from bandwidth-optimized long tunnels, replacing existing connections only when measurable improvements exceed a stability threshold. This lower layer is responsible for bootstrapping, membership management, fault recovery, and connectivity maintenance, and is designed to be independent from higher-level dissemination mechanisms. Multicast trees are then implicitly extracted from the base overlay: new sources initially flood the network, and participating nodes locally suppress redundant or low-quality paths based on observed duplicate traffic. Membership information is maintained through an epidemic gossip mechanism that uniformly spreads random peer identifiers and supports scalable joins without requiring global knowledge, although a bootstrap node is used in practice. The system explicitly targets churn, and experimental results show very high delivery ratios even under aggressive failure rates, indicating that the separation between a resilient random base overlay and adaptive dissemination structures yields both robustness and efficiency.

E. Bulut and B. K. Szymanski [41] addresses the problem of constructing **limited** scale-free overlays that closely follow a target power-law degree distribution while remaining practical and cost-efficient for peer-to-peer systems. The main contribution is the introduction of two parameterized growth algorithms, SRA and SDA, which allow the designer to explicitly control the desired scale-free exponent, thereby achieving a high adherence to scale-freeness without creating extreme hubs. Both approaches rely on incremental node addition and focus exclusively on link creation at join time, avoiding costly global rewiring operations. The Semi-Randomized Growth Algorithm (SRA) operates without global knowledge and uses randomized degree targets derived from the desired power-law distribution. A joining node samples target degree values, broadcasts a request, and connects to responding peers whose current degrees match these targets, relaxing the constraints toward nearby degree values when necessary. In contrast, the Semi-Deterministic Growth Algorithm (SDA) assumes knowledge of the total network size and deterministically computes the degree that each node should maintain to preserve the target distribution. Nodes advertising matching degrees respond to new peers, which then establish connections accordingly. While both algorithms can guarantee scale-free properties only for a bounded range of exponents and do not handle churn, they demonstrate that accurate and tunable power-law overlays can be constructed efficiently using join-time decisions alone.

V. Marza, M. Dehghan, and B. Akbari [42] proposes a hybrid overlay topology for peer-to-peer video streaming that explicitly combines insights from complex network theory with awareness of the underlying physical topology. Rather than relying solely on abstract graph properties, the approach introduces a position-based construction mechanism aimed at aligning the logical P2P overlay with physical proximity, thereby reducing communication costs and improving streaming efficiency. The authors argue that, since many real-world networks simultaneously exhibit scale-free and small-world properties, an overlay that integrates both characteristics is better suited to practical deployment than models that enforce only one of these properties. The topology construction starts from a minimal

motif, a fully connected triangle, which represents an initial set of servers at a CDN-like level and is deliberately chosen such that the three nodes are physically far apart. New peers are then added incrementally and connect to their three closest neighbors, creating a recursive spatial partitioning of the network. Each new insertion subdivides the existing regions into smaller zones, leading to a hierarchical structure that reflects both physical location and logical connectivity. As the network grows, this iterative process yields an overlay that naturally combines clustering, short path lengths, and heterogeneous degree distribution. Experimental results indicate that this hybrid scale-free and small-world topology provides higher robustness to random failures and malicious attacks, outperforming classical small-world and scale-free overlays in terms of resilience while remaining well adapted to the requirements of video streaming applications.

J.-H. Park [43] introduces a distributed algorithm for constructing scale-free networks through preferential rewiring, without requiring network growth. In this model, all nodes start with an inherent attractiveness reflecting their computational power or availability, and initially maintain only a single link. At each time step, nodes randomly sample a limited set of peers and attempt to establish a bidirectional connection with the most attractive candidate. If the candidate node accepts—by comparing the requesting node's attractiveness with that of its existing neighbor—both nodes rewire their links, replacing connections to less attractive nodes. This iterative process drives the emergence of a power-law degree distribution with a scaling exponent of 2.5, as confirmed analytically and via Monte Carlo simulations. Notably, the resulting networks exhibit ultra-small diameters, scaling as $O(\ln \ln N)$ for $2 < \gamma < 3$. The algorithm operates without churn, relying solely on local decisions and random sampling, yet efficiently produces highly heterogeneous, scale-free topologies that reflect intrinsic node attributes.

C. T. Diggans, J. Fish, and E. M. Bollt [44] investigate how constraints on node connectivity influence the emergence of hierarchical structures in networks. Building on the classical Barabási–Albert preferential attachment model [45], they introduce conductance-based limits on the number of connections a node can maintain. By systematically restricting link capacity, the study demonstrates that bottlenecks in connectivity naturally induce hierarchical organization, where high-degree nodes occupy central positions while lower-degree nodes are relegated to peripheral roles. This work highlights the interplay between local degree constraints and global network topology, showing that hierarchy can emerge as an intrinsic property of scale-free systems when structural limitations are enforced.

As seen, the diversity of approaches in the litterature is striking, but a careful reading reveals that most protocols in this family fail to meet one or more of our requirements. We begin by setting aside those that are incompatible with our constraints before assessing the remaining ones.

Several protocols must be dismissed immediately. Gia [30] and SG-1 [31] introduce the notion of superpeers, but in doing so they break the symmetry of the overlay: not all nodes participate equally in the protocol, and the distinction between clients and superpeers is either statically assigned or requires a separate coordination layer. A second group of protocols — including LLR [33], and the bootstrapping-dependent schemes of [37] — rely on an external oracle or bootstrap server not merely for initial contact but as a structural component of the construction process, reintroducing a dependency that we seek to eliminate. A third group — including Phenix [32] — requires a continuous influx of new nodes to sustain the scale-free structure: the power-law degree distribution emerges from network growth, and the protocol has no mechanism to maintain it in a stable or churning network where arrivals and departures are balanced. Finally, several protocols explicitly work against the emergence of hubs: [36] enforces hard degree caps, [35] actively avoids highly connected nodes, and [41] introduces parameterized controls to limit hub formation. These protocols pursue a different objective than ours and are therefore incompatible with our design.

What remains after these eliminations is a small set of protocols that are genuinely decentralized, do not require continuous growth, and allow hubs to emerge — notably T-MAN [38], VICINITY [39], and the rewiring-based approach of [43]. We now assess them on our standard criteria.

Do scale-free overlays achieve small diameter and fast information propagation? Yes — this is their primary advantage over uniform gossip overlays. The presence of highly connected hub nodes dramatically reduces the network diameter, enabling faster information dissemination. Protocols such as [43] analytically establish ultra-small diameters scaling as $O(\ln \ln N)$ for appropriate power-law exponents. Hub nodes act as natural relay points, accelerating the spread of information across the network.

Are they resilient to crash failures and churn? Partially. Scale-free networks are well known to be resilient to random failures: because most nodes have low degree, a randomly selected failing node is unlikely to be a hub, and the network remains connected. However, they are highly vulnerable to targeted failures: removing even a small fraction of the highest-degree nodes can rapidly fragment the network. Under churn, maintaining the power-law structure is non-trivial — protocols that rely on growth dynamics lose their structural properties when the network stabilizes, and active maintenance mechanisms such as those in VICINITY and T-MAN are needed to preserve degree heterogeneity over time.

Are they resilient to Byzantine failures? No — and the situation is arguably worse than in uniform gossip overlays. Hub nodes, precisely because they are highly connected and central, represent high-value targets for Byzantine adversaries. A Byzantine node that successfully impersonates or corrupts a hub can disproportionately affect information dissemination and aggregation across the entire network. Furthermore, protocols based on preferential attachment are inherently susceptible to Sybil-style attacks, where a malicious node artificially inflates its apparent degree or attractiveness to become a hub.

Is the maintenance overhead acceptable? It varies significantly across protocols. Growth-based approaches incur low ongoing overhead but cannot maintain the topology under churn. Active maintenance protocols such as T-MAN and VICINITY require continuous gossip exchanges to preserve the target degree distribution, with overhead comparable to standard gossip protocols. Rewiring-based approaches such as [43] are lightweight but operate under the assumption of a stable membership.

Are scale-free overlays flexible and application-agnostic? Partially. T-MAN and VICINITY are notably flexible: the ranking function that drives topology construction is application-defined, allowing the same protocol to target different structural objectives. However, this flexibility comes at the cost of requiring the application to specify a meaningful distance or ranking criterion, which is not always straightforward.

Do they natively support hub election for aggregation? This is the critical point. While scale-free overlays do produce nodes with elevated connectivity, this elevated connectivity is a passive structural property — it is not coupled to any active role in the protocol. High-degree nodes in a scale-free overlay are not aware of their status, do not volunteer for aggregation duties, and are not explicitly used as aggregators by the learning layer. The emergence of hubs in the topological sense does not translate into a mechanism for structured aggregation. Furthermore, the identity of hubs changes continuously as the topology evolves, making it difficult for the learning layer to rely on them in a stable manner.

Conclusion on scale-free overlays. This family of protocols comes closest to our objective among all existing approaches, and the intuition underlying it — that degree heterogeneity can accelerate information dissemination — is directly relevant to our work. However, none of the existing protocols simultaneously achieves full decentralization, stability under churn, Byzantine resilience, and explicit hub election with an active aggregation role. The structural emergence of high-degree nodes is a

necessary but not sufficient condition for efficient decentralized learning: what is needed is a protocol that not only produces hubs but actively elects them, assigns them a defined aggregation responsibility, and maintains this structure robustly under failures and adversarial conditions. To the best of our knowledge, no existing peer sampling protocol achieves this combination. This gap defines the precise contribution of the Elevator protocol. Before introducing it, however, we examine one remaining family of peer sampling protocols that addresses a dimension not yet covered in our analysis: Byzantine resilience. While none of the protocols surveyed so far provide meaningful guarantees against adversarial participants, a dedicated line of work has tackled this problem directly. Understanding its contributions and limitations will complete our picture of the state of the art and further motivate the design choices made in Elevator and Lift.

3.5 Byzantine-Resilient protocols

Byzantine attacks [46] refer to arbitrary and potentially malicious behaviors by nodes in a distributed system. Unlike crash failures or omission faults, Byzantine nodes may deviate from the protocol in unpredictable ways, such as sending inconsistent messages to different peers, forging data, or coordinating with other malicious nodes to subvert the system. These attacks pose a significant threat to the robustness and correctness of distributed protocols, especially in decentralized environments where trust assumptions are minimal. In the context of peer sampling and hub selection protocols, Byzantine nodes can manipulate their local views to influence the overlay topology and gain disproportionate visibility. Designing protocols resilient to such behaviors is therefore essential to maintain reliability, fairness, and convergence guarantees under adversarial conditions.

As noted in the introduction to this chapter, Byzantine-resilient peer sampling is a young and narrow research area. Because the number of protocols is limited and their designs are closely derived from the gossip protocols already described in detail above, we present them here more concisely, focusing on their key contributions and differences rather than their full algorithmic descriptions.

Among Byzantine fault-tolerant protocols, the Brahms protocol [47] relies on a gossip-based peer sampling algorithm resistant to flooding attacks, in which Byzantine nodes attempt to propagate their identifiers widely to bias local views. Brahms achieves unbiased ID sampling from potentially biased histories using min-wise independent permutations. Similarly, [48] proposes a mechanism capable of producing uniform ID samples while adapting to dynamic environments. Basalt [49] builds upon Brahms and introduces a ranking function to determine cache updates. Secure Peer Sampling [50] extends the Newscast gossip protocol [21] by incorporating cryptographic keys, a certificate authority, and a hub-detection mechanism to mitigate malicious behavior. SecureCyclon [51], derived from Cyclon [23], introduces additional security features enabling nodes to detect and blacklist malicious participants that violate the peer sampling protocol. Finally, AUPE [52] leverages trusted hardware components (e.g., Intel SGX) to monitor and control the dissemination of identifiers within the system.

Byzantine-resilient peer sampling represents a relatively young and narrow research area. Rather than proposing fundamentally new overlay architectures, the existing literature is predominantly focused on hardening existing gossip protocols against adversarial participants. The set of protocols is small, and most contributions follow a common pattern: take an established gossip-based peer sampling protocol, identify a specific Byzantine attack vector, and introduce a targeted countermeasure. We apply the same evaluative lens as before, focusing only on the criteria where this family differs from standard gossip-based overlays.

Before doing so, one protocol must be set aside. Phenix [32], already discussed in the scale-free section, is the only protocol in this family that targets a power-law topology while incorporating Byzantine resilience mechanisms. However, as noted earlier, Phenix relies on continuous node arrivals to sustain

its scale-free structure, and its Byzantine defenses are designed around this growth assumption. It therefore does not constitute a viable foundation for a stable, churn-resilient, hub-based overlay.

The remaining protocols — Brahms [47], Basalt [49], Secure Peer Sampling [50], SecureCyclon [51], and AUPE [52] — all target random graph topologies and share the same structural baseline as the gossip protocols from which they derive. Their behaviour with respect to diameter, information propagation speed, crash and churn resilience, flexibility, and hub election is therefore identical to that of standard gossip overlays, as assessed in the previous section. Only three criteria differ meaningfully.

Are they resilient to Byzantine failures? Yes — this is their primary and defining contribution. Brahms [47] resists flooding attacks using min-wise independent permutations to produce unbiased samples from potentially biased histories. SecureCyclon [51] introduces blacklisting mechanisms to detect and exclude nodes that violate the protocol. Secure Peer Sampling [50] adds cryptographic verification to Newscast. However, the Byzantine resilience guarantees remain protocol-specific and are not always formally proven under realistic threat models. The field is young, and the coverage of adversarial scenarios remains incomplete.

Is the maintenance overhead acceptable? It is higher than for standard gossip protocols. Cryptographic operations, certificate management, and identity verification all introduce additional computation and communication per gossip round. AUPE [52] goes further by relying on trusted hardware components such as Intel SGX, which introduces a dependency on specific hardware.

Is decentralization preserved? Not always. The reliance on certificate authorities in Secure Peer Sampling [50] and on trusted hardware in AUPE [52] reintroduces centralized trust assumptions that partially undermine the decentralization objective.

Conclusion on Byzantine-resilient peer sampling. This literature confirms that Byzantine resilience in peer sampling is achievable, but at a cost — in overhead, in complexity, and in some cases in decentralization. More importantly, all existing protocols inherit the fundamental limitations of the gossip paradigm: no global aggregation, no hub structure, and slow information dissemination. The field remains young, with limited coverage of adversarial scenarios and no protocol that simultaneously achieves Byzantine resilience, hub election, and aggregation efficiency.

This completes our survey of the state of the art. The gap identified across all three families — gossip, scale-free, and Byzantine-resilient — defines the design space that Elevator and Lift are built to occupy, and to which we now turn.

3.6 Conclusion

The survey presented in this chapter reveals four complementary families of overlay management approaches: structured overlays providing deterministic routing, unstructured peer sampling protocols sustaining low-cost random-like graphs, power-law topologies leveraging controlled heterogeneity for efficient dissemination, and Byzantine-resilient mechanisms securing peer sampling against malicious behavior.

Table 1: Comparison of overlay protocol families against the criteria relevant to decentralized learning. No existing family simultaneously achieves small diameter, fast dissemination, strong resilience, low overhead, full decentralization, and hub election.

Family	Diameter	Dissemination	Crash resilience	Churn resilience	Byzantine resilience	Overhead	Decentralized	Flexible	Hub election
Structured	Small	Fast	Weak	Weak	Weak	High	Yes	No	No
Hierarchical / Oracle	Small	Fast	Moderate	Moderate	Moderate	Moderate	No	No	No
Gossip / Random walk	Small	Slow	Strong	Strong	Weak	Low	Yes	Yes	No
Power-law / Scale-free	Small	Fast	Moderate	Weak	Weak	Moderate	Partial	Partial	No
Byzantine-resilient gossip	Small	Slow	Strong	Strong	Strong	High	Partial	Yes	No

Table 1 summarizes the findings of this survey across the criteria most relevant to decentralized learning. As the table illustrates, each family of protocols offers a distinct trade-off: structured overlays achieve small diameter and fast dissemination but lack resilience and flexibility; gossip-based protocols are robust and fully decentralized but suffer from slow dissemination and high overhead under Byzantine conditions; power-law topologies improve dissemination efficiency but remain fragile under churn and provide no Byzantine guarantees; and Byzantine-resilient gossip protocols offer strong security at the cost of increased overhead and partial decentralization. Crucially, no existing family simultaneously satisfies all the criteria we require — in particular, none supports hub election while maintaining full decentralization, strong resilience, and low overhead.

Beyond this core gap, the design of next-generation decentralized overlays raises several additional open challenges, including fairness and incentive alignment, physical-topology awareness, secure bootstrapping, and resource efficiency on edge devices. These questions, while important, fall outside the scope of this thesis. Our work focuses specifically on the challenge identified throughout this chapter: the absence of a fully decentralized mechanism for controlled hub election and formation. The contribution presented in Chapter 4 addresses this precise challenge by proposing a distributed protocol that steers the overlay toward a target number of hubs, guarantees bounded recovery times after hub failures, and maintains stability under churn.

Chapter 4

Emergent Peer-to-Peer Multi-Hub Topology

4.1 Introduction

As established in Chapter 3, no existing overlay management protocol simultaneously achieves full decentralization, controlled topology shaping, and hub election. Yet, the presence of highly connected nodes in a peer-to-peer network would be highly desirable in a number of practical settings. In decentralized federated learning, such nodes could serve as aggregators, collecting and redistributing machine learning models across the network, playing a role analogous to that of the central server in classical federated learning. Beyond federated learning, similar benefits would arise in file sharing systems, where well-connected nodes could act as high-availability relay nodes, or in distributed storage architectures, where they could serve as replication anchors. In all these settings, the key property is the same: a small number of nodes acting as bridges between the rest of the network, ensuring that any two nodes are at most two hops apart and thus keeping the network diameter at two.

However, relying on statically designated nodes introduces well-known vulnerabilities: a fixed, publicly known hub is a natural target for adversarial attacks. What is needed instead is a mechanism that allows such nodes to emerge organically from the network itself, without central coordination, while remaining controllable in number and resilient to failures and churn.

This chapter presents Elevator, a protocol designed precisely for this purpose. Elevator allows selected nodes to naturally ascend to hub status through a process we term *hub sampling*, by hybridizing two fundamental concepts: *preferential attachment* and *random attachment*. This combination promotes a balanced network structure where hubs emerge organically based on connectivity patterns, yet adapt to dynamic network changes. The parameter h, representing the desired number of hubs, provides flexibility and control over the resulting topology, enabling tailored configurations to suit specific application requirements.

4.2 Elevator Protocol

Elevator introduces a new abstraction in overlay network management, called the **hub sampling service**. This service can be viewed as a generalization of the traditional peer sampling service: instead of returning arbitrary random peers, it aims to enable the random emergence and maintenance of a controlled number of hub nodes within the overlay.

Definition 4.2.1 (Hub)

Let $G = (V, E)$ be a directed overlay graph representing the state of the network, where each node $v \in V$ maintains a partial view $P(v) \subseteq V$.

In the context of the Elevator protocol, a node $h \in V$ is defined as a **hub** if its identifier appears in the partial view of every node in the network, including itself:

$$\forall v \in V, \quad h \in P(v)$$

♣

Equivalently, a hub is a node to which all other nodes are connected, functioning as a highly accessible focal point in the overlay. In practical terms, this is analogous to the role of a server in a centralized network, providing a shortcut for communication and information dissemination.

The objective of a hub sampling service is to promote the appearance of globally reachable nodes that structurally reduce communication distances while preserving decentralization. Unlike centralized mechanisms, this service operates without global knowledge and must adapt dynamically to network changes.

Definition 4.2.2 (Decentralized Hub Sampling Service)

Let V be the set of nodes in the network and let $h \in \mathbb{N}$ be the desired number of hubs, with $1 \leq h \leq |V|$.

A decentralized hub sampling service is a protocol that, starting from an arbitrary overlay configuration, induces the emergence of a subset $H \subseteq V$ such that $|H| = h$ and every node in H satisfies the hub property defined in Definition 4.2.1.

♣

Ideally, the resulting hub set H is drawn according to a uniform distribution over all subsets of V of size h, that is:

$$\Pr(H = S) = \frac{1}{\binom{|V|}{h}}$$

for every subset $S \subseteq V$ with $|S| = h$.

In practice, the decentralized protocol approximates this ideal uniform selection using only local information and without global coordination.

Elevator is a decentralized peer-to-peer overlay management protocol designed to implement this hub sampling service. It autonomously guides the network toward a desired number of hubs while maintaining structural efficiency. Its operation relies on a combination of preferential attachment and random attachment mechanisms to balance hub emergence with connectivity.

At a high level, hub emergence is driven by a preferential attachment dynamic. At each protocol step, nodes tend to select the most frequently observed identifiers within their local neighborhood. As this behavior is executed independently by all nodes, highly referenced nodes become increasingly visible, which amplifies their selection probability and leads to the natural emergence of hubs after a few protocol cycles.

In parallel, hubs are used as sources of randomness: since they are present in the partial view of every node, requesting a random identifier from a hub is equivalent to sampling a node uniformly at random in the network. This mechanism allows the protocol to preserve uniformly random outgoing connections alongside the preferential ones. As a result, after a small number of cycles, the overlay stabilizes around the desired topology consisting of h hubs and uniformly random remaining connections.

Having established a formal definition of hubs and of a hub sampling service, we now turn to the desired properties of the Elevator protocol, which characterize its behavior and performance beyond the aspect of hub emergence.

4.2.1 Desired Properties

The Elevator protocol is designed to satisfy a set of fundamental structural and dynamical properties that characterize the quality and usefulness of the maintained overlay.

Connectivity is an indispensable property. An overlay management protocol that fails to maintain global connectivity is of limited practical value, as network partitions prevent nodes from communicating and undermine any distributed application running on top of the overlay. Therefore, Elevator must ensure that the network remains connected despite churn and topology adaptations.

Low-diameter is a central objective of our approach. The introduction of hubs is precisely motivated by the need to reduce communication distances across the network. A small diameter implies that messages can reach any node within a limited number of hops, enabling fast information dissemination, efficient aggregation, and improved responsiveness. By promoting the emergence of well-connected hub nodes, Elevator aims to maintain short paths between arbitrary pairs of nodes.

Convergence refers to the protocol's ability to autonomously drive the network toward the desired topology, characterized by a controlled number of hubs. Starting from an arbitrary initial configuration, the overlay should progressively evolve toward the target structural properties in a timely manner. Rapid convergence is particularly important in dynamic environments where network conditions continuously change.

Stability ensures that once the desired structural properties are reached, they are maintained over time. The overlay should not exhibit excessive oscillations or structural instability that could degrade performance or increase maintenance overhead.

Finally, **Robustness** captures the resilience of the protocol to churn and targeted attacks. Since hubs play a central structural role, the protocol must tolerate their potential failure and allow new hubs to emerge when necessary, thereby preserving connectivity and low-diameter properties even under adverse conditions.

Some of these properties will be formally analyzed in the theoretical study preceding the simulation section, where we provide analytical arguments and proofs for key structural guarantees. The remaining aspects will be empirically evaluated through simulation experiments to assess the overall effectiveness and reliability of the Elevator protocol.

4.2.2 Approach

To achieve both robustness and a low network diameter, we integrate two fundamental concepts: preferential attachment and random attachment, each serving distinct yet complementary roles in shaping the network topology.

Preferential Attachment. Drawing from the concept pioneered by Barabási and Albert [45], preferential attachment dictates that new connections in the network are established preferentially with nodes possessing a higher number of existing connections. In our adaptation, we modify this concept to elevate certain nodes to the status of hubs without requiring the network to continuously grow. Instead of new nodes joining and preferentially connecting to highly connected nodes, each existing node leverages information from its neighbors to identify and connect to the most frequently connected nodes (up to a predefined number h). This mechanism enables the organic emergence of hubs within the network, with selected nodes naturally assuming central roles based on their connectivity without any explicit distinction other than their number of incoming links.

Random Attachment. Inspired by gossip-based peer sampling algorithms [18], [21], random attachment ensures that nodes maintain connections with a representative and diverse subset of the network. This strategy promotes network robustness by preventing excessive clustering and dependency on specific nodes (hubs). When existing hubs disappear (e.g., due to failures or departure), other nodes within the network are opportunistically elevated to hub status, ensuring continuity and adaptability of the network topology over time.

Our target is to obtain a topology of the network that has the following properties: *(i)* There are h defined hubs, with h a parameter defined before the start of the network and common to all nodes, *(ii)* ignoring hubs, the distribution of the remaining connections is random, and *(iii)* each node has c connections, consisting of h connections to hubs and c-h connections to random nodes.

4.2.3 Service API

The API of the hub sampling service mirrors that of classical peer sampling service [21], comprising two key methods: *(i) init()* that initializes the service on a given node, *i.e.*, initializes the list of outgoing connections of a node (Indeed, we assume that a given node starts connected to a random subset of nodes in the network, the actual initialization procedure being implementation-dependent), and *(ii) getPeer()* that returns a random peer address from the node list of peers.

The focus of this work is to present an implementation of the *getPeer()* method, Elevator, as a gossip-based algorithm, and to study the performance of its implementation. In addition to these two methods, we add a third method to the API called *getHub()* that returns a random hub. The *getHub()* method can be easily derived from *getPeer()* by filtering the output of *getPeer()* to only select the h nodes acting as hubs in the network. This method can be useful for applications that only need to contact a hub.

4.2.4 Elevator detailed description

Having outlined the underlying principles and the structural properties we aim to achieve, we now provide a formal and operational description of the Elevator protocol, detailing the parameters, data structures, and iterative procedures that implement the hybrid preferential–random attachment mechanism described above.

The algorithm uses the following parameters and data structures:

- *Parameter c*: The maximum number of outgoing connections (its default value for all nodes is 20).
- *Parameter h*: The number of preferential attachment connections (its default value for all nodes is *c/2*).
- *Parameter maxsize_buffer_backward*: The maximum number of backward connections to send (its default value for all nodes is 100).
- *Structure cache*: The list of outgoing connections. The list is implemented as an array of size *c*. The list is initialized with random existing addresses (random connections to other nodes of the network).
- *Structure backward_peers*: The list of other nodes that have tried to connect to the node. The list is implemented as a linked list (initially empty).

Additionally, we have three temporary structures: *(i) frequency_map* holds the frequency of occurrences for all neighbors of neighbors, implemented as a map (node ⇒ integer), *(ii) preferred* holds the list of preferred nodes, implemented as a linked list, and *(iii) preferred_backward* holds the list of backward connections of the preferred nodes, implemented as a linked list.

The proposed protocol executes the following actions at each run:

(i) **Retrieve neighbors at distance two**: Each node collects the neighbor lists of its neighbors (i.e., all nodes at distance two).

(ii) **Build frequency map and select preferred nodes**: From this data, the node builds an ordered list of the most frequent peers (the frequency map) and contacts the top h nodes, referred to as the **preferred** nodes.

(iii) **Receive backward lists**: The node asks all **preferred** nodes for their backward list. Each preferred node contacted sends back a maximum of *maxsize_buffer_backward* addresses from its backward list (**backward_peers**) and adds the contacting node to its own backward list. The contacting node creates a **preferred_backward** list containing all the received identifiers.

(iv) **Reset cache**: The node reset its cache to an empty array.

(v) **Fill cache with hubs and random peers**: The node selects the **preferred** nodes and c - h random peers from the **preferred_backward** list to populate its cache.

(vi) **Complete cache if needed**: If the cache is still not full, the node adds random peers from the frequency map until the cache reaches size *c*.

(See Figure 9 and Figure 10 for detailed pseudocode of the algorithm.)

```
   initial peer list: cache
   cache size: c
   number of hubs desired: h
   initial backward list: backward_peers (empty)
1  loop
2  |  for peer in backward_peers
3  |  |  if peer not responding
4  |  |  |  backward_peers.remove(peer)
5  |  for peer in cache
6  |  |  if peer not responding
7  |  |  |  cache.remove(peer)
8  |  frequency_map ← {}
9  |  for peer in cache
10 |  |  peer_cache ← send(CACHE_REQUEST, peer)
11 |  |  frequency_map ← frequency_map ∪ peer_cache
12 |  preferred ← frequency_map.sortByFrequency().select(number = h)
13 |  frequency_map.remove(preferred)
14 |  preferred_backward ← {}
15 |  for peer in preferred
16 |  |  peer_backward_peers ← send(BACKWARD_REQUEST, peer)
17 |  |  preferred_backward ← preferred_backward ∪ peer_backward_peers
18 |  preferred_backward.shuffle()
19 |  cache ← {}
20 |  cache ← preferred + preferred_backward[1..(c - h)]
21 |  while cache.size() < c
22 |  |  peer ← frequency_map.selectRandom()
23 |  |  cache.append(peer)
```

Figure 9: Elevator algorithm (active thread).

```
   max number of backward connections to send: maxsize_buffer_backward
1  loop
2  |  request, peer ← receive()
3  |  if request == CACHE_REQUEST
4  |  |  send(cache, peer)
5  |  |  backward_peers.add(peer)
6  |  if request == BACKWARD_REQUEST
7  |  |  backward_peers.shuffle()
8  |  |  send(backward_peers[1..maxsize_buffer_backward], peer)
```

Figure 10: Elevator algorithm (background thread).

4.2.5 Byzantine protocols

The Elevator protocol was not designed to be resilient to Byzantine attacks, and the protocol assumes that each node is honest and returns reliable information. Since in Elevator each node modifies its cache based on the cache of its neighbors, having one or more Byzantine nodes among its neighbors significantly changes the local behavior of the protocol (for a given node) and therefore the overall convergence toward the h hubs.

To describe how such malicious behavior can affect the protocol, we adopt the Byzantine failure model defined in Chapter 2, namely the **Byzantine Synchronous Message-Passing model** (BSMP ⟨n, t⟩ [∅]). Our Byzantine model assumes that a certain percentage of nodes are Byzantine from the start. These malicious nodes try to break the Elevator protocol by sending false information during cache exchanges, manipulating the hub selection process.

The goal of Byzantine attackers is to get selected as hub by the correct nodes (that genuinely execute the protocol). Obviously, if there is a fraction p of Byzantine nodes overall, then it is trivial for the Byzantine nodes to obtain a fraction p of the hubs (they should just behave as correct nodes). So, the Byzantine nodes strive to obtain a higher fraction of the hubs than their fraction of the nodes.

Attack Mechanism: When legitimate nodes ask Byzantine nodes for their cache contents or backward peers information (i.e., the nodes who contacted them in the past), the Byzantine nodes respond with fake data designed to help malicious nodes become hubs.

This attack works because Elevator relies on nodes honestly reporting their connectivity information. Apart from that, Byzantine nodes perform the protocol like other nodes. Byzantine nodes modify their behavior in order to achieve the goal of having a large proportion of hubs be Byzantine, but their objective is also to avoid detection. If their behavior deviates too much from that of a normal node, they could easily be detected and blacklisted.

We are studying several types of Byzantine nodes:

(i) Passive unique Byzantine: A single Byzantine sending an empty cache.

(ii) Active unique Byzantine: A single Byzantine who sends his modified cache with a reference to himself (to increase his probability of being chosen as a hub).

(iii) Non-coordinating Byzantine nodes: Multiple Byzantine nodes who send their modified cache with a reference to themselves but do not include references to other Byzantine nodes.

(iv) Coordinated Byzantine nodes: Each Byzantine node maintains a coordinated fake cache containing references to all other Byzantine participants in the network. When responding to legitimate cache requests, Byzantine nodes return sublists of this coordinated cache, effectively creating an artificial preference for Byzantine nodes in the sampling process.

In terms of pseudo-code for the Byzantine nodes, this amounts to replacing the background Elevator process (Figure 10) with the following algorithms: Figure 11 for the passive unique Byzantine, Figure 12 for the active unique Byzantine attack and the multiple non-coordinating Byzantines, and Figure 13 for the multiple coordinated Byzantines.

```
   backward list: backward_peers
1  loop
2  |  (request, peer) ← receive()
3  |  if request == CACHE_REQUEST
4  |  |  backward_peers.add(peer)
5  |  |  backward_peers.shuffle()
6  |  |  send([], None, peer)
```

Figure 11: Do-nothing attack.

```
   my address: my_address
   backward list: backward_peers
1  loop
2  |  (request, peer) ← receive()
3  |  if request == CACHE_REQUEST
4  |  |  backward_peers.add(peer)
5  |  |  backward_peers.shuffle()
6  |  |  modified_cache ← cache.remove(random()).add(my_address)
7  |  |  send(modified_cache, my_address, peer)
```

Figure 12: Non-coordinating attack.

```
   addresses of all byzantine nodes: all_byzantines
   backward list: backward_peers
1  loop
2  |  (request, peer) ← receive()
3  |  if request == CACHE_REQUEST
4  |  |  backward_peers.add(peer)
5  |  |  backward_peers.shuffle()
6  |  |  all_byzantines.shuffle()
7  |  |  modified_cache ← all_byzantines[0:c]
8  |  |  random_backward ← all_byzantines.random_value()
9  |  |  send(modified_cache, random_backward, peer)
```

Figure 13: Coordinated attack.

4.2.6 Lift protocol

To address Elevator's vulnerability to Byzantine attacks, we propose a deterministic hub redistribution mechanism (that we name Lift) that activates after the network has converged to its initial hub configuration. Our approach leverages the fact that node identifiers are assigned randomly and cannot be modified by Byzantine nodes. If Byzantine nodes are active, we hope that our new protocol will be more efficient than Elevator in terms of resilience, and if Byzantine nodes are not active, we hope that the protocol will have no impact on protocol performance and convergence towards hubs.

The counter-attack operates in two phases: an initial convergence phase using standard Elevator, followed by a deterministic hub redistribution phase.

Phase 1 – Initial Convergence: The network runs the standard Elevator protocol for a predetermined number of cycles to allow hub formation. During this phase, Byzantine nodes may successfully infiltrate hub positions through coordinated attacks.

Phase 2 – Hub Redistribution: After convergence, all correct nodes simultaneously execute the following deterministic process (see Figure 14 for detailed pseudocode):

(i) Each correct node retrieves the identifiers of the h current hubs (which may be Byzantine). Since Elevator has converged, the first h elements of each correct node's cache correspond to the addresses of the h hubs (a hub may contain itself in its cache).

(ii) Each node builds a seed by concatenating the h hub identifiers. Because these identifiers are already sorted, the resulting seed is identical for every correct node.

(iii) Each node initializes a pseudo-random number generator (see Definition 2.3.7) using this seed. The PRNG used is Java's default implementation, namely a linear congruential generator [53]. Since both the seed and the PRNG are identical for all correct nodes, the generated sequence is identical, effectively creating a shared random list of values.

(iv) Each correct node generates h new random values using the PRNG, corresponding to h node identifiers in the network. If a generated value has already been selected, the PRNG is invoked again until a fresh identifier is obtained.

(v) Each correct node replaces the first h identifiers in its cache (corresponding to the potentially Byzantine hubs) with the h identifiers generated by the PRNG. The old hub connections are therefore removed and replaced with new hubs chosen uniformly at random.

```
   hub list: H
   network size: N
   target hubs: h
1  seed ← getSortedHubIDs(H)
2  prng ← Random(seed)
3  selectedIDs ← {}
4  while selectedIDs.size() < h
5  |  randomID ← prng.nextInt(N)
6  |  if randomID not in selectedIDs
7  |  |  selectedIDs ← selectedIDs ∪ {randomID}
8  H ← selectedIDs
```

Figure 14: Lift: Deterministic Hub Redistribution.

Since all nodes use the same seed derived from the initial hub selection, they deterministically select identical new hub sets. Because node identifiers are randomly assigned and immutable, each node has equal probability $\frac{h}{N}$ of becoming a hub, regardless of Byzantine status. The algorithm replaces the cache contents entirely: the first h positions are filled with the deterministically selected new hubs, while the remaining positions are populated with random non-hub nodes to maintain cache diversity.

The critical hypothesis is that Byzantine nodes cannot manipulate their node identifiers, which are assigned uniformly at random during network initialization and remain immutable thereafter.

We further assume that the pseudo-random number generator (PRNG) used in the protocol is correct and cannot be biased or influenced by Byzantine nodes. In particular, the seed provided to the PRNG is obtained by concatenating the identifiers of the selected hubs. Since each node identifier is indepen-

dently and uniformly generated at initialization, this concatenation can be modeled as a uniformly random seed.

Therefore, even if Byzantine nodes dominate the initial hub selection process, the subsequent deterministic redistribution treats all nodes equally, as it depends solely on immutable identifiers and on the output of an unbiased PRNG.

4.3 Theoretical Analysis

We initially introduced the intuition behind hub emergence, explaining how a combination of preferential attachment and random sampling can naturally lead to the formation of hubs. We then provided a detailed description of the Elevator protocol together with its pseudo-code, specifying its operational mechanisms at the local level.

However, while this description clarifies how the protocol functions, it does not formally establish its properties. In particular, the convergence toward the desired number of hubs, the preservation of connectivity, and the resulting structural guarantees remain to be demonstrated. The objective of the following section is therefore to provide a theoretical analysis of Elevator and to formally study its emergent behavior.

First we instantiate the formal framework introduced in the Chapter 2. We consider a peer-to-peer system composed of a set of nodes V, modeled as in Definition 2.1.3, and evolving according to the global system Definition 2.3.1. The overlay is represented as a time-varying graph $G(t) = (V, E(t))$ in the sense of Definition 2.3.8, and each node maintains a partial view as defined previously. For both the theoretical analysis and simulation experiments, we assume that the initial overlay forms a random k -out graph, i.e., each node selects k distinct nodes uniformly at random to populate its partial view. Similarly, when a new node joins the network, it initializes its partial view by connecting to k nodes chosen uniformly at random. Although the protocol operates asynchronously, we analyze its evolution using the notion of protocol cycles defined in Definition 2.3.4, where each node executes one protocol step per cycle.

All nodes execute the same peer-to-peer protocol, namely Elevator.

In order to analyze and model the Elevator protocol, we adopt several simplifying assumptions. Without these assumptions, a formal analysis would be extremely difficult, if not impossible.

Assumptions:

- We assume a failure-free network with a constant number of nodes. In particular, Byzantine behavior and the Lift protocol are not considered in this analysis.
- The random identifiers returned to hubs through the `BACKWARD_REQUEST` mechanism are assumed to be equivalent to identifiers drawn from a uniform distribution.
- When selecting preferred nodes, if a node encounters two or more candidates with the same occurrence frequency, it deterministically selects the candidate with the smallest identifier.
- All nodes are assumed to execute the protocol synchronously and correctly at every cycle.
- Prior to the first cycle, the network is assumed to be highly connected and its topology is modeled as a uniform k-out random graph, where $k = c$ corresponds to the cache size shared by all nodes.
- The parameter h (the target number of hubs) is assumed to be identical for all nodes.

Under these assumptions, our objective is to analyze the stability and the convergence of the Elevator protocol, and to characterize its convergence speed.

4.3.1 Stability

We define the stability of the Elevator algorithm as the property that, once convergence to a set of h hubs has been reached, both the list of hubs and their number h remain (with high probability) constant over time.

Formally, let $H(t)$ denote the set of hubs identified at time t, and let $C_v(t)$ be the ordered list of outgoing connections (the cache) of node $v \in V$. The algorithm is said to be *stable* if, after convergence time T, the following holds with high probability:

$$\Pr[\forall t \geq T, H(t) = H(T) \wedge \forall v \in V, C_v(t)[1:h] = H(t)] = 1$$

In other words, after convergence, every node in the network maintains the same h leading entries in its cache, corresponding to the stable hub set $H(T)$, and this structure remains fixed over time.

It is important to note that only the top-h entries of the cache are relevant for stability. The remaining $c - h$ entries of each cache can still fluctuate over time, as they typically contain random samples of other nodes in the network. Since these entries do not influence the global hub set, their variability does not affect the stability of the Elevator algorithm.

Proposition 4.3.1

Once Elevator has converged to a stable state, it remains in that state. To prove this, we must establish that:

(i) the number of hubs cannot exceed h,
(ii) the number of hubs cannot fall below h, and
(iii) The set of hubs remain the same.

♠

Proof. We prove point (i) as follows: At each new iteration of the algorithm, each node selects the first h elements of its *frequency_map*. Therefore, even if in the previous cycle an additional node temporarily became a hub alongside the h existing hubs, in the next cycle the number of hubs will be restored to at most h.

To prove point (ii), it should be noted that for the number of hubs to decrease, one or more nodes must not choose one of the nodes already selected as a hub in their list of potential hubs.

Given that we are in a stable state, with each node having the same h nodes as hubs in its list, the only way for a node not to choose one of the previous hubs as a potential hub for the next cycle is for a random node to appear in the list of connections of the node's c outgoing neighbours, i.e., as frequently as a hub, and it would also need to have a smaller ID than a hub already present.

If this happens, and since there can only be a maximum of h nodes chosen as potential hubs by a node n, then one of the potential hubs of node n will be removed and replaced by a random node. We therefore need to calculate the probability of this event occurring.

Let us consider a randomly selected node i that could be chosen as a potential hub. We first evaluate the probability that i appears in the list of successors of one of the successors m of a given node n.

Since the network contains N nodes in total, and there are $c - h$ remaining slots in the list of successors of m (the h hub positions are already occupied), the probability that i appears in this list can be written as: $\Pr[i \in \text{Succ}(m)] \approx \frac{c-h}{N}$.

Here we assume independence in the selection of the $c - h$ nodes, which is not strictly true (as once a node has been chosen as a successor, it cannot be selected again). However, this approximation becomes reasonable when N is large.

Next, we calculate the probability that node i appears in the list of all c successors of node n. Since the choices of successors are independent across the c neighbours of n, this probability is:

$$\Pr\left[i \in \bigcap_{m \in \text{ Succ}(n)} \text{Succ}(m)\right] = \left(\frac{c-h}{N}\right)^c.$$

This represents the probability that node i is chosen as a potential hub by node n: indeed, if i is present in the cache of all successors of n, and we assume that its identifier is smaller than at least one of the existing hubs, then i will replace that hub and become a new potential hub for n.

Let N be the number of nodes, h the number of hubs, and c the cache (outgoing neighbors) size. Fix a node n. For a given non-hub node i (there are $N - h$ such nodes), the probability that i appears in the list of successors of a given successor m of n is approximately

$$p_1 \approx \frac{c-h}{N},$$

using the large-N approximation (independent sampling without replacement $\approx$ with replacement).

Assuming independence across the c successors of n, the probability that i belongs to *all* those successor-lists is

$$p_2 \approx p_1^c = \left(\frac{c-h}{N}\right)^c.$$

Thus the expected number (or by union bound, an upper bound on the probability) that *at least one* non-hub i satisfies this property for node n is bounded by

$$(N-h)p_2 = (N-h)\left(\frac{c-h}{N}\right)^c.$$

If we now take the union over all N nodes n (assuming independence for an upper bound), we obtain an upper bound on the probability that some cache (in the whole network) receives such a random node that could replace an existing hub:

$$P_{[\text{new potential hub}]} \approx N(N-h)\left(\frac{c-h}{N}\right)^c. \tag{1}$$

This quantity is to be interpreted as a (pessimistic) upper bound on the probability that a potential hub appears at random and replaces an existing hub during one cycle.

Assume that $c \ll N$ and $h \ll N$. Then the probability that a new potential hub appears, as computed in Equation (1), is very low. Consequently, the complementary probability, i.e., that no new potential hub appears, is practically equal to 1.

Suppose a new potential hub does appear. This event can at most reduce the number of hubs for a single cycle, because the new potential hub affects only one node's cache. In the subsequent cycle, when this node updates its cache using the Elevator algorithm, it selects the same list of potential hubs as all other nodes, restoring the set of hubs to its previous configuration.

Furthermore, the probability that a new potential hub appears in two consecutive cycles is approximately the square of the already small probability, making it even less likely. Therefore, we can safely neglect these events and conclude that, with high probability, the number of hubs cannot decrease after convergence.

Finally, we consider the scenario where a potential hub is elected randomly by all nodes in the network, and this potential hub has an ID smaller than one of the existing hubs. In this case, the potential hub may replace an existing hub, causing the set of hubs to change. To prove (iii), we need to prove that with high probability, this scenario will not happen.

We aim to compute the probability $p(x)$ that the intersection of all N subsets A_k contains exactly x elements, where $x \in \{0, 1, ..., m\}$, and $m = c - h$.

We first define $q(x)$, the probability that the intersection of the N subsets A_k contains at least x elements:

$$q(x) = \frac{\binom{n}{x} \cdot \left(\binom{n-x}{m-x}\right)^N}{\left(\binom{n}{m}\right)^N},$$

where

- $\binom{n}{x}$ is the number of ways to choose the x elements that are common to all subsets A_k,
- $\binom{n-x}{m-x}$ is the number of ways to choose the remaining $m - x$ elements for each subset A_k, ensuring that the x common elements are included,
- $\binom{n}{m}$ is the total number of ways to choose m elements for each subset A_k.

The probability $p(x)$ that the intersection contains exactly x elements is then

$$p(x) = q(x) - q(x+1), \quad x = 0, 1, ..., m-1,$$

and for the special case $x = m$:

$$p(m) = q(m) = \frac{1}{\left(\binom{n}{m}\right)^{N-1}}.$$

Here,

- $q(x)$ is the probability that the intersection contains at least x elements,
- $q(x+1)$ is the probability that the intersection contains at least $x+1$ elements,
- Subtracting $q(x+1)$ from $q(x)$ yields the probability that the intersection contains exactly x elements.

The probability that a new potential hub replaces an existing hub is extremely small for typical parameters $c \ll N$ and $h \ll N$.

Indeed, we have

$$q(0) = P[X \geq 0],$$

which is the probability that there are at least 0 elements common to all subsets A_k. This can be computed as

$$q(0) = \frac{\binom{n}{0}\left(\binom{n}{m}\right)^N}{\left(\binom{n}{m}\right)^N}.$$

Since

$$\binom{n}{0} = 1,$$

we obtain

$$q(0) = \frac{1 \cdot \left(\binom{n}{m}\right)^N}{\left(\binom{n}{m}\right)^N} = 1.$$

For the other values, such as $q(1), q(2), \ldots$, these probabilities are extremely small when $c \ll N$ and $h \ll N$. Therefore, we have

$$p(0) = q(0) - q(1) \approx 1,$$

where $p(0)$ represents the probability that the set of hubs does not change. This shows that, with very high probability, a randomly selected potential hub cannot replace any existing hub, confirming the stability of the set of hubs in the network.

Combining these three cases, we conclude that the Elevator algorithm is stable: once the network has converged to h hubs, this set remains unchanged with high probability in subsequent cycles.

□

4.3.2 Convergence

To reason about the convergence of the Elevator algorithm, we consider three possible scenarios after a protocol cycle:

(i) **Network disconnection:** the network becomes disconnected, in which case the system cannot converge to a stable state because nodes in different components cannot coordinate on a common set of hubs.

(ii) **Multiple hub clusters:** two or more clusters form in the network, with each cluster choosing a distinct set of hubs. In this case, there is no convergence towards a stable state with a single set of h hubs in the network.

(iii) **Stable convergence:** the network converges towards a stable state with exactly h hubs shared by all nodes.

To prove convergence with high probability, it is therefore sufficient to show that the first two scenarios, i.e., network disconnection and formation of multiple hub clusters, are highly unlikely to occur in practice. Once these cases are ruled out, the system will converge to the stable set of h hubs with high probability.

Proposition 4.3.2

If the network contains at least one hub, then the network is strongly connected with high probability.

♠

Proof. The presence of at least one hub ensures that every node has an outgoing connection to the hub, which makes the network weakly connected. Additionally, each node has at least one random successor, chosen uniformly across the network, because each node requests a random incoming connection from the hub. Therefore, starting from any node n, any other node can be reached by following a sequence of random outgoing connections.

It is possible that the network temporarily forms two or more clusters, in which case some nodes might not be reachable from others. However, this is unlikely:

- If a node is in a small cluster, there is a high probability that it's random outgoing connection points to a node in another cluster.
- If a node is in a big cluster, there are many nodes in its cluster, so it is likely that at least one node has a random outgoing connection to a node in another cluster.

Even if after a protocol cycle the network is temporarily not strongly connected, it will regain strong connectivity with high probability in the next cycle, thanks to new random outgoing connections. Thus, the network may lose this property momentarily, but it will eventually recover it after a few cycles. □

Proposition 4.3.3

If the network contains at least one hub, an additional hub will eventually appear with high probability.

Proof. Each hub provides a random outgoing connection to every node in the network. This means there is a small, but non-zero, probability that all nodes points to the same node. If this occurs, that node will be selected as a new hub.

Although the probability for all nodes to select the same node simultaneously is low, it is strictly greater than zero. Therefore, given enough protocol cycles, this event is bound to happen eventually. Consequently, the network will eventually contain at least one additional hub, beyond the original hub. □

Proposition 4.3.4

Once the network reaches a state with a number of hubs between 1 and $h - 1$, this number cannot decrease.

Proof. Consider a system state where the number of hubs is h_i, with $1 \leq h_i \leq h - 1$. If a node is chosen as a hub, it will be included in the caches of other nodes according to the algorithm. In this scenario:

- Any newly selected node as a hub increases the total number of hubs by 1, but does not remove any of the previously selected hubs from the network.
- Therefore, the set of existing hubs is preserved in subsequent cycles.

As a consequence, the number of hubs in the network cannot decrease while it remains below h. This establishes that, once the system enters a state with $1 \leq h_i \leq h - 1$ hubs, the number of hubs is non-decreasing until it reaches h. □

Proposition 4.3.5

Let h be the desired number of hubs in the network. Once the network contains at least one hub, it will converge with high probability to a stable state containing exactly h hubs.

Proof. From Proposition 4.3.2, we know that if the network contains at least one hub, it remains strongly connected with high probability. From Proposition 4.3.3, we know that an additional hub will eventually appear with high probability. From Proposition 4.3.4, we know that once the number of hubs is between 1 and $h - 1$, it cannot decrease.

Combining these results, we can conclude the following:

- Starting from a state with at least one hub, the network remains strongly connected.
- Since the number of hubs cannot decrease, and there is always a non-zero probability for new hubs to appear (Proposition 4.3.3), the number of hubs will eventually increase until it reaches h.
- Once the network reaches h hubs, the system has converged to the desired stable state (as the number of hubs cannot exceed h due to the algorithm's selection mechanism).

Therefore, starting from at least one hub, the network converges with high probability to the stable state with exactly h hubs. □

Proposition 4.3.6

With sufficiently large c, the Elevator algorithm generates at least one hub after a finite number of cycles with high probability.

Proof. Initially, each node selects c neighbors uniformly at random. With high probability, these selections are spread across the network and are representative of the network as a whole, ensuring that no isolated cluster is formed before the first cycle.

Each node then selects h nodes as potential hubs, following the preferential attachment rules of the Elevator protocol. With high probability, the chosen potential hubs form a subset representative of the entire network. Each node subsequently requests its potential hubs for random incoming connections. Since the potential hubs are representative of the network, with high probability, each node receives $c - h$ random incoming connections from a subset that is also representative of the network. Thus, after a cycle of the protocol, the list of successors of each node remains representative of the network, maintaining connectivity and minimizing the risk of cluster formation. This state is preserved in subsequent cycles.

In the following cycles, nodes select the top h candidates from their frequency maps. The number of potential hubs gradually decreases, until at least one node emerges as a hub.

Therefore, with sufficiently large c, the probability of creating disconnected clusters is very low, and the algorithm converges to a state with at least one hub after a finite number of cycles with high probability. □

Proposition 4.3.7

Starting from a random initial network, the Elevator algorithm converges with high probability to a stable state containing exactly h hubs.

Proof. From Proposition 4.3.6, we know that with sufficiently large c, the Elevator algorithm generates at least one hub after a finite number of cycles with high probability. From Proposition 4.3.5, we know that once the network contains at least one hub, it will converge with high probability to a stable state containing exactly h hubs.

Combining these two results, we conclude that:

- Starting from a random network, the algorithm generates the first hub with high probability.
- Once at least one hub exists, the network remains strongly connected, and additional hubs appear until the number of hubs reaches h without decreasing.
- Therefore, the system converges with high probability to the desired stable state containing exactly h hubs.

□

4.3.3 Time to Convergence

Studying the convergence time of the Elevator algorithm directly on the full system is extremely challenging. The network can consist of thousands of nodes, each maintaining multiple outgoing connections, resulting in a dynamic graph with stochastic elements. Modeling the system using a

Markov chain would be prohibitively complex, as it is practically impossible to compute the transition probabilities between all possible system states.

Therefore, we adopt a simplified model that captures the essential dynamics of the system while remaining analytically tractable. Our goal is to develop a model that reflects the behavior observed in simulations and experiments, namely the very rapid convergence observed in networks of 1,000 nodes, typically within 4 to 5 protocol cycles.

Model A, Geometric Sequence

We provide a simple analytical model to estimate the upper bound on the convergence time of our protocol. The idea is to track the evolution of the node with the highest initial indegree, and observe how quickly its popularity can grow due to the recursive selection mechanism.

We consider a recursive mechanism where each node, at each cycle, selects as new successor the node that appears most frequently among its 2-hop neighbors. This induces a reinforcement effect: popular nodes attract more attention.

Let us assume that a node v has indegree $d_0 = i$ at cycle $t = 0$. Each of these i predecessors has roughly K neighbors (with $K = c$ the size of the cache). Hence, v appears in up to $i \cdot K$ 2-hop paths. If a fraction p of these paths leads to new incoming edges for v at the next cycle, then the indegree evolves as:

$$d_{t+1} = p \cdot K \cdot d_t$$

Such an equation has the general solution given by a geometric progression:

$$d_t = d_0 \cdot (p \cdot K)^t,$$

where $d_0 = i$ is the initial indegree at cycle $t = 0$.

This shows that the indegree evolves exponentially with respect to the number of cycles t, growing or shrinking depending on the value of $p \cdot K$.

We define convergence as the moment when a node has indegree comparable to the size of the network, i.e.,

$$d_t \geq N.$$

Using the closed-form expression

$$d_t = i \cdot (p \cdot K)^t,$$

we solve for t:

$$i \cdot (p \cdot K)^t \geq N \Rightarrow (p \cdot K)^t \geq \frac{N}{i} \Rightarrow t \geq \frac{\log\left(\frac{N}{i}\right)}{\log(p \cdot K)}.$$

This provides an upper bound on the number of cycles required for convergence.

This bound captures a snowball effect: once a node accumulates a small advantage in indegree, it can quickly dominate due to positive feedback. Even if the exact dynamics involve noise and competition, this reasoning shows that convergence is fast (logarithmic in N), and largely driven by early centrality fluctuations.

The geometric sequence model provides a first approximation of the convergence dynamics of the Elevator protocol, but it suffers from several important limitations. First, it assumes that the probability of being selected as a hub is strictly proportional to the current indegree, ignoring other structural

effects of the network. Second, it neglects the presence of duplicate nodes in the second-degree neighborhoods, which can bias the distribution of choices. Third, the exponential growth curve derived from this model only reflects an average behavior, without capturing the stochastic fluctuations that arise in real network dynamics. Finally, the parameter p, representing the fraction of second-degree neighbors selecting a given hub, is chosen arbitrarily. These limitations highlights the need for a more refined model that naturally accounts for the saturation effect as the indegree approaches its maximum possible value. To address this issue, we introduce a logistic-based model, which provides a more realistic description of the slowdown in growth near the system's capacity.

Model B, Logistic Function with Dynamic Rate

We decided to use a logistic function with a dynamic rate of growth to study the convergence time of the Elevator algorithm. This choice is well-suited to our case because the system is dynamic, with one or several hubs increasing their in-degree rapidly, initially in an approximately exponential manner. The rate of increase is not constant, but itself grows over time, before eventually slowing down as the in-degree approaches its upper bound, i.e., the size of the network.

We model the evolution of the in-degree growth (of the slowest hub) using the following function:

$$d(t) = \frac{N}{1 + a \cdot \exp\left(-\left(r_0 t + \left(\frac{1}{2}\right)\delta_r t^2\right)\right)}$$

where N is the network size, a is a scaling parameter, r_0 is the initial growth rate, and δ_r controls the acceleration of the rate.

For our case, we selected the following values:

$$r_0 = 2 + \frac{h}{10}, \quad \delta_r = \frac{K}{N}, \quad a = \frac{N}{K} - 1,$$

with $K = c$ is the size of the cache and N the size of the network.

These choices are motivated as follows: a larger h increases the speed of convergence, since having multiple hubs accelerates the dissemination of hub information in the network. Although K has little influence in practice, in theory a larger cache allows nodes to explore more candidates, which can also increase the convergence speed. Finally, a larger network size N naturally increases the convergence time, as more nodes are required to reach consensus; however, the growth remains exponential, and thus convergence is still very fast in practice.

Evaluation of models

We evaluate the two proposed models (Model A: Geometric Growth Model and Model B: Logistic Function) by comparing them with the results obtained from simulations of the Elevator protocol. The goal of this comparison is to examine whether the theoretical models reproduce the same qualitative behavior observed in practice, in particular the progression curve of the hub node's indegree over time. We will quantitatively assess the models by computing the mean absolute error (MAE) and the root mean squared error (RMSE) between the predicted curves and the simulation data. As we can see in Figure 15, Figure 16 and Figure 17, the Logistic Model is closer to the data from the simulation, and in particular it's more accurate in situations where the values of K and h are changed. As we can see in Table 2, Table 3 and Table 4, the Logistic has almost always a better RMSE and MAE compared to the Geometric model, and sometimes with values very small, indicating that our model is very good at fitting to the data. If we look at convergence times (in Table 5, Table 6 and Table 7), the geometric model is often too fast in terms of convergence time. The logistic model is more pessimistic, but this suits us because we want to have an upper bound on convergence time, and in any case, convergence times

remain very close to the simulation results. It should be noted that when calculating the convergence time, we used an approximation of $10^{\{-3\}}$ relative to the simulation value, given that the Logistic model never reaches the limit value but comes as close to it as possible.

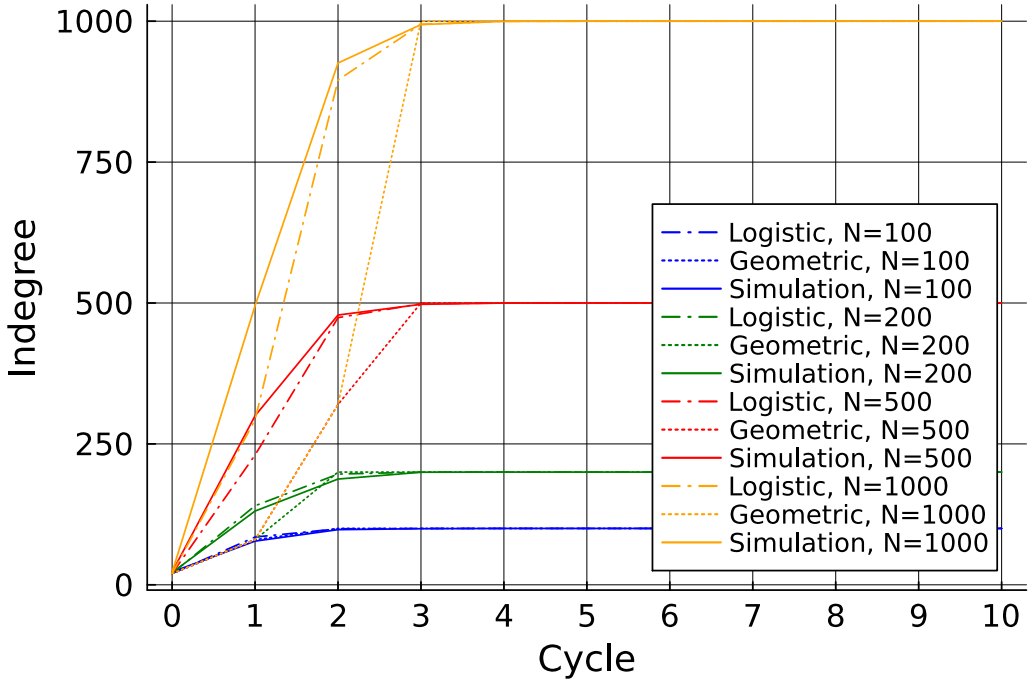

Figure 15: In-degree evolution of the hub, comparing models with simulation data, with N from 100 to 1000. K=20. h=10.

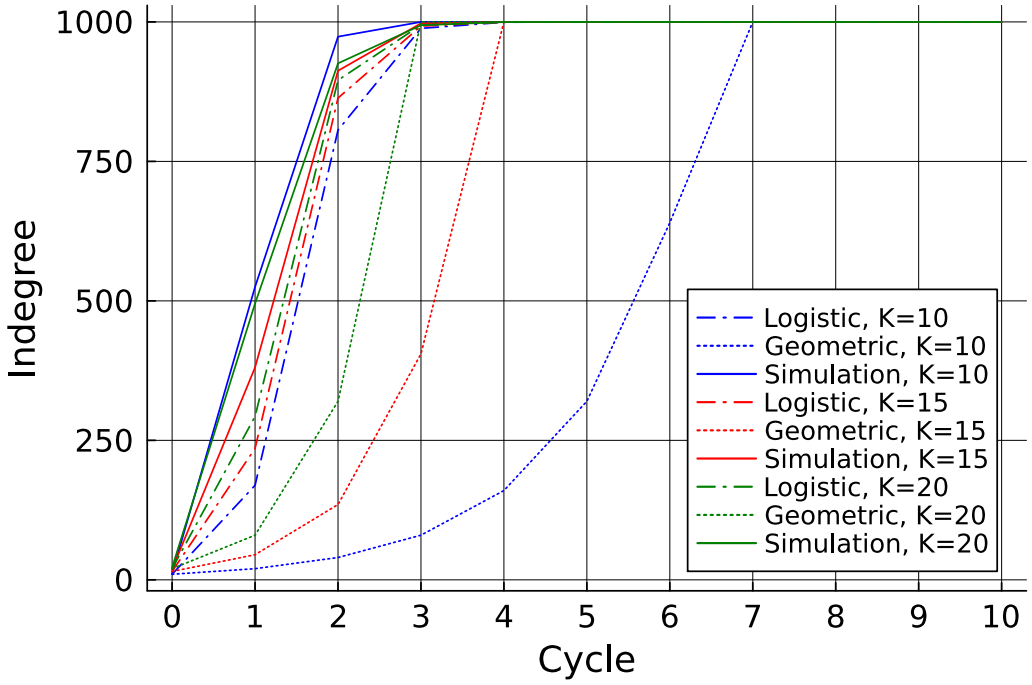

Figure 16: In-degree evolution of the hub, comparing models with simulation data, with varying values for K. N=1000, h=10.

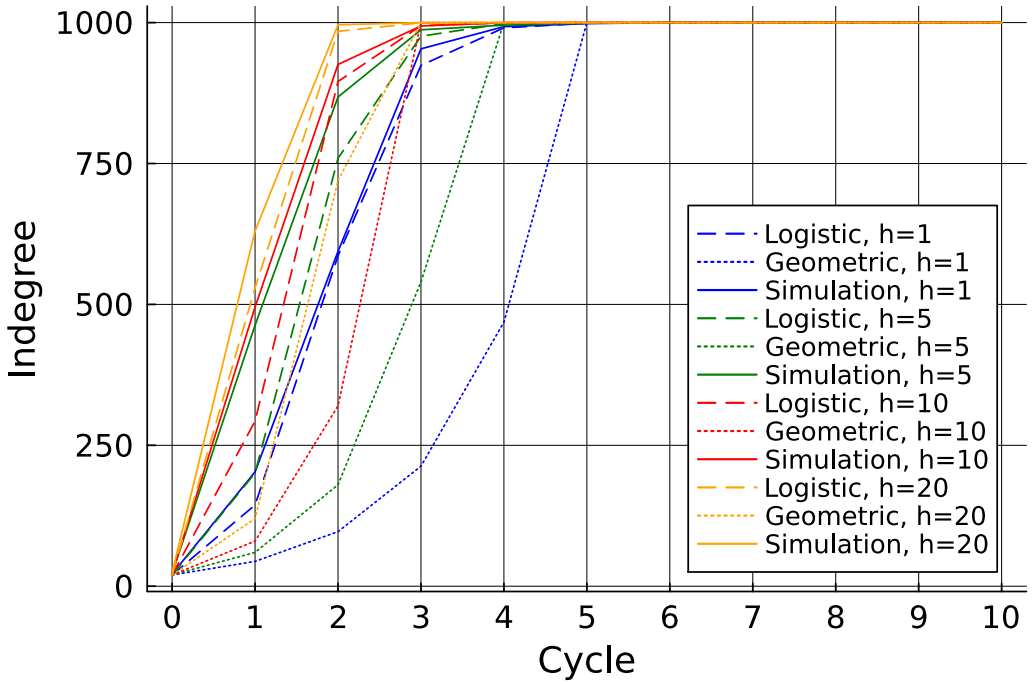

Figure 17: In-degree evolution of the hub, comparing models with simulation data, with varying values for h. K=20, N=1000.

Table 2: Comparison of Logistic and Geometric Models for Different N

N	RMSE (Logistic)	RMSE (Geometric)	MAE (Logistic)	MAE (Geometric)
100	2.38	1.30	1.09	0.72
200	3.92	15.79	1.82	5.93
500	21.04	81.75	7.08	34.86
1000	61.72	221.44	21.40	93.59

Table 3: Comparison of Logistic and Geometric Models for Different K

K	RMSE (Logistic)	RMSE (Geometric)	MAE (Logistic)	MAE (Geometric)
10	118.58	545.27	48.77	385.44
15	46.23	311.61	18.46	155.40
20	61.72	221.44	21.40	93.59

Table 4: Comparison of Logistic and Geometric Models for Different h

h	RMSE (Logistic)	RMSE (Geometric)	MAE (Logistic)	MAE (Geometric)
1	19.95	315.60	9.13	174.91
5	85.72	275.66	35.31	140.56
10	61.72	221.44	21.40	93.59
20	30.61	174.94	10.64	71.88

Table 5: Cycles to reach N for different network sizes.

N	Logistic	Geometric	Simulation
100	4	2	5
200	5	2	4
500	6	3	5
1000	6	3	5

Table 6: Cycles to reach N for different values of K.

K	Logistic	Geometric	Simulation
10	7	7	4
15	6	4	5
20	6	3	5

Table 7: Cycles to reach N for different values of h.

h	Logistic	Geometric	Simulation
1	9	5	7
5	7	4	6
10	6	3	5
20	5	3	3

4.4 Simulation-Based Evaluation

We evaluate our proposal by carrying out a simulation campaign. All simulations use the Java **PeerSim** simulator [54]. We have modified the simulator to add parallelism to accelerate computations. With Peersim, we implemented our algorithm Elevator, and state-of-the-art PROOFS [18] and Phenix [32] algorithms[2]. Also, we used the implementation of Newscast provided by PeerSim.

We compared the performance of Elevator with these 3 algorithms. We chose to compare our proposed algorithm to these three algorithms as they are widely used in the literature. Newscast is used for gossip learning [55], PROOFS is a foundational algorithm, as Secure Cyclon [51], one of the latest peer sampling algorithm in the literature, is based on Cyclon [23], itself based on PROOFS. Phenix is interesting as it has especially been conceived to be resilient to failures and Byzantine attacks and also to construct networks that have a low diameter. We did not include recent algorithms [41], [56], [57] that primarily focus on improving the power law distribution of the in-degrees [41], [56], [57], as they are expected to behave similarly to Phenix [32].

All simulations were run with a network of size $\mathbf{n}$ = 1000. As the Phenix network needs a growing network to work, we started the Phenix algorithm with a network size of 20 and capped the size of the network to 1000. The simulations were run during 1000 cycles, and we repeated each simulation 100 times. All simulations were started with a network initialized as a k-out random graph, with $k = c = 20$. All simulations were run on 16 vCPU, using 64G of memory, on a cluster composed of 10 servers, described in Table 8.

[2] https://gitlab.lip6.fr/legheraba/elevator

Table 8: Description of the cluster

Machine	Memory	Processors	Cores
DELL PowerEdge XE8545	2 To	2 x AMD EPYC 7543	128 threads @ 2.80 GHz
DELL PowerEdge R750xa	2 To	2 x Intel Xeon Gold 6330	112 threads @ 2.00 GHz

We evaluated the following metrics: in-degree distribution, clustering coefficient, average shortest path length, and diameter.

The degree distributions of Newscast and PROOFS exhibit patterns akin to a normal distribution. We see similar results for Elevator, except for a distinct group of 10 hubs with an in-degree of 1000. By contrast, the Phenix protocol's degree distribution conforms to a power-law distribution.

PROOFS and Newscast maintain a low clustering coefficient during all simulations, as seen in Figure 18. On the contrary, Phenix and Elevator have both a clustering coefficient of around 0.6. For Phenix, the value is related to the power-law distribution of in-degree, and for Elevator, the value is linked to the presence of hubs, that are connected to everyone, and this automatically increases the value of the coefficient.

As we can see in Figure 19, Elevator has a very low average path length, with a value below 2. This value is due to the presence of hubs in the network, that permit to have a maximum distance of 2 between any 2 nodes. Phenix has the same value. PROOFS is very close, with a value around 2.15 and Newscast is a bit below 2.6. All these values are very good and thus we need to compute the diameter to discriminate between algorithms.

In Figure 20, we see that Elevator gives a network with a diameter equal to 2. Again, this value is due to the presence of hubs in the network. The Phenix algorithm yields similar results. This is better than PROOFS and Newscast, which output respectively 3 and 4 for this metric.

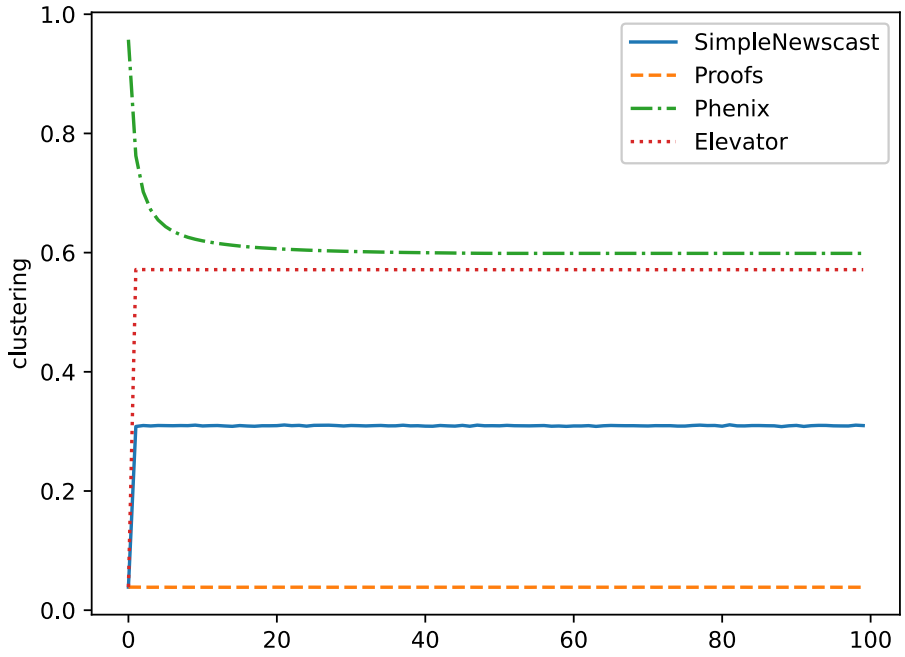

Figure 18: Clustering coefficient computed during the simulation (no failures), for each algorithm, everv 10 cvcles

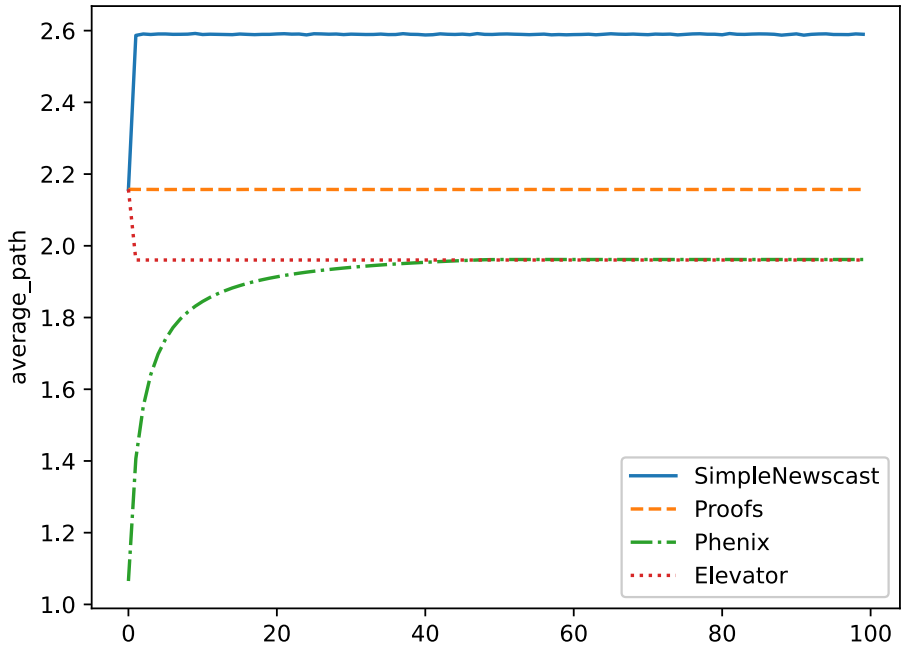

Figure 19: Average path length computed during the simulation (no failures), for each algorithm, every 10 cycles

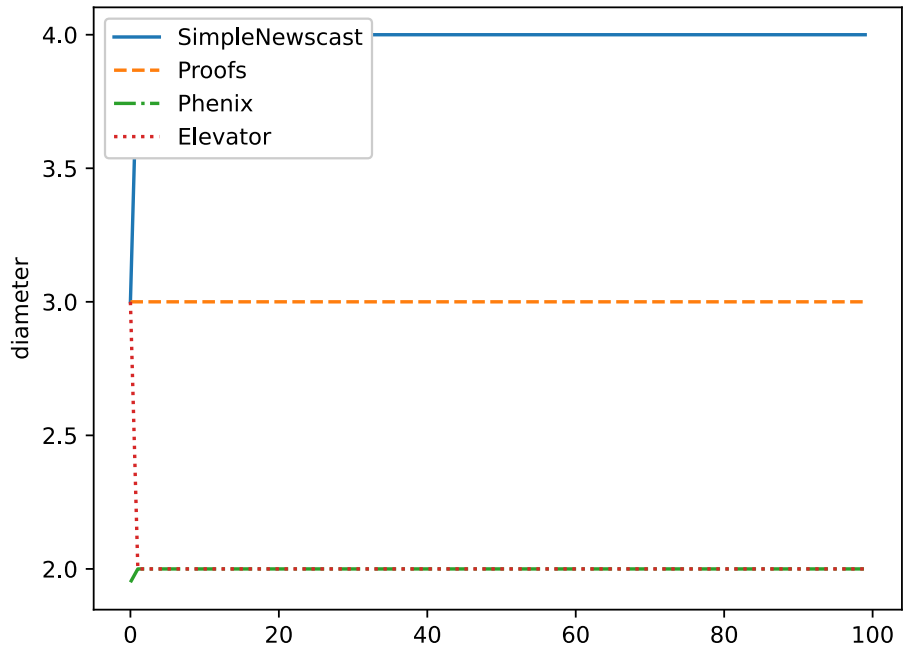

Figure 20: Diameter computed during the simulation (no failures), for each algorithm, every 10 cycles

We also compared the algorithms according to their resilience to crashes, churn, and byzantine attacks, as shown below.

4.4.1 Resilience to crashes

We analyze the performance of the four algorithms when the network suffers crashes. To simulate a brutal failure we disconnected 50% of the nodes in the middle of the simulation, **i.e.**, in this case, we have disconnected 500 nodes at cycle 500 (as there are 1000 nodes in total and 1000 cycles).

The performance of Elevator is not affected, as the in-degree distribution is still the same, and we have 10 hubs with an in-degree of 500. The degree distribution is also the same for Newscast and PROOFS. For Phenix, the degree distribution remains the same, with values going to a max of 1000, even if there are only 500 nodes in the network. It's because the nodes have kept in their cache the addresses of (old) nodes who are no longer in the network. In Figure 21, the clustering coefficient evolution shows that it is not affected by the crashes, as we have almost the same results as those obtained without a crash. The same observation holds for the average path length and the diameter, as we can see in Figure 22 and Figure 23.

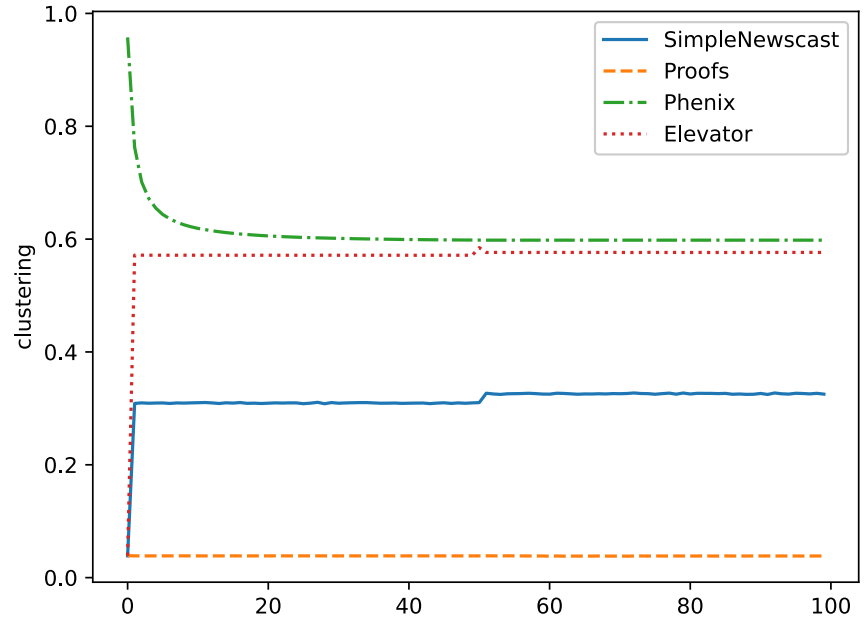

Figure 21: Clustering coefficient computed with a 50% crash, for each algorithm, every 10 cycles

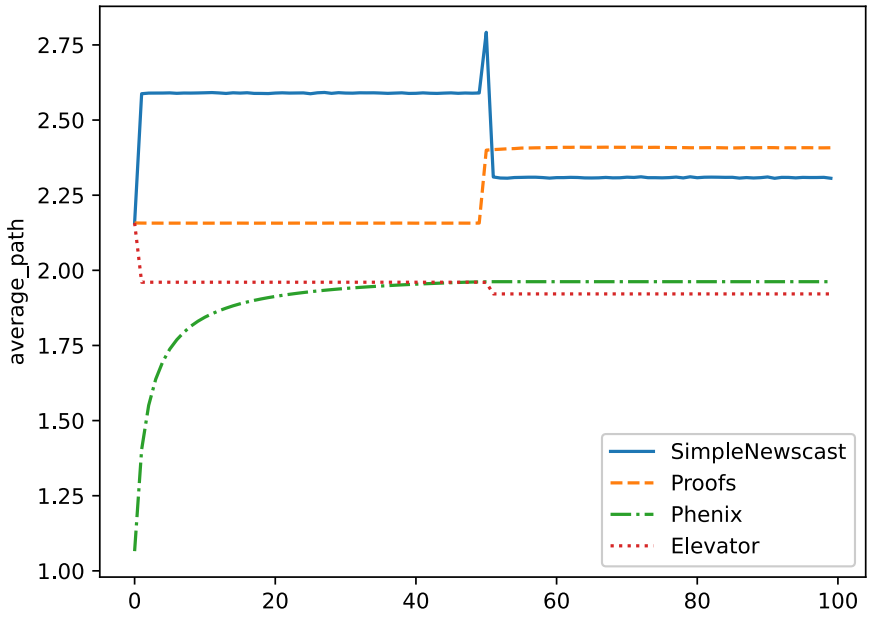

Figure 22: Average path length computed with a 50% crash, for each algorithm, every 10 cycles

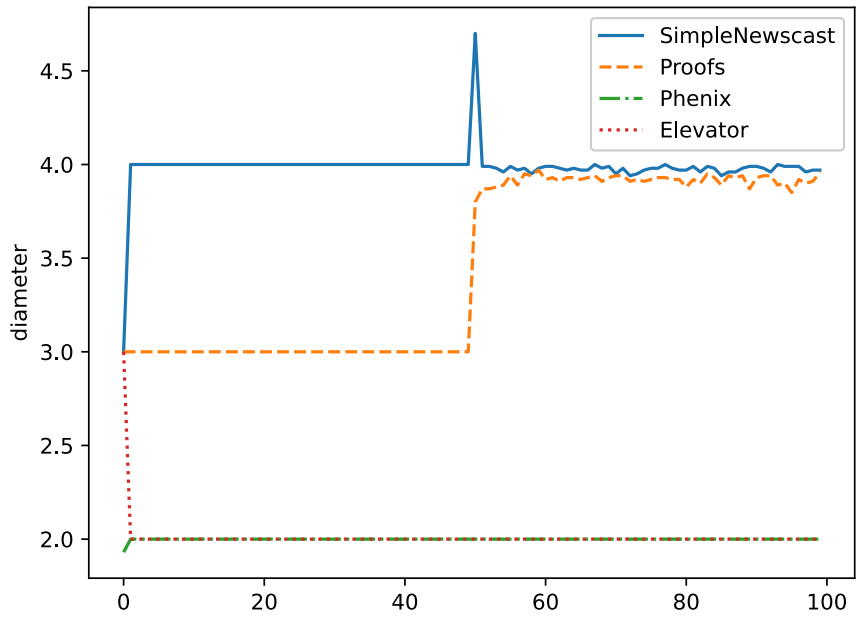

Figure 23: Diameter computed with a 50% crash, for each algorithm, every 10 cycles

4.4.2 Resilience to churn

We now analyze the performance of the four algorithms when the network is subject to churn. To simulate churn, we disconnected 10% of the nodes at each cycle and replaced them with the same amount of new nodes, each connected to 20 nodes uniformly at random. The churn occurs during 500 cycles, between cycle n°250 and cycle n°750. As the Phenix algorithm needs a growing network to work, the way we implement churn differs. Following previous work [32], in the case of Phenix, we implement churn having the number of removed nodes less than the number of added nodes at each cycle, assuming nodes are removed following a normal distribution $\mathcal{N}(0, 1)$, for all cycles of the simulation.

The in-degree distribution of Elevator remains the same, with 10 hubs. PROOFS seems affected by churn, as the mean degree distribution goes to 10 instead of 20 without churn. In Figure 24 we can observe that we have almost the same results as the results obtained without churn for the clustering coefficient. For the average path length, PROOFS is the most affected, with a value going from 2.25 without churn to a value of 2.5 with churn, and the value keep increasing after the end of the churn, going up to 2.75, as we can see in Figure 25. In Figure 26, we can see that the diameter varies with churn, with a mean going up to 3.25 instead of 2.0, but the values for Phenix and Elevator remain below the ones of Newscast and PROOFS.

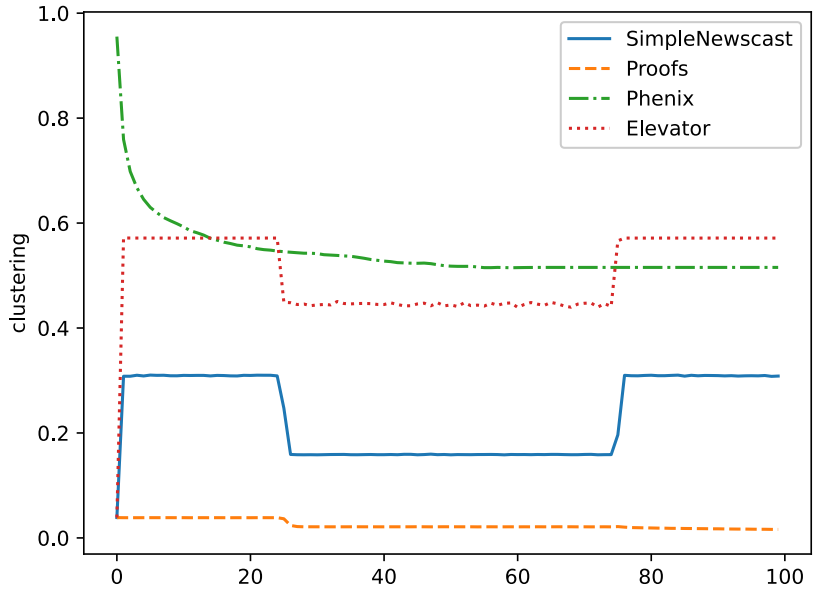

Figure 24: Clustering coefficient computed with churn, for each algorithm, every 10 cycles

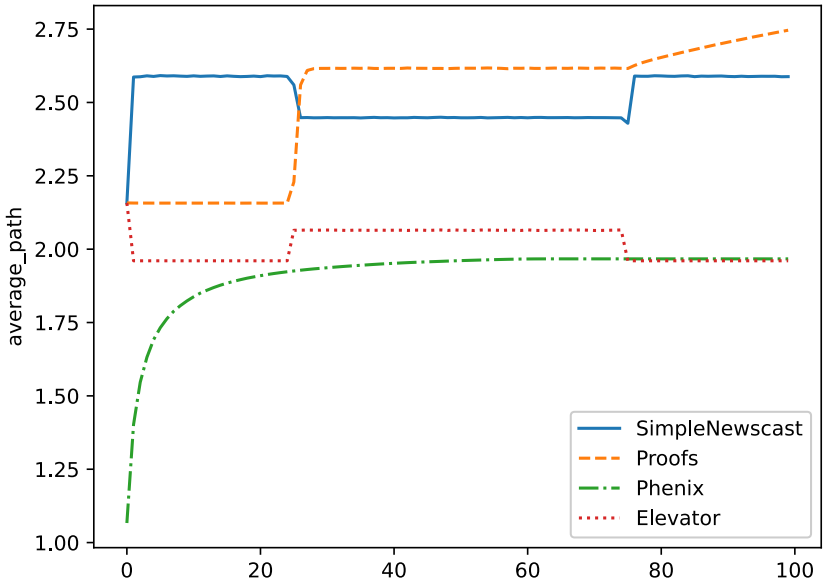

Figure 25: Average path length computed with churn, for each algorithm, every 10 cycles

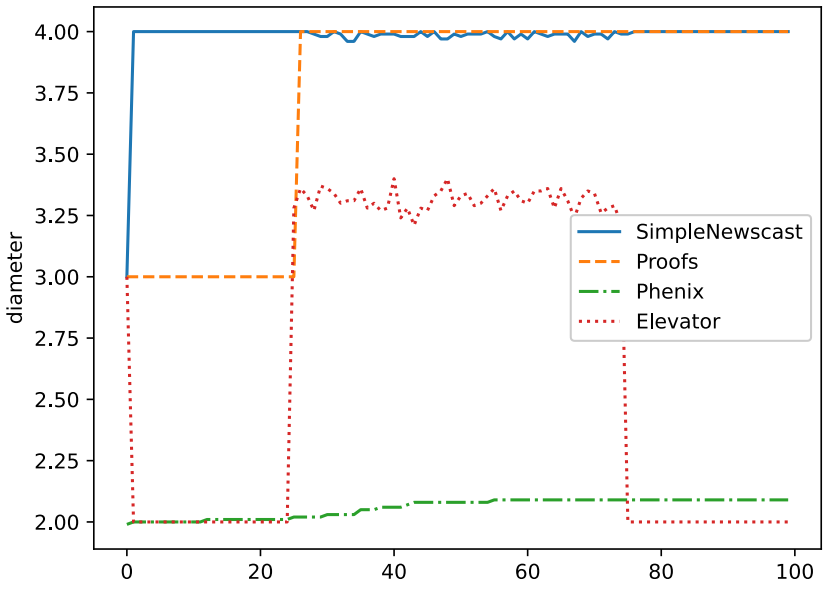

Figure 26: Diameter computed with churn, for each algorithm, every 10 cycles

4.4.3 Resilience to hub-targeted failures

We hereby analyze the performance of the four algorithms after a failure on the hubs during the execution of the simulation. To simulate it, we disconnected 10 nodes that have the highest in-degree in the middle of the simulated scenario.

Logically, Newscast and PROOFS are not affected by the failure, as there are no hubs in the networks built by these algorithms. For Elevator, the in-degree distribution remains similar, with 10 high-in-degree peers that have each an in-degree of 990. We are thus confident in the capacity of our algorithm to promote new nodes to the position of hubs if the previous hubs were disconnected. In Figure 27 we can see that we have almost the same results as the results obtained without crashes for the clustering coefficient. Its the same for the average path length and the diameter, there is no impact, as we can see in Figure 28 and Figure 29.

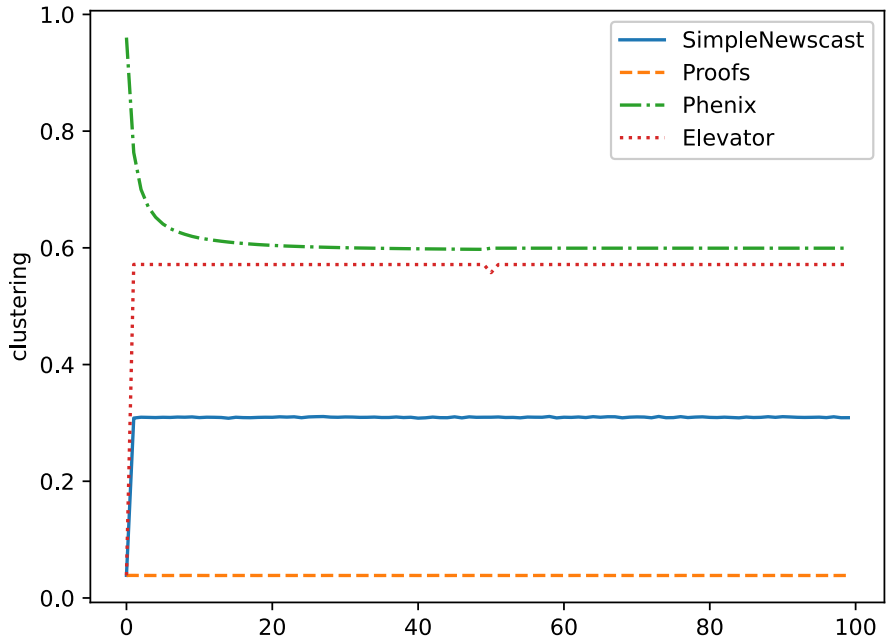

Figure 27: Clustering coefficient computed with a hub-targeted failure, for each algorithm, every 10 cycles

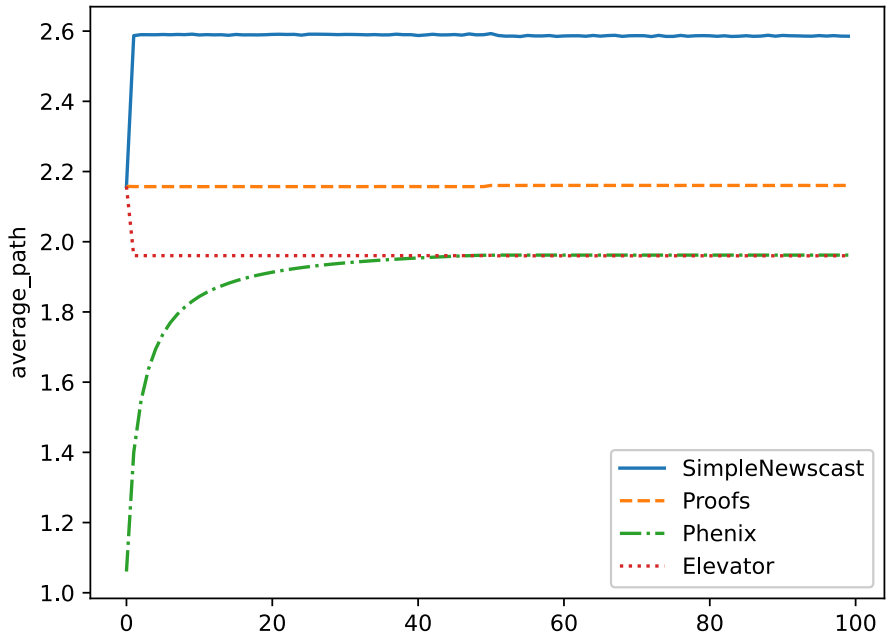

Figure 28: Average path length computed with a hub-targeted failure, for each algorithm, every 10 cycles

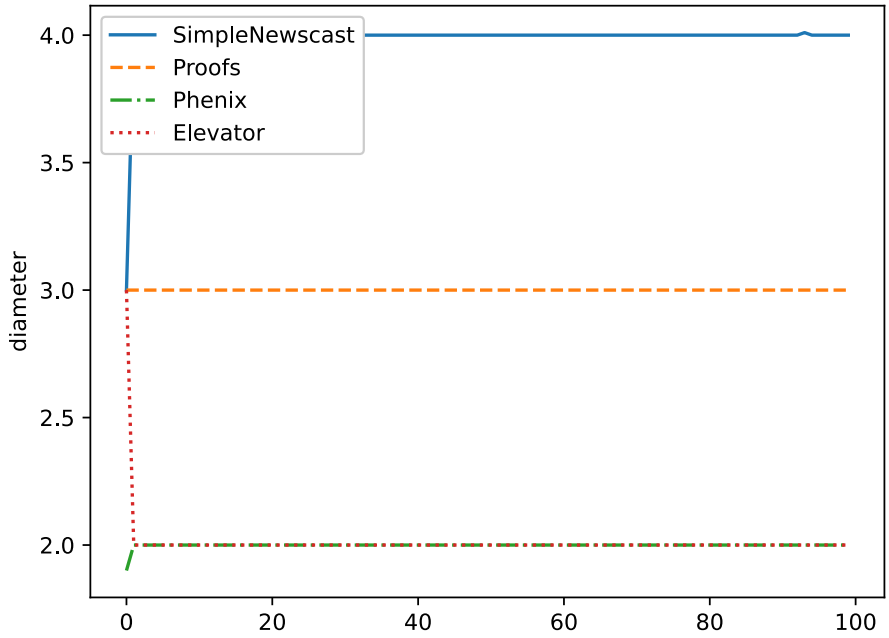

Figure 29: Diameter computed with a hub-targeted failure, for each algorithm, every 10 cycles

4.4.4 Resilience to Byzantine attacks

We measure Elevator's Byzantine resilience using two key metrics: (i) *Hub formation rate* — the number of hub positions held by legitimate nodes versus attackers (Byzantine nodes), and (ii) *Network topology stability* — whether hub formation continues to function correctly under attack. Each test runs for 1000 cycles to ensure network stabilization, and results are averaged over 100 independent simulations to account for randomness in network initialization and protocol execution.

Our experimental evaluation of Elevator under Byzantine attacks reveals several important insights regarding its resilience and limitations. When the protocol runs without malicious nodes, convergence to the 10 hubs occurs very quickly — in fewer than 4 cycles on average (Figure 30). Introducing a single Byzantine node in a 1,000-node network shows minimal disruption: in the passive case, the malicious node becomes a hub only 2 times out of 100 simulations, while in the active case, it becomes a hub 7 times out of 100; in both cases, the total number of hubs remains 10 (Figure 31, Figure 32). This confirms that Elevator is robust against isolated adversarial behavior.

When multiple non-coordinated Byzantine nodes are introduced randomly in the network, their impact remains limited. On average, only 0.95 out of 10 hubs are Byzantine, meaning that although the attackers represent 5% of the nodes, they account for 9.5% of hubs (Figure 33).

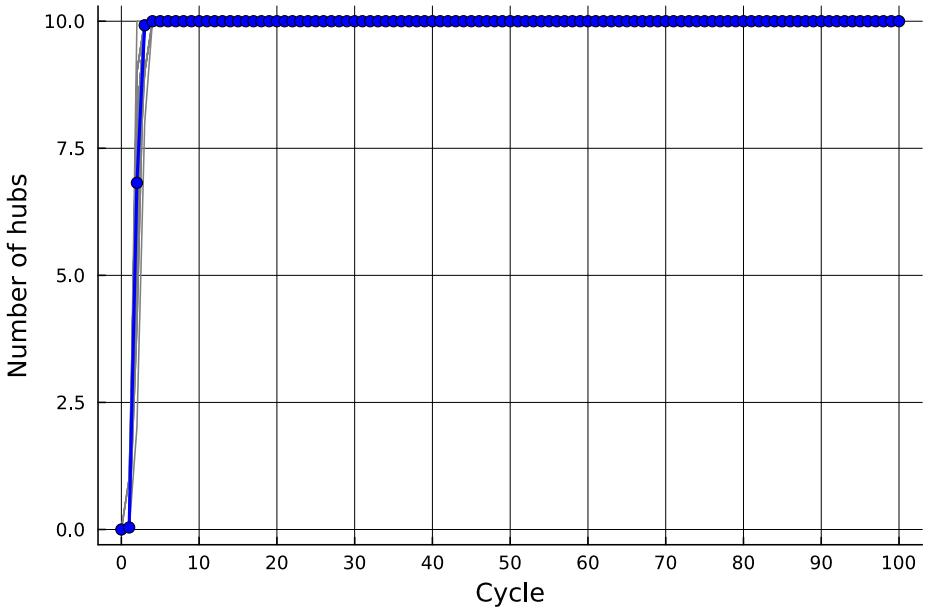

Figure 30: Running of Elevator without attack.

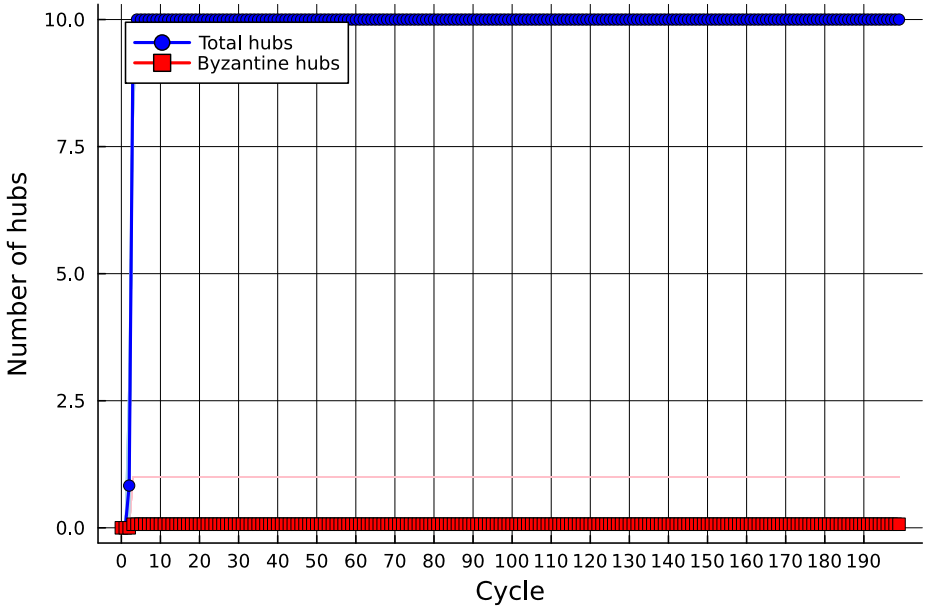

Figure 31: Active Byzantine behavior.

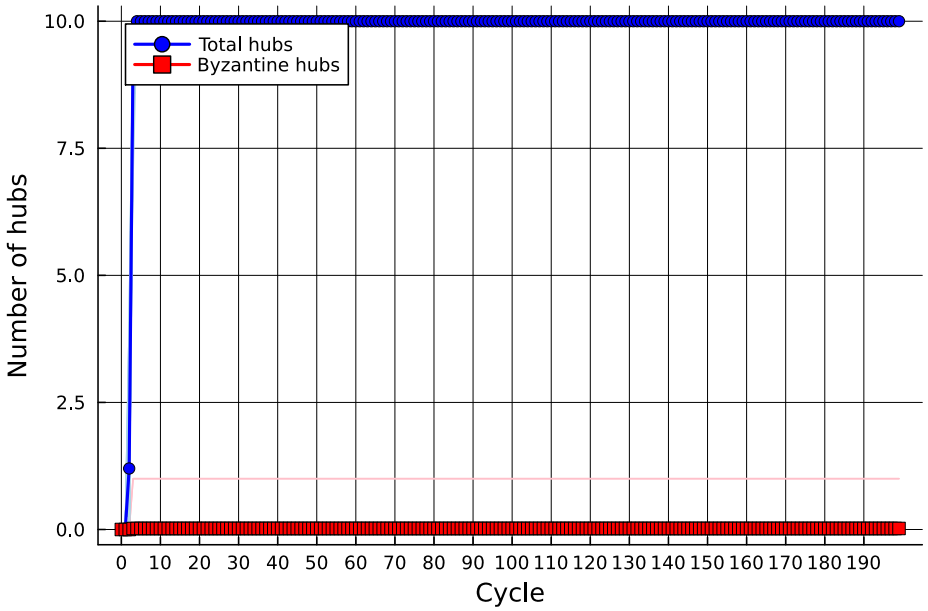

Figure 32: Passive Byzantine behavior.

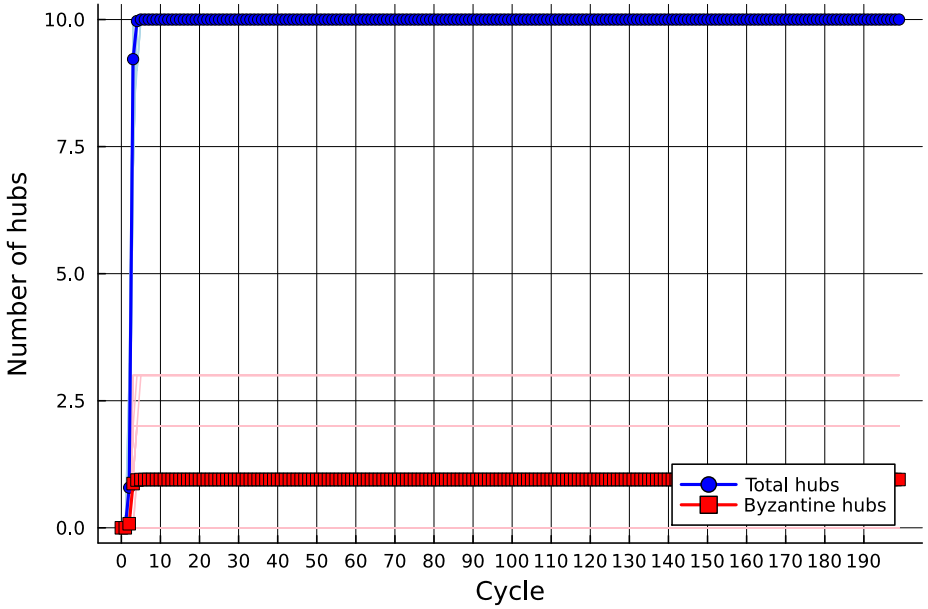

Figure 33: Independent Byzantine attack at 5% rate.

This highlights that coordination is a critical factor for a successful attack. Indeed, coordinated Byzantine nodes — each aware of all other Byzantine nodes and sharing this information when responding to cache requests — dramatically increase the risk of hub capture. Our experiments show a sharp vulnerability threshold around 2% Byzantine participation (Figure 34, Figure 35, Figure 36): at 1%, the proportion of Byzantine hubs rises from 1% of nodes to 13.4% of hubs, and at 5%, all 10 hubs become Byzantine. This threshold aligns closely with the cache size parameter ($c = 20$), indicating that coordinated attackers need to approach or exceed the cache size to overwhelm the random sampling mechanism effectively.

These findings demonstrate that while Elevator is resilient to individual or independent attacks, its main vulnerability lies in coordinated misinformation. Consequently, it is necessary to implement a defense mechanism that mitigates the influence of Byzantine nodes and restores fairness.

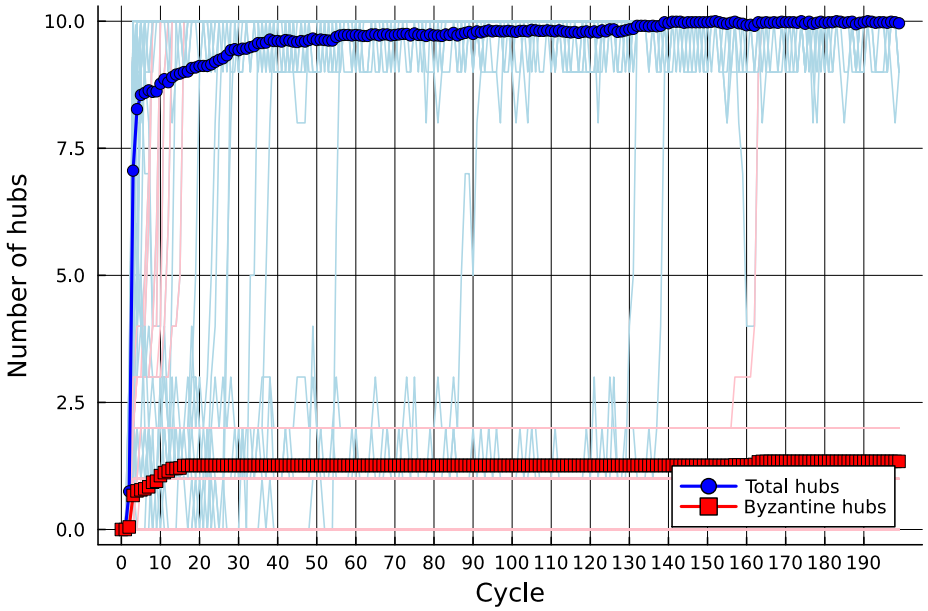

Figure 34: Byzantine hub infiltration at 1% rate.

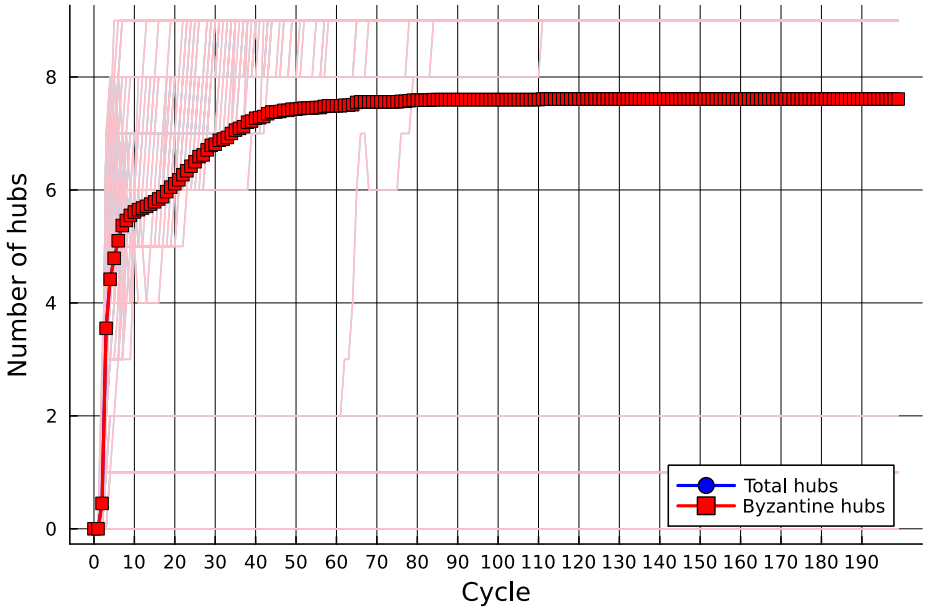

Figure 35: Byzantine hub infiltration at 2% rate.

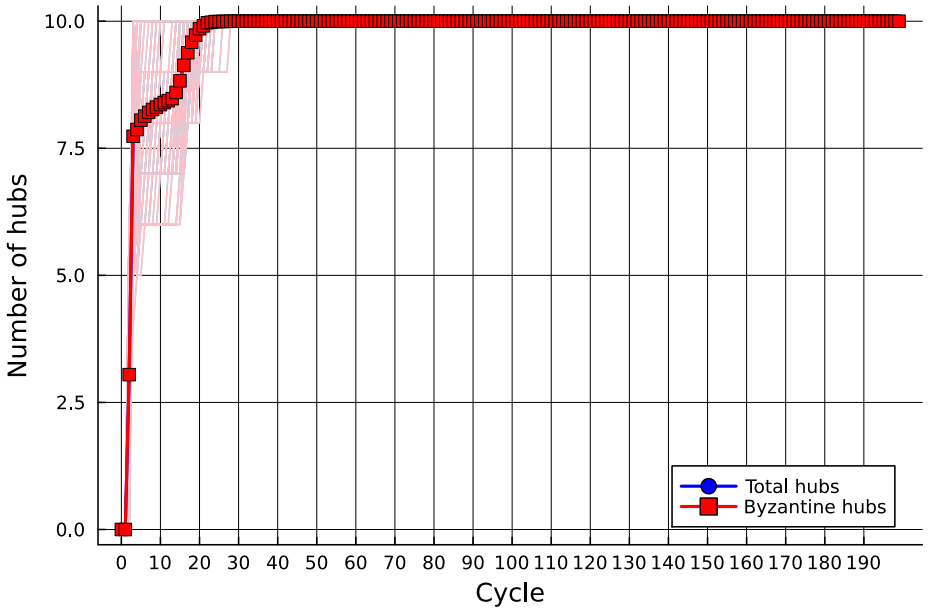

Figure 36: Byzantine hub infiltration at 5% rate.

4.4.5 Effectiveness of Lift countermeasure

We evaluate the effectiveness of our Lift protocol across different Byzantine participation rates, using the same experimental setup as in the vulnerability analysis. The countermeasure is activated at cycle 100, after which we observe its impact on hub formation and Byzantine infiltration.

At 5% Byzantine participation, the counter-attack is highly effective. After activation, Byzantine hubs are rapidly eliminated and remain at a minimal level for the rest of the simulation. The average total number of hubs decreases slightly to 9.58, while the average number of Byzantine hubs falls to 0.34. In other words, we go from 5% Byzantine nodes to 3.4% Byzantine hubs, representing an almost complete recovery from Byzantine infiltration with only a minor reduction in overall hubs (Figure 37).

For 10% Byzantine participation, the countermeasure initially removes Byzantine hubs effectively at cycle 100, but over subsequent cycles, Byzantine nodes gradually regain hub positions. By the end of the simulation, the network has on average 3.19 Byzantine hubs, and the total number of hubs has decreased from 10 to 7.82. This corresponds to approximately 40% of hubs being Byzantine. Although the majority of hubs remain non-Byzantine, the effectiveness is noticeably reduced compared to the 5% case (Figure 38).

At 15% Byzantine participation, the Lift countermeasure's effectiveness diminishes further. While the initial elimination at cycle 100 is successful, Byzantine nodes progressively reestablish themselves as hubs, reaching an average of 4.21 Byzantine hubs by the end. The total number of hubs also decreases from 10 to 6.71, meaning roughly 62% of hubs are now Byzantine. At this level, the countermeasure fails to maintain effective control over hub formation (Figure 39).

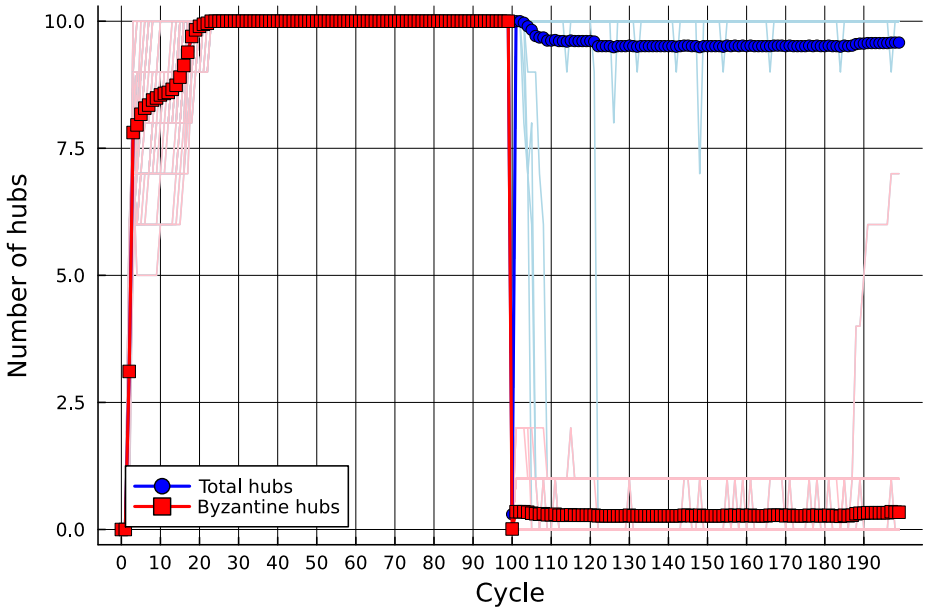

Figure 37: Counter-attack effectiveness at 5% rate.

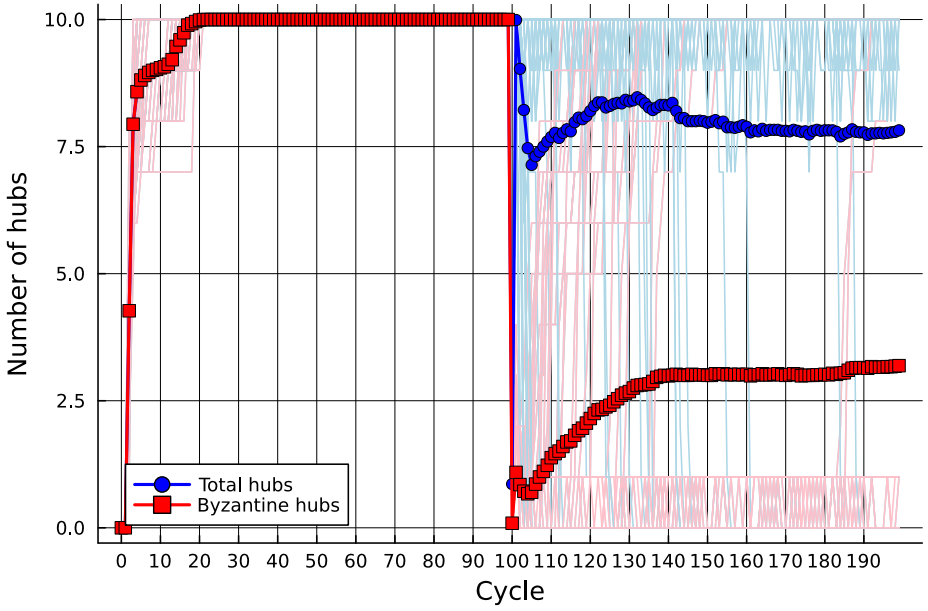

Figure 38: Counter-attack effectiveness at 10% rate.

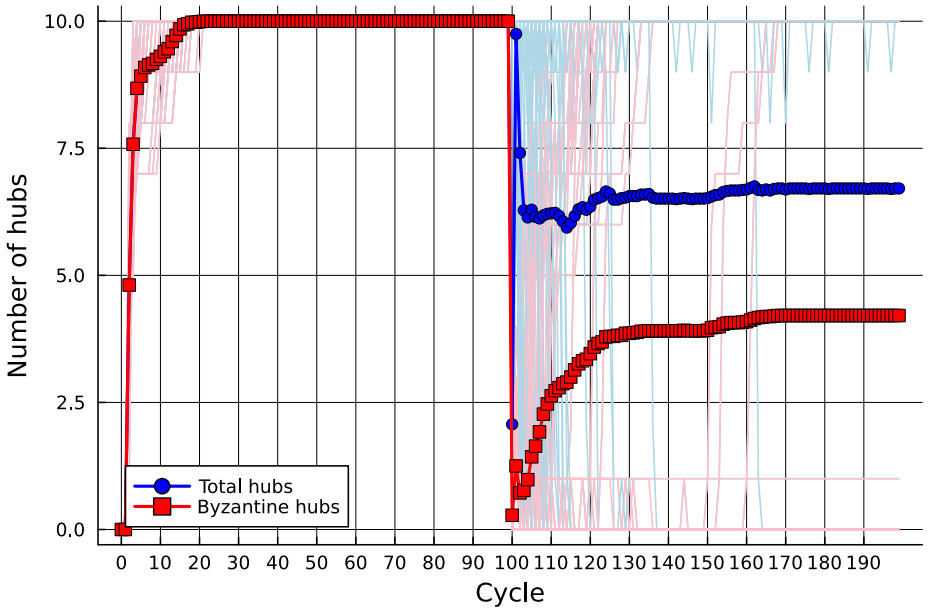

Figure 39: Counter-attack effectiveness at 15% rate.

4.4.6 Summary

We first analyzed the structural properties of the network produced by the Elevator protocol. The in-degree distribution remains consistent across different numbers of hubs (see Figure 40 and Figure 41), except in the extreme case where $h = c = 20$. In this configuration, nodes connect exclusively to hubs, resulting in a multi-star topology and eliminating random connections. This behavior is fully aligned with the protocol definition, where all outgoing links become preferential.

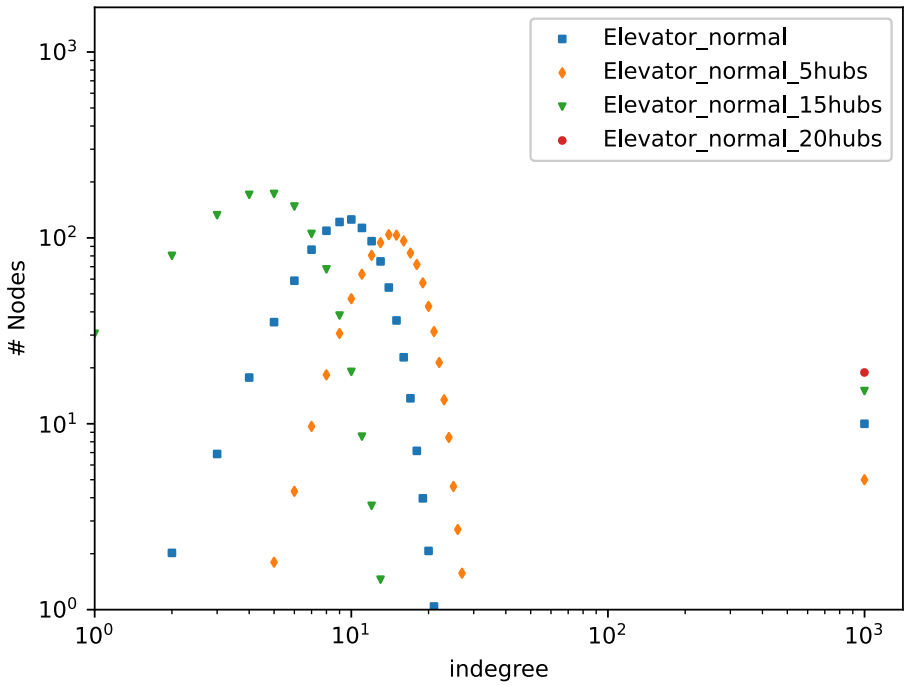

Figure 40: In-degree distribution of the network, after the run of the Elevator algorithm, with a variable number of hubs (5 hubs. 10 hubs. 15 hubs. 20 hubs). no failures.

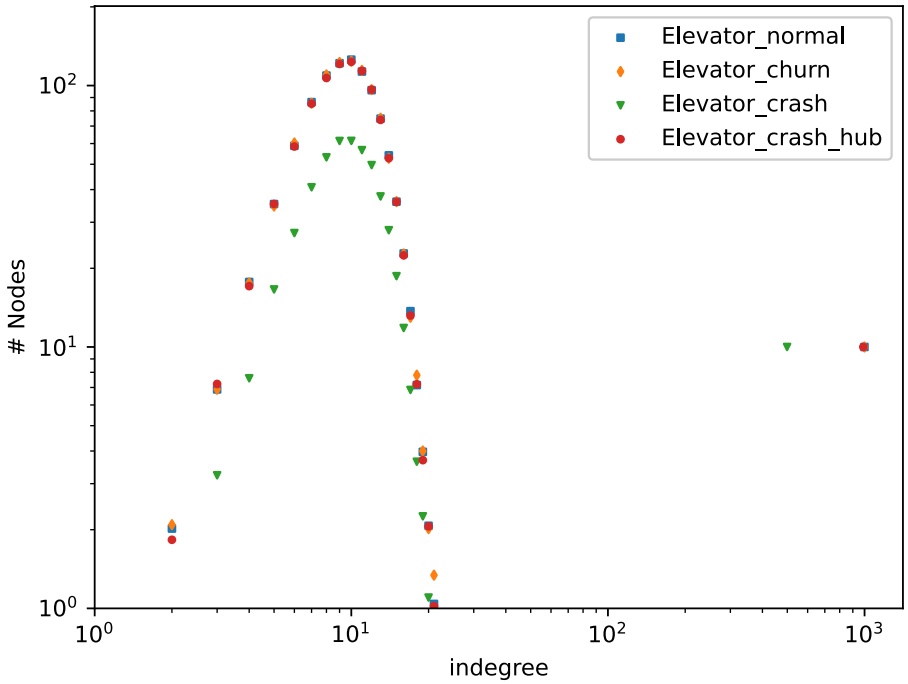

Figure 41: In-degree distribution of the network, after the run of the Elevator algorithm, during each context (no failures, 50% crash, churn, and hub-targeted failure).

Across different failure contexts, the overall distribution shape and structural metrics remain stable. As illustrated in Figure 42, Figure 43, and Figure 44, the clustering coefficient, average path length, and diameter exhibit only minor variations. This stability is an intrinsic property of the protocol: once hubs emerge, they remain stable over time (except in the presence of failures), which explains the robustness of global metrics. In this regard, Elevator demonstrates greater structural stability than Phenix, particularly concerning diameter and average path length.

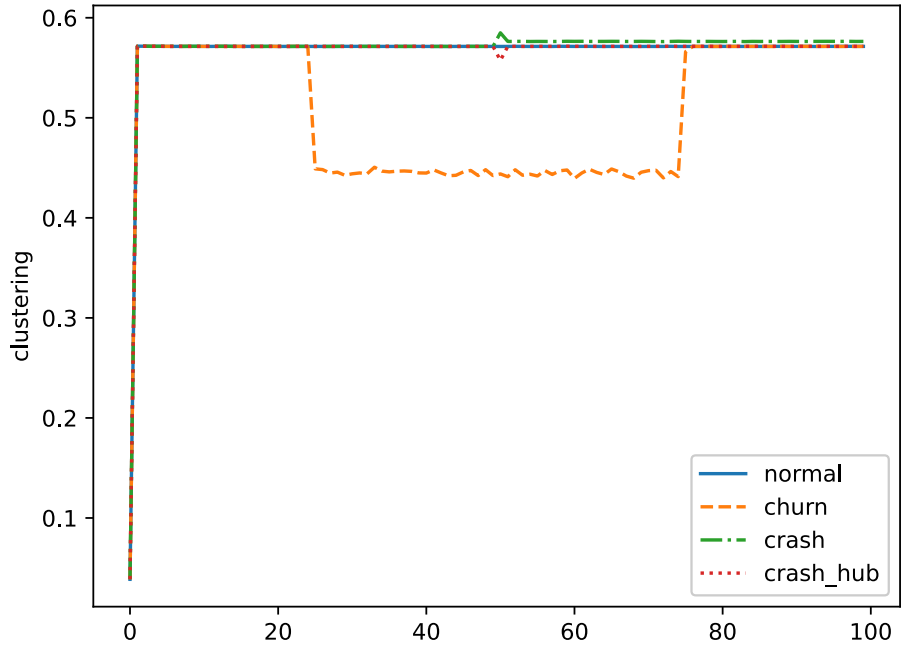

Figure 42: Clustering of the network, after the run of the Elevator algorithm, during each context (no failures, 50% crash, churn, and hub-targeted failure).

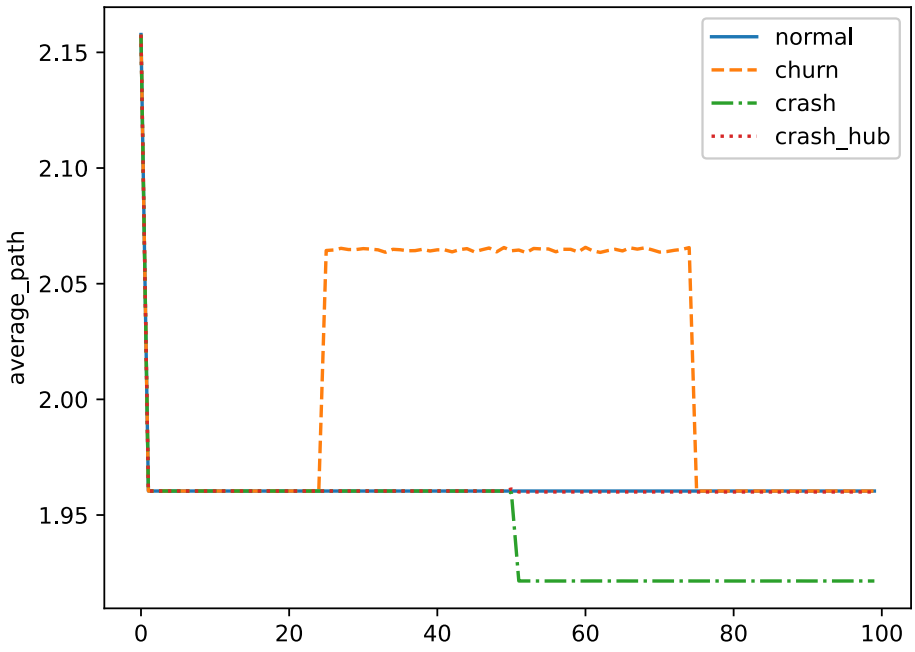

Figure 43: Average path length of the network, after the run of the Elevator algorithm, during each context (no failures, 50% crash, churn, and hub-targeted failure).

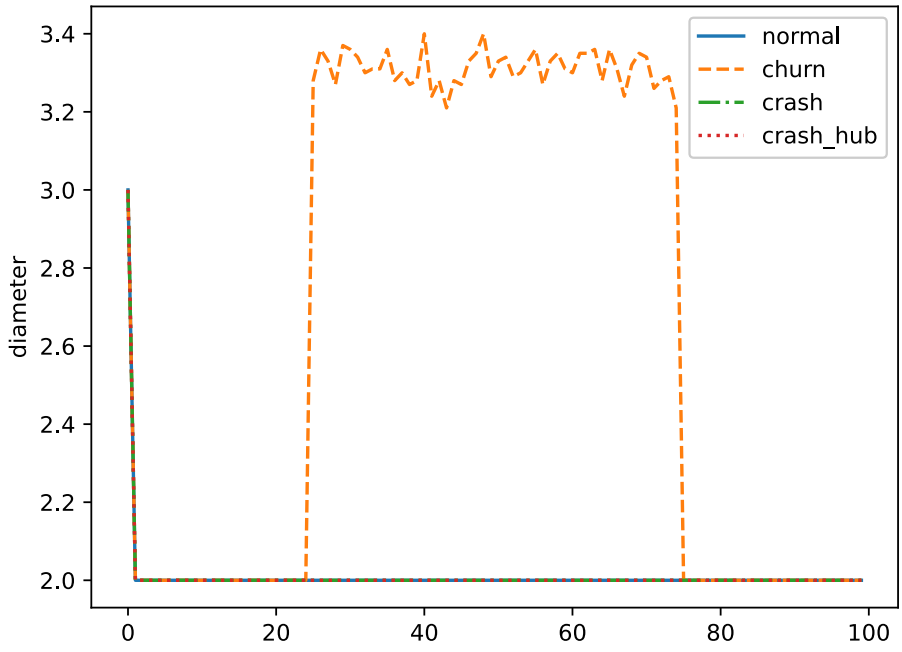

Figure 44: Diameter of the network, after the run of the Elevator algorithm, during each context (no failures, 50% crash, churn, and hub-targeted failure).

We then evaluated the protocol under Byzantine behavior and assessed the effectiveness of the Lift counter-attack. The results show that Lift successfully disrupts coordinated Byzantine hub capture at lower participation rates (e.g., 5%) by introducing a deterministic hub redistribution mechanism. However, as Byzantine participation increases (10% and 15%), its effectiveness decreases: malicious nodes progressively regain hub positions after the countermeasure is triggered. Additionally, the total number of hubs may decrease, indicating that Byzantine interference can prevent some correct nodes from maintaining their hub status.

Interestingly, even after activation of the countermeasure, Byzantine nodes continue attempting hub capture and achieve partial success, leading to slight deviations from the theoretical expectation of an average of $\frac{B}{N}$ Byzantine hubs. Nevertheless, Lift significantly reduces Byzantine influence while remaining lightweight, as it operates as a one-shot solution.

Overall, our simulation results demonstrate that Elevator achieves the targeted structural properties, including the emergence of a controlled number of hubs, bounded degree, and low network diameter. The protocol proves resilient to crash failures and churn, maintaining stable global metrics under dynamic conditions. However, it remains vulnerable to coordinated Byzantine attacks. The proposed Lift countermeasure increases resilience against such attacks without compromising the decentralization or the performance of the protocol.

4.5 Implementation over TCP/IP

To complement the simulation-based evaluation, we implemented a fully operational version of the Elevator protocol over real TCP/IP networks. This implementation (available at https://github.com/MohamedLEGH/elevator-algorithm) serves two main purposes: (i) validating the feasibility of Elevator in a realistic peer-to-peer environment, and (ii) assessing its behavior under asynchronous execution, failures, and heterogeneous deployment conditions.

4.5.1 Implementation choices and technological stack

The implementation relies on the Go programming language[3] and the **libp2p** networking framework[4]. Go was chosen primarily for its strong support for concurrency through goroutines and channels, which naturally fits the highly concurrent nature of peer-to-peer protocols. In addition, Go provides efficient networking primitives and a mature ecosystem for building distributed systems.

The **libp2p** library was used to implement the peer-to-peer communication layer. It provides abstractions for peer identities, transport protocols, stream multiplexing, and protocol negotiation, allowing the Elevator algorithm to be deployed over unstructured peer-to-peer overlays without relying on any centralized component. Communication between peers is performed over TCP/IP using libp2p streams, each stream being associated with a specific protocol identifier.

In parallel, the standard `net/http` library was used to expose a lightweight HTTP interface on each node. This interface serves two roles: (i) enabling external control and monitoring of nodes (e.g., initialization, data collection, experiment orchestration), and (ii) facilitating the bootstrapping phase of the network.

4.5.2 Node architecture

Each peer in the network is an autonomous process characterized by:

- a **libp2p port**, used exclusively for peer-to-peer communication and protocol execution;
- an **HTTP port**, used for external control, initialization, and data collection.

[3] https://go.dev/
[4] https://libp2p.io/

Upon startup, a node initializes its libp2p host and registers stream handlers for the Elevator protocol. Each handler corresponds to a specific message type (e.g., cache request or backward request) and defines the logic executed upon reception of a stream. At the same time, an HTTP server is launched in a separate goroutine, allowing the node to receive external commands without blocking protocol execution. A third goroutine is optionally used to process user input from the terminal, mainly for debugging and manual control during experiments.

The node remains idle until its local cache has been fully initialized. Only once this condition is met does it start executing the Elevator protocol.

4.5.3 Cache initialization and network bootstrapping

Since the libp2p implementation targets unstructured peer-to-peer networks, no predefined topology is assumed. The initial overlay is therefore constructed externally in a bootstrapping phase.

First, a configuration file listing all participating nodes (IP addresses and HTTP ports) is generated. Using this information, each node is assigned an initial cache of size (c), corresponding to a random (c)-out graph. The initial caches are generated and then distributed to the nodes through HTTP POST requests. Upon reception, each node stores the received cache locally and acknowledges successful initialization.

This approach ensures that all nodes start from a well-defined and controlled initial state, while remaining faithful to the assumptions of the theoretical model.

4.5.4 Execution of the Elevator protocol

Once initialized, each node repeatedly executes the Elevator protocol in cycles. During each cycle, the following steps are performed:

(i) **Frequency map construction** The node queries all peers in its local cache for their respective caches using libp2p streams. The responses are aggregated into a frequency map that counts how often each peer appears.

(ii) **Hub selection** The node selects the top-(h) peers with the highest frequencies as potential hubs and removes them from the frequency map.

(iii) **Backward exploration** For each selected hub, the node requests a backward list (i.e., incoming neighbors) using a dedicated protocol message. These lists are merged to enrich the candidate set.

(iv) **Cache reconstruction** A new cache of size (c) is built by combining the selected hubs with a subset of backward peers and, if necessary, additional randomly selected nodes.

This process closely mirrors the algorithmic description introduced earlier, but operates over real network connections and asynchronous message exchanges.

4.5.5 Execution modes and synchronization strategies

To explore different execution semantics, three variants of the protocol were implemented:

- **Synchronous start, synchronous cycles** All nodes start at a predefined time and execute each cycle in lockstep, waiting a fixed duration between cycles.
- **Externally synchronized execution** A centralized controller periodically triggers the start of each cycle by sending HTTP requests to all nodes. While the Elevator protocol itself remains decentralized, this mode facilitates controlled experiments and reproducibility.
- **Asynchronous execution** Nodes start simultaneously but wait a random duration between cycles. This mode reflects more realistic conditions, where nodes are not synchronized and operate independently.

These variants allow us to study the robustness of Elevator under both idealized and realistic timing assumptions.

4.5.6 Experimental validation

The implementation was validated through a series of experiments on small- to medium-scale networks (ranging from 20 to 50 nodes), executed either on a single machine or distributed across two machines. Experiments confirmed the rapid emergence of hubs within the first few cycles, in line with the theoretical analysis and simulation results, as seen in Figure 45 and Figure 46.

In Figure 47, we show experiments that simulated hub failures by forcibly disconnecting the highest-degree nodes during execution. In all cases, new hubs emerged naturally after a short transient phase, demonstrating the self-healing properties of the protocol. The presence of random connections in the cache played a crucial role in maintaining connectivity and enabling recovery.

From a systems perspective, the implementation revealed a high degree of concurrency, with a large number of goroutines active at runtime. This behavior is expected, as libp2p internally spawns goroutines for stream handling, connection management, and message processing. Despite this, the system remained stable and responsive throughout the experiments.

Overall, this TCP/IP implementation confirms that Elevator is not only theoretically sound and effective in simulation, but also practical and robust when deployed over real peer-to-peer networks. It further demonstrates that the protocol tolerates asynchronous execution, node failures, and dynamic network conditions, making it suitable for realistic distributed environments.

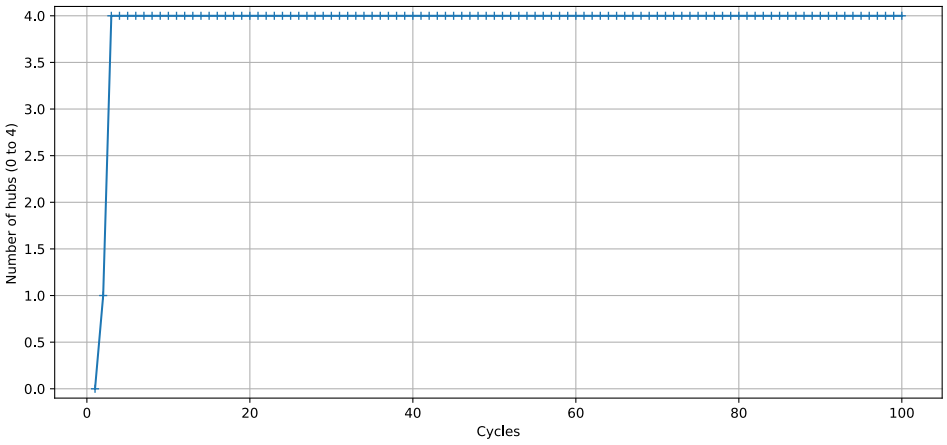

Figure 45: Number of hubs at each cycle, with $N = 20$, $c = 10$ and $h = 4$

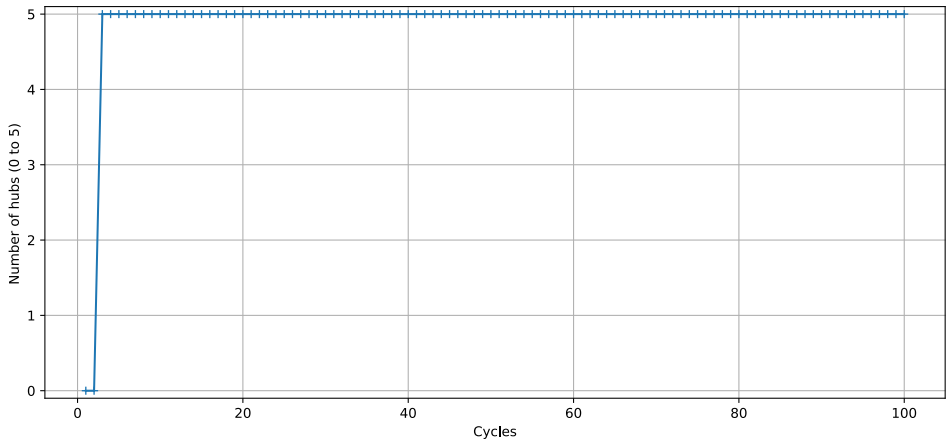

Figure 46: Number of hubs at each cycle, with $N = 50$, $c = 10$ and $h = 5$

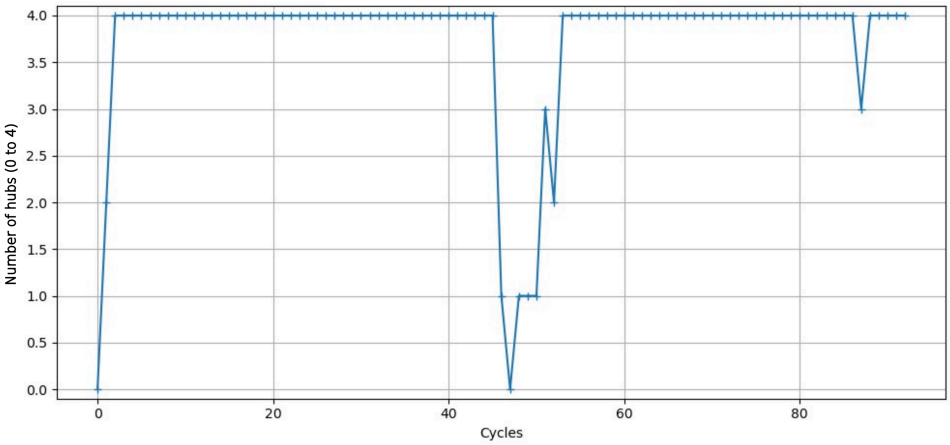

Figure 47: Crash of the hubs in the middle of the experiment, with $N = 50$, $c = 10$ and $h = 4$

4.5.7 CPU Information Collection

Monitoring CPU usage is a critical aspect of evaluating the performance and behavior of each node in the network. Metrics such as CPU utilization (%CPU), CPU time, and memory allocation provide insight into the resource consumption of individual processes. To automate this process, we developed the script `info.py`, which collects these metrics for all nodes and stores them in a CSV file for subsequent analysis. The script is executed at the end of the `launch_nodes.sh` script to ensure that metrics are captured throughout the lifetime of the experiment.

The `info.py` script identifies all processes named `main` and retrieves their process identifiers (PIDs). Using these PIDs, it executes system commands to extract the desired metrics, including CPU and memory statistics. This approach enables precise monitoring of the computational load imposed by the Elevator protocol on each node.

Following preliminary tests on a personal machine, the implementation and scripts were adapted to conduct experiments in a dedicated Linux environment. This allows for more controlled and scalable evaluation of the protocol under realistic system conditions.

For the single-machine experiments, three configurations of the Elevator protocol were tested. In all configurations, the network consisted of 100 nodes executing 100 protocol cycles, with each node maintaining a cache of size 20. The three versions differed in the number of hubs: Version 1 used 10 hubs, Version 2 used 5 hubs, and Version 3 used a single hub. These experiments allowed us to evaluate

the impact of varying the number of hubs on the stabilization and performance of the protocol while keeping other parameters constant. For all three versions, the experiments were conducted using 100 nodes with a cache size of 20 and 100 protocol cycles, while varying the number of hubs. The resulting graphs (Figure 48, Figure 49 and Figure 50) were consistent with those presented in the previous section, showing rapid stabilization of hubs within the first cycles, regardless of parameter variations. Analysis of CPU metrics revealed that certain nodes consumed nearly twice the %CPU and CPU time compared to others. These nodes were identified as the selected hubs, which aligns with the intrinsic definition of a hub: a node maintaining a large number of connections to other peers. Indeed, hubs transmit their caches to a larger subset of nodes, explaining the increased computational load observed.

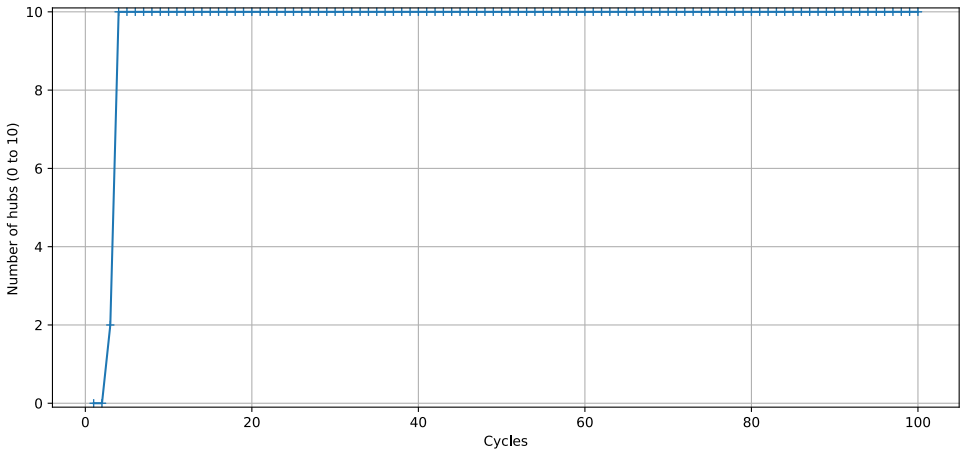

Figure 48: Number of hubs at each cycle, semi-synchronous, with $N = 100$, $c = 20$ and $h = 10$

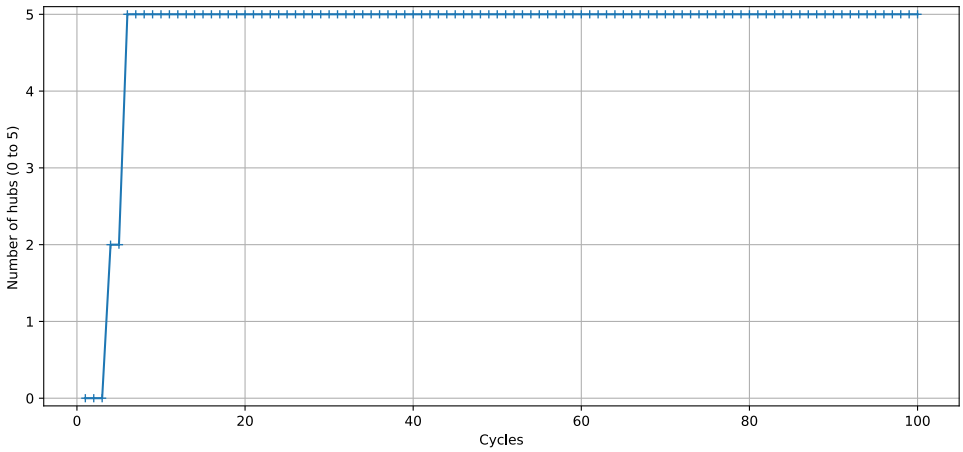

Figure 49: Number of hubs, synchronous mode, with $N = 100$, $c = 20$ and $h = 5$

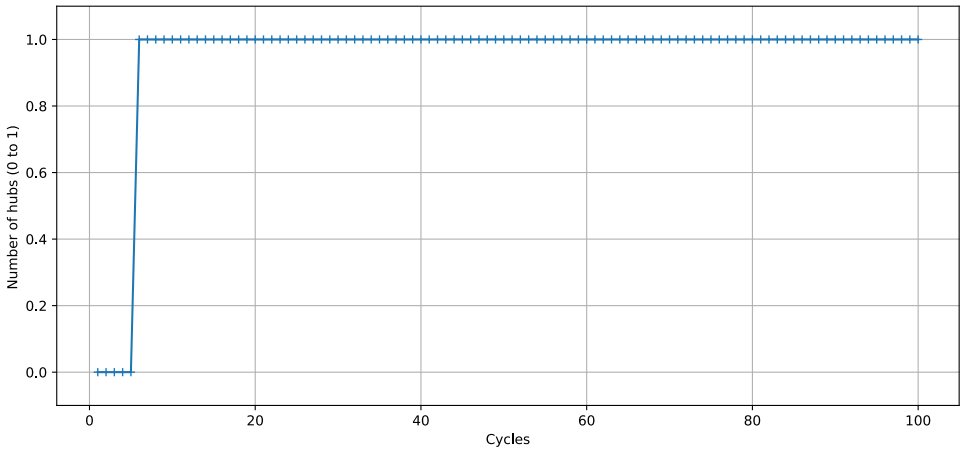

Figure 50: Number of hubs, asynchronous mode, with $N = 100$, $c = 20$ and $h = 1$

For the two-machine experiments, the network was distributed across a server and a local machine. The server hosted 99 nodes, while the local machine hosted a single node, resulting in a total of 100 nodes. All nodes executed 100 protocol cycles, and each maintained a cache of size 20. The experiment used 10 hubs. This configuration allowed us to observe the behavior and stabilization of hubs in a distributed setup spanning multiple machines, providing insight into the protocol's robustness under a heterogeneous deployment. The results obtained mirrored those of the single-machine experiments. As seen in Figure 51, Figure 52 and Figure 53, hubs consistently stabilized within the first cycles, demonstrating that the protocol behavior is robust under a distributed setup spanning multiple machines.

Experimental results confirmed theoretical expectations, with rapid convergence to the preconfigured number of hubs across all tested scenarios. Variations in node parameters did not affect the overall stabilization behavior, illustrating the robustness of the Elevator protocol. Future work may involve scaling the experiments to larger networks distributed across more machines to assess performance at a greater scale and to compare results under more heterogeneous deployment conditions.

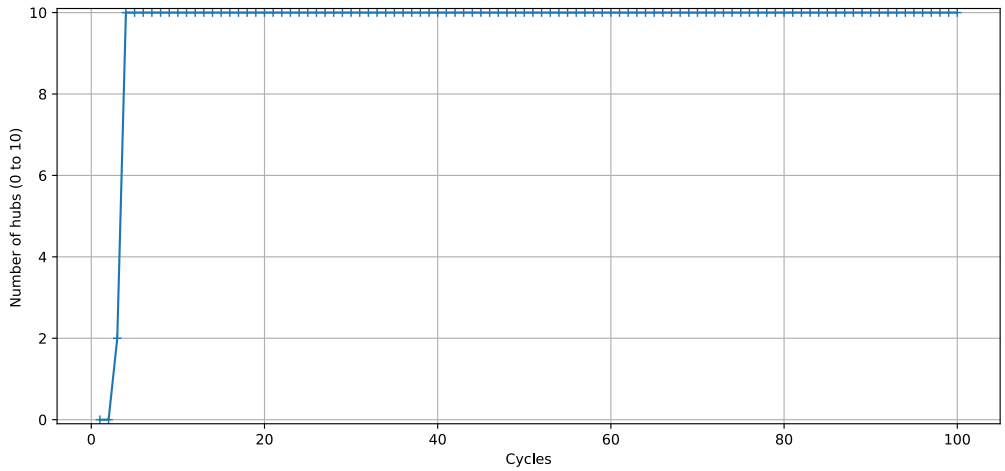

Figure 51: Experiments on a cluster of 2 machines, semi-synchronous mode, with $N = 100$, $c = 20$ and $h = 10$

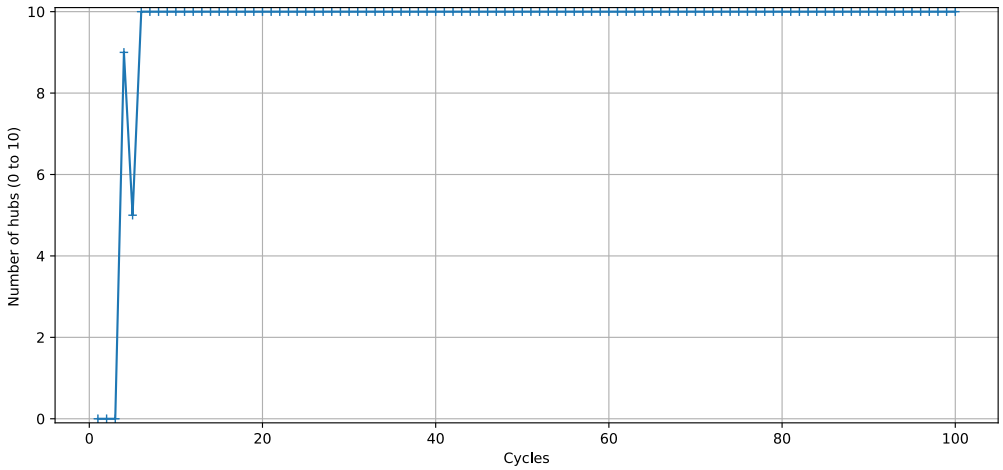

Figure 52: Experiments on a cluster of 2 machines, synchronous mode, with $N = 100$, $c = 20$ and $h = 10$

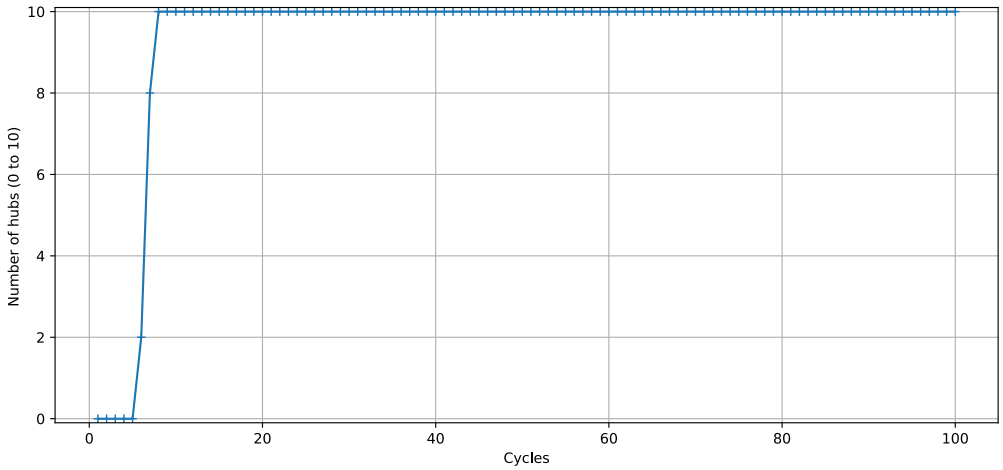

Figure 53: Experiments on a cluster of 2 machines, asynchronous mode, with $N = 100$, $c = 20$ and $h = 10$

4.6 Conclusion

We proposed a novel peer sampling algorithm, Elevator, designed for unstructured P2P networks, which enables the organic emergence of a controlled number of hub nodes while preserving decentralization and bounded degree.

Our study combines theoretical analysis, simulation-based evaluation, and real-world experimentation. First, we conducted a formal analysis of the algorithm, establishing its convergence, its stability properties, and providing insights into its convergence speed. This theoretical investigation shows that the protocol drives the system toward a topology characterized by a predefined number of hubs h, while maintaining randomness among the remaining connections.

We then validated these properties through extensive simulations. The results demonstrate that Elevator achieves the targeted structural objectives: low network diameter, stable hub formation, bounded degree, and robustness under crash failures and churn. The protocol maintains stable global metrics even under dynamic conditions, confirming the soundness of its design.

Beyond simulations, we implemented Elevator on a real network, confirming its practical feasibility and validating that its theoretical and simulated properties hold in realistic environments.

We also investigated the vulnerability of Elevator to Byzantine attacks. Our analysis shows that, while the protocol is resilient to failures and churn, it remains vulnerable to coordinated Byzantine strategies aiming at capturing hub positions. To address this limitation, we proposed a modification of the algorithm, Lift, which increases resilience against Byzantine behavior through a deterministic redistribution mechanism. Importantly, this countermeasure improves robustness without compromising decentralization or degrading the performance of the protocol.

Elevator opens the way to a new class of algorithms that we refer to as hub sampling algorithms, where structural centrality is deliberately engineered within unstructured overlays. One particularly promising application domain is artificial intelligence, and federated learning in particular, where controlled hub structures may accelerate model aggregation and dissemination. Before presenting this contribution, Chapter 5 surveys the state of the art in decentralized learning. This use case (and our associated contribution in the field) is then studied in detail in Chapter 6.

Chapter 5

Decentralized Machine Learning

This chapter focuses on a less explored branch of machine learning: decentralized learning. While the dominant research direction assumes the existence of a central authority to coordinate the learning process, decentralized learning investigates how a set of nodes can collaboratively train a model without any central coordinator. Throughout this chapter, we assume familiarity with the foundational concepts of supervised learning — in particular the notions of loss function, gradient descent, and standard model architectures — which are covered in detail in Appendix A.3. In brief, supervised learning consists in finding the parameters θ of a model f_θ that minimize a loss function $L(\theta)$ over a labeled dataset, using gradient descent as the optimization procedure.

Machine learning models can be trained under different assumptions regarding data availability, computational resources, and the organization of the learning process. The most straightforward paradigm is **centralized learning**, in which all training data are collected and processed at a single location. This setting enables exact gradient computation over the full dataset and provides strong theoretical guarantees, but it raises fundamental challenges: data must be transferred to a central location, which is costly, raises privacy concerns, and becomes impractical at the scale of modern distributed systems. **Online learning** relaxes the assumption that the full dataset is available before training begins, allowing the model to update incrementally as new data arrives. **Ensemble learning** shows that combining multiple independently trained models can yield better predictive performance than any single model — a principle that foreshadows the aggregation mechanisms central to decentralized learning, where nodes repeatedly exchange and average local model updates to collectively optimize a shared objective without any node having access to the full dataset. **Distributed learning** addresses the scalability limitations of centralized learning by spreading data and computation across multiple nodes, but still relies on a coordinating entity — typically a parameter server. A detailed treatment of these paradigms is provided in Appendix A.4.

The natural next step, and the focus of this chapter, is to remove this central coordinator entirely: decentralized learning requires nodes to collaborate over data that remains private and local to each node, emerging naturally at the intersection of machine learning and peer-to-peer systems, as illustrated subsequently in Figure 54.

From a distributed systems perspective, it extends decentralized computation to the learning setting: instead of collaboratively computing a global statistic, each node maintains a local model and participates in the learning process exclusively through peer-to-peer interactions. From a machine learning perspective, it relaxes the centralisation assumption by keeping data local and moving models across nodes rather than data. We now formalise this setting.

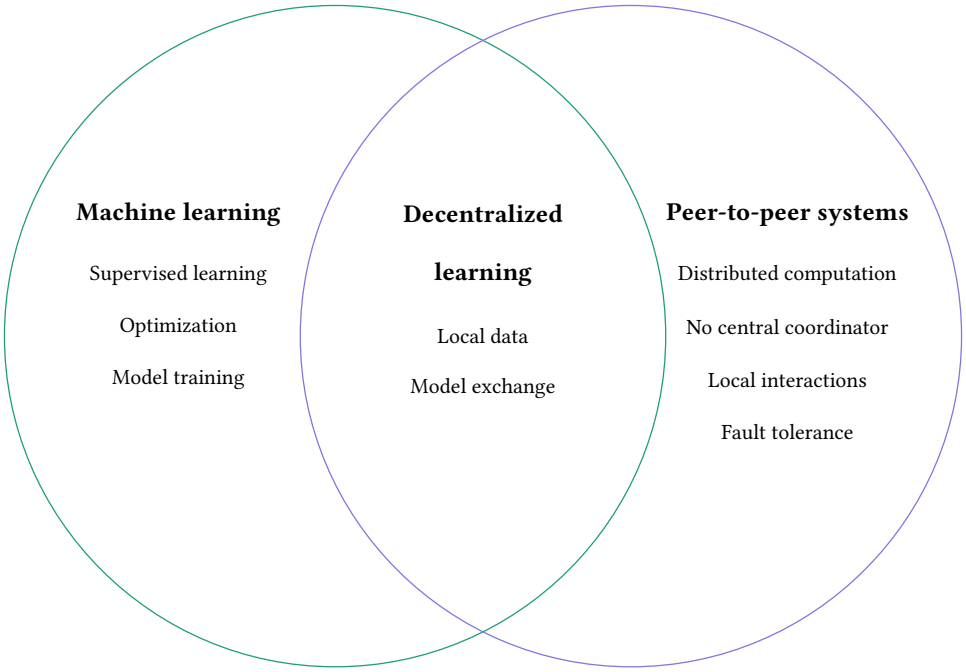

Figure 54: Decentralized learning at the intersection of machine learning and peer-to-peer systems.

5.1 Formal Framework

Having situated decentralized learning at the intersection of machine learning and peer-to-peer systems, and distinguished it from centralized and distributed paradigms, we now turn to a formal treatment of the problem. We begin by clarifying how data are partitioned across nodes, which conditions all subsequent definitions.

5.1.1 Data Partitioning

Decentralized learning approaches can be broadly categorized into two settings, commonly referred to as horizontal and vertical, depending on how data are partitioned across nodes [58].

In horizontal decentralized learning, all nodes share the same feature space but hold different subsets of data instances. Each node trains a local model on its own dataset, and learning proceeds by combining these local models through parameter-wise aggregation — for instance by averaging corresponding parameters across nodes. This setting is particularly well suited to peer-to-peer and federated environments, where data are naturally distributed across participants but follow a common schema.

In vertical decentralized learning, nodes observe the same set of data instances but with disjoint feature subsets. No single node has access to the full feature vector of an instance; learning therefore requires exchanging intermediate representations or partial gradients computed on complementary feature subsets. This setting involves stronger coordination constraints and more complex communication patterns than the horizontal case.

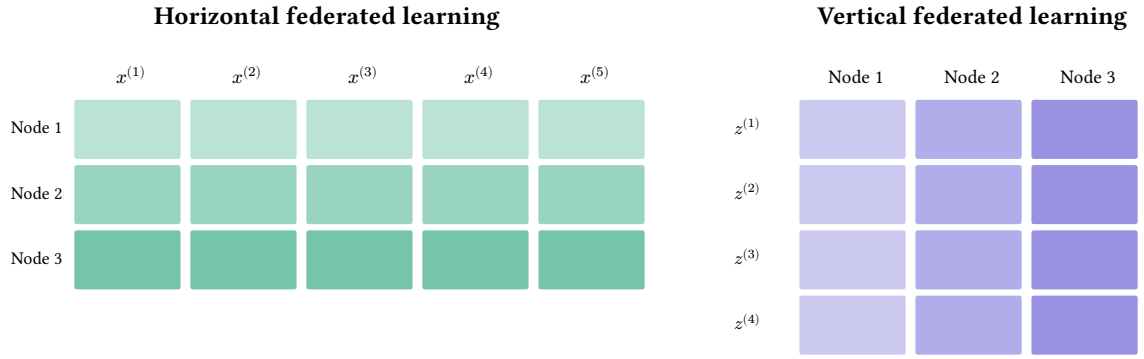

Figure 55: Horizontal federated learning (left): nodes share the same feature space but hold disjoint sets of instances. Vertical federated learning (right): nodes observe the same instances but hold disjoint feature subsets.

In this thesis, we focus exclusively on **horizontal decentralized learning**, which is by far the most prevalent setting in the literature and aligns naturally with peer-to-peer systems where nodes independently collect data instances under a shared feature schema.

5.1.2 Assumptions

The decentralized learning system inherits the assumptions established in the peer-to-peer model of Chapter 2. In particular, we assume that all nodes are identical in terms of computational capabilities and memory, that communication channels are reliable and instantaneous, and that message transmission incurs no latency or bandwidth constraints.

Specifically:

- **Model size**: the size of the local model f_{θ_i} — that is, the number of parameters $p = |\theta_i|$ — is assumed to be identical across all nodes and imposes no memory or transmission constraint. Model exchanges between nodes are therefore treated as instantaneous, regardless of the number of parameters.
- **Training time**: the time required to perform a local model update, such as computing a gradient step or aggregating received parameters, is assumed to be negligible. Local learning computations are therefore considered instantaneous, consistently with the abstract execution model of Definition 2.1.3.

These assumptions allow us to isolate the algorithmic and theoretical properties of decentralized learning protocols from hardware and network effects, and to focus on the convergence and communication behavior of the system.

5.1.3 Adversarial models

In addition to the network-level failure models introduced in Chapter 2 — crash failures and Byzantine failures at the communication layer — decentralized learning systems are exposed to a second class of perturbations that operate at the learning level. Even when the underlying network functions correctly, the aggregation process can be compromised by adversarial behaviors of participating nodes with respect to their model updates.

Several types of adversarial actions are commonly considered in the literature [59]:

- **Privacy attacks**: a node attempts to infer or reconstruct the private data of other nodes by analyzing received model updates [60].
- **Poisoning attacks**: a node intentionally manipulates its local model updates to degrade the performance of the global model [61].

- **Backdoor attacks**: a malicious node injects hidden triggers or patterns into the model during training, aiming to influence the model's behavior on specific inputs while preserving normal performance on clean data [62].
- **Free-riding**: a node benefits from the aggregated models of others without contributing meaningful updates, for instance by sending stale or null parameters [59].

These behaviors are orthogonal to the crash and Byzantine failure models of Chapter 2: a node may be honest at the network level — forwarding messages correctly and remaining available — while behaving adversarially at the learning level. Conversely, a Byzantine node in the network sense may also corrupt model updates.

In this thesis, we do not study these adversarial behaviors. We assume that all nodes are honest and follow the prescribed learning protocol correctly. This allows us to focus on the convergence and dynamic properties of decentralized learning protocols under the assumption of fully cooperative participants, and leave the study of robustness to adversarial settings as a direction for future work.

5.1.4 Abstract learning node

Building on the abstract node model introduced in Definition 2.1.1 and Definition 2.1.3, we now define the notion of an **abstract learning node** by enriching the general peer-to-peer node with machine learning components.

Definition 5.1.1 (Learning Node)

A **learning node** is an abstract node (see Definition 2.1.1) whose local state s_i is extended with two additional components:

(i) a **local dataset** $\mathcal{D}_i = \{(x_j, y_j)\}_{j=1}^{n_i}$, where $x_j \in \mathbb{R}^d$ are feature vectors and y_j are labels. The dataset is private: it is stored exclusively on node i and is never transmitted to other nodes.

(ii) a **local model** $f_{\theta_i} : \mathbb{R}^d \to \mathcal{Y}$, parameterized by $\theta_i \in \mathbb{R}^p$, which represents the current state of the model maintained by node i.

The local state of node i is thus $s_i = (\mathcal{D}_i, \theta_i, P(i))$, where $P(i)$ denotes its partial view of the network (see Definition 2.2.21).

The dataset $\mathcal{D}_i$ is a static component of the local state: it does not change across protocol cycles. The model parameters θ_i, by contrast, constitute the dynamic component of the state and are updated at each protocol step.

♣

Remark

From a multi-agent systems perspective [63], each decentralized learning node can be viewed as an autonomous agent: its local state s_i corresponds to the agent's internal memory, its neighbourhood $P(i)$ and the evolving overlay topology constitute its local environment, and the empirical loss defines the objective to be minimised. Although this multi-agent framing establishes a natural conceptual bridge to cooperative reinforcement learning and decentralized control, formalising our nodes as agents within a multi-agent learning framework lies explicitly outside the scope of this work.

5.1.5 Decentralized learning objective

Having defined the learning node, we can now state the global learning objective. Each learning node i defines a local empirical loss $L_i(\theta)$ (see Definition 1.3.3).

The global objective of the decentralized learning system is to collectively minimise the aggregate loss over all nodes, in the sense of Definition 2.3.5:

$$\min_{\theta \in \mathbb{R}^p} L_{\text{global}}(\theta), \quad L_{\text{global}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} L_i(\theta),$$

Remark

No single node has access to the full loss $L_{\text{global}}(\theta)$, since $\mathcal{D}_i$ is local to node i. The minimisation must therefore be achieved collaboratively, through the exchange of model parameters θ_i or gradients $\nabla L_i(\theta_i)$ between neighbouring nodes, without any node ever observing the data of another.

Building on the notion of convergence introduced in Definition 2.3.6, we say that a decentralized learning system has converged if the global loss falls below a prescribed threshold $\varepsilon > 0$.

Definition 5.1.2 (Convergence of Decentralized Learning)

A decentralized learning system is said to have **converged** if there exists a time $T \geq 0$ such that for all $t \geq T$:

$$L_{\text{global}}(S(t)) = \sum_{i=1}^{N} \alpha_i L_i(\theta_i(t)) \leq \varepsilon,$$

where $S(t) = (\theta_i(t))_{i \in V}$ is the global state of the system at time t (see Definition 2.3.1), and $\varepsilon > 0$ is a convergence threshold fixed a priori.

♣

Remark

In practice, exact convergence in the sense of Definition 5.1.2 is rarely studied directly. Instead, one typically fixes a time horizon T and evaluates the global loss $L_{\text{global}}(S(T))$ achieved after T protocol cycles. This allows one to compare decentralized learning protocols in terms of their convergence speed: a protocol that reaches a lower loss within the same number of cycles is considered more efficient.

5.1.6 Performance metrics

While convergence in the sense of Definition 5.1.2 is defined in terms of the global loss L_{global}, it is useful in practice to complement this criterion with a more interpretable metric. Building on Definition 1.3.4, we define the global accuracy of the decentralized learning system as the average accuracy across all nodes, evaluated on their respective local test datasets.

Definition 5.1.3 (Global accuracy)

Let $\mathcal{D}_i^{\text{test}}$ denote the local test dataset of node i, and let $\hat{y}_j = f_{\theta_i}(x_j)$ be the predicted label for input x_j. The **local accuracy** of node i at time t is:

$$\text{Acc}_i(t) = \frac{1}{|\mathcal{D}_i^{\text{test}}|} \sum_{(x_j, y_j) \in \mathcal{D}_i^{\text{test}}} \mathbb{1}\{\hat{y}_j = y_j\}.$$

The **global accuracy** of the system at time t is the average local accuracy across all nodes:

$$\text{Acc}(t) = \frac{1}{N} \sum_{i=1}^{N} \text{Acc}_i(t).$$

♣

We evaluate the performance of decentralized learning protocols through two complementary criteria.

- **Final accuracy**: the global accuracy $\text{Acc}(T)$ measured after a fixed number of protocol cycles T. This criterion captures the asymptotic quality of the learned model and allows direct comparison between protocols under a fixed computational budget.
- **Time to accuracy**: the number of protocol cycles required for the global accuracy to reach a predetermined threshold $\tau \in (0, 1)$, formally defined as:

  $$T_\tau = \min\{t \geq 0 \mid \text{Acc}(t) \geq \tau\}.$$

  This criterion measures the convergence speed of the protocol, independently of its final performance level.

> **Remark**
>
> Final accuracy and time to accuracy are complementary: a protocol may converge quickly to a moderate accuracy (low T_τ, moderate $\text{Acc}(T)$), while another may converge more slowly but ultimately reach a higher accuracy. Both criteria are therefore necessary to fully characterize the behavior of a decentralized learning protocol.

5.2 Aggregation Strategies

Having defined the formal framework of decentralized learning, we now turn to the two central design questions that govern any decentralized learning protocol: how local models should be combined, and which nodes should communicate with whom. We address these questions in turn, first through the choice of aggregation operator, then through the choice of collaboration topology.

5.2.1 Aggregation operator

A first question in decentralized learning concerns the aggregation operator itself: given that nodes have exchanged their local model parameters, how should these be combined into an improved model ? As established in Definition 5.1.1, each node i maintains a local model f_{θ_i} trained exclusively on its private dataset $\mathcal{D}_i$, and since the amount of data available at a single node is generally insufficient to achieve low loss, collaboration between nodes is required to leverage the information distributed across the network.

In horizontal decentralized learning (see Definition 5.1.1), all nodes share the same feature space and parameter space $\mathbb{R}^p$, which makes parameter-wise aggregation well-defined: the parameters of multiple local models can be directly combined.

Several aggregation operators have been proposed in the literature, ranging from weighted averaging to more sophisticated strategies based on gradient correction or momentum. In this work, we focus on the simplest and most widely studied: **model averaging**, in which nodes exchange their local parameters and compute their arithmetic mean [64]. Despite its simplicity, this operator forms the basis of most decentralized and federated learning algorithms and admits strong theoretical guarantees under standard assumptions.

Definition 5.2.1 (Average SGD)

At each iteration t, every node i performs a local stochastic gradient descent step on its local objective L_i (see Definition 1.3.3):

$$\theta_{t+\frac{1}{2}} = \theta_t - \eta \nabla_\theta L_i(\theta_t),$$

After a synchronization step, the local models are aggregated by averaging:

$$\theta_{t+1} = \frac{1}{N} \sum_{i=1}^{N} \theta_{t+\frac{1}{2}}.$$

♣

Remark

Under standard regularity assumptions on L_i and IID data distribution across nodes, Average SGD converges to the same optimum as centralized SGD [65].

Two design choices govern the practical behavior of Average SGD. The first is **aggregation frequency**: nodes may aggregate at every iteration or after several local gradient steps, trading off communication cost against convergence speed. The second is the **synchronization scope**: rather than averaging model parameters, an alternative is to aggregate gradients directly — nodes exchange $\nabla L_i(\theta_i(t))$, average them, and apply the result to a shared model. This is equivalent to parameter averaging when models are synchronized at every step, but the two strategies diverge when local updates accumulate over several steps before aggregation. In this work, we restrict our study to parameter-based aggregation for simplicity.

The statistical properties of local datasets also play a crucial role in the convergence behavior of Average SGD. Under IID distributions, model averaging performs comparably to centralized training. In the non-IID setting, however, local data distributions may differ significantly across nodes, causing local models to drift in different directions — a phenomenon known as **client drift** [66]. In such cases, arithmetic averaging may no longer be appropriate, and more robust aggregation operators have been proposed, such as geometric median-based aggregation [67], which is less sensitive to outlier models induced by heterogeneous data distributions. Since this thesis focuses on the interaction between aggregation and network dynamics, we operate under the IID assumption throughout, and leave the non-IID setting as a direction for future work.

5.2.2 Topology-driven Aggregation

A central design question in decentralized learning is determining which nodes should exchange and aggregate their models, and according to what structure. While the aggregation operator — here Average SGD (see Definition 5.2.1) — defines **how** models are combined, the collaboration topology defines **who** communicates with whom.

The choice of topology, ranging from centralized star-shaped architectures to fully decentralized peer-to-peer overlays, directly impacts convergence speed, robustness to failures, scalability, and resilience

to churn. Understanding these trade-offs is therefore essential for the design and analysis of decentralized learning protocols.

Here, we look at the main aggregation strategies encountered in the literature, organized according to their underlying network topologies and coordination mechanisms. We progressively move from centralized and hierarchical approaches, such as Federated Learning and its multi-server extensions, to fully decentralized schemes based on gossip and local interactions.

We focus on aggregation strategies defined by network topology and communication patterns. Orthogonal aspects such as robust aggregation rules, privacy mechanisms, or incentive schemes are not discussed.

Federated learning

Federated learning (FL) [1] is a collaborative learning paradigm in which multiple clients train a shared model without exchanging their raw data. Each client performs local training on its private dataset and communicates only model parameters to a central coordinating entity, referred to as the server. The paradigm was introduced to address privacy, bandwidth, and data ownership constraints in large-scale systems such as mobile devices and edge computing environments.

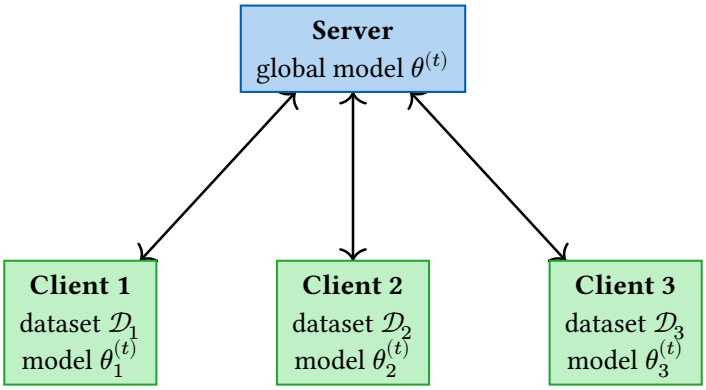

Figure 56: Federated learning with a star topology: the server broadcasts $\theta^{(t)}$, clients train locally on $\mathcal{D}_i$, and return updated parameters $\theta_i^{(t+1)}$ for aggregation.

The FedAvg algorithm [1] generalizes Average SGD (see Definition 5.2.1) by allowing each client to perform $E \geq 1$ local gradient steps between communication rounds, rather than a single step. This reduces communication frequency at the cost of introducing a potential divergence between local and global objectives when E is large.

From a network perspective, federated learning relies on a star-shaped topology: a single server communicates with all clients, collects their local parameters, aggregates them — typically as a weighted average $\theta^{(t+1)} = \sum_{i=1}^{N} w_i \theta_i^{(t)}$ with $w_i = n_i / \sum_j n_j$ — and broadcasts the updated global model back to all participants.

While this architecture enables efficient coordination and simplifies convergence analysis, it introduces a strong centralization point. Although federated learning avoids centralizing data, it does not eliminate central control: the server must be reliable and trusted, and its failure or compromise may disrupt the entire learning process.

Input: number of clients N, local datasets $\{\mathcal{D}_1, \ldots, \mathcal{D}_N\}$, learning rate η, communication rounds T, local steps per round E, weights $w_i = n_i / \sum_j n_j$, initial model $\theta^{(0)}$

for $t = 0$ **to** $T - 1$ **do**
  server broadcasts $\theta^{(t)}$ to all clients
  for each client i **in parallel do**
    $\theta_i^{(t,0)} \leftarrow \theta^{(t)}$
    for $e = 1$ **to** E **do**
      sample minibatch $\xi_i \subset \mathcal{D}_i$
      $\theta_i^{(t,e)} \leftarrow \theta_i^{(t,e-1)} - \eta \nabla_\theta L_i\left(\theta_i^{(t,e-1)}; \xi_i\right)$
    end for
    send $\theta_i^{(t,E)}$ to server
  end for
  server aggregates:
    $\theta^{(t+1)} \leftarrow \sum_{i=1}^{N} w_i \theta_i^{(t,E)}$
end for
return $\theta^{(T)}$

Figure 57: Federated averaging (FedAvg) [1].

Multi-star federated learning

Multi-star federated learning extends the classical federated learning paradigm by replacing the single central server with multiple coordinating servers. Each server acts as a local aggregation point for a subset of clients, forming multiple star-shaped subnetworks that operate in parallel. This architecture is motivated by scalability, fault tolerance, and geographical distribution — as illustrated by Gaia [68], a system designed for geographically distributed machine learning in which workers send their updates to an assigned regional server, and servers synchronize across geographical zones. It is commonly encountered in large-scale industrial deployments where a single server would become a performance bottleneck.

In a multi-star setting, clients are assigned to one or more servers and perform local training in the same way as in standard federated learning. Each server collects the updated parameters from its associated clients and performs a local aggregation, for instance using FedAvg (see Figure 57). Compared to the single-star topology, this reduces communication load and latency, and allows the system to scale to a much larger number of clients.

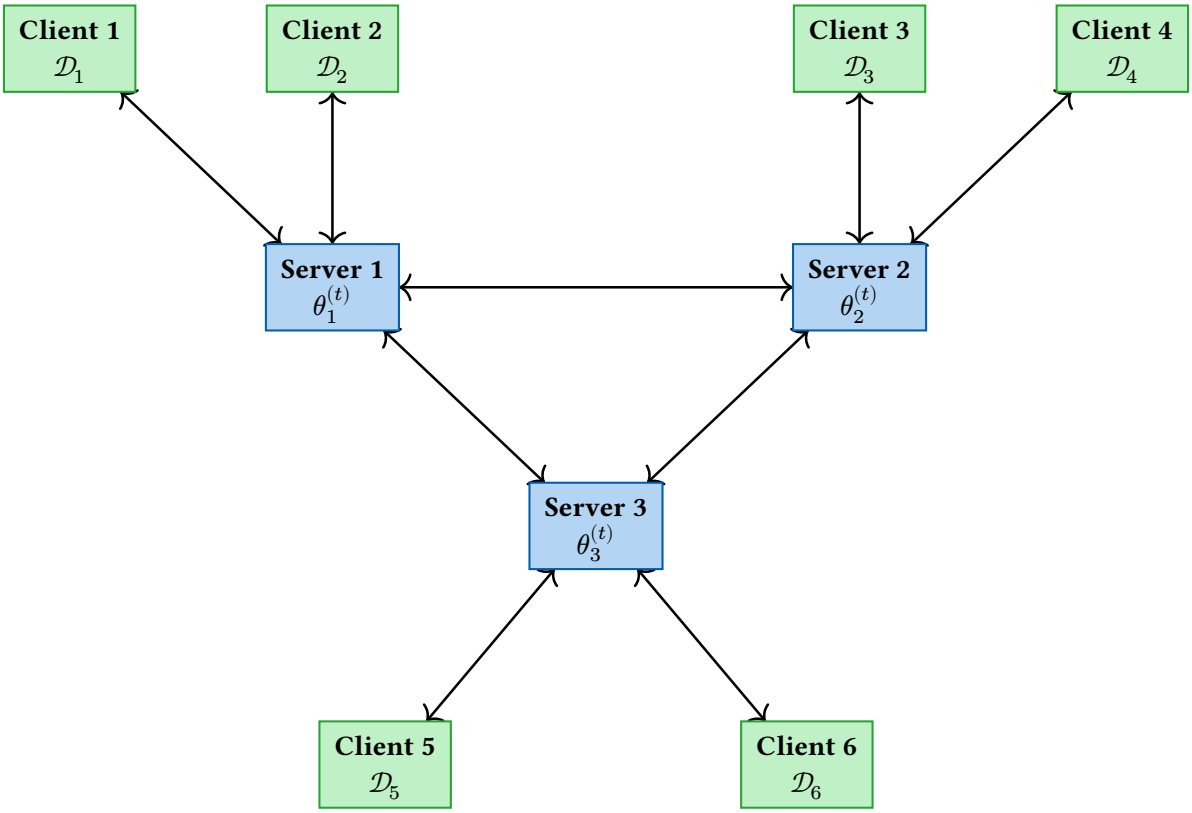

Figure 58: Multi-star federated learning: three servers form a fully connected clique for cross-server synchronization, each coordinating a local star of two clients. Clients communicate only with their assigned server; servers exchange aggregated models with one another.

However, the presence of multiple servers raises fundamental design questions regarding global consistency and convergence. In particular, servers must be synchronized to prevent model drift between different regions of the network. Several synchronization strategies can be considered:

- **Server-level aggregation**: servers periodically exchange their aggregated models and perform a second-level aggregation [68].
- **Client-to-multiple-servers**: clients send their local models to multiple servers, increasing redundancy and robustness at the cost of higher communication overhead.

From a topological perspective, multi-star federated learning corresponds to a two-level hierarchy. While it removes the single point of failure of classical federated learning, each server still represents a critical coordination node for its associated clients. If a server fails or behaves in a Byzantine manner, the learning process of its local star can be compromised, and inconsistencies may propagate to other servers during synchronization.

Hierarchical federated learning

Hierarchical federated learning (HFL) generalizes the multi-star architecture by organizing servers into a tree-shaped topology, forming a hierarchy of aggregation levels [69]. At the lowest level, clients perform local training and send their model updates to intermediate servers, which act as local aggregators. These intermediate servers then forward partially aggregated models upward in the hierarchy, until a final aggregation is performed at a root server.

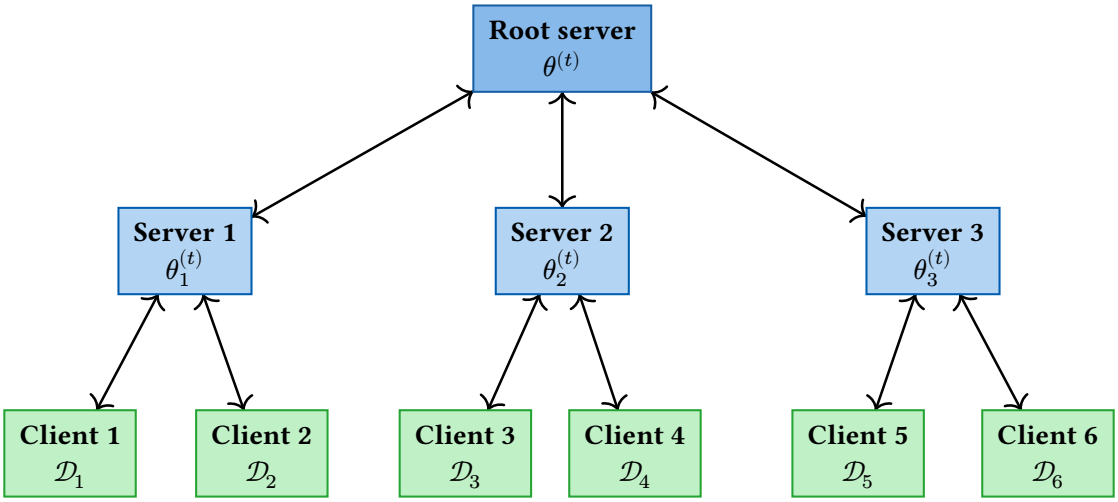

Figure 59: Hierarchical federated learning: a root server coordinates three intermediate servers, each aggregating updates from two local clients. Aggregated models propagate upward level by level until a global model is produced at the root.

From a graph-theoretic perspective, this architecture corresponds to a tree topology, where each internal node performs aggregation over the models received from its children. This structure enables scalable learning over very large populations of clients by distributing the aggregation workload across multiple levels, thereby reducing communication and computational pressure on the root server [69].

Despite these scalability benefits, HFL remains fundamentally centralized and inherits several limitations from tree-based topologies. The structure is typically rigid, with predefined parent–child relationships. Failures of intermediate aggregation nodes can disconnect entire subtrees, temporarily preventing a large number of clients from contributing to the global model. Similarly, failures or Byzantine behavior at higher levels of the hierarchy may corrupt or block the learning process for all downstream nodes [70].

Blockchain-based federated learning

Blockchain-based federated learning combines federated learning with blockchain-based distributed ledger technologies in order to remove the reliance on a single trusted coordinator, motivated by the promise of decentralization, auditability, and trust minimization [71].

Several architectural strategies have been proposed. In a first approach, each participant submits its local model update to a smart contract deployed on the blockchain [72]. Once a sufficient number of updates has been received, the smart contract performs the aggregation and publishes the resulting global model, acting as a decentralized coordinator that enforces participation rules and aggregation logic. An alternative approach relies on the block validation process: instead of performing aggregation on-chain, the block validator designates a node or a small committee as aggregator for a given round [73]. Participants send their local models to the selected aggregator, which computes the aggregated model and disseminates it to the network, while the blockchain records the selection process and ensures accountability.

Despite their conceptual appeal, these approaches inherit significant limitations from blockchain technology. First, blockchain systems require consensus on an exact global state, whereas machine learning optimization only requires convergence toward a sufficiently low loss. Enforcing strict consensus at every learning round introduces substantial overhead without providing proportional benefit to the learning process. Second, storing model parameters directly on-chain is often impractical due to storage constraints and associated costs, particularly for large models.

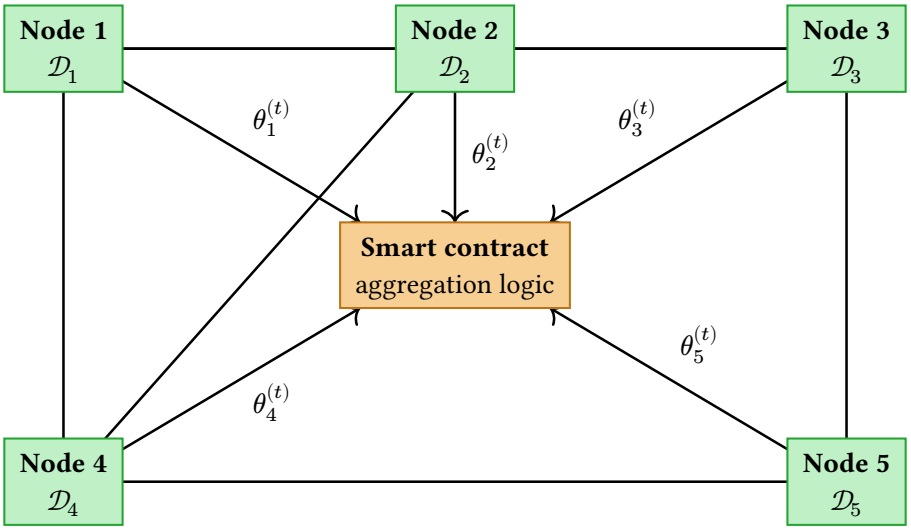

Figure 60: Blockchain-based federated learning: nodes are interconnected in a peer-to-peer overlay and submit their local model parameters $\theta_i^{(t)}$ to a smart contract, which performs aggregation and publishes the updated global model to all participants.

Most practical implementations therefore resort to off-chain storage or aggregation, which reintroduces trust assumptions and partially undermines the decentralization objective [71]. Finally, when aggregation is delegated to a single node or a small committee selected by the block validator, the system remains vulnerable to centralization risks, contradicting the original motivation for using a blockchain.

From a topological perspective, blockchain-based federated learning relies on a globally accessible coordination layer when smart contracts are used, as all participants interact through a shared ledger. When off-chain aggregators are employed, the resulting communication structure resembles a star topology, with the aggregator acting as a temporary central node.

In summary, the high communication latency, storage overhead, and consensus costs of blockchain systems are poorly aligned with the iterative and approximate nature of distributed machine learning, and the practical deployment of such systems remains an open challenge [71].

Gossip learning

Gossip Learning [55] is a fully decentralized learning paradigm in which nodes exchange models through randomized peer-to-peer interactions. At each communication round, a node selects one of its neighbors uniformly at random and sends its current local model to that neighbor. There is no central coordinator and no notion of a global aggregation phase.

Upon receiving a model from a neighbor, a node performs a local aggregation between the received model and its own local model, typically by computing a weighted or uniform average of their parameters. The resulting aggregated model is then refined by performing one or several local learning steps using the node's private dataset. This interaction pattern is repeated asynchronously across the network, leading to a gradual diffusion of information.

In the most common formulation, aggregation is performed before the local learning step. An alternative variant applies a local learning update independently to both models before merging them, which can improve robustness in non-IID data settings. Another extreme variant removes aggregation altogether: the local model is simply replaced by the received model. In this case, models effectively perform random walks over the network, and learning corresponds to successive local updates applied along these trajectories.

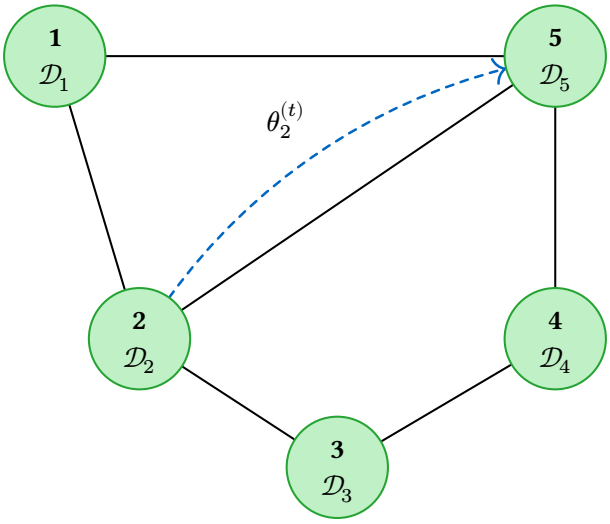

Figure 61: Gossip learning: nodes are connected in a random peer-to-peer overlay. At each round, a node selects a neighbor uniformly at random and sends its current local model.

Gossip learning is typically deployed over random graph topologies, where the randomized communication pattern ensures sufficient mixing properties. Aggregation remains strictly local, and no global model is ever explicitly computed. Nevertheless, under suitable assumptions on the learning rate, loss function, and network connectivity, the local models are known to converge toward a common global solution. This convergence, however, is significantly slower than in Federated Learning due to the absence of coordinated global synchronization and the limited bandwidth of local interactions.

Despite its slower theoretical convergence, gossip learning offers strong advantages in terms of system robustness. The absence of any central entity makes the scheme inherently resilient to node failures, network partitions, and churn. Nodes can join or leave the system dynamically without disrupting the learning process, provided the underlying communication graph remains connected on average. These properties make gossip learning particularly attractive for large-scale, dynamic, and failure-prone environments where centralized or hierarchical approaches are impractical. Importantly, empirical results suggest that the gap with federated learning may be smaller than theoretical bounds indicate: [74] show that gossip learning can match the convergence quality of federated learning in practice, while operating without any central coordinator.

Gossip Learning (main thread)

```
   ML model: model
   List of neighbors : cache
1  loop
2  |  Wait for Δ time units
3  |  peer ← selectRandom(cache)
4  |  send(peer, model)
```

Figure 62: Gossip Learning

Gossip Learning (background thread)

```
   ML model: model
   local dataset: data
1  loop
2  |  peer_model ← receive()
3  |  model ← merge(model, peer_model)
4  |  model.update(data)
```

Figure 63: Gossip Learning (background thread)

Beyond the foundational work of [55], gossip learning has attracted a substantial body of follow-up research aimed at improving its convergence speed, robustness, and applicability to diverse settings.

Several works address the efficiency of the gossip communication pattern itself. [75] propose a token-account approach that improves convergence speed by better controlling the flow of models across the network, and [76] further refine the model merging strategy to amplify the benefits of token-based

flow control, reporting significant improvements over prior solutions in simulations based on real-world smartphone availability traces. [77] extend the base algorithm to account for node heterogeneity: since faster nodes send more models than slower ones, they propose storing one model per neighbor and selecting from this cache before merging, though this extension assumes a fixed neighbor set. [78] improve decentralized SGD by introducing a matching decomposition sampling strategy that constructs better communication topologies, while [79] provide a unified theoretical framework for analyzing the convergence of gossip-based SGD under changing topologies and local updates.

The algorithm has also been extended to non-standard learning tasks. [80] adapt gossip learning to k-means clustering, demonstrating the generality of the paradigm beyond supervised learning. [81] propose a segmented gossip approach in which nodes exchange only a subset of model parameters rather than the full model, reducing communication overhead, though their setting is closer to a distributed cluster than a fully decentralized peer-to-peer network.

A distinct line of work focuses on non-IID data settings and personalization. [82] introduce PENS, a performance-based neighbor selection algorithm in which nodes evaluate received models on their local test set and preferentially gossip with peers whose models perform best locally, effectively steering communication toward nodes with similar data distributions. In a related direction, [83] argue that approximating a global distribution is not always necessary, and propose PEPPER, a gossip-based personalized recommender system in which each node trains a model tailored to its own user rather than optimizing a global objective.

Epidemic learning

Epidemic learning is a decentralized learning scheme in which each node exchanges its local model with all of its neighbors at every communication round [84]. Contrary to classical gossip learning, where interactions are pairwise and asynchronous, this approach requires each node to wait for the models of all its neighbors before performing aggregation. As a result, the learning process is inherently synchronous.

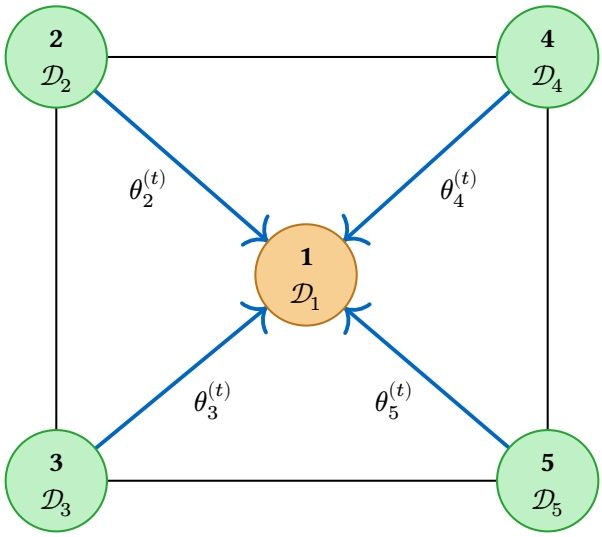

Figure 64: Epidemic learning: node 1 (orange) collects the models $\theta_2^{(t)}, \ldots, \theta_5^{(t)}$ from all its neighbors simultaneously. Once all models are received, node 1 aggregates them and performs a local update on $\mathcal{D}_1$.

At each round, a node broadcasts its current model to all adjacent nodes and collects the models received from its neighborhood. Once all expected models have been received, the node computes an aggregation, typically by averaging the parameters of its own model with those of its neighbors.

The aggregated model is then updated using a local learning step on the node's private dataset. This process is repeated synchronously across the network.

Epidemic learning can be deployed over various network topologies. On random graphs, it preserves some of the decentralization benefits of gossip learning while accelerating convergence thanks to richer local aggregation. On a fully connected topology, where every node is connected to all others, the scheme becomes equivalent to a global aggregation performed in a fully decentralized manner.

However, the increased degree of connectivity comes at a significant cost. The communication and aggregation overhead grows linearly with the number of neighbors, making the approach poorly scalable for high-degree nodes. In fully connected networks, the communication cost per round becomes prohibitive as the number of nodes increases. Furthermore, the synchronous nature of the protocol makes it sensitive to stragglers and node failures, as a single slow or unavailable neighbor can delay the entire aggregation step.

While epidemic learning offers faster convergence than pairwise gossip schemes, it sacrifices robustness and scalability due to its synchronous nature and high communication overhead at high-degree nodes.

Epidemic learning has served as a foundation for a growing body of work addressing practical limitations of the base protocol. These extensions target a range of challenges including privacy, anonymity, energy efficiency, data heterogeneity, scalability, and stragglers.

On the privacy and anonymity front, Zip-DL [60] introduces resistance to privacy attacks by adding carefully calibrated noise to model updates before transmission, while Shatter [85] takes a complementary approach by introducing the concept of virtual nodes: instead of transmitting models under their true identity, nodes split their model across virtual identities, thereby concealing which physical node carries which model and preventing adversaries from linking model updates to specific participants.

From an energy efficiency perspective, SkipTrain [86] proposes alternating between training phases and transmission phases, allowing nodes to skip local training during transmission rounds. This decoupling reduces the energy consumption of the protocol, making epidemic learning more suitable for resource-constrained environments such as mobile or edge devices.

The non-IID setting is addressed by Facade [87], which adapts epidemic learning to heterogeneous data distributions by clustering nodes according to the similarity of their local datasets. By preferentially exchanging models within clusters of nodes sharing similar data distributions, Facade mitigates the client drift problem that arises when models trained on heterogeneous data are naively averaged. DivShare [88] addresses a related challenge by improving resilience to stragglers: when slow nodes delay the aggregation step, DivShare adapts the protocol to tolerate late or missing model transmissions without blocking the learning process.

Finally, two works address the scalability of epidemic learning at larger network sizes. Plexus [89] selects a subset of nodes at each cycle to participate in training and model construction, reducing the per-round communication and computation cost while preserving convergence properties, thereby enabling epidemic learning to scale to larger networks. Mosaic Learning [90] fragments models into pieces and strategically disseminates the fragments that differ most across nodes, exploiting model diversity to accelerate convergence time.

Ring-based decentralized learning

Decentralized learning can also be implemented over a ring topology [91], although this approach is relatively uncommon in the machine learning literature. In a ring-based system, each node maintains connections with exactly two neighbors, typically referred to as its left and right neighbors, forming

a closed cycle. This topology is simple, deterministic, and requires each node to store only a constant number of connections, which makes it attractive from a maintenance and routing perspective.

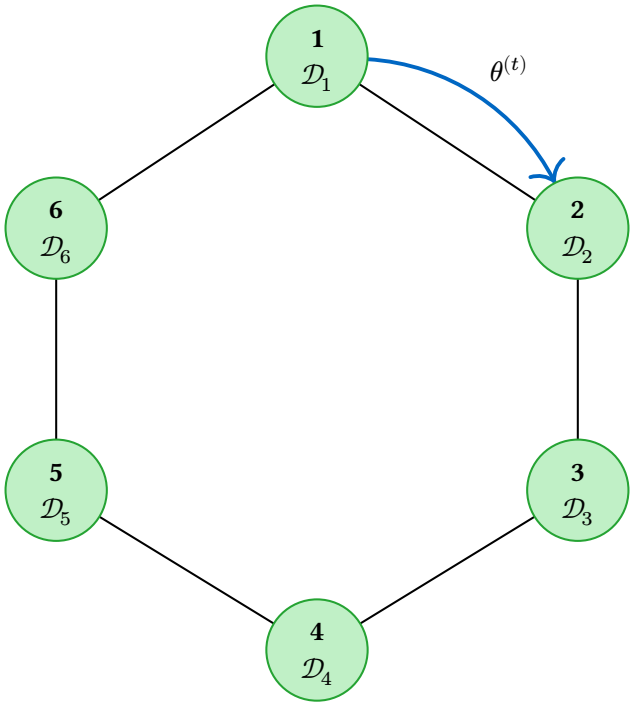

Figure 65: Decentralized learning on a ring topology: nodes are connected to exactly two neighbors, forming a closed cycle. A model $\theta^{(t)}$ circulates sequentially along the ring — here from node 1 to node 2 — and is updated at each node using its local dataset $\mathcal{D}_i$ before being forwarded to the next neighbor.

Several aggregation strategies can be employed in a ring topology. A first approach consists in **local neighborhood aggregation**, where each node periodically exchanges model parameters or updates with its immediate neighbors and aggregates them, for instance by averaging its own model with those received from the left and right neighbors.

A second strategy relies on **model circulation**. In this approach, a single model is passed sequentially from node to node along the ring. Each node locally updates the received model using its own dataset before forwarding it to the next neighbor. After a full traversal of the ring, the model has effectively been trained on the data of all nodes, in a manner reminiscent of incremental or online learning.

An illustrative example of a ring-based decentralized learning protocol is Fedlay [91], which organizes nodes into **virtual rings**. In Fedlay, each node maintains connections to 2ℓ neighbors across ℓ distinct virtual rings, where $\ell \in \mathbb{N}^*$ is a configurable protocol parameter. At each training cycle, every node performs model aggregation by merging its local model with those received from its neighbors in each virtual ring, typically through weighted averaging. This multi-ring structure increases connectivity without sacrificing the simplicity of ring-based routing, allowing for more robust information propagation compared to a single-ring topology. However, the reliance on virtual rings still inherits some of the fundamental limitations of ring structures, particularly regarding latency and sensitivity to node churn.

Despite its conceptual simplicity, decentralized learning on a ring topology suffers from significant limitations. The rigid structure of the ring makes the system particularly vulnerable to failures and churn. The failure of a single node or link may break the ring and disconnect the network unless additional repair mechanisms are employed [91]. Frequent joins and leaves further complicate the maintenance of the ring structure and may disrupt the learning process.

Summary

The aggregation strategies surveyed in this section differ fundamentally in their underlying network topology, which directly shapes their convergence speed, scalability, and fault tolerance properties. Table 9 summarises the main strategies discussed, together with their associated topologies.

Table 9: Main aggregation strategies and their associated network topologies.

Aggregation strategy	Associated topology
Federated learning	Star (single central server)
Multi-star federated learning	Multiple stars
Hierarchical federated learning	Tree / hierarchical topology
Blockchain-based federated learning	Complete graph
Gossip learning	Random graph or complete graph
Epidemic learning	Random graph or complete graph
Ring-based decentralized learning	Ring

5.3 Conclusion

This chapter has examined decentralized learning as a response to a fundamental limitation shared by centralized and distributed learning alike: the requirement that data be aggregated or made accessible at a central location, which raises insurmountable challenges in terms of privacy, scalability, and data ownership. Decentralized learning addresses this limitation at its root by keeping data strictly local to each node, enabling collaborative model training without any node ever observing the data of another. We formalized this setting by grounding it in the node model of Chapter 2: a decentralized learning node is a node enriched with a local dataset and a local model, and the global learning objective is to collectively minimize the aggregate loss L_{global}. We characterized convergence in terms of this global objective, and introduced complementary performance metrics — final accuracy and time to accuracy — that will serve as evaluation criteria throughout the remainder of this thesis.

We then surveyed the main topology-driven aggregation strategies proposed in the literature, ranging from the star-shaped architecture of federated learning and its hierarchical extensions, to fully decentralized schemes such as gossip learning and epidemic learning. This survey revealed a fundamental tension that runs through the field: centralized and hierarchical approaches such as federated learning benefit from efficient coordination and fast convergence, but rely on a central point of control that introduces fragility, scalability limitations, and trust requirements. Fully decentralized approaches such as gossip learning eliminate this central dependency and offer strong resilience properties, but typically converge more slowly due to the limited bandwidth of local pairwise interactions.

Beyond this architectural tension, several broader challenges remain for the next generation of decentralized learning systems:

- **Security against privacy and Byzantine attacks.** Decentralized architectures are inherently exposed to data inference, membership inference, and model poisoning attacks. Designing protocols that guarantee robust aggregation or differential privacy without relying on central trust remains largely open.

- **Theoretical performance guarantees.** While empirical results abound, formal convergence bounds and complexity analyses for fully decentralized, asynchronous, and highly heterogeneous settings are still fragmented. Bridging empirical practice with rigorous theoretical foundations is a critical unsolved problem.
- **Adaptation to large language models (LLMs).** Training or fine-tuning billion-parameter models in a decentralized setting clashes with severe bandwidth, memory, and compute constraints at the edge. Developing communication-efficient, parameter-optimized strategies tailored to LLMs without sacrificing convergence is an emerging frontier.
- **Resilient and efficient decentralized learning systems.** Building fully peer-to-peer learning frameworks that simultaneously achieve high fault tolerance, rapid convergence, and low communication overhead — without relying on fragile coordinators or hierarchical structures — remains a core systems-level challenge.

Among these open research directions, this thesis deliberately narrows its scope to the fourth challenge: **the design of a resilient and efficient decentralized learning system**. The following chapter (Chapter 6) presents our contribution to this problem.

Chapter 6

Efficient and Resilient Decentralized Learning Protocols

The previous chapter established a fundamental tension in decentralized learning: centralized approaches such as federated learning achieve fast convergence through coordinated aggregation, but at the cost of a single point of failure, scalability limitations, and trust requirements on the central server. Fully decentralized approaches such as gossip learning and epidemic learning eliminate this dependency and offer strong resilience properties, but typically converge more slowly due to the limited bandwidth of pairwise or local interactions.

In this chapter, we introduce HEAL (for **H**ub **E**nhanced **A**daptive **L**earning), a novel decentralized learning framework designed to bridge this gap. HEAL is the first cross-layer decentralized learning framework that exploits an optimized, self-organizing peer-to-peer overlay — specifically the Elevator protocol introduced in Chapter 4 — to dynamically promote a controlled number of nodes to act as aggregators. Unlike traditional federated learning, aggregator nodes are not pre-selected and hence HEAL becomes extremely resilient to the network dynamicity (nodes crashes and churn). That is, HEAL self-heals and self-adapts by promoting new hubs while continuing the learning process without significant losses. We challenged HEAL, Federated Learning, Gossip Learning and Epidemic Learning with various topologies in static and dynamic environments (crash and churn prone) on two tasks: a binary classification task using a logistic regression [92] on the Spambase dataset [93] and on a multinomial classification task using the LeNet5 model [94] on the MNIST dataset [95]. HEAL outperforms Gossip and Epidemic Learning in both accuracy and convergence time. Moreover, HEAL is resilient to churn and crashes while Federated Learning cannot cope with these faults.

Table 10: Comparison of different decentralized learning algorithms.

Decentralized Learning Protocol	**Topology**	**Fault resilience**	**Churn resilience**	**Aggregation speed**
Federated Learning	Static (Star [1] and Multi-Star [68])	No	No	quick & centralized
Gossip Learning	Static (Random Regular [74])	No	No	slow & local
	Dynamic (Newscast [55])	Yes	Yes	slow & local
Epidemic Learning	Dynamic (Newscast [96])	Yes	Yes	slow & local
FedLay [91]	Dynamic (based on virtual rings)	Yes (only one)	No	slow & local
HEAL	**Dynamic (Elevator)**	**Yes**	**Yes**	**quick, using hubs**

6.1 HEAL Protocol

6.1.1 Desired Properties

HEAL is designed to satisfy a set of fundamental properties that characterize a practical, efficient, and resilient decentralized learning framework.

Model convergence is the primary objective. A decentralized learning framework that fails to drive participating nodes toward a common, accurate model is of no practical utility. HEAL must therefore ensure that the collaborative training process converges to a model of quality comparable to what centralized training would yield. In strict adherence to the federated learning paradigm, raw data must remain local at all times, authorizing exclusively the exchange of model parameters. This constraint ensures privacy and security by design, preventing sensitive information from leaking during the training process.

Fast dissemination is a central motivation for HEAL. The framework aims to propagate model updates across the network within a small number of communication rounds. Rapid dissemination directly translates into faster convergence, reduced training time, and improved responsiveness to distributional shifts in the data.

Model agnosticism ensures that HEAL imposes no structural assumptions on the learning task or the model architecture. The framework should support any supervised learning model, from linear classifiers to deep neural networks, provided that the model parameters can be exchanged and aggregated. This generality is essential for broad applicability across heterogeneous deployment scenarios.

Resilience to failures and churn captures the framework's ability to sustain learning progress in the presence of node departures, crashes, and unpredictable network dynamics. Ensuring continuity of the learning process under such adverse conditions requires a fully decentralized design, where no single node acts as an indispensable coordinator. Any architecture that relies on a central point of control introduces a single point of failure, whose loss would halt the entire training process. HEAL must therefore distribute both coordination and aggregation responsibilities across the network, so that the failure of any individual node, regardless of its role in the network, does not compromise the overall learning dynamics.

Resource efficiency and algorithmic simplicity reflect the practical constraints under which decentralized learning systems operate. HEAL should minimize communication overhead, avoid redundant model transmissions, and keep memory and computation requirements within reasonable bounds to remain viable on resource-constrained devices. Beyond computational constraints, the design must adhere to a minimalist philosophy. A simpler algorithmic approach reduces implementation complexity and facilitates auditing, ensuring that the system is not only performant but also transparent and verifiable.

Addressing these properties in a cohesive manner requires a structured design. HEAL therefore follows a **layered architecture**, drawing inspiration from the OSI model [97]. This approach isolates concerns into distinct layers — from network overlay management to model aggregation — enabling modularity, easier verification, and incremental refinement. The next subsection presents the detailed architecture of HEAL.

6.1.2 HEAL Architecture

HEAL is structured as a four-layer protocol stack, designed to enforce modularity, separation of concerns, and extensibility. From bottom to top, these layers are: the **Network Layer**, responsible for low-level peer-to-peer communication and message routing; the **Overlay Layer**, which maintains the logical topology and manages neighbor discovery; the **Aggregation Layer**, implementing the core decentralized learning logic such as model merging and synchronization; and the **Application Layer**,

which interfaces with the local learning task, handles data loading, and orchestrates the training loop. Each layer operates independently, interacting with adjacent layers only through well-defined interfaces. This strict decoupling enables modular development, simplifies debugging and auditing, and allows individual components to be replaced or optimized without affecting the rest of the system. In the following paragraphs, we detail the responsibilities and internal mechanisms of each layer, and describe how they interact to achieve efficient and resilient decentralized learning.

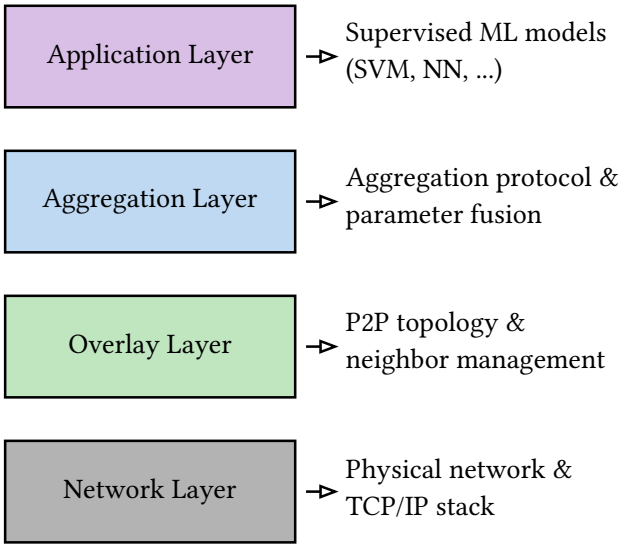

Figure 66: Layered architecture of HEAL

Network Layer

The Network Layer forms the foundation of the HEAL protocol stack. It is responsible for low-level peer-to-peer communication, providing the basic message passing primitives upon which all higher layers depend. This layer directly implements the network assumptions formalized in Chapter 2. In particular, we assume the underlying network is connected, reliable, and provides bidirectional communication channels between nodes.

For all practical purposes, the Network Layer is equivalent to a standard TCP/IP stack. Each node is identified by a unique network address (e.g., an IP address and port), and messages are transmitted over reliable, bidirectional channels that guarantee in-order delivery without loss or corruption. This abstraction aligns with our modeling assumptions, where message transmission is treated as instantaneous and reliable, allowing us to focus on overlay-level protocols without accounting for network-level failures.

The Network Layer exposes two primary operations to the layer above:
- `send(peer_id, message)`: transmits a message to a specified peer identified by its network address,
- `receive()`: listens for incoming messages and delivers them to the upper layer for processing.

By delegating all physical networking concerns to this layer, HEAL ensures that higher-level components — overlay maintenance, aggregation, and learning orchestration — remain agnostic to the underlying transport mechanisms. This separation enables the protocol to operate over any standard network infrastructure without modification, while also allowing future extensions (e.g., support for unreliable transports or encrypted channels) to be implemented at this layer without affecting the rest of the system.

Overlay Layer

The Overlay Layer implements the logical topology that enables efficient decentralized learning in HEAL. This layer is powered by the Elevator protocol, whose design and theoretical guarantees are detailed in Chapter 4. Elevator builds upon the reliable communication primitives provided by the Network Layer to dynamically elect a subset of nodes as **hubs**, forming a structured overlay with desirable properties for model dissemination and aggregation.

Elevator operates in a fully decentralized manner and converges within $O(\log N)$ communication cycles, where N is the network size. The protocol is resilient to node churn and failures: the departure or crash of any individual node — including a hub — does not compromise the overlay's connectivity, as replacement mechanisms automatically restore the desired topology.

The overlay exposes two configurable global parameters:
- $c \in \mathbb{N}^*$: the total size of each node's partial view, i.e. the number of outgoing connections it maintains,
- $h \in \mathbb{N}^*$ with $c > h$: the number of preferential connections maintained by each node, targeting the most connected peers in the neighborhood.

Each node maintains a partial view of c outgoing connections to other nodes, structured as follows:
- h preferential connections, targeting the most connected peers discovered in the local neighborhood,
- $c - h$ connections to randomly selected peers, refreshed periodically to ensure good mixing properties.

As an emergent property of this local attachment rule, exactly h nodes spontaneously rise to hub status at the system level, forming a dense interconnected core to which all other nodes maintain a direct outgoing connection.

This hybrid structure combines the low-diameter benefits of a hub-based topology with the robustness of random gossip-style connections. Hubs serve as high-visibility coordination points, while random edges provide redundancy and mitigate the risk of partitioning.

The Overlay Layer exposes the following API to the Aggregation Layer above:
- `getHub()`: returns a uniformly random hub from the current hub set, useful for sampling an aggregator without global knowledge,
- `getHubs()`: returns the complete, consistent list of all h hubs, identical across all nodes in the system,
- `getRandomPeers()`: returns the set of $c - h$ random neighbors for gossip-based exchanges,
- `isHub()`: returns `true` if the local node currently holds hub status, `false` otherwise.

The primary objective of this overlay design is to enable efficient model aggregation in the layer above. Hubs act as natural aggregation points: nodes can push model updates to hubs, which then merge and redistribute aggregated models.

By decoupling overlay management from learning logic, HEAL ensures that the aggregation strategy can be modified or extended without affecting the underlying topology maintenance, and vice versa. This separation of concerns is central to the modularity and extensibility of the overall architecture.

Aggregation Layer

The Aggregation Layer defines the strategy by which locally trained models are combined into a global model and redistributed across the network. As surveyed in Chapter 5, the literature offers a variety of aggregation strategies for decentralized learning, ranging from gossip-based averaging to more structured federated approaches. These strategies differ in their communication patterns, convergence guarantees, and tolerance to heterogeneous data distributions.

HEAL adopts an aggregation design inspired by Federated Learning [1], but departs from the classical single-server assumption by distributing the aggregation responsibility across multiple coordinators. Specifically, HEAL leverages the hub set maintained by the Overlay Layer to instantiate h concurrent

aggregators, where h is a global parameter of the underlying Elevator protocol. This design choice directly addresses the single point of failure inherent to centralized federated approaches, and distributes the aggregation load evenly across the network. Each node in the network executes the HEAL aggregation learning protocol, in addition to the Elevator protocol (that dynamically assigns "normal" (client) or "hub" (server) status to the participating nodes).

The aggregation process unfolds in five successive phases.

(i) **Local training.** Each non-hub node trains its current model on its local dataset, producing an updated set of parameters ready for aggregation.

(ii) **Model transfer.** Each non-hub node selects s hubs uniformly at random and transmits its current local model to these hubs (with s a global parameter of the protocol). This randomized assignment ensures that the incoming load is balanced across all hubs in expectation.

(iii) **Hub aggregation.** Upon receiving model submissions, each hub waits for a configurable delay $\delta \in \mathbb{R}^+$ before proceeding. This window allows the hub to collect contributions from a sufficient number of nodes before aggregating. After the delay, each hub independently computes a local aggregate from the models it has received, using Average SGD (as defined in Definition 5.2.1).

(iv) **Inter-hub coordination.** Once local aggregation is complete, all hubs enter a synchronous coordination phase. Since hubs form a complete graph — every hub is connected to every other hub by construction of the Elevator overlay — each hub broadcasts its local aggregate to all other hubs and receives their aggregates in return. Each hub then computes the global model from the full set of hub aggregates (again, using Average SGD). Because all hubs perform this computation on the same inputs, the resulting global model is identical across all hubs.

(v) **Redistribution.** Finally, each hub transmits the global model back to the non-hub nodes that submitted their local model to it during the model transfer phase. Since each non-hub node has submitted its model to s hubs, and all hubs compute an identical global model during the inter-hub coordination phase, each node receives s copies of the same global model. The node retains the first received copy and discards the remaining ones, then proceeds to the next training round.

This five-phase process is repeated over successive rounds until the global model converges, mirroring the iterative communication structure of Federated Learning [1].

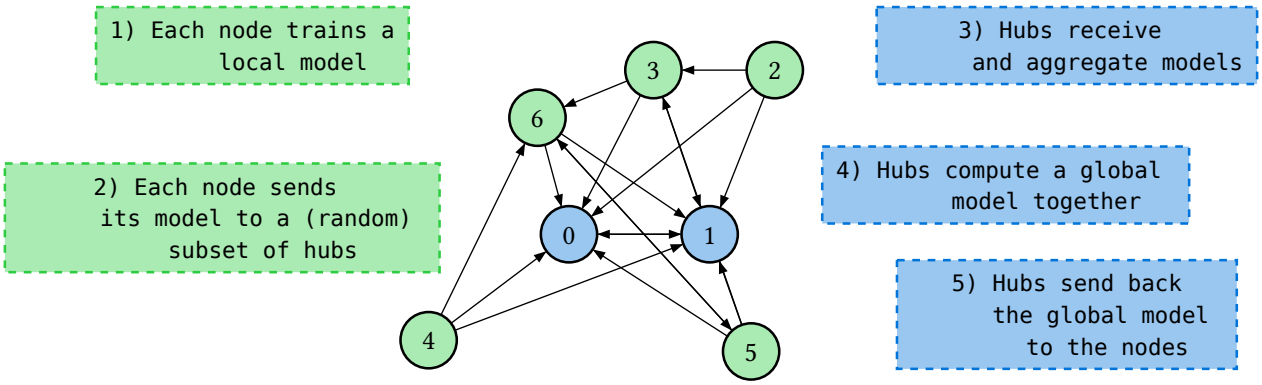

Figure 67: HEAL learning protocol: the five phases

The number of aggregators h is inherited directly from the Elevator protocol, making it a tunable parameter that jointly governs overlay topology and aggregation granularity. Increasing h reduces the per-hub load and improves fault tolerance, at the cost of additional inter-hub communication during the reconciliation phase. The parameter s offers a flexibility-redundancy trade-off: setting $s = 1$ minimizes bandwidth usage, while $s > 1$ provides redundancy in the event of hub failures. Since each

node contributes its model the same number of times regardless of s, the aggregated global model is unaffected by this choice.

In the event of one or more hubs failing during a protocol cycle, the remaining hubs can temporarily manage the nodes without a dedicated hub until a new hub emerges, which typically occurs within two cycles. If all hubs fail simultaneously, the aggregation process halts but resumes as soon as new hubs appear. Elevator also establishes random connections between nodes in addition to hub connections. In HEAL, model aggregation relies solely on hub connections and inter-hub exchanges. These random connections could potentially be exploited to accelerate model convergence or mitigate malicious behavior, but this lies beyond the scope of the present work. We currently assume that all network nodes are honest, and leave the investigation of Byzantine-resilient aggregation to future research.

Figure 68 and Figure 69 present the detailed pseudo-code of HEAL Aggregation.

```
   duration to wait for model: delta_time
   list of all hubs (obtained by the HEAL overlay, Elevator protocol): hubs_list
1  loop
2      nb_hubs ← hubs_list.size()
3      models ← {}
4      time ← time.now()
5      backwards_list ← {}
6      while time.now() < time + delta_time
7          peer_model, peer ← receive()
8          models.append(peer_model)
9          backwards_list.append(peer)
10     average_model ← average(models)
11     send(hubs_list, average_model)
12     models_hubs ← {}
13     models_hubs.append(average_model)
14     nb_receive ← 0
15     while nb_receive < nb_hubs − 1
16         hub_model ← receive()
17         nb_receive ← nb_receive +1
18         models_hubs.append(hub_model)
19     global_model ← average(models_hubs)
20     send(backwards_list, global_model)
```

Figure 68: HEAL Learning: The Hub algorithm.

duration to wait for model: **delta_time**
The model, weights or parameters initialized at random: **model**
The local data of the node: **data**
list of all hubs (obtained by HEAL overlay, Elevator protocol): **hubs_list**
number of hubs to send the model to: **s**

```
loop
    model ← trainModel(model, data)
    hubs ← chooseRandom(hubs_list, s)
    for hub in hubs
        send(hub, model)
    hubs_models ← list()
    for hub in hubs
        model_hub ← receive()
        hubs_models.append(model_hub)
    model ← hubs_models[0]
```

Figure 69: HEAL Learning: The client algorithm.

The Aggregation Layer exposes two primary artifacts to the Application Layer above: the current model, updated at the end of each aggregation round, and access to the node's local dataset, which the Application Layer uses to drive the local training loop.

Application Layer

The Application Layer constitutes the topmost component of the protocol stack, interfacing with the Aggregation Layer to retrieve the current global model and contribute locally trained updates. In HEAL, the application is a machine learning model itself, as formally defined in Chapter 5.

HEAL is designed to support any supervised machine learning model (as defined in Definition 1.3.2), without imposing structural constraints on the model architecture. Linear regressors, support vector machines, and deep neural networks are all valid instantiations, provided that three conditions are satisfied. First, the model must be trainable via gradient descent, as local training relies on iterative parameter updates driven by a differentiable loss function. Second, each node must hold a local dataset partitioned into a training set, used to update the model parameters, and a test set, used to evaluate model quality independently of the training process. Third, all nodes must represent their models in a compatible parameter format, so that model averaging during the aggregation phase is well-defined.

The aggregation function used in HEAL, Average SGD (defined in Definition 5.2.1), computes a simple uniform average of the received model parameters. This approach implicitly assumes that the local datasets are independently and identically distributed (IID) across nodes, meaning that each node's data is drawn from the same underlying distribution. While this assumption simplifies the convergence analysis and is standard in introductory federated learning literature [1], it does not always hold in practice: nodes in a real deployment may hold data with significantly different statistical properties, a setting commonly referred to as non-IID or heterogeneous data. The study of decentralized learning under non-IID data distributions, and the design of aggregation strategies robust to such heterogeneity, lie beyond the scope of this thesis and are left as directions for future work.

In practice, model size imposes implicit constraints on both local computation and network transfer costs. Large models require more memory, longer training times, and higher communication bandwidth. HEAL does not address these engineering concerns directly; the model is treated as an abstract

parameterized function throughout the protocol. In our simulations, we restrict experiments to small-scale models for practical reasons, but nothing in the protocol design precludes the use of larger architectures.

Once the iterative training process has converged and a satisfactory global model has been reached, the HEAL protocol is no longer needed. Each node retains its local copy of the global model and can use it autonomously for inference, without any further communication with the rest of the network.

6.1.3 Architectural properties

The layered architecture presented in the preceding subsection directly supports the desired properties established in Section 6.1.1. Model convergence follows from the aggregation design: by distributing the federated averaging computation across h concurrent hubs and ensuring that all hubs reach an identical global model through the inter-hub coordination phase, HEAL reproduces the communication structure of FedAvg [1] and inherits its convergence guarantees under IID data distributions. Fast dissemination is achieved through the Elevator overlay, whose $O(\log N)$ convergence diameter ensures that model updates propagate to all nodes within a small number of communication rounds. Model agnosticism is enforced by the Application Layer interface, which imposes no structural constraints beyond gradient-based trainability and a compatible parameter format, accommodating any supervised learning model from linear classifiers to deep neural networks. Resilience to failures and churn is provided by Elevator, which tolerates individual node departures — including hub failures — and restores the desired topology within a bounded number of cycles; the aggregation process may pause if all hubs fail simultaneously, but resumes automatically once new hubs are elected. Finally, resource efficiency and algorithmic simplicity are reflected in the two-phase aggregation design: each non-hub node transmits its model to at most s hubs per round, and the hub algorithm requires no coordination beyond a single broadcast among hubs, keeping both communication overhead and implementation complexity minimal.

6.1.4 Communication overhead

The number of models exchanged per cycle varies depending on the topology type and the aggregation algorithm used. We derive the theoretical message count for each framework as follows. In Federated Learning, each of the $n-1$ non-server nodes sends its local model to the server, which then broadcasts the aggregated global model back to all $n-1$ nodes, yielding $2(n-1)$ exchanges per cycle. In Gossip Learning, each node sends its model to one neighbor per cycle, resulting in n exchanges in total. Epidemic Learning follows the same principle, except that each node contacts all c of its outgoing neighbors, giving $n \cdot c$ exchanges. For HEAL, the exchange proceeds in three steps: first, each of the $n-h$ non-hub nodes sends its model to s hubs, contributing $(n-h) \cdot s$ messages; the hubs then exchange their aggregated models with one another, producing $h(h-1)$ messages; finally, each hub redistributes the global model back to the $n-h$ non-hub nodes that contributed to it, adding another $(n-h) \cdot s$ messages. The total per-cycle overhead for HEAL is therefore $2(n-h) \cdot s + h(h-1)$.

Table 11: Comparison of decentralised learning frameworks in terms of communication overhead per cycle, where n is the total number of nodes (including server/hubs), c is the number of outgoing connections, h is the number of hubs, and s is the number of hubs to which each node sends its model.

Aggregation framework	**Number of messages per cycle**
Federated Learning	$2(n-1)$
Gossip Learning	n
Epidemic Learning	$n \cdot c$
HEAL	$2(n-h) \cdot s + h(h-1)$

6.1.5 Experimental setup

Having established the design and theoretical properties of HEAL, we now turn to its empirical evaluation. The protocol is assessed through simulation, which allows us to control network conditions, vary key parameters, and measure convergence behavior in a reproducible setting.

This section describes the experimental setup used to evaluate HEAL. We detail the simulation environment, the learning tasks, the baselines, the configuration of each protocol, and the fault scenarios considered.

Simulation environment

All experiments were conducted using Gossipy[5], a cycle-based peer-to-peer learning simulator. In Gossipy, each cycle consists of every node sequentially executing the protocol. The learning component is not simulated — models are trained using real gradient updates via PyTorch [98] — while the networking layer is fully simulated: nodes are Python objects with direct in-memory access, and all model exchanges are virtual. This design allows for reproducible and controlled experiments without the overhead of a real network infrastructure.

By default, Gossipy only supports static topologies defined ahead of time. To accommodate dynamic topologies, we integrated it with the PeerSim simulator [54], which handles dynamic peer sampling and topology evolution. At each cycle, the graph generated by PeerSim is retrieved and passed to Gossipy, which uses it to determine the virtual connections between nodes and consequently the aggregation partners for that cycle.

All simulations were run on 16 vCPU, using 64G of memory, on a cluster composed of 10 servers, already described in Table 8.

Learning Tasks

We evaluate our protocol on two learning tasks: 1) binary classification and 2) multinomial classification, as defined in the previous chapter (Definition 1.3.5 and Definition 1.3.6). Binary classification constitutes a straightforward baseline that allows us to verify the correctness of the protocol, whilst multinomial classification is more complex, enabling us to compare our protocol more precisely with other decentralised protocols.

The binary classification task is evaluated on the Spambase dataset (Appendix A.3.1), which comprises 4601 samples split into 90% for training and 10% for testing. We use a logistic regression model (Definition 1.3.10) — a simple model that is sufficient for this dataset — with 57 parameters (excluding the bias) and a binary output. The model is trained using SGD with a learning rate of 0.1 and a batch size of 32.

The multinomial classification task is evaluated on the MNIST dataset (Appendix A.3.1), which provides 60,000 training images and 10,000 test images. We use a LeNet5 [94] convolutional neural network, originally designed for this dataset, with approximately 60,000 parameters and an output over 10 classes. The model is trained using the Adam optimiser [99] with a learning rate of 0.001, a weight decay of 0.01, and a batch size of 32.

In both cases, the dataset is partitioned across the network nodes in an IID fashion. The data partition is performed randomly and uniformly across nodes.

[5] https://github.com/makgyver/gossipy

Table 12: Summary of the learning tasks and models used in our experiments.

Task	Dataset	Model	Parameters
Binary classification	Spambase	Logistic regression	57
Multinomial classification	MNIST	LeNet5	60,000

Baselines

The decentralised learning protocols considered in the literature were surveyed in Chapter 5. Revisiting them through the lens of HEAL's layered architecture reveals that a given protocol is in fact the result of two independent choices: a network topology and an aggregation strategy. Not all combinations are valid — some aggregation strategies presuppose a particular topology — but the decomposition clarifies the design space. Furthermore, a fundamental distinction exists between static and dynamic topologies. Static topologies are simpler to deploy but offer no resilience to node failures or churn. Dynamic topologies are more complex to maintain, yet they enable greater model mixing in gossip-based networks and provide inherent resilience to failures.

Table 13: Valid combinations of network topology and aggregation strategy. ✓ indicates a supported combination and × an incompatible one. FL denotes Federated Learning.

Topology	Central aggregation	Gossip	Epidemic
Star	✓ (FL)	×	×
Ring	×	✓	✓
Random	×	✓	✓
Complete	×	✓	✓
Elevator	✓ (HEAL)	✓	✓

The baselines retained for our evaluation cover a representative subset of this design space: Federated Learning [1] (star topology with central aggregation), Gossip Learning [55] and Epidemic Learning [96] (dynamic random topology), Epidemic Learning on a Chord [9] topology, Epidemic Learning on a ring topology, Gaia [68] (multi-star static topology with central aggregation), and Fedlay [91] (dynamic topology). HEAL combines the Elevator dynamic topology with central aggregation at the hubs level, and is the only protocol in our evaluation that spans all three aggregation strategies.

Configuration

Gossipy natively provides implementations of Federated Learning, Gossip Learning, and Epidemic Learning. We extended the simulator with the following contributions: Epidemic Learning on Chord and ring topologies, Gaia, Fedlay, and HEAL. The Elevator component of HEAL was implemented in PeerSim, while the aggregation logic was implemented directly in Gossipy. Static topologies (Federated Learning, ring, Chord, and Gaia) were generated using the NetworkX[6] Python library.

In Elevator (used by HEAL), the connections are directional. However, to compare them with other algorithms (which assume an undirected graph), we modified the underlying graph of the topology generated to make it undirected.

All evaluations were conducted with a network of 100 nodes. For Elevator, we used 5 hubs, as we found this number to be a good balance between performance and resilience. For Gaia, we had 5 servers responsible for aggregation (to compare with the 5 hubs) and 19 nodes (or workers) attached to each server. Each algorithm was evaluated 5 times, and we present the average results obtained.

[6] https://networkx.org/

Fault scenarios

HEAL is evaluated under five scenarios. The first is a fault-free baseline, against which all other scenarios are compared, allowing us to assess the performance of HEAL under normal operating conditions. The remaining scenarios follow the fault taxonomy introduced in Chapter 4, with the exception of Byzantine failures, which are outside the scope of this thesis for the decentralised learning component.

The second scenario simulates the failure of 20% of nodes, testing whether the protocol maintains acceptable learning performance when a non-negligible fraction of participants becomes unavailable. The third and fourth scenarios simulate the failure of a single hub and the failure of all 5 hubs respectively, verifying that HEAL does not exhibit a single point of failure — a key design requirement for any decentralised protocol. Finally, the fifth scenario introduces churn, where 10% of nodes leave the network and are replaced by new nodes at each cycle, testing whether the protocol remains functional under the dynamic membership conditions typical of real peer-to-peer networks.

For Federated Learning and Gaia, which are centralised by nature, node failures are applied exclusively to non-server nodes, as the failure of a server unconditionally halts training in these protocols. This asymmetry is inherent to their architecture and further motivates the need for fully decentralised alternatives such as HEAL.

6.1.6 Results

Learning in a crash-free, churn-free environment

For simulations without failures and churn, we ran all algorithms over 1000 cycles, on the two learning tasks (Spambase, MNIST). As shown in Figure 70 and Figure 71, when there are no failures in the network, Federated Learning performs best, which was expected. Surprisingly, the ring topology performs second best despite lower connectivity. Other topologies based on random graphs perform less well. HEAL, however, performs very well for both the Spambase and MNIST datasets.

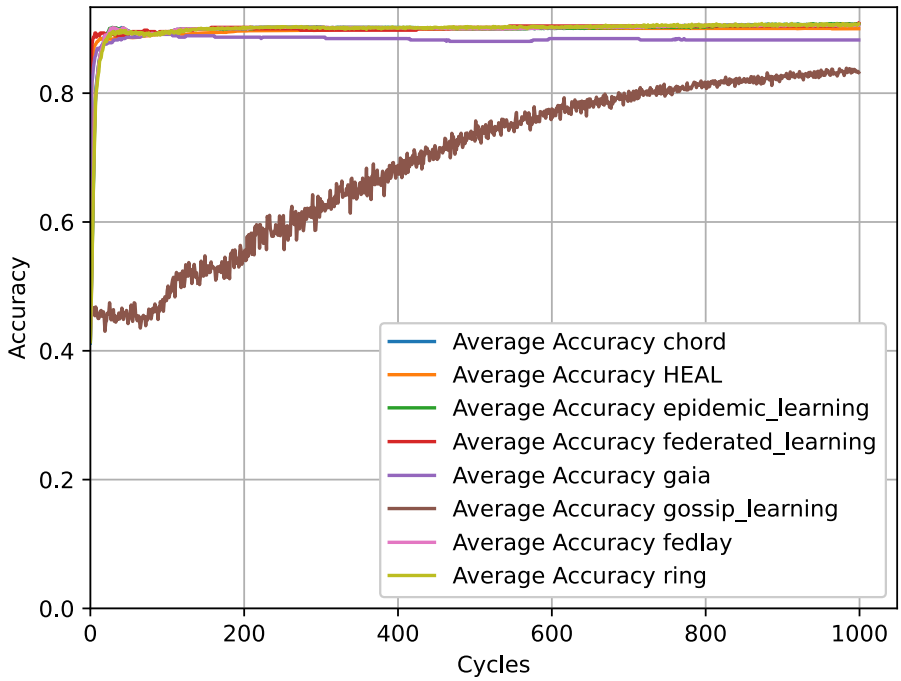

Figure 70: Accuracy of various communication protocols, with a network of 100 nodes, during 1000 cycles. HEAL overlay has 5 hubs, each node sends its model to one hub, for the Spambase dataset, no failures

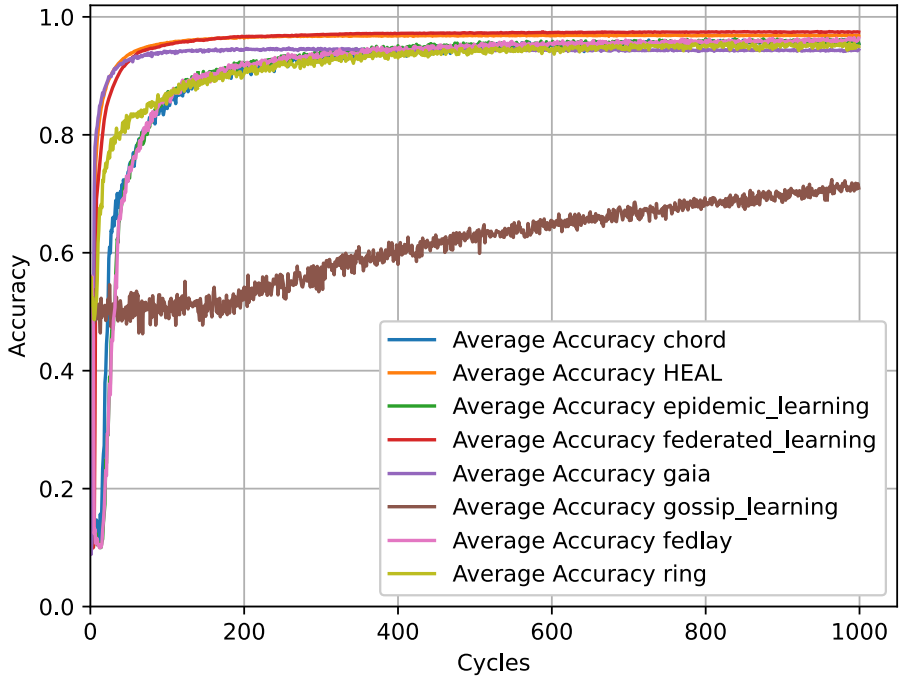

Figure 71: Accuracy of various communication protocols, with a network of 100 nodes, during 1000 cycles. HEAL overlay has 5 hubs, each node sends its model to one hub, for the MNIST dataset, no failures

In Table 14, Table 15 and Table 16 we have compiled the results for all learning algorithms, for the two learning tasks, with the final accuracy obtained after 1000 cycles. Federated Learning, Gaia, and HEAL have very similar results, with a final accuracy of around 0.88 on Spambase, and 0.95 on MNIST. Gossip-based approaches achieve values of 0.85 and 0.91 on Spambase and MNIST.

Table 14: Final accuracy by communication method and dataset used. HEAL overlay with 5 hubs, each node sends its model to one hub (fault and churn free scenario).

Method	**Spambase**	**MNIST (LeNet)**
Federated Learning	0.9087	0.9742
Gaia	0.8826	0.9442
Gossip Learning	0.8322	0.7098
Epidemic Learning	0.9548	0.9111
Ring	0.9076	0.9499
Chord	0.9063	0.9542
FedLay	0.9056	0.9639
HEAL	0.9001	0.9687

Table 15: Number of cycles required to achieve different accuracy levels on the Spambase dataset, for a network of 100 nodes, without failures. N/A indicates that the protocol did not achieve the target accuracy within 1000 cycles.

Method	**0.85**	**0.90**
Federated Learning	2	135
Gaia (5 servers)	6	N/A
Gossip Learning	N/A	N/A
Epidemic Learning	12	25
Ring	13	141
Chord	12	27
FedLay	12	26
HEAL (5 hubs, $s = 1$)	4	339

Table 16: Number of cycles required to achieve different accuracy levels on the MNIST dataset, for a network of 100 nodes, without failures. N/A indicates that the protocol did not achieve the target accuracy within 1000 cycles.

Method	**0.85**	**0.90**	**0.95**
Federated Learning	22	37	91
Gaia (5 servers)	13	28	88
Gossip Learning	N/A	N/A	N/A
Epidemic Learning	90	137	372
Ring	74	157	569
Chord	98	159	451
FedLay	89	136	422
HEAL (5 hubs, $s = 1$)	15	27	76
HEAL (7 hubs, $s = 3$)	5	10	33

Convergence time is very fast for aggregator-based approaches: on Spambase, Federated Learning converges to 0.85 in 2 cycles, and HEAL takes 4 cycles. On the other hand, to converge on 0.90 accuracy, HEAL takes 339 cycles while Federated Learning takes 135 cycles, hinting at possible HEAL optimization, e.g. adjusting the learning rate. On MNIST, HEAL performs very well, even better than Federated Learning, and it converges to 0.95 accuracy in 76 cycles.

We ran simulations changing the number of hubs over 2000 cycles. On Figure 72, we observe that increasing the number of hubs (and the number of hubs to which clients send their model) has almost no impact on accuracy, which is expected since hubs aggregate models. There is a slight drop in accuracy when the number of hubs is increased significantly, due to the fact that only non-hubs are learning, not hubs. Increasing the number of hubs to which we send our model, from 1 to $\frac{\text{nb\_hubs}}{2}$, slightly increases convergence speed.

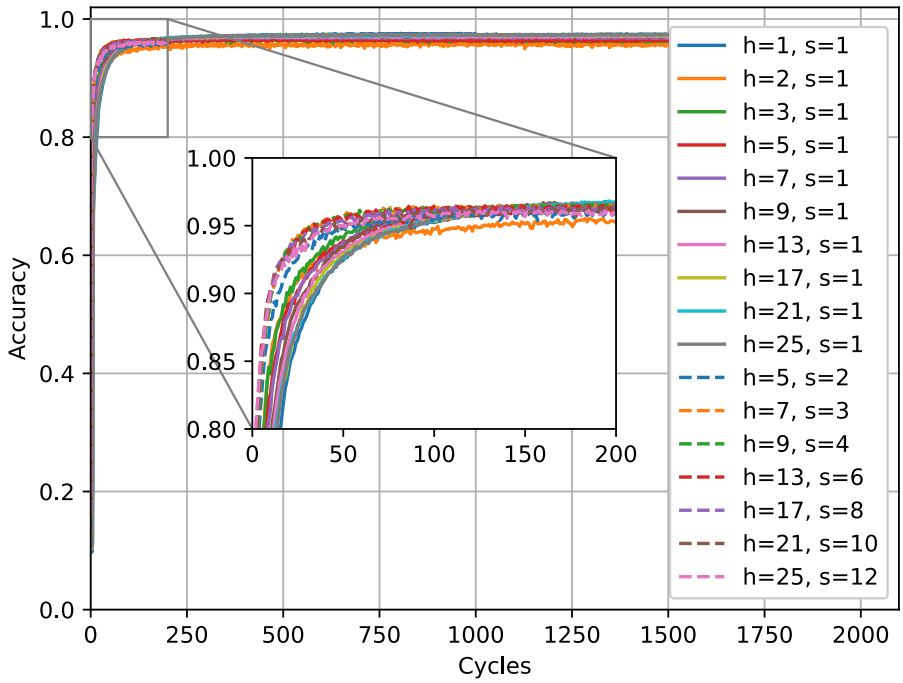

Figure 72: HEAL with different numbers of hubs (h), from 1 to 25, each node sent its model to (s) hubs, with (s) equals to 1 or $\frac{h}{2}$, no failures, 2000 cycles

We have computed the number of models exchanged by cycle for each algorithm. The results match the theoretical values, and we have summarized the results in the Table 17. HEAL is very close to Federated Learning, with a number of models sent equal to 210, compared with 198 for Federated Learning. Epidemic Learning is much higher, with 1000 messages per cycle.

Table 17: Number of models exchanged per cycle for each protocol, with $n = 100$, $c = 10$, $h = 5$, and $s = 1$.

Algorithm	**Messages per cycle**
Federated Learning	198
Gaia	210
Gossip Learning	100
Epidemic Learning	1000
Epidemic Learning on ring	200
Epidemic Learning on Chord	1000
FedLay	956
HEAL	**210**

Learning despite crashes

We analyze the performance of the algorithms when the network suffers crashes. To simulate a brutal failure we disconnected 20% of the nodes chosen uniformly at random, just after the start of the learning process, i.e., in this case, we have disconnected 20 nodes at cycle 10, and we compared HEAL with Chord, Gaia and Fedlay. HEAL is the algorithm with the best results, although Gaia remains very close, as seen in Figure 73.

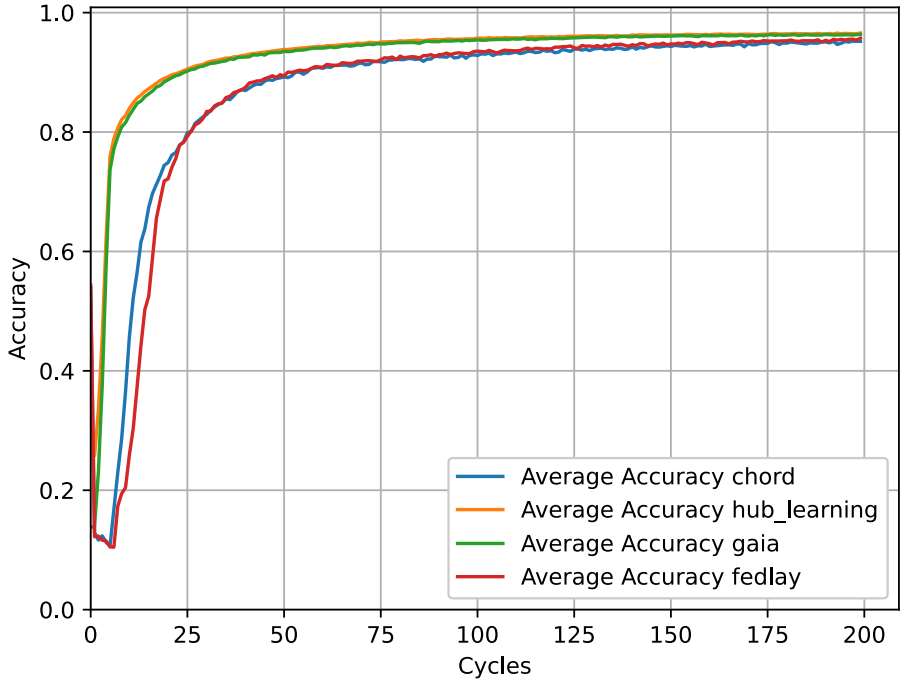

Figure 73: Accuracy of various communication protocols, with a network of 100 nodes, during 1000 cycles. HEAL overlay has 5 hubs, each node sends its model to one hub. for the MNIST dataset, when 20% of the nodes fail at cycle 10

We also compared HEAL subjected to different level of crashes (20%, 30%, 40%, 50%), as seen in the Figure 74. HEAL remain resilient even under a crash level of 50%. We have summarized our results in Table 18. The final accuracy, at cycle n°200, is very close to the accuracy obtained without crashes for a crash level of 20%. For greater crash level, the drop in accuracy is greater, but the algorithm still manages to converge. For a crash level of 40%, accuracy reaches 0.9, in a number of cycles of 107.

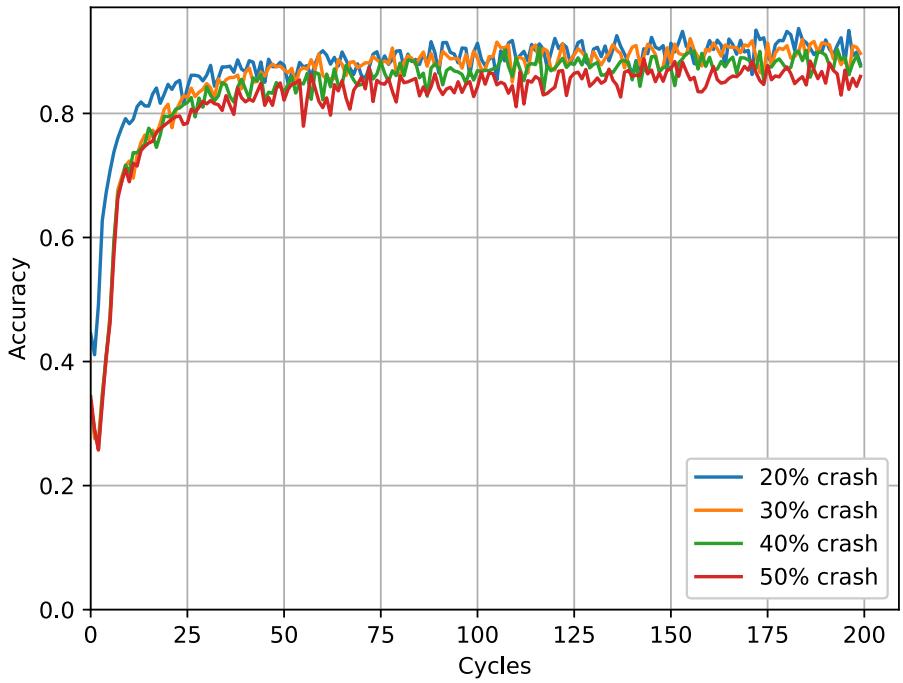

Figure 74: Accuracy of HEAL for the MNIST dataset for different levels of crash, with 100 nodes and 5 hubs, each node sent its model to one hub, 200 cycles

Table 18: Accuracy after a crash occurring at cycle 10, on the MNIST dataset. Final accuracy is measured at cycle 200. N/A indicates that the protocol did not achieve the target accuracy within the simulation.

Method	**Fault scenario**	**Final accuracy (cycle 200)**	**Cycles to 0.90 accuracy**
Gaia (5 servers)	20% node crash	0.9637	37
Chord	20% node crash	0.9520	53
FedLay	20% node crash	0.9563	51
FedLay	30% node crash	0.9540	51
FedLay	40% node crash	0.9521	51
FedLay	50% node crash	0.9537	51
HEAL (5 hubs, $s = 1$)	20% node crash	0.9658	30
HEAL (5 hubs, $s = 1$)	30% node crash	0.9178	59
HEAL (5 hubs, $s = 1$)	40% node crash	0.8987	107
HEAL (5 hubs, $s = 1$)	50% node crash	0.8719	N/A
HEAL (5 hubs, $s = 1$)	1 hub crash	0.9638	28
HEAL (5 hubs, $s = 1$)	5 hub crash	0.9629	28

HEAL under churn environment and hub-targeted attacks

We further analyzed the performance of HEAL under network churn conditions. To simulate churn, we disconnected 10% of the nodes at each cycle and replaced them with an equal number of new nodes, each connected to 20 nodes uniformly at random, between cycles 50 and 150. We also analyzed the

performance of the main learning algorithms after a targeted attack on the hubs during the execution of the simulation. We tested two scenarios, one where we disconnected one of the 5 hubs, and another where we disconnected all 5 hubs at the same time. In both cases the failure happened in round 10. In Figure 75, we have compared the execution of HEAL without failures, and with different failure scenarios (crash of 20 nodes, crash of one hub, crash of all 5 hubs, churn). As can be seen, there is no significant impact when 20 nodes or hubs crash. Indeed, thanks to the Elevator overlay, even when all the hubs are shutdown, 5 new nodes are elected very quickly as hubs, and the training job continues as if no catastrophic event had happened. During churn, model accuracy falls slightly, but rises again very quickly once churn is over, back to the level without failures.

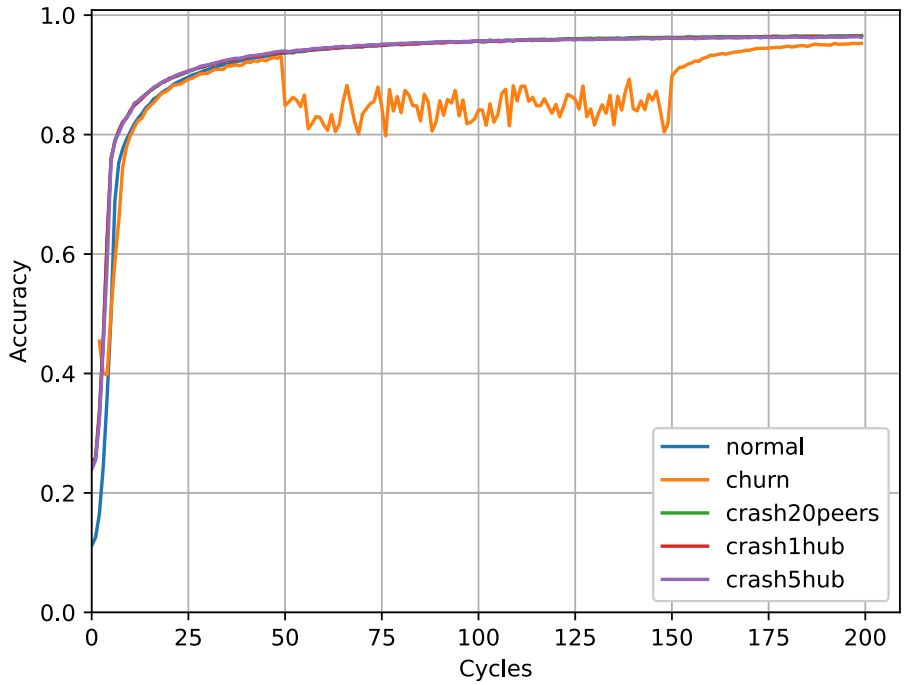

Figure 75: HEAL for all contexts (no failures, crash of 20 peers, crash of 1 hub, crash of all hubs, churn), with 5 hubs, each node sent its model to one hub, 200 cycles

We also compared HEAL subjected to different level of churn (10%, 20%, 30%), as seen in the Figure 76. HEAL remain resilient even under a churn level of 30%. The accuracy level drops sharply during the churn phase, but rises again almost immediately when the churn is over.

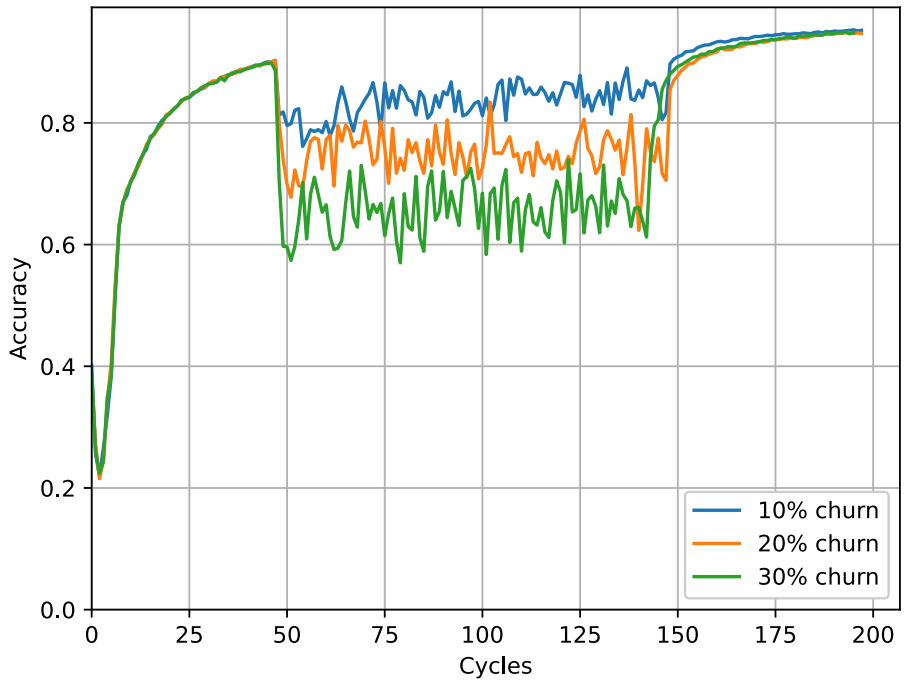

Figure 76: Accuracy of HEAL for the MNIST dataset for different levels of churn, with 100 nodes and 5 hubs, each node sent its model to one hub, 200 cycles.

We have summarized our results in Table 19. The final accuracy, at cycle n°200, is very close to the final accuracy obtained without churn, even for a churn level of 30%. We also computed the mean drop level of accuracy during churn. With a churn level of 30%, the drop is 0.28, which is sharp, but the final accuracy is not affected.

Table 19: Final accuracy at cycle 200 under churn conditions between cycles 50 and 150, on the MNIST dataset.

Method	**Churn context**	**Final accuracy (cycle 200)**	**Mean accuracy drop during churn**
HEAL (5 hubs, $s = 1$)	10% churn	0.9507	0.0955
HEAL (5 hubs, $s = 1$)	20% churn	0.9477	0.1777
HEAL (5 hubs, $s = 1$)	30% churn	0.9500	0.2767
FedLay	10% churn	0.9510	0.0045
FedLay	20% churn	0.9505	0.0140
FedLay	30% churn	0.9501	0.0232

6.1.7 Summary

The experimental results demonstrate that HEAL achieves competitive learning performance whilst providing resilience properties that centralised and gossip-based approaches cannot offer simultaneously.

In a fault-free environment, HEAL reaches a final accuracy of 0.90 on Spambase and 0.97 on MNIST, comparable to Federated Learning (0.91 and 0.97) and significantly above gossip-based approaches such as Gossip Learning (0.83 and 0.71). In terms of convergence speed on MNIST, HEAL ($s = 1$) reaches

0.95 accuracy in 76 cycles, faster than all gossip-based baselines, and HEAL ($s = 3$) further reduces this to 33 cycles, outperforming even Federated Learning (91 cycles). The number of models exchanged per cycle remains low (210), close to Federated Learning (198) and far below Epidemic Learning (1000).

Under crash conditions, HEAL maintains a final accuracy of 0.97 even after 20% of nodes fail, and remains functional up to a crash level of 50%. Crucially, hub-targeted attacks — including the simultaneous failure of all 5 hubs — have no measurable impact on accuracy, thanks to the hub re-election mechanism provided by the Elevator overlay. This confirms that HEAL has no single point of failure, in contrast to Federated Learning and Gaia, where server failure unconditionally halts training.

Under churn, HEAL experiences a temporary accuracy drop during the churn phase, but recovers to its pre-churn level almost immediately once churn subsides. Even at a churn rate of 30%, the final accuracy at cycle 200 remains above 0.95, demonstrating the robustness of the protocol under the dynamic membership conditions typical of real peer-to-peer networks.

6.2 FLAIR Protocol

A key claim of HEAL's layered architecture is that each layer can be substituted independently, yielding a different protocol without redesigning the system from scratch. To validate this modularity, we present FLAIR (*Federated Learning with Adaptive Integrity-preserving Randomness*), an alternative instantiation of the same architecture targeting wireless edge networks.

FLAIR departs from HEAL in three layers. At the network layer, FLAIR operates over WiFi rather than a general-purpose internet overlay. At the overlay layer, nodes are organised into clusters using a protocol inspired by LEACH [100], a well-known cluster-based routing protocol designed for energy-constrained networks, in which cluster heads are elected periodically and rotate among nodes. At the aggregation layer, model aggregation is performed locally within each cluster, rather than globally across all hubs as in HEAL. This design reduces communication costs and is well-suited to scenarios where nodes are geographically or topologically grouped.

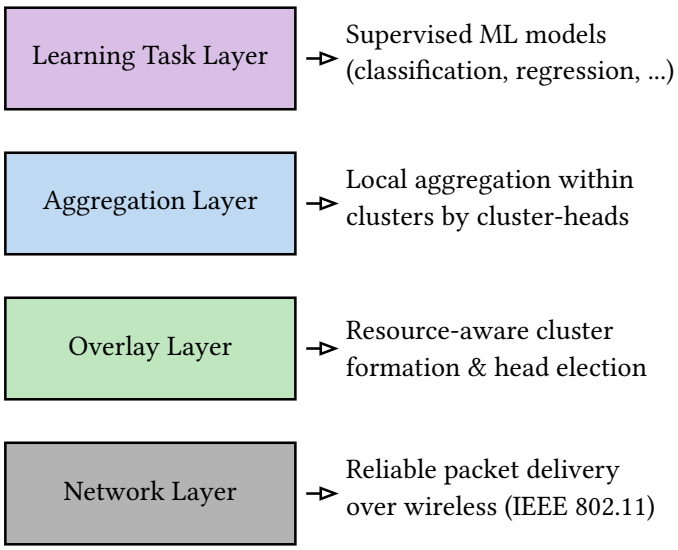

Figure 77: Layered architecture of FLAIR

6.2.1 FLAIR Architecture

Network Layer

The Network Layer forms the foundation of the FLAIR protocol stack. It is responsible for low-level wireless communication, providing the basic message passing primitives upon which all higher layers depend. Unlike HEAL, which operates over a standard TCP/IP infrastructure, FLAIR targets infrastruc-

ture-less ad-hoc wireless networks where nodes communicate over a shared wireless medium, without relying on any fixed network infrastructure.

Each node is equipped with a wireless interface compliant with the IEEE 802.11 standard [101] and can only communicate directly with nodes within its transmission range. Communication is therefore inherently single-hop: a node cannot relay messages through intermediaries at this layer. This constraint shapes the overlay design, as cluster members communicate directly with their cluster-head, and cluster-heads broadcast directly to their members, without any inter-cluster coordination at the communication level.

Nodes are heterogeneous in both their computational resources (CPU, memory) and their communication capabilities (bandwidth, transmission range). Each node is identified by its wireless interface address. The Network Layer exposes the same two primitive operations as in HEAL:

- `send(peer_id, message)`: transmits a message to a peer within direct wireless range,
- `receive()`: listens for incoming messages and delivers them to the upper layer for processing.

Overlay Layer

The Overlay Layer implements the logical topology that enables efficient decentralized learning in FLAIR. Rather than relying on a pre-configured or manually maintained topology, this layer provides a fully decentralized and automatic clustering mechanism inspired by LEACH [100]. Built on top of the Network Layer, it dynamically organizes nodes into clusters and elects a subset of nodes as **cluster-heads** (CHs), which serve as local aggregation points. The entire process — from CH election to cluster formation — proceeds autonomously at each round, without any central coordinator or global knowledge.

The protocol operates in rounds. Two mechanisms are central to its design. First, a target fraction $p \in (0, 1)$ of nodes is expected to serve as CHs in each round, ensuring probabilistic load balancing across the network. Second, every node evaluates a Verifiable Random Function (VRF) [102], which produces a publicly verifiable random value in $[0, 1]$. This primitive prevents manipulation of elections and guarantees fairness, while remaining lightweight enough for resource-constrained environments.

Phase 1: Cluster-head selection

Let G be the set of eligible nodes at round r. A node n is eligible to become a CH only if it has not acted as one during the last $1/p$ rounds. Each eligible node computes a threshold $T(n)$ and samples $x \sim \text{Uniform}(0, 1)$ through a VRF. Node n elects itself as CH if $x < T(n)$, where

$$T(n) = \begin{cases} \dfrac{p \cdot R_n}{1 - p \cdot \left(r \bmod \frac{1}{p}\right)} & \text{if } n \in G, \\ 0 & \text{otherwise,} \end{cases}$$

and the resource score R_n is defined as

$$R_n = \alpha \cdot \text{CPU}_n + \beta \cdot \text{RAM}_n + \gamma \cdot \text{GPU}_n + \delta \cdot \text{BW}_n,$$

with $\alpha + \beta + \gamma + \delta = 1$. The four components of R_n are normalized indicators collected locally by each node from kernel-level system metrics, without requiring any external coordination:

- $\text{CPU}_n \in [0, 1]$: the fraction of CPU capacity currently available on node n, derived from processor utilization statistics exposed by the operating system kernel,
- $\text{RAM}_n \in [0, 1]$: the fraction of available memory on node n, obtained from the kernel memory subsystem,

- $\mathrm{GPU}_n \in [0,1]$: the fraction of GPU capacity available on node n, when a GPU is present; set to 0 otherwise,
- $\mathrm{BW}_n \in [0,1]$: the available wireless bandwidth of node n, estimated from link-layer statistics reported by the network interface.

Each node computes its own resource score R_n independently and autonomously, using only local information from its operating system. No global resource monitoring or centralized collection is required. This weighted combination biases elections toward resource-rich nodes while preserving the probabilistic load-balancing properties of LEACH. The use of VRF guarantees that elections remain tamper-resistant and publicly verifiable.

target CH ratio: **p**
current round: **r**
eligibility set: **G**
resource vector: $(\mathbf{CPU}_n, \mathbf{RAM}_n, \mathbf{GPU}_n, \mathbf{BW}_n)$
weights: **alpha, beta, gamma, delta**
for each node $n \in \mathcal{N}$ **in parallel do**
  if $n \notin G$ **then**
    $T(n) \leftarrow 0$
  else
    $R_n \leftarrow \alpha \cdot \mathrm{CPU}_n + \beta \cdot \mathrm{RAM}_n + \gamma \cdot \mathrm{GPU}_n + \delta \cdot \mathrm{BW}_n$
    $T(n) \leftarrow \dfrac{p \cdot R_n}{1 - p \cdot \left(r \bmod \frac{1}{p}\right)}$
  end if
  $x \leftarrow \mathrm{VRF}_{\mathrm{Uniform}(0,1)}$
  if $x < T(n)$ **then**
    broadcast `CH-ADV`
    mark n as CH
  end if
end for

Listing 1: Resource-aware CH selection with verifiable randomness.

Phase 2: Cluster formation

Once CHs have been elected, each CH broadcasts a cluster-head advertisement (`CH-ADV`) within its transmission range. Non-CH nodes listen for incoming advertisements, evaluate a distance-based cost for each candidate CH, and join the most suitable one by sending a `JOIN-REQ` message. The CH responds with a `JOIN-ACK` and updates its membership list. The outcome is a stable partition of the network into clusters with balanced resource allocation and minimized communication costs.

```
for each non-CH node u in parallel do
    listen for CH-ADV beacons; collect candidate set C
    compute cost(u, c) for all c ∈ C
    c* ← arg min_{c∈C} cost(u, c)
    send JOIN-REQ to c*
end for
for each CH c do
    for each JOIN-REQ from u do
        if capacity allows then
            send JOIN-ACK to u
            update membership list
        end if
    end for
end for
```

Listing 2: Cluster formation.

The Overlay Layer exposes the following API to the Aggregation Layer above:

- `isCH()`: returns `true` if the local node is currently elected as a cluster-head,
- `getMembers()`: returns the list of member nodes in the local cluster (CH only),
- `getCH()`: returns the address of the cluster-head the local node has joined,
- `broadcast(message)`: sends a message to all members of the local cluster (CH only),

Aggregation Layer

Once clusters are formed, decentralized learning begins within each cluster independently, removing the need for any global coordination between clusters. Unlike HEAL, where hubs exchange aggregated models with one another before redistribution, FLAIR performs aggregation exclusively at the cluster-head level, with no inter-cluster communication. Each cluster thus runs a fully self-contained learning process.

Phase 1: Local training

Every node u trains the current model $\boldsymbol{w}^{(t)}$ on its private dataset $\mathcal{D}_u$ for E local epochs, producing a locally updated model $\boldsymbol{w}_u^{(t+1)}$. Only model updates are shared; raw data always remains private.

Phase 2: Model aggregation

Each CH aggregates the updates received from its members using a simple averaging rule (see Definition 5.2.1):

$$\boldsymbol{w}^{(t+1)} = \frac{1}{|\mathcal{S}_c|} \sum_{u \in \mathcal{S}_c} \boldsymbol{w}_u^{(t+1)},$$

where $\mathcal{S}_c$ is the set of members of cluster c. The aggregated model is then redistributed to all cluster members for the next iteration.

Phases 1 and 2 are repeated for a fixed number of in-cluster learning epochs per round, denoted E_{round}. After each iteration, members receive the aggregated model from their CH, perform local training, and send updated weights back. This iterative loop continues until all E_{round} epochs are completed, forming one full round of in-cluster decentralized learning.

initial model: $\boldsymbol{w}^{(0)}$
batch size: $\boldsymbol{B}$
in-cluster epochs: $\boldsymbol{E}_{\text{round}}$
for each cluster c **in parallel do**
  $t \leftarrow 0$
  for $e = 1$ **to** E_{round} **do**
    for each member $u \in \mathcal{S}_c$ **in parallel do**
      $\boldsymbol{w}_u^{(t+1)} \leftarrow \text{LocalTrain}\big(\boldsymbol{w}^{(t)}, \mathcal{D}_u, B\big)$
      send $\boldsymbol{w}_u^{(t+1)}$ to CH
    end for
    CH computes $\boldsymbol{w}^{(t+1)} \leftarrow \frac{1}{|\mathcal{S}_c|} \sum_{u \in \mathcal{S}_c} \boldsymbol{w}_u^{(t+1)}$
    CH broadcasts $\boldsymbol{w}^{(t+1)}$ to $\mathcal{S}_c$
    $t \leftarrow t + 1$
  end for
end for

Figure 78: In-cluster decentralized learning.

6.2.2 Architectural properties

The layered architecture of FLAIR directly supports a set of desirable properties for decentralized learning over wireless networks. Several of these properties are shared with HEAL, whilst others are specific to the wireless and cluster-based setting.

Model convergence is promoted by the iterative in-cluster aggregation design: within each cluster, the CH performs repeated averaging over member updates across E_{round} epochs per round, driving local consensus. Although no global aggregation occurs between clusters, convergence across the full network is achieved through the rotation of cluster-heads at every round. Since any node may become a CH in a subsequent round, locally aggregated models are progressively redistributed across the network, causing models from different clusters to merge over time without requiring explicit inter-cluster communication.

The number of in-cluster epochs E_{round} introduces a tunable trade-off: a large value of E_{round} accelerates local convergence within each cluster but delays inter-cluster model mixing, as cluster-heads rotate less frequently relative to the amount of training performed. Conversely, a small value of E_{round} favors rapid cluster-head rotation and therefore faster global mixing, at the cost of slower local convergence per round. This parameter allows FLAIR to be adapted to the requirements of a given deployment.

Fast dissemination is achieved through the combination of in-cluster broadcasting and CH rotation. Within a cluster, the CH redistributes the aggregated model to all members in a single broadcast step. Across clusters, the rotation mechanism ensures that aggregated models gradually propagate throughout the network over successive rounds.

Model agnosticism is preserved by the Aggregation Layer interface, which imposes no structural constraints on the learning model beyond gradient-based trainability and a compatible parameter format. FLAIR accommodates any supervised learning model, from linear classifiers to deep neural networks, in the same manner as HEAL.

Resilience to failures and churn is provided by the clustering protocol. Since cluster-heads are re-elected at every round using a probabilistic, resource-aware mechanism, the failure of any individual node — including a current CH — only affects the current round. A new CH is elected at the start of the following round, and the learning process resumes automatically without manual intervention or global coordination.

Scalability and wireless compatibility are native properties of FLAIR's architecture. The protocol operates entirely over IEEE 802.11 [101] wireless links and requires only single-hop communication within each cluster, making it deployable on standard Wi-Fi hardware without any additional infrastructure. Since each cluster operates independently, the protocol scales naturally with the number of nodes: adding nodes to the network increases the number of clusters or their size, without introducing any centralised bottleneck.

6.2.3 Experimental setup

This section describes the experimental setup used to evaluate FLAIR. We detail the simulation environment, the learning tasks, the baselines, and the experimental scenarios considered.

Simulation environment

All experiments were conducted using ns-3 [103], a discrete-event network simulator that provides faithful modeling of IEEE 802.11 wireless communications, including ad-hoc mode and single-hop transmissions. Unlike Gossipy, which simulates the networking layer whilst performing real model training, ns-3 simulates the full network stack, including wireless channel conditions, interference, and packet scheduling. All baseline protocols were re-implemented in ns-3 (in C++) to ensure strict comparability under identical network conditions.

All simulations were run on a dedicated server equipped with two Intel Xeon E5-2660 v3 processors (10 cores, 2 threads per core, 2.6 GHz base frequency), 125 GB of RAM, and 456 GB of storage, running Arch Linux (kernel 6.12.4-arch1-1).

Learning tasks

We evaluate FLAIR on two binary classification tasks. Both tasks use a logistic regression model [92] with cross-entropy loss, and data is partitioned across nodes such that each node holds a private local subset that never leaves the device.

The first task uses the Spambase dataset (Appendix A.3.1), which was also used in the HEAL experiments.

The second task uses the "Predicting Watering the Plants" dataset [104], a Kaggle dataset tailored for intelligent irrigation systems. It contains 100,000 samples described by features capturing soil state and ambient conditions (e.g., soil moisture, temperature, humidity), with a binary label indicating whether watering is required. The class distribution is 53.6% positive and 46.4% negative, representing a mildly imbalanced scenario. This dataset was selected to demonstrate the applicability of FLAIR to real-world edge deployments such as smart farming.

Table 20: Summary of the learning tasks used in the FLAIR experiments.

Task	Dataset	Model	Samples
Binary classification	Spambase	Logistic regression	4,601
Binary classification	Watering the Plants	Logistic regression	100,000

Baselines

FLAIR is compared against four representative protocols, each capturing a different architectural paradigm. Centralized Federated Learning [1] (C-FL) serves as the canonical client–server baseline, with a single global server aggregating all updates at each round. Gaia [68] represents the hierarchical paradigm, with 5 local servers each coordinating 20 clients. HEAL provides a hybrid decentralized reference point, with 5 dynamically elected hubs per round performing two-stage aggregation. Finally, Gossip Learning [55] represents the fully decentralized paradigm, where each node contacts 3 random peers per round.

Table 21: Configuration of protocols in Experiment 1.

Protocol	**Aggregation**	**Configuration**
C-FL	Global averaging	$E = 3, \eta = 0.01$
Gaia	Local + global averaging	5 local servers, $E = 3$
HEAL	Hub + inter-hub	5 hubs per round
Gossip Learning	Pairwise averaging	3 peers per round
FLAIR	In-cluster averaging	$E_{\text{round}} = 3$

Experimental scenarios

Four experiments were designed to assess different aspects of FLAIR's performance. Performance was assessed in terms of final accuracy, recovery speed, and stability of convergence. To capture the effect of round duration, two configurations were considered: longer rounds of $E_{\text{round}} = 3$ FL epochs per round, and shorter rounds of $E_{\text{round}} = 1$ FL epoch per round.

Table 22: Overview of experimental scenarios for FLAIR evaluation.

Experiment	**Objective**	**Setup**
Experiment 1	Comparative evaluation in static networks	100 static nodes; comparison with C-FL, Gaia, HEAL, Gossip Learning
Experiment 2	Resilience to node dropouts	Permanent, temporary, and random crashes (up to 90% nodes); $E_{\text{round}} = 1$ and $E_{\text{round}} = 3$
Experiment 3	Impact of mobility on learning performance	5 mobility models; perfect vs. range-limited connectivity
Experiment 4	Smart farming with heterogeneous nodes	80 fixed sensors + 20 mobile robots; Watering the Plants dataset

Experiment 1 evaluates learning performance in a static network of 100 nodes under fault-free conditions, serving as the primary comparative benchmark against C-FL, Gaia, HEAL, and Gossip Learning.

Experiment 2 examines resilience under three types of node dropout: permanent crashes, where nodes leave the system at the start of the second round and do not return; temporary crashes, where nodes remain inactive for 15 time units (equivalent to 3 rounds of 5 epochs) before resuming, with repeated failures injected once accuracy begins to stabilize; and random dropouts, where nodes intermittently disconnect and reconnect throughout training, creating highly unpredictable availability patterns.

Experiment 3 investigates the effect of node mobility on learning performance. Two connectivity scenarios are considered. In the perfect connectivity scenario, all nodes communicate regardless of their physical positions. In the range-limited connectivity scenario, nodes can only communicate within a fixed transmission radius, resulting in temporary disconnections as nodes move. Five well-

known mobility models [105] are simulated: RandomWaypoint, RandomWalk, RandomDirection, Gauss-Markov, and ConstantVelocity.

Experiment 4 demonstrates FLAIR in a realistic smart farming application. The network consists of 80 fixed sensors that continuously sample environmental parameters (soil moisture, temperature, humidity) and 20 mobile robotic nodes that autonomously navigate the field and relay model updates between clusters, bridging connectivity gaps. Learning is performed on the Watering the Plants dataset [104].

6.2.4 Results

Comparative evaluation in static networks

Figure 79 shows the accuracy evolution over training cycles in a static network of 100 nodes. FLAIR achieves the highest final accuracy (≈ 0.91), surpassing C-FL, HEAL, and Gossip Learning (≈ 0.90), and clearly outperforming Gaia (≈ 0.88). These results demonstrate that the clustering-based design of FLAIR accelerates convergence whilst sustaining higher steady-state accuracy. Compared to Gaia, convergence is up to 2.5 $\times$ faster, and compared to Gossip Learning, the protocol requires significantly fewer cycles to stabilize. Overall, FLAIR combines the scalability of decentralized designs with the efficiency of clustering, providing superior performance in static deployments.

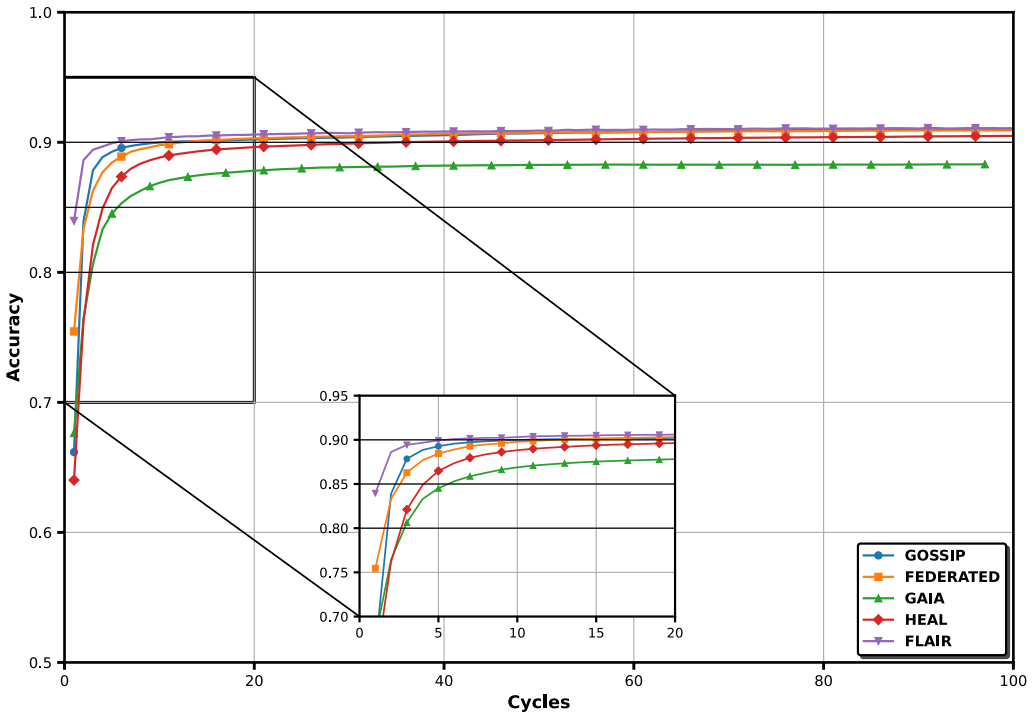

Figure 79: Accuracy evolution of FLAIR and baselines in static networks (100 nodes).

Resilience to node dropouts

Figure 80, Figure 81, Figure 82, Figure 83, Figure 84, and Figure 85 report the average accuracy evolution under permanent, temporary, and random crashes for both round duration settings ($E_{\text{round}} = 1$ and $E_{\text{round}} = 3$). In the baseline case without dropout, FLAIR stabilized around 0.90 accuracy.

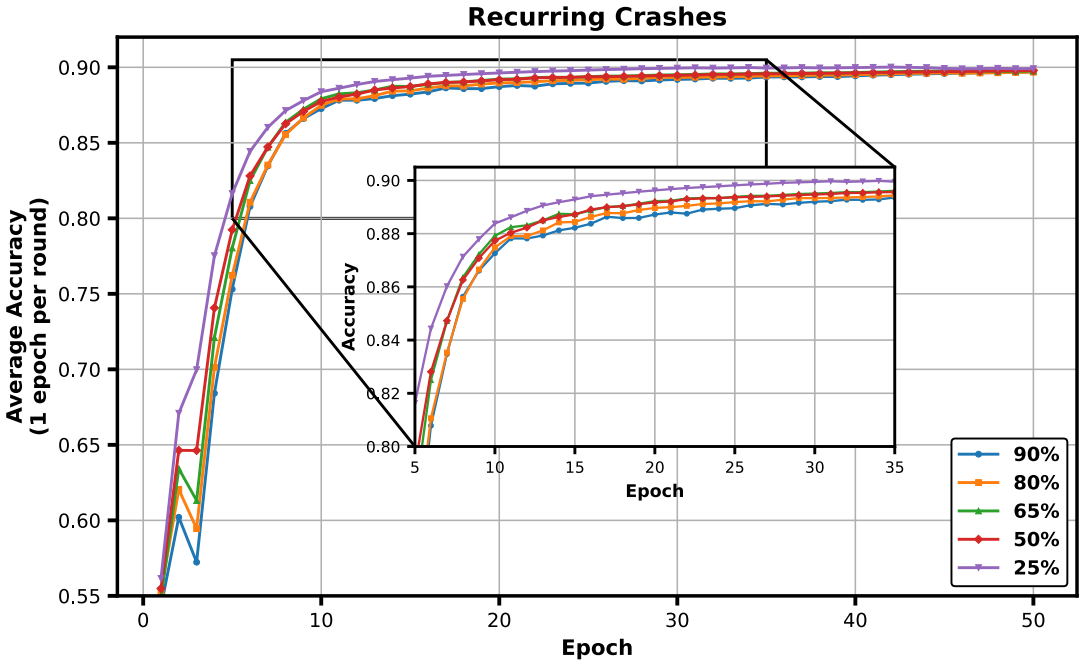

Figure 80: Accuracy evolution of FLAIR under recurring fault conditions (1 epoch per round).

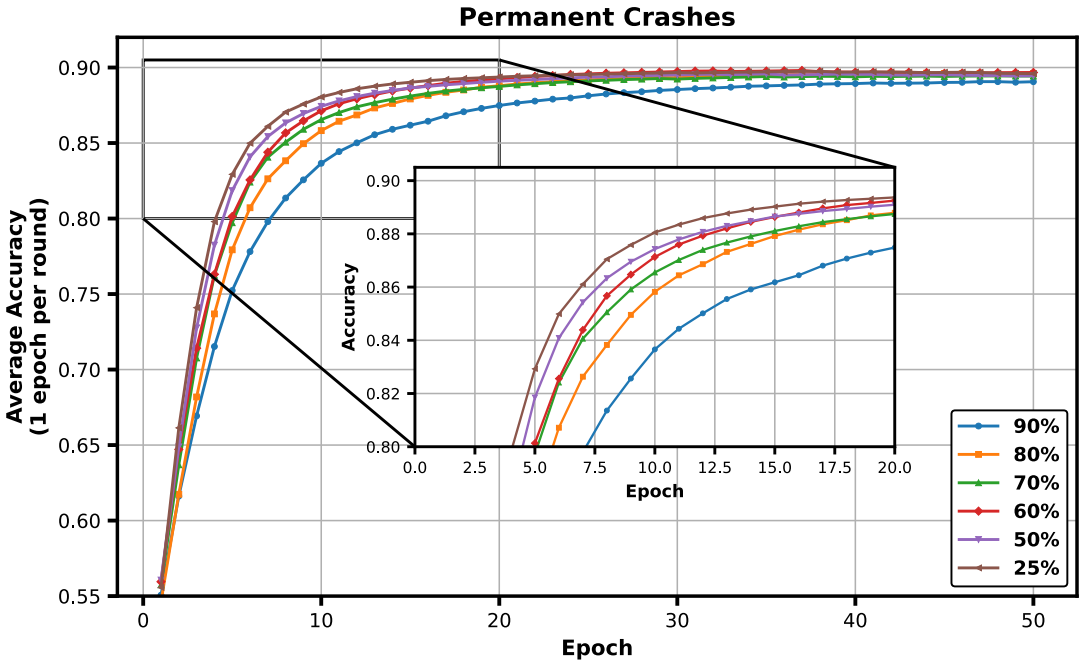

Figure 81: Accuracy evolution of FLAIR under permanent fault conditions (1 epoch per round).

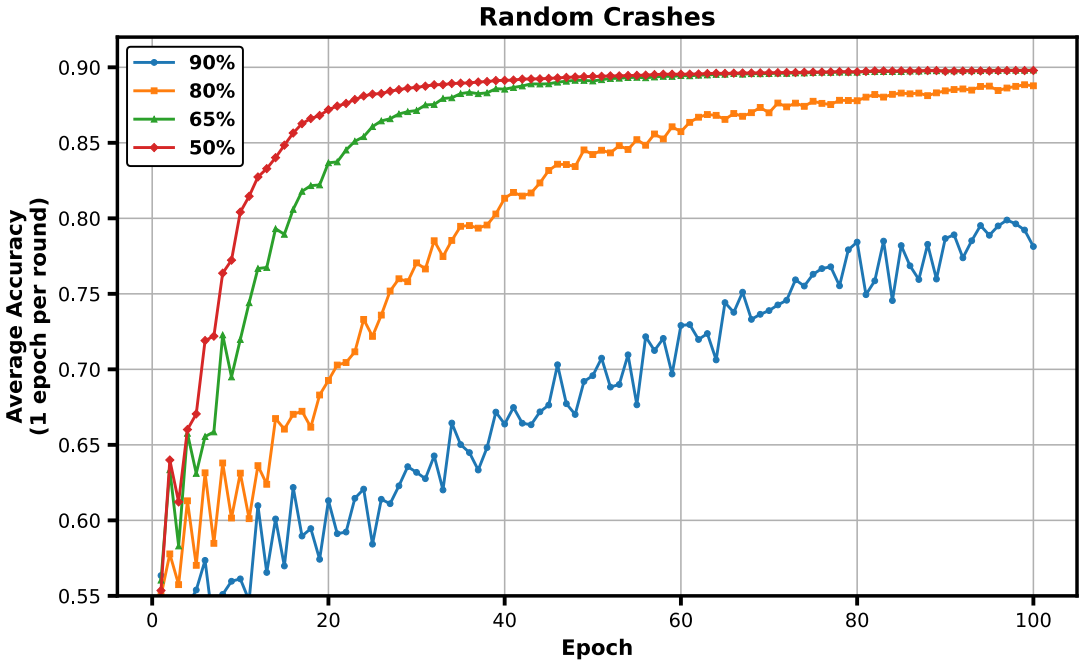

Figure 82: Accuracy evolution of FLAIR under random fault conditions (1 epoch per round).

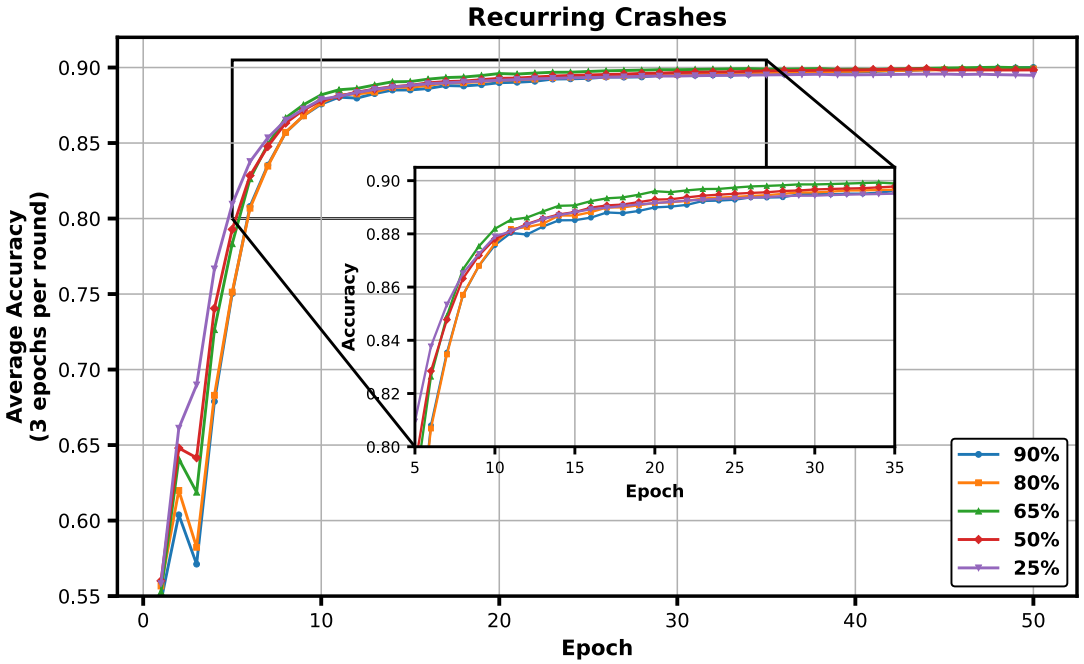

Figure 83: Accuracy evolution of FLAIR under recurring fault conditions (3 epochs per round).

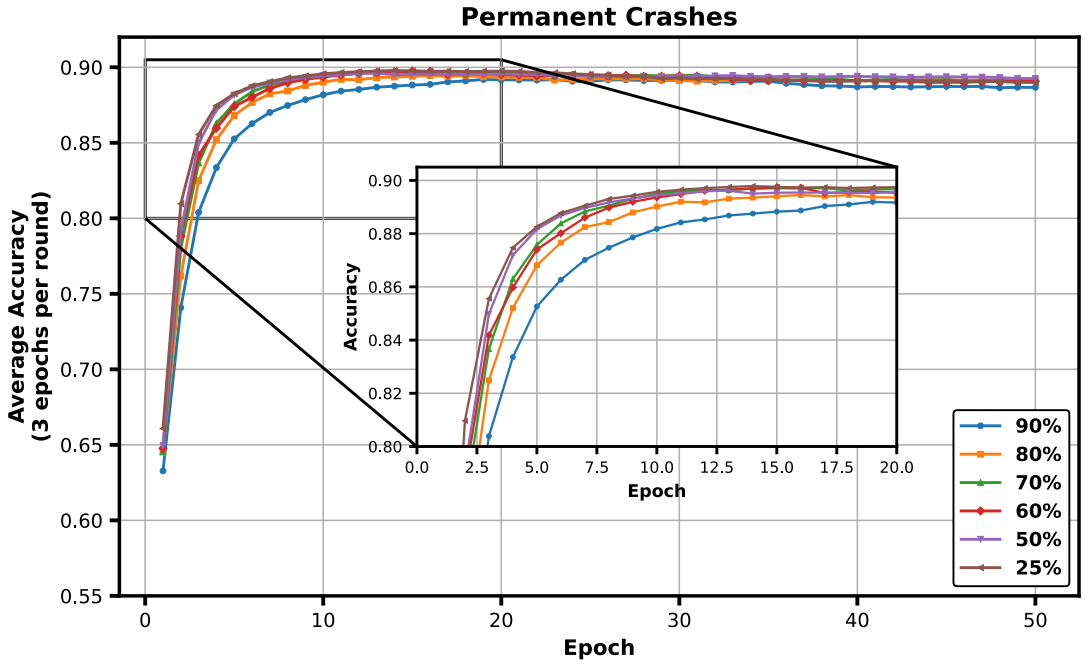

Figure 84: Accuracy evolution of FLAIR under permanent fault conditions (3 epochs per round).

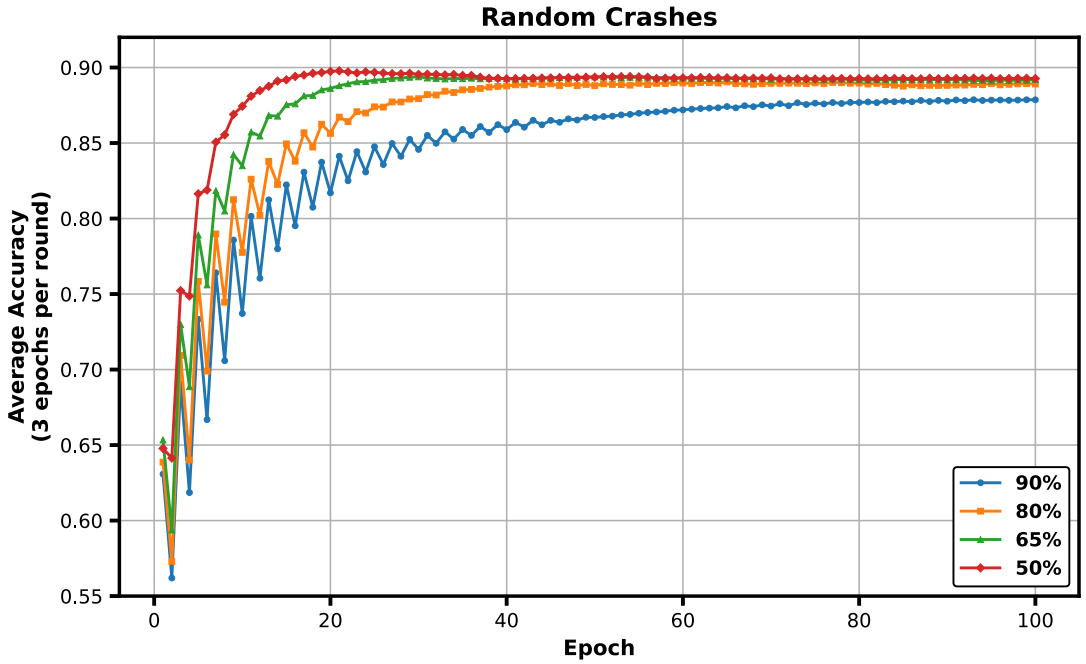

Figure 85: Accuracy evolution of FLAIR under random fault conditions (3 epochs per round).

Under permanent crashes, even when 90% of nodes were removed, the system still converged above 0.88, as summarized in Table 23, demonstrating graceful degradation. Convergence was slightly faster with longer rounds, as multiple local epochs helped amortize the impact of node removals.

Temporary crashes caused only short-lived perturbations, with accuracy rapidly recovering once nodes rejoined and maintaining a trajectory close to the baseline. With $E_{\text{round}} = 3$, the system quickly returned to baseline accuracy, whilst with $E_{\text{round}} = 1$, perturbations persisted longer and induced

minor oscillations. An interesting observation is that once accuracy stabilized, subsequent temporary crashes had only a marginal effect.

Random crashes proved the most disruptive, especially under the short-round configuration. At high dropout rates (e.g., 90%), convergence was significantly delayed and oscillations were frequent. In contrast, with $E_{\text{round}} = 3$, the dynamics were smoother and recovery more stable, since longer aggregation intervals absorbed much of the instability. Overall, FLAIR consistently maintained accuracy above 0.85 across all scenarios, confirming strong resilience even under extreme dropout conditions.

Table 23: Number of rounds required to reach 0.80 and 0.85 accuracy under different dropout scenarios and round duration settings.

Scenario	$E_{\text{round}} = 3$		$E_{\text{round}} = 1$	
	0.80	**0.85**	**0.80**	**0.85**
Permanent crashes (80%)	3	4	5	9
Temporary crashes (80%)	4	8	6	9
Random crashes (80%)	4	10	39	55
Permanent crashes (90%)	4	6	8	13
Temporary crashes (90%)	5	8	7	10
Random crashes (90%)	12	29	/	/

Impact of mobility on learning performance

Figure 86 and Figure 87 shows the accuracy evolution under both connectivity scenarios across five mobility models. Under perfect connectivity, mobility had no measurable impact on convergence speed or final accuracy, which remained comparable to the static network baseline. When communication was range-limited, occasional disconnections caused minor perturbations and slightly slower convergence, yet overall accuracy remained within 2% of the static case, staying above 0.88 for all mobility patterns. These results indicate that FLAIR is resilient to mobility effects and that its clustering mechanism effectively adapts to dynamic topologies.

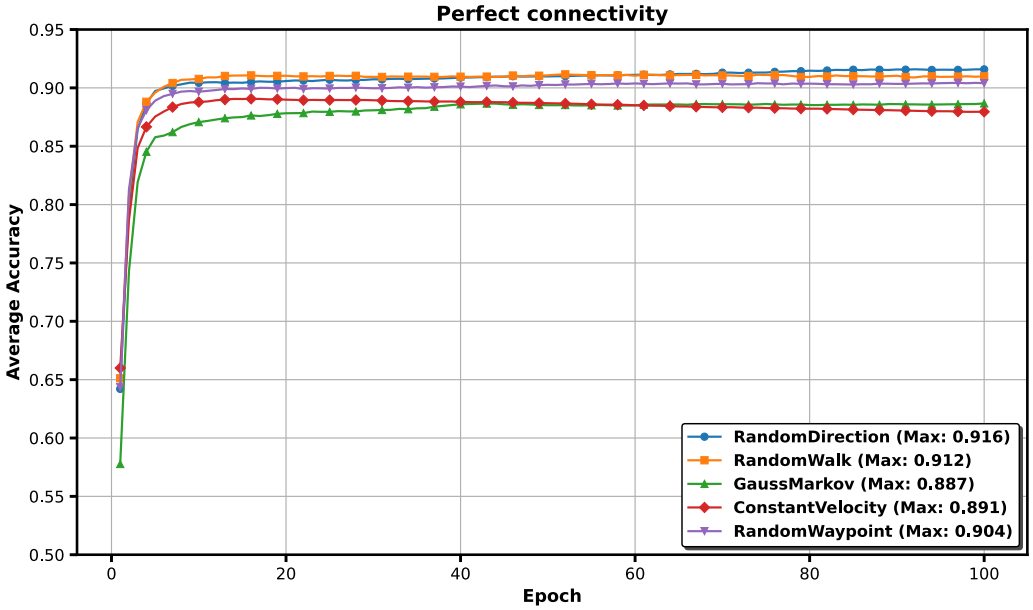

Figure 86: Accuracy evolution of FLAIR under five mobility patterns with perfect connectivity.

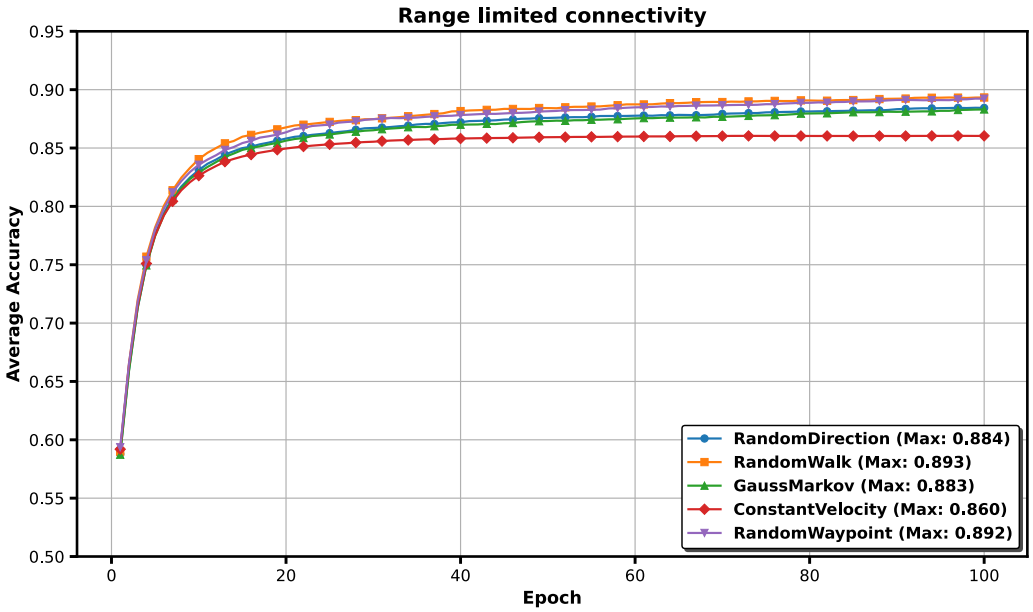

Figure 87: Accuracy evolution of FLAIR under five mobility patterns with range-limited connectivity.

Smart farming with heterogeneous nodes

Figure 88 shows the accuracy evolution under two local update settings. With $E_{\text{round}} = 3$, convergence is faster in early stages, exceeding 70% within 10 epochs, whilst $E_{\text{round}} = 1$ initially converges more slowly but eventually closes the gap. Both configurations converge near the centralized baseline of 71.9%, with final accuracies of 71.2% and 71.4% respectively.

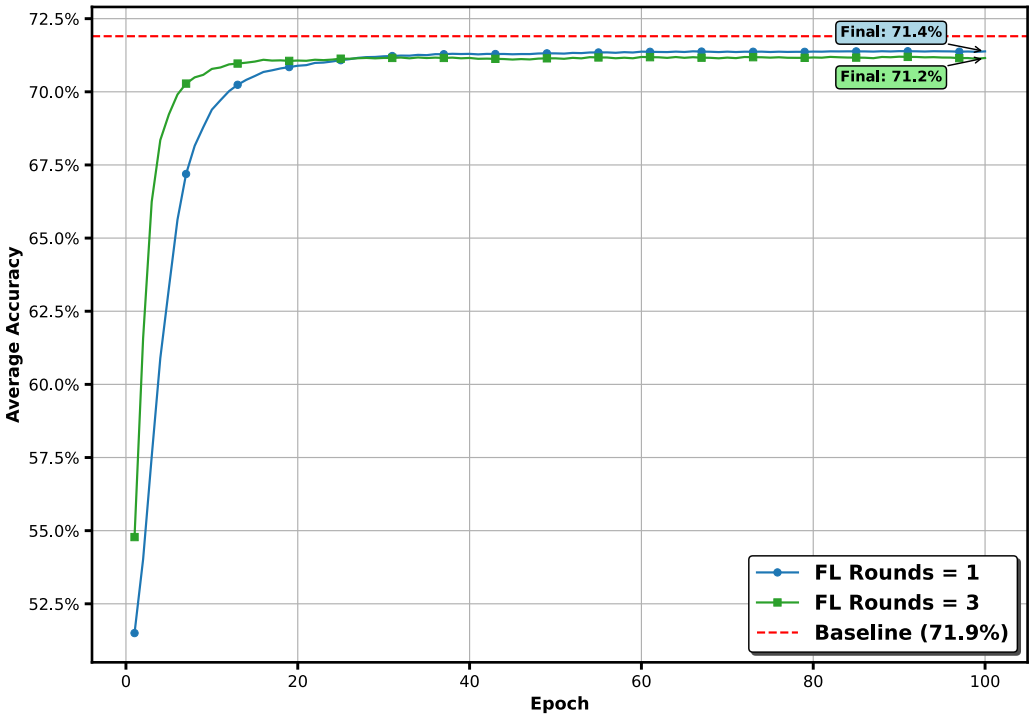

Figure 88: Accuracy evolution on the Watering the Plants dataset under the smart farming setup (80 fixed sensors + 20 mobile robots). Two local update settings ($E_{\text{round}} = 1$ vs. $E_{\text{round}} = 3$) are compared against the centralized baseline (71.9%).

To further validate robustness in realistic deployments, the dropout experiments from Section 6.2.4.2 were extended to the smart farming setup. The same failure types — permanent, temporary, and random crashes — were injected under the $E_{\text{round}} = 3$ setting, as shown in Figure 89, Figure 90 and Figure 91. These results confirm that the resilience properties identified in controlled static networks extend to heterogeneous, application-driven scenarios. Even in the presence of mobility and partial connectivity, FLAIR demonstrates graceful degradation and rapid recovery, underscoring its practicality for real-world IoT deployments.

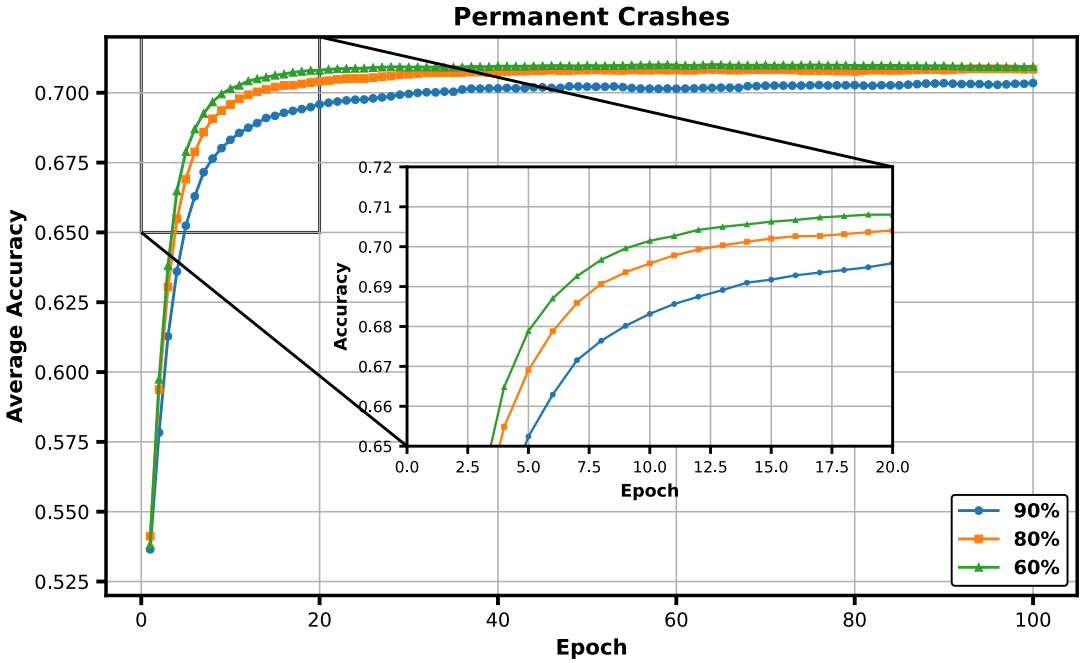

Figure 89: Accuracy evolution of FLAIR under permanent fault conditions in the smart farming scenario.

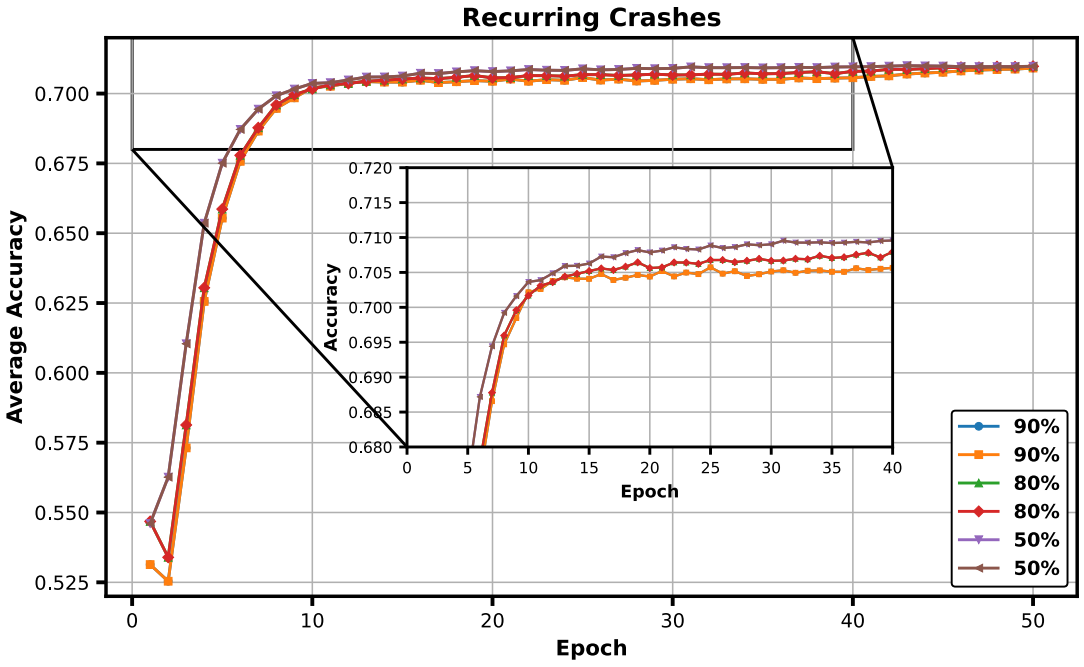

Figure 90: Accuracy evolution of FLAIR under recurring fault conditions in the smart farming scenario.

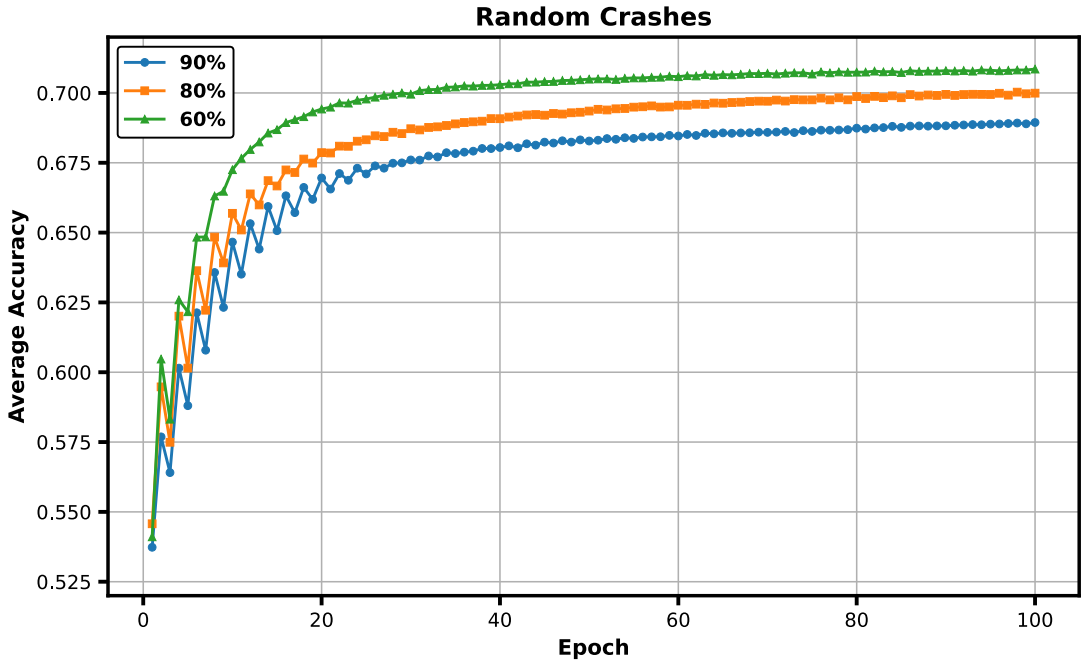

Figure 91: Accuracy evolution of FLAIR under random fault conditions in the smart farming scenario.

6.2.5 Summary

FLAIR demonstrates that the layered architecture introduced in this thesis is genuinely modular: by substituting the network layer, the overlay protocol, and the aggregation strategy independently of one another, a protocol well-suited to resource-constrained ad-hoc wireless networks emerges naturally from the same architectural foundations as HEAL.

The core contribution of FLAIR lies in its dynamic, resource-aware, and verifiable cluster-head election mechanism, which facilitates load balancing and enhances system robustness without requiring any fixed infrastructure. By confining aggregation to the cluster level and rotating cluster-heads at every round, FLAIR achieves progressive global model mixing without inter-cluster communication, keeping communication overhead low whilst preserving convergence.

The experimental evaluation conducted in ns-3 validated these properties across four scenarios. In static networks, FLAIR achieved faster convergence and higher final accuracy than C-FL, Gaia, HEAL, and Gossip Learning. Under extreme node dropout conditions — up to 90% of nodes failing — the protocol demonstrated graceful degradation, consistently maintaining accuracy above 0.85. Under all five mobility models considered, accuracy remained within 2% of the static baseline, confirming resilience to dynamic topologies. Finally, the smart farming experiment showed that FLAIR achieves performance near the centralized baseline (71.9%) in a heterogeneous IoT deployment with fixed sensors and mobile robots.

6.3 Conclusion

This chapter presented two original contributions to decentralized learning: HEAL and FLAIR. Both protocols were designed and evaluated as part of this thesis, and both instantiate the same layered architecture, demonstrating that cross-layer design is a principled and productive approach to building decentralized learning systems.

HEAL addresses the fundamental tension between convergence speed and fault resilience by leveraging the Elevator overlay to dynamically elect hub nodes as distributed aggregators. Simulation results on the MNIST dataset show that HEAL achieves 99% of the accuracy of Federated Learning whilst

remaining fully decentralized and fault-tolerant. With 5 hubs, HEAL reaches 0.95 accuracy in 76 cycles — comparable to Gaia and significantly faster than all gossip-based baselines. With 7 hubs and $s = 3$, this reduces to 33 cycles, making HEAL $2.3\times$ faster than the second-best result. HEAL continues to operate in the presence of node crashes, hub failures, and churn, with final accuracy remaining above 98% of the fault-free baseline in all tested scenarios. The communication overhead (210 messages per cycle) remains close to that of Federated Learning (198), and far below gossip-based alternatives.

FLAIR validates the modularity claim of the layered architecture by instantiating it in a different operational context: resource-constrained ad-hoc wireless networks. By substituting the Elevator overlay with a resource-aware, verifiable clustering protocol and confining aggregation to the cluster level, FLAIR yields a protocol with no fixed infrastructure and no inter-cluster coordination, yet competitive learning performance. In static networks, FLAIR surpasses all baselines in final accuracy. Under extreme dropout conditions of up to 90% node failures, it consistently maintains accuracy above 0.85. Under five mobility models, accuracy remains within 2% of the static baseline. In the smart farming scenario, FLAIR approaches the centralized baseline (71.9%) with final accuracies of 71.2% and 71.4% for $E_{\text{round}} = 1$ and $E_{\text{round}} = 3$ respectively.

The central finding of this chapter is that the layered architecture itself is the primary contribution: HEAL and FLAIR are two existence proofs that the same design principles — separation of concerns, modular substitution, and cross-layer composability — can yield protocols adapted to fundamentally different deployment contexts without redesigning the system from scratch. This opens a broad research avenue, as future protocols targeting other network environments (satellite networks, vehicular networks, underwater sensor networks) could be designed by composing existing or new layer implementations within the same framework.

Several directions remain open. Both protocols currently assume IID data distributions; extending them to non-IID settings through personalized aggregation or adaptive weighting is a natural next step. Reinforcing both protocols against Byzantine and adversarial nodes — including model poisoning and inference attacks — is equally important for real-world deployments. Finally, a formal convergence analysis of FLAIR under non-stationary cluster topologies, and of HEAL under non-IID data, would strengthen the theoretical foundations of both contributions.

Chapter 7

Conclusions and Outlook

7.1 General conclusions

This thesis was motivated by a fundamental question: can federated learning be conducted in a truly decentralised manner? The initial intuition pointed toward blockchain as a natural substrate for decentralised coordination. However, a systematic survey of the state of the art revealed an existing paradigm — gossip learning — that already eliminates the central server. The problem, well-documented in the literature, is that gossip learning consistently underperforms federated learning in both convergence speed and final model accuracy, precisely because it lacks any form of structured aggregation. This observation shifted the research question to a lower level of the stack: rather than modifying the learning algorithm itself, we asked whether the underlying communication topology could be engineered to recover the benefits of structured aggregation, while preserving full decentralisation. This led us to survey the peer sampling and overlay management literature, in search of protocols capable of shaping the network toward a topology that would support efficient global aggregation.

Several families of protocols have been proposed, yet none was designed with the explicit goal of structuring the overlay to support global aggregation — in particular, none considered the deliberate emergence of hub nodes as a design objective. Random-walk and multi-hop gossip protocols accelerate information propagation by reducing the effective diameter of the communication graph, but they do not produce a structured topology and their gains in practice are limited by increased message complexity and latency. Power-law overlay protocols produce degree distributions that are more favourable to fast dissemination, yet they fall short of the aggregation efficiency achievable with explicit hub nodes. Byzantine-resilient peer sampling protocols address a different concern — security rather than performance — and do not consider decentralised learning as a target use case. More broadly, very few peer sampling protocols have been designed with decentralised machine learning as an explicit operational context, as documented in Chapter 3.

The machine learning literature exhibits a complementary blind spot. The dominant research direction in distributed learning is federated learning, where a central server coordinates model aggregation; the question of full decentralisation receives comparatively little attention. Blockchain-based approaches do eliminate the central coordinator, but they rely on complete communication graphs and do not scale to large networks. Gossip learning protocols are scalable and genuinely decentralised, yet their lack of global aggregation leads to systematic underperformance relative to federated learning, as documented in Chapter 5. The result is a clear gap: no existing protocol simultaneously achieves decentralisation, scalability, and aggregation efficiency.

This gap shaped the central research question of the thesis: *how can the efficiency of gossip learning be improved so as to approach federated learning, without sacrificing decentralisation?*

The survey of the peer sampling literature confirmed our intuition that the answer lies not at the application layer but one level below, at the peer sampling layer: by shaping the communication topology, it is possible to introduce structured aggregation without relying on any pre-assigned coordinator. However, as documented in Chapter 3, no existing protocol was designed with this objective in mind — none considered the deliberate elevation of nodes to the role of hubs as a design goal. This absence left us no choice but to design such a protocol from scratch.

The result is **Elevator**, a decentralised peer-to-peer overlay protocol that organises nodes through a lightweight, self-organising random election mechanism. Crucially, Elevator is application-agnostic: it provides a general-purpose structured overlay that any distributed algorithm can exploit. The evaluation of Elevator rests on three complementary pillars. First, a theoretical analysis characterises the protocol's behaviour in terms of overlay properties — diameter, average shortest path length, and clustering coefficient — and derives analytical bounds on convergence and hub election dynamics. Second, these theoretical results are validated through large-scale simulations conducted on PeerSim, a peer-to-peer network simulator widely used and recognised by the research community. Third, a real implementation over TCP/IP confirms that the protocol behaves as predicted at the network level, bridging the gap between simulation and deployment. The consistency across all three levels of evaluation strengthens confidence in the correctness and robustness of the protocol.

This multi-level methodology did not come for free. Peer-to-peer networks with emergent hub structures of the kind introduced by Elevator are a novelty, and the research community has not converged on standard evaluation practices for such systems. A substantial methodological effort was therefore devoted to establishing appropriate scientific rigour: selecting and justifying the evaluation metrics, designing visualisations that faithfully represent the structural properties of the overlay, and adapting PeerSim — a simulator that is over twenty years old and was not designed with hub-based overlays in mind — to the requirements of this work. In particular, scaling simulations to networks of thousands or even hundreds of thousands of nodes, which is the operational target of a general-purpose peer-to-peer protocol, demanded significant engineering effort to make PeerSim tractable at that scale. The evaluation framework developed in the course of this work — comprising the selection of appropriate overlay metrics, the design of visualisations adapted to hub-based structures, and the engineering of a scalable simulation infrastructure — is itself a contribution to the community. In the absence of established practices for evaluating hub-based peer-to-peer overlays, this thesis proposes and justifies a methodology that future work in this area can build upon.

Building on Elevator, the **HEAL** protocol (Hub Enhanced Adaptive Learning) instantiates federated learning directly atop the structured overlay. Hubs serve as aggregators, and the inter-hub communication ensures that locally aggregated models are further reconciled across the network. This two-level aggregation scheme is the mechanism by which HEAL recovers the efficiency of federated learning while preserving decentralisation.

The evaluation of HEAL relies on simulation, but the machine learning component is not simulated: multiple real model architectures are trained on several benchmark datasets covering different learning tasks, ensuring that the measured accuracy figures reflect genuine model behaviour rather than approximations. The combination of diverse models, datasets, and task types provides strong evidence that the results are not artefacts of a particular experimental setup. Accuracy — a standard metric in the federated learning literature — is complemented by overlay topology metrics inherited from the Elevator evaluation, giving a joint view of network-level and learning-level performance. HEAL achieves competitive accuracy under fault-free conditions, and its resilience is confirmed under both crash failures and churn, owing to the redundant hub submission mechanism. Integrating the machine learning workload into large-scale peer-to-peer simulations further required adapting the simulation infrastructure to handle the additional computational and memory demands of real model training within a distributed event-driven framework.

The HEAL framework was subsequently extended in two directions, each addressing a different deployment constraint. The **Lift** protocol addresses a subtle but critical vulnerability in Elevator: the hub election mechanism is precisely the highest-value attack surface in the stack, since an adversary that controls hub election effectively controls the entire aggregation process. A systematic study of Byzantine resilience in Elevator reveals that the protocol withstands a range of adversarial behaviours

without modification — including passive byzantines that return empty neighbour lists, single active byzantines that inflate their own election probability by self-reporting, and multiple non-colluding byzantines employing the same strategy. However, when byzantines collude — sharing knowledge of one another and returning fellow byzantines in their neighbour lists — as few as two percent of malicious nodes are sufficient to guarantee that all byzantines are elected as hubs, fully compromising the aggregation layer. Lift addresses this threat through a dedicated defence mechanism at the overlay level, extending the collusion tolerance threshold to ten percent of byzantine nodes while preserving the decentralised nature of the election process. Beyond this threshold, colluding byzantines are again able to capture hub roles, which defines the boundary of the current security guarantee. Crucially, Lift operates entirely at the peer sampling layer and is transparent to the learning layer: HEAL requires no modification to benefit from the stronger security guarantees provided by the hardened overlay.

The **FLAIR** protocol (Federated Learning with Adaptive Integrity-preserving Randomness) validates the modularity of the layered architecture by instantiating it in a fundamentally different operational context: resource-constrained ad-hoc wireless networks, where the overlay topology is shaped by radio range, interference, and node mobility rather than by an explicit election mechanism. In place of Elevator, FLAIR employs a resource-aware cluster-head election mechanism inspired by LEACH, in which each node associates with the cluster head from which it receives the strongest signal. Aggregation is confined to the cluster level, and cluster heads rotate at every round, so that progressive global model mixing emerges from topology dynamics alone, without any inter-cluster coordination. The evaluation on the ns-3 network simulator validates these properties across static, fault-prone, mobile, and heterogeneous deployment scenarios. Taken together, HEAL and FLAIR constitute two existence proofs that the layered architecture can yield protocols adapted to fundamentally different deployment contexts — a result that opens a broad design space for future protocols targeting other network environments such as satellite, vehicular, or sensor networks.

Considered as a whole, the contributions of this thesis establish a coherent framework for decentralised, efficient, and resilient machine learning over peer-to-peer networks. The progression from the state of the art, through Elevator, to HEAL, Lift, and FLAIR constitutes a principled answer to the motivating question: structured aggregation can be achieved without centralisation, provided that the overlay layer is designed with that goal in mind.

It is worth reflecting on the scope and conditions under which this answer holds. The results established in this thesis are most conclusive under conditions that approximate the idealised setting of federated learning: data distributions that are not severely heterogeneous, network scales of up to one thousand nodes for the overlay layer and one hundred nodes for the learning layer, and fault rates that remain moderate. Under these conditions, HEAL demonstrably closes the accuracy gap with centralised federated learning while operating without any coordinator. Beyond these conditions — in the presence of strongly non-IID data, at very large scales, or under sustained Byzantine attack at the learning layer — the framework provides resilience mechanisms and encouraging empirical results, but formal guarantees are not yet established. The contributions of this thesis should therefore be understood as a proof of concept and a rigorous foundation, rather than a fully deployed solution: they demonstrate that structured aggregation without centralisation is achievable and practically effective, and they identify precisely the conditions under which the approach succeeds and the boundaries beyond which further work is needed.

More broadly, this thesis should be read not as an incremental contribution to an existing body of work, but as the opening of a new research territory. By combining a novel peer sampling layer with a modular, application-agnostic learning architecture, the framework departs from both the federated learning tradition — which has largely taken centralisation for granted — and the gossip learning tradition — which has accepted limited aggregation efficiency as an inherent constraint. The resulting

design space, in which overlay structure and learning performance are co-designed from the ground up, had not, to the best of the author's knowledge, been explored in a principled way prior to this work. The protocols, evaluation methodology, and open problems identified in this work collectively define a new research agenda, and it is the author's hope that they will serve as a foundation for a broader community effort toward truly decentralised, efficient, and trustworthy machine learning over peer-to-peer networks.

> "Diviser chacune des difficultés que j'examinerais en autant de parcelles qu'il se pourrait et qu'il serait requis pour les mieux résoudre."
>
> — René Descartes, *Discours de la méthode*, 1637

7.2 Limitations

Despite the contributions presented in this thesis, several limitations must be acknowledged.

The theoretical analysis of Elevator's convergence relies on idealised assumptions: the absence of failures during overlay construction, and several reliability assumptions on the underlying physical network, such as reliable message delivery and stable connectivity. Relaxing these assumptions — for instance, by accounting for transient link failures or message loss at the network layer — would yield a more realistic characterisation of the protocol's behaviour and remains an open theoretical problem.

On the simulation side, the Elevator experiments do not model the underlying network layer, abstracting away latency, bandwidth, and packet loss. Furthermore, the largest simulations conducted in this work reached networks of one thousand nodes; while this is sufficient to observe the structural properties of the overlay, a general-purpose peer-to-peer protocol should ideally be validated at scales of tens or hundreds of thousands of nodes. Reaching such scales would require a more capable simulation infrastructure; this is precisely the motivation behind the new unified simulator currently under development, which is described among the near-term research directions. Regarding fault tolerance, the fault scenarios considered remain moderate in intensity; higher fault rates and more sustained churn patterns would stress-test the resilience mechanisms more thoroughly. Similarly, the Byzantine fault experiments consider relatively straightforward attack strategies; more sophisticated adversarial behaviours — such as coordinated collusion or adaptive attacks — are left for future work.

The real network experiments over TCP/IP were conducted on a modest testbed in terms of both node count and geographic distribution. A larger and more geographically dispersed deployment would provide stronger evidence of the protocol's behaviour under realistic wide-area network conditions.

The HEAL experiments were limited to networks of one hundred nodes and to relatively simple machine learning models. Evaluating HEAL with larger node populations and more complex model architectures — such as deeper convolutional networks or transformer-based models — would better reflect the demands of real-world decentralised learning deployments and would allow a more rigorous assessment of the framework's scalability at the learning layer. Beyond model complexity, the evaluation relies solely on accuracy as a performance metric; complementary measures such as communication cost, convergence speed, or energy consumption would provide a more complete picture of the framework's practical efficiency. Furthermore, no formal convergence guarantee is established for HEAL: the observation that the protocol reaches competitive accuracy levels is empirical, and a theoretical proof of convergence — even under simplified assumptions — remains an open problem.

The experimental methodology also presents limitations. Each experimental configuration was evaluated over only five simulation runs, which may be insufficient to fully account for the variance introduced by random initialisation and stochastic training dynamics; a larger number of runs would strengthen the statistical reliability of the reported results. The range of configurations explored is

likewise limited: the number of hubs, the number of local training rounds between successive aggregation phases, and the role of hubs as potential training participants were not systematically varied. A broader parameter study would clarify the sensitivity of HEAL's performance to these design choices, some of which were partially explored in the context of FLAIR.

All experiments were conducted in simulation, using a cycle-based notion of time that does not reflect real execution costs. Deploying HEAL on a physical testbed would allow the actual overhead of the inter-hub aggregation phase to be measured — in terms of wall-clock time and network traffic — and would enable a direct comparison with baseline algorithms under identical hardware and network conditions. At a more fundamental level, the current design assumes that the inter-hub aggregation phase is synchronous, meaning that all hubs must coordinate within a shared round boundary. Achieving a fully asynchronous variant of HEAL — where hubs aggregate independently without any global synchronisation primitive — remains an open challenge, and would remove the implicit dependency on an external coordination mechanism, whether centralised or distributed.

The Lift protocol, while effective up to a collusion threshold of ten percent of byzantine nodes, presents several limitations. The adversarial model considered remains relatively constrained: byzantine nodes operate independently or collude as a single coordinated group, but more sophisticated threat models are not addressed. Furthermore, Lift hardens only the overlay layer: the learning layer remains unprotected against poisoning attacks, in which malicious nodes submit manipulated model updates, and against privacy attacks such as gradient inversion or membership inference. The security guarantees of Lift should therefore be understood as a necessary but not sufficient condition for deploying HEAL in fully adversarial environments.

The FLAIR protocol presents its own set of limitations. Several are shared with HEAL: the experimental evaluation assumes IID data distributions across nodes, the behaviour of the protocol under non-IID conditions remains an open question, and no formal convergence guarantee is established under non-stationary cluster topologies — as cluster heads rotate and node associations change, the conditions under which the global model converges to a good solution are not theoretically characterised. Beyond these shared limitations, the network model underlying the FLAIR evaluation remains simplified despite the use of ns-3. Message loss, message corruption, propagation delay, response latency, and the impact of message size on communication overhead are not explicitly accounted for in the experimental setup. While ns-3 provides higher-fidelity modelling of wireless channel conditions than the simulators used for HEAL, the evaluation still abstracts away several aspects of real wireless network behaviour that could significantly affect protocol performance in deployment.

7.3 Outlook

The work presented in this thesis opens several research directions, spanning immediate extensions of the existing protocols to longer-term theoretical and applied challenges.

In the near term, two directions are already under active investigation. The first concerns the hub election mechanism in Elevator. In its current form, election is based on a random process, which ensures fairness but does not account for the heterogeneity of nodes in real deployments — nodes differ widely in computational capacity, memory, bandwidth, and availability. An ongoing line of work extends the election mechanism to incorporate node capability metrics, so that hubs are elected with a bias toward the most capable participants while preserving the decentralised nature of the process. Preliminary results are encouraging and suggest that capability-aware election yields overlays with better sustained aggregation throughput. A complementary direction explores the sensitivity of Elevator to the initial network topology. The current protocol is initialised from a random graph; preliminary experiments investigating whether Elevator converges toward a hub-based structure from alternative starting topologies — such as ring graphs or scale-free networks — have yielded encouraging results,

and a systematic study of this convergence behaviour is underway. In parallel, ongoing work targets the efficiency of the Elevator protocol itself: reducing message complexity, lowering memory overhead, and simplifying the algorithm without compromising its structural guarantees. These optimisations are in progress and have not yet been fully validated.

The second near-term direction concerns data heterogeneity in HEAL. The current aggregation scheme does not account for statistical heterogeneity across nodes, which can cause the global model to drift away from the local data distributions of individual participants. Adapting the aggregation strategy — for instance through personalisation or locally-weighted aggregation — to mitigate this effect is a natural and pressing extension.

A third near-term priority is the development of a new simulation infrastructure. The experimental work in this thesis relied on two separate simulators — PeerSim for the peer sampling layer and Gossipy for the machine learning layer — whose integration introduced both engineering complexity and methodological constraints. A new simulator, written from scratch in Python, is currently under development. It is designed to handle peer sampling, dynamic network evolution, and machine learning workloads within a single unified framework, using NumPy for numerical computation. The objective is twofold: to achieve better performance than the existing tools at the scales of interest, and to provide a simpler and more accessible experimentation platform for future work in decentralised learning.

At medium term, several challenges stand out. The first is the integration of robustness against Byzantine attacks into HEAL itself. While Lift addresses Byzantine resilience at the overlay level, the learning layer remains vulnerable to poisoning attacks, in which malicious participants submit manipulated model updates, as well as to privacy attacks such as gradient inversion or membership inference. Equipping HEAL with defences against these threat models — whether through robust aggregation rules, differential privacy mechanisms, or secure aggregation protocols — is a necessary step toward deployment in adversarial environments. This hardening effort extends naturally to the Elevator layer itself: adding message encryption and digital signatures would protect against Byzantine participants attempting to manipulate inter-node communication, and deploying the stack over a privacy-preserving overlay such as Tor would further protect participant identities and message confidentiality.

The second medium-term objective is a large-scale real-world deployment of Elevator and HEAL. Such a deployment would validate the scalability assumptions made in the simulation studies — in particular the behaviour of the protocol at node counts that could not be reached within the simulation infrastructure — and would expose practical challenges such as churn patterns, network heterogeneity, and hardware diversity that are difficult to reproduce faithfully in simulation.

A third medium-term direction is the evaluation of HEAL with large-scale machine learning models. The experiments in this thesis were deliberately limited to lightweight architectures in order to keep simulation tractable. Assessing whether HEAL can sustain the aggregation of models with hundreds of millions of parameters — of the scale of contemporary large language models — would expose fundamental limitations in terms of communication bandwidth, aggregation latency, and memory requirements at hub nodes, and would motivate targeted protocol adaptations for this regime.

In the longer term, several directions define a broader research programme. First, the scope of HEAL is currently limited to supervised learning tasks; extending the framework to unsupervised learning and reinforcement learning would significantly broaden its applicability, particularly for settings where labelled data is scarce or where agents must learn from interaction rather than from a fixed dataset. Second, HEAL operates under the horizontal federated learning assumption, where all participants share the same feature space. Adapting the framework to vertical federated learning — where different

participants hold different features of the same instances — raises fundamentally different challenges in terms of aggregation, communication, and privacy, and would open the framework to a wider class of collaborative learning scenarios. Third, and most fundamentally, establishing formal convergence guarantees for HEAL remains an open theoretical problem. While the limitations section establishes that the current results are purely empirical, a rigorous convergence analysis would need to go further: beyond confirming the empirical observations of this thesis, such guarantees would need to account for realistic operating conditions — partial participation, heterogeneous data distributions, and dynamic network topologies — and would provide actionable bounds on the number of aggregation rounds required to reach a target model quality.

A fourth long-term challenge is the design of an incentive mechanism to reward active participation and deter free-riding. In any open peer-to-peer system, nodes may choose to consume the outputs of collaborative learning without contributing computational resources or local data. Addressing this problem requires a mechanism that can attribute contributions, measure effort, and distribute rewards in a decentralised manner. One approach is to design such a mechanism as a standalone protocol layer within the existing stack. An alternative — and potentially complementary — direction is to integrate HEAL with a blockchain system or a layer-two payment network such as the Lightning Network, so that node contributions are rewarded through on-chain or off-chain transactions. Such a hybrid architecture would combine the scalability of the HEAL learning stack with the trust and incentive guarantees of a decentralised ledger.

Finally, a long-term applied objective is the deployment of the full stack — Elevator, HEAL, and associated security and incentive layers — in a concrete industrial use case. Domains such as autonomous vehicles and drone swarms present particularly demanding requirements: high mobility, stringent latency constraints, heterogeneous hardware, and safety-critical model quality. Designing an end-to-end system architecture for such a setting, and validating it under realistic operational conditions, would constitute a significant step toward the industrial deployment of fully decentralised machine learning.

> "Our knowledge can only be finite, while our ignorance must necessarily be infinite."
>
> — Karl Popper

Publications

The works presented in this manuscript gave rise to multiple publications.

The contributions of Chapter 4 have been published in the proceedings of the following international conferences:

[106] M. A. Legheraba, M. Potop-Butucaru, and S. Tixeuil, "Brief Announcement: A Self-* and Persistent Hub Sampling Service," in *International Symposium on Stabilizing, Safety, and Security of Distributed Systems*, 2024, pp. 461–465.

[107] M. A. Legheraba, M. Potop-Butucaru, S. Tixeuil, and S. Fdida, "Emergent Peer-to-Peer Multi-Hub Topology," in *2024 22nd International Symposium on Network Computing and Applications (NCA)*, 2024, pp. 219–226.

[108] M. A. Legheraba, N. Rachdi, M. Potop-Butucaru, and S. Tixeuil, "LIFT: Byzantine Resilient Hub-Sampling," in *2025 Thirteenth International Symposium on Computing and Networking (CANDAR)*, 2025, pp. 1–9.. This publication received the **Outstanding Paper Award** in its category.

Part of the contributions of Chapter 6 have been published in the proceedings of the following international and national conferences:

[109] M. A. Legheraba, S. Galkiewicz, M. Potop-Butucaru, and S. Tixeuil, "HEAL: Resilient and Self-* Hub-based Learning," in *International Conference on Advanced Information Networking and Applications*, 2025, pp. 200–211.

The contributions of Chapter 4 and Chapter 6 have also been presented in the proceedings of the following French national congresses, respectively:

[110] M. A. Legheraba, M. Potop-Butucaru, S. Tixeuil, and S. Fdida, "Des noeuds anonymes hissés au rang de stars," in *ALGOTEL 2025–27èmes Rencontres Francophones sur les Aspects Algorithmiques des Télécommunications*, 2025.

[111] M. A. Legheraba, S. Galkiewicz, M. Potop-Butucaru, and S. Tixeuil, "Les étoiles montantes de l'apprentissage distribué," in *ALGOTEL 2025–27èmes Rencontres Francophones sur les Aspects Algorithmiques des Télécommunications*, 2025.

Appendix

A.1 Common network topologies

Network topology refers to the structural organization of a network: the way nodes are interconnected and how links are arranged between them. Different topologies lead to fundamentally different properties in terms of connectivity, robustness, routing efficiency, and scalability. In the context of overlay networks, topologies are not viewed as static structures but as reference models describing the possible shapes of snapshot graphs $G(t)$ induced by peer-to-peer protocols over time.

Network topologies can be broadly divided into two categories. **Deterministic topologies** are defined by explicit construction rules that fully determine the presence of each edge from the position or role of nodes — stars, rings, trees, grids, and meshes fall into this category. **Random topologies** are generated according to probabilistic rules, and include classical random graphs as well as more advanced models such as small-world networks, power-law networks, and stochastic block models.

Mesh

A mesh network topology corresponds to a complete graph, in which every node is directly connected to every other node in the network. This topology offers optimal communication properties, as any node can reach any other node in a single hop, resulting in a graph diameter equal to one and minimal latency for message dissemination. Such full connectivity also provides high redundancy, making the network inherently robust to individual link failures. However, these advantages come at a prohibitive cost in large-scale systems. Each node must maintain a connection with all other nodes, leading to a quadratic growth in the number of links and significant overhead in terms of bandwidth, memory, and connection management. Moreover, a mesh topology requires each node to know the complete list of participants in the network, which is impractical or impossible in dynamic environments where nodes frequently join and leave. As a result, mesh networks are inherently static and do not scale well, limiting their applicability to small, tightly controlled systems rather than large peer-to-peer or highly dynamic overlay networks.

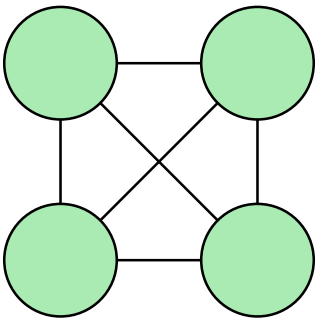

Figure 92: A mesh network with 4 nodes.

Star

A star network topology corresponds, from a graph-theoretic perspective, to a graph in which all nodes are connected to a single central node, often referred to as the hub. This topology naturally maps to a client–server architecture, where the central node acts as a server and all other nodes act as clients. Communication between any two clients must pass through the central node, which results in a graph diameter equal to two and enables fast message exchanges with low hop count. The simplicity of this topology makes it easy to deploy, manage, and control, and clients only need to maintain a single connection to participate in the network. However, the central node constitutes a critical bottleneck, as it must handle all communications and can become overloaded as the network

grows. More importantly, it represents a single point of failure: if the server crashes, is disconnected, or is compromised, the entire network becomes unavailable.

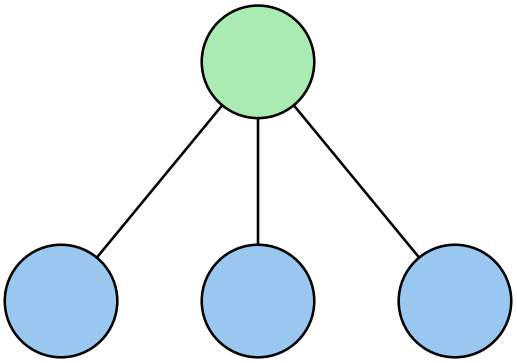

Figure 93: A star topology.

Multi-star

A multi-star topology can be seen as an extension of the star topology in which several central nodes coexist, each forming a local star with a subset of clients. From a graph-theoretic point of view, this corresponds to a collection of star subgraphs that may or may not be interconnected. This topology is widely used in cloud systems, for instance in distributed databases where a primary server is supported by one or more replica servers that can take over in case of failure or overload. Multi-star architectures are also common in geographically distributed systems, where services are replicated across multiple regions to reduce latency and provide better quality of service to users worldwide. Several variants of multi-star topologies exist: in some designs, clients are connected to all available servers, while in others each client is connected to a single server at a time; similarly, servers may be fully interconnected, partially connected, or completely isolated from each other. Compared to a single-star topology, multi-star networks improve scalability, fault tolerance, and availability, but they still rely on centralized components at the level of each star. As a result, they remain more structured and less decentralized than peer-to-peer topologies, and require coordination mechanisms for load balancing, leader election, or consistency among servers.

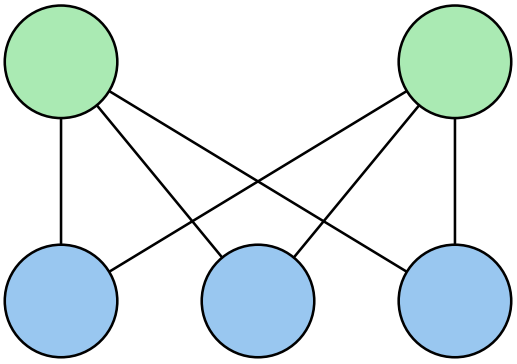

Figure 94: A multi-star topology, with 2 servers and 3 clients. Each client is connected to each server.

Ring

A ring topology corresponds to a graph in which each node maintains exactly two connections, typically referred to as its left and right neighbors, forming a closed cycle. From a graph-theoretic perspective, this structure is a simple cycle graph. Ring topologies require coordination mechanisms between nodes, as well-defined rules are needed to handle the addition or removal of one or more nodes without breaking the ring and disconnecting the network. In particular, join and leave operations must ensure that neighbor relationships are consistently updated to preserve connectivity. The diameter

of a ring network grows linearly with the number of nodes, i.e., it is proportional to N, which leads to potentially high communication latency as information may need to traverse many intermediate nodes. To mitigate this limitation, many ring-based systems introduce additional long-range links, often inspired by skip lists, allowing nodes to bypass large portions of the ring and significantly reduce routing and propagation times. Such enhancements improve efficiency while preserving the simplicity and locality properties of the underlying ring structure.

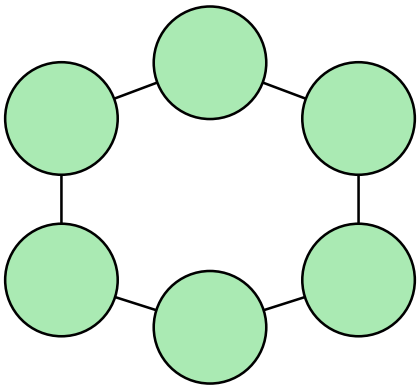

Figure 95: A ring topology, with 6 nodes.

Grid / Lattice

A grid (or lattice) network topology arranges nodes in a regular d-dimensional structure, where each node is connected only to its immediate neighbors along each dimension. In a d-dimensional lattice, each interior node has exactly $2d$ neighbors: two per dimension, corresponding to its left and right neighbors along each axis. The most common case encountered in practice is the two-dimensional grid, where each interior node has exactly four neighbors, while boundary and corner nodes have three and two neighbors respectively.

It is worth noting that the one-dimensional grid corresponds exactly to the ring topology discussed earlier, where each node is connected to exactly two neighbors forming a closed cycle. At the other extreme, the **hypercube** is a d-dimensional grid over $N = 2^d$ nodes, in which each node has exactly $d = \log_2 N$ neighbors. This topology offers a favorable trade-off between degree and diameter: the diameter grows as $O(\log N)$, significantly better than the two-dimensional grid while maintaining a moderate and uniform degree.

The local nature of connections in grid topologies comes at the cost of communication efficiency. The diameter of a two-dimensional $n \times n$ grid grows as $O(n) = O\left(\sqrt{N}\right)$, meaning that messages may need to traverse many hops to reach distant nodes. Information dissemination and aggregation processes are therefore significantly slower than in mesh or scale-free topologies.

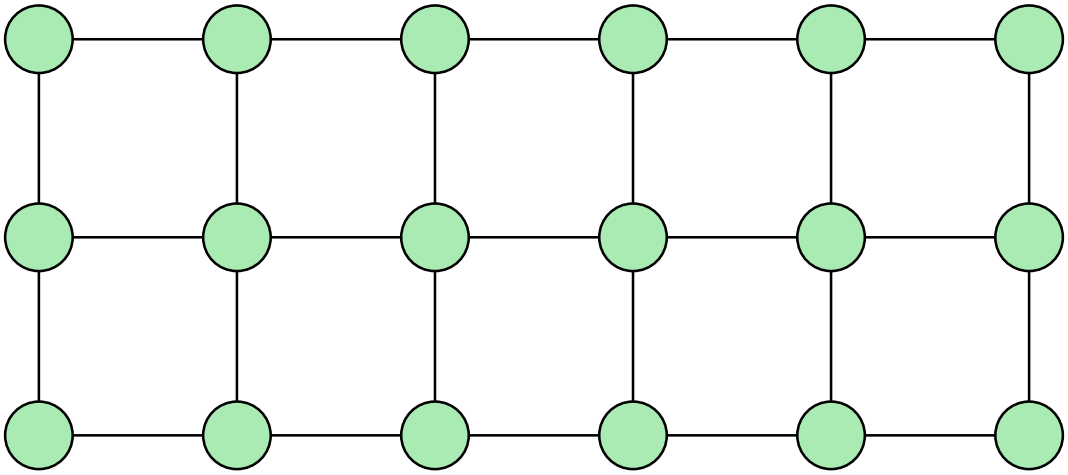

Figure 96: A 6×3 two-dimensional grid network.

Hierarchical

A hierarchical network topology corresponds to a graph structured as a tree, where nodes are organized into different levels with parent–child relationships. This topology is commonly used in large-scale systems such as the Domain Name System (DNS), which relies on a hierarchical structure of authorities, ranging from root servers at the top level to top-level domain servers and authoritative name servers below. Compared to a star topology, hierarchical networks scale more effectively, as intermediate nodes distribute and absorb part of the workload, reducing the burden on any single central node. However, this topology remains largely static and is inherently fragile to failures. If an intermediate node fails, all its descendants become disconnected from the rest of the network, and a failure at the root level can impact the entire system. As a result, hierarchical topologies often require additional mechanisms such as redundancy, replication, or failover strategies to improve fault tolerance and availability.

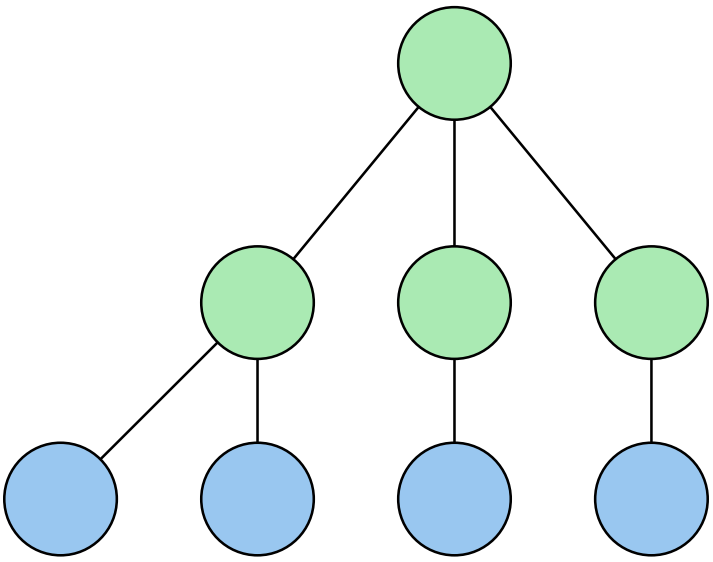

Figure 97: A hierarchical (or tree) topology, with 2 levels, the root server and the intermediate servers.

Random network

In contrast to the deterministic topologies presented above, real-world networks are often not explicitly organized but instead emerge in a largely random manner. Social networks, for instance, are formed through independent and uncoordinated interactions between individuals, leading to structures that

are difficult to predict or control globally. To model such systems, random network models have been widely studied, among which the Erdős–Rényi random graph [112] is the most classical and intuitive. In this model, edges are created at random, either by fixing the probability of connection between any pair of nodes or by fixing the expected number of connections per node. Despite the apparent lack of structure, random graphs exhibit several desirable properties. When the average degree k is greater than a small constant (typically slightly above 2), the probability that the graph is connected rapidly approaches one as the network size grows. Moreover, the diameter of the graph remains relatively small, scaling logarithmically with the number of nodes, which ensures efficient information propagation. In the directed case, k usually denotes the out-degree of each node, while the in-degree follows a binomial distribution centered around k. These properties make random graphs attractive as baseline models for large-scale decentralized systems, even though they do not capture heterogeneity or hub formation observed in many real networks.

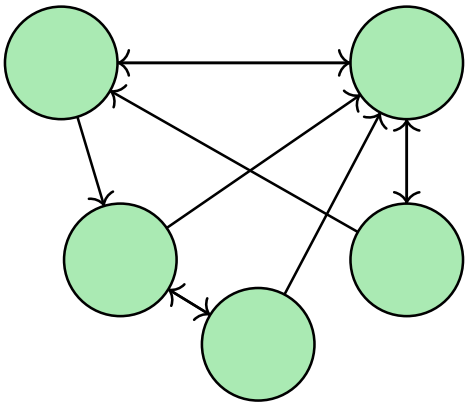

Figure 98: A random directed graph, $k = 2$, with k the outdegree of each node.

Definition 1.1.1 (Erdős–Rényi Random Graph G(n, p))

Let $n \in \mathbb{N}$ be the number of vertices and $p \in [0, 1]$. An Erdős–Rényi random graph $G(n, p)$ is defined as a random graph $G = (V, E)$ where:

- $V = \{1, 2, ..., n\}$ is the set of vertices;
- for every unordered pair $\{i, j\} \subset V$ with $i \neq j$, the edge $\{i, j\}$ is included in E independently with probability p:

$$\forall i \neq j, \quad \Pr(\{i, j\} \in E) = p.$$

♣

Definition 1.1.2 (Directed Random Graph with Fixed Outdegree)

Let $n \in \mathbb{N}$ be the number of nodes and $k \in \mathbb{N}$ such that $k < n$. A directed random graph $G = (V, E)$ with fixed outdegree k is defined as follows:

- $V = \{1, 2, ..., n\}$;
- for each node $i \in V$, exactly k outgoing edges are created;
- the k distinct destination nodes are selected uniformly at random from V, without replacement.

Formally, for each node i, the set of outgoing neighbors $N_{\text{out}(i)}$ satisfies:

$$|N_{\text{out}(i)}| = k,$$

and each subset of size k of V is equally likely.

♣

Small world

Small-world networks provide a more realistic representation of many real-world networks compared to classical random graphs. In such networks, the neighbors of a node are often also neighbors of each other, reflecting the common social phenomenon that "friends of my friends are also friends." This property results in a high clustering coefficient, which contrasts with the Erdős–Rényi random graph, where clustering is typically very low. The Watts–Strogatz model [113] formalizes this concept by starting from a regular lattice and randomly rewiring a fraction of edges. These random long-range connections create shortcuts between distant parts of the network, significantly reducing the graph diameter while preserving local clusters. This combination of high clustering and small diameter makes small-world networks highly relevant for modeling social networks, communication systems, and peer-to-peer overlays, where local connectivity and fast information propagation are both crucial.

Definition 1.1.3 (Watts-Strogatz Small-World Network)

A Watts-Strogatz small-world network is generated by the following procedure:

(i) Start with a regular ring lattice with N nodes, each connected to K nearest neighbors ($\frac{K}{2}$ on each side).

(ii) For each edge (i, j), rewire it with probability p:
- Remove the edge (i, j).
- Connect node i to a randomly chosen node k (excluding i and avoiding duplicate edges).

(iii) Repeat for all edges.

Parameters:
- N: number of nodes in the network.
- K: initial number of neighbors per node (must be even).
- p: rewiring probability, controlling the randomness of the network.

Properties:
- High clustering coefficient compared to random graphs.
- Small average shortest-path length due to long-range shortcuts.

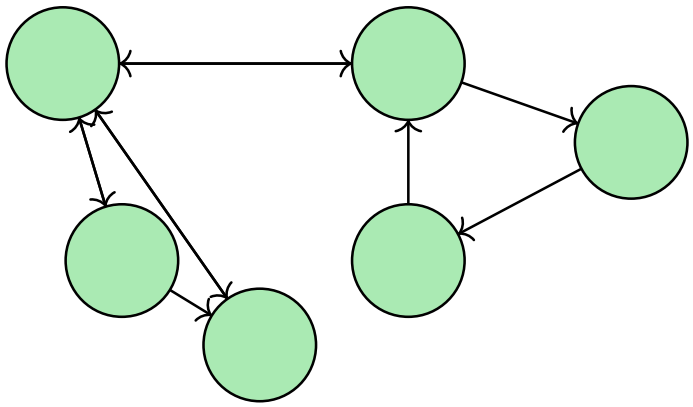

Figure 99: A directed small world network, with 2 clusters (left and right).

Power-law and scale-free networks

The Erdős–Rényi and Watts–Strogatz models are useful approximations, but many real-world networks are more complex than simple random graphs. In particular, they are commonly described as **scale-free** networks [45]: their structural properties remain invariant across scales, meaning that the same organizational patterns can be observed regardless of the network size, with no characteristic degree scale dominating the topology.

In practice, scale-free behavior is most often associated with degree distributions that follow a **power-law**. While the notions of scale-free and power-law are not strictly equivalent, the latter provides a precise mathematical framework to characterize the absence of a characteristic scale in the degree distribution.

Definition 1.1.4 (Power-law network)

Let $G = (V, E)$ be a graph with $|V| = N$. Let K be the random variable denoting the degree of a node chosen uniformly at random from V. The network is said to be power-law if

$$P(K = k) \sim k^{-\gamma}$$

where $\gamma > 1$ is the power-law exponent.

♣

A defining characteristic of power-law networks is their heavy-tailed degree distribution: most nodes have small degree, while a small but non-negligible fraction — commonly referred to as **hubs** — exhibit very large degree. This structural heterogeneity sharply contrasts with Erdős–Rényi graphs, whose degree distribution is binomial. Empirical studies of real-world systems, including peer-to-peer overlays, communication networks, and social graphs, frequently report power-law exponents in the range $2 < \gamma < 3$ [45], a regime in which the average degree remains finite while the variance diverges in the limit of large network size.

This structural heterogeneity has direct consequences for information dissemination. Hubs act as communication shortcuts, significantly reducing the average path length and facilitating rapid message exchange. As a consequence, broadcast, gossip, and aggregation processes may converge faster than in more homogeneous topologies. Many scale-free networks also exhibit the **ultra-small world** property [114]: while the diameter of classical random graphs typically scales as $\log N$, scale-free networks may display diameters scaling as $\log \log N$, further accelerating information spreading.

A classical generative model for scale-free networks is the Barabási–Albert (BA) model [45], which produces power-law degree distributions through **preferential attachment**: when a new node joins the network, it connects to existing nodes with probability proportional to their current degree,

$$\Pi(i) = \frac{k_i}{\sum_j k_j},$$

where k_i is the degree of node i. Starting from a small initial network of m_0 nodes and adding one node with $m \leq m_0$ edges at each step, this rich-get-richer mechanism naturally leads to the emergence of hubs and a power-law degree distribution, reproducing the structural properties observed in many real-world networks.

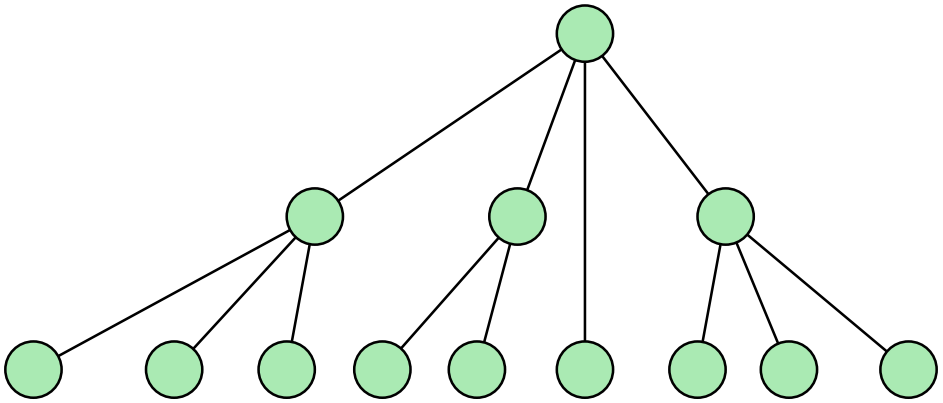

Figure 100: Example of a scale-free network

Stochastic block model

While power-law and scale-free models capture degree heterogeneity, they do not explicitly account for community structure — the tendency of nodes to form densely connected groups with sparser connections between them. Stochastic Block Models (SBM) [115] address this limitation by providing a generative framework in which nodes are partitioned into communities (or blocks), and the probability of a link between two nodes depends solely on the communities to which they belong. This allows the generation of networks that appear random globally but exhibit strong local structure, with explicit control over the number and size of communities as well as intra- and inter-community connection probabilities. SBM generalizes the Erdős–Rényi random graph, which corresponds to the special case of a single block, and is particularly well suited to model overlays organized around hubs or clusters of nodes sharing common interests. Because it is probabilistic, multiple network instances can be generated from the same parameters, making it a valuable tool for benchmarking community detection algorithms and studying networks with varying modularity.

Definition 1.1.5 (Stochastic Block Model)

A Stochastic Block Model (SBM) is a generative model for random graphs with community structure. Consider a graph $G = (V, E)$ with N nodes, and let the nodes be partitioned into K disjoint blocks (or communities) $C_1, C_2, ..., C_K$. The probability of an edge between two nodes depends only on the blocks to which they belong.

Formally, let $B \in [0,1]^{\{K \times K\}}$ be a matrix of connection probabilities between blocks, where $B_{\{ab\}}$ is the probability that a node in block C_a connects to a node in block C_b. Then, for each pair of nodes (i, j):

- If node $i \in C_a$ and node $j \in C_b$, the edge (i, j) exists independently with probability B_{ab}: $P((i, j) \in E) = B_{ab}$.

Special cases include:

- **Intra-block probabilities**: B_{aa}, the probability of connection between nodes within the same community.
- **Inter-block probabilities**: B_{ab}, $a \neq b$, the probability of connection between nodes of different communities.

SBM generalizes the Erdős–Rényi random graph, which corresponds to the case $K = 1$.

♣

The topologies surveyed above — from deterministic structures such as meshes, stars, and rings, to probabilistic models such as random graphs, small-world networks, power-law networks, and stochastic block models — provide a rich vocabulary for characterizing the structural properties of peer-

to-peer overlays. Having established this repertoire, we now turn to the formal modeling of overlay networks as graphs, and introduce the abstractions that will be used throughout this manuscript to reason about connectivity, communication, and protocol execution.

A.2 History and Use cases of Peer to Peer networks

The client–server architecture is the most common communication model on the Internet. It is a natural fit for protocols such as HTTP, FTP, or SSH, where one central server provides services or data to multiple clients that request them. The main advantage of client–server architecture lies in its simplicity. The server manages client requests and coordinates their interactions, while users only need to establish a connection to this central point. Security is also easier to enforce, since it mainly involves securing the server.

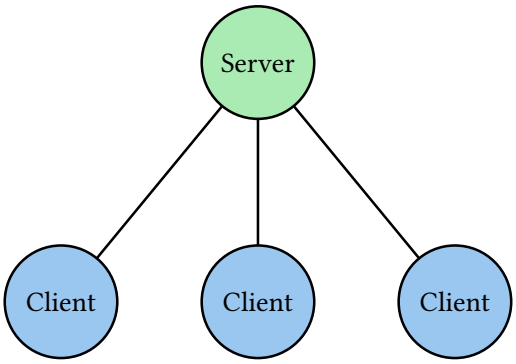

Figure 101: A client-server architecture, with 1 server and 3 clients.

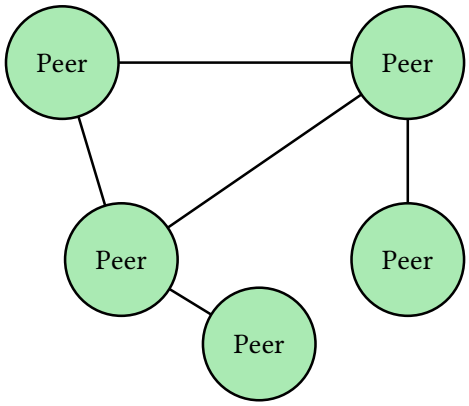

Figure 102: A peer-to-peer architecture.

However, this apparent simplicity comes at a significant cost: the server represents a single point of failure. If it crashes, the entire service becomes unavailable. If it is compromised, all clients are potentially affected. In addition, the computing and networking load is concentrated on the server, while the clients' capabilities (bandwidth, CPU, storage) often remain underutilized.

To overcome these limitations, peer-to-peer (P2P) architectures emerged as an alternative model. In a P2P system, all nodes (or peers) can act both as clients and servers, directly sharing resources, data, and computation. This decentralization enhances resilience, as there is no single point of failure, and promotes a fairer use of global resources by distributing the workload across participants. P2P systems can also scale naturally, since each new peer contributes additional resources to the network.

Nevertheless, these benefits come at the cost of increased complexity. Moving from a 1–N to an N–N communication model introduces significant challenges in coordination, data consistency, and peer discovery. Security and trust management also become more difficult, as there is no central authority to authenticate or regulate interactions. Moreover, peers are heterogeneous, with varying reliability and performance. As a result, P2P systems must rely on adaptive and fault-tolerant protocols capable of handling a wide range of network conditions and potential attacks.

Although today's digital services (e.g. GAFAM) are mostly based on centralized architectures, the Internet itself was originally conceived as a decentralised system, as illustrated by the ARPANET network (see Figure 103). While the Internet Protocol (IP) does not form a single decentralised network — but rather a federation of interconnected operator networks — it inherently supports decentralisation, as any node can directly reach another by its IP address without relying on a central server to route messages. Among the first Internet protocols, several exhibited decentralised or hybrid characteristics rather than a purely client–server model. SMTP and NNTP, for instance, rely on direct communication between independent servers — making them peer-to-peer at the inter-server level — while still following a client–server model for end users connecting to their local instance. Similarly,

DNS introduced a distributed yet hierarchical naming system, in which authority is delegated across multiple autonomous zones rather than centralised in a single entity. Moreover, long before the Internet, human societies relied on decentralised networks of exchange, such as medieval trade routes[7] or the Universal Postal Union[8]. In that sense, peer-to-peer architectures reflect a natural and recurring pattern of human organisation.

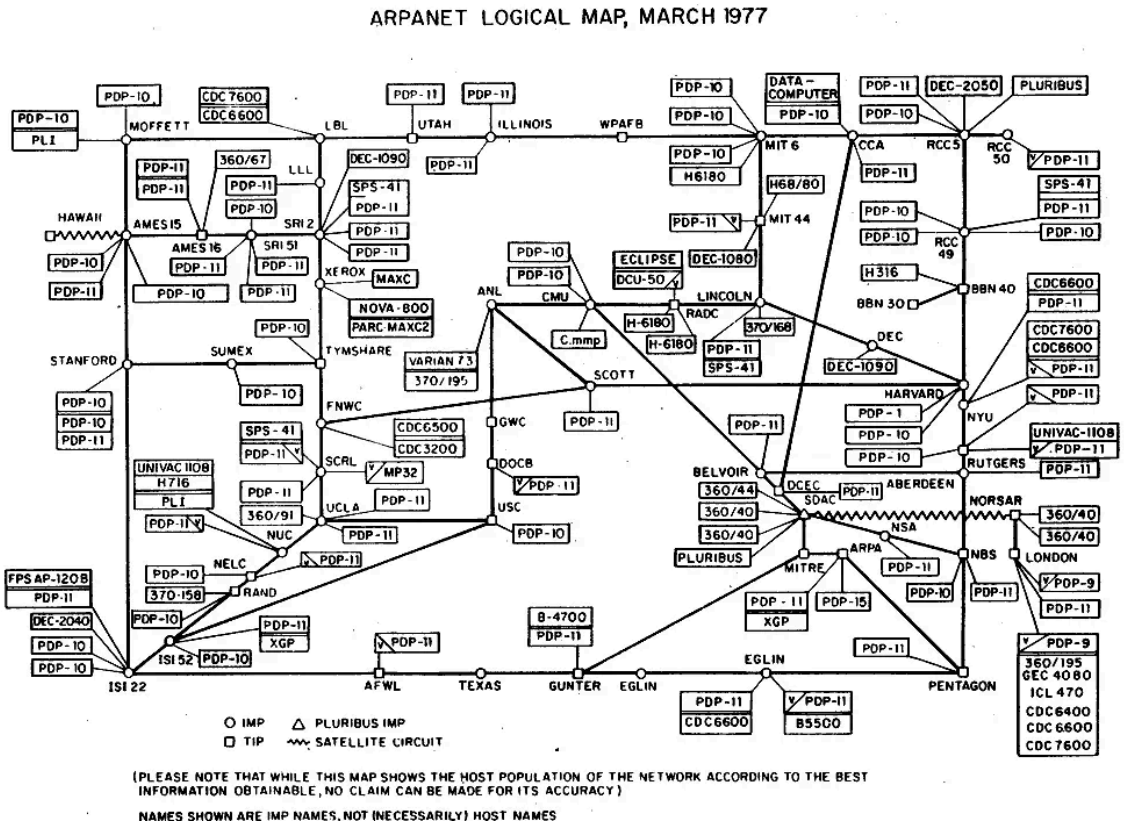

Figure 103: ARPANET, a network with a peer to peer architecture.

The idea of decentralisation, initially present in the Internet's underlying protocols, resurfaced more visibly in the late 1990s as peer-to-peer applications began empowering users to exchange data directly with one another. At that time, the growing demand for large-scale multimedia sharing—combined with limited computing and networking resources (CPU, memory, bandwidth, and storage)—made it difficult for any single server to handle massive numbers of simultaneous downloads. Peer-to-peer networks addressed this limitation by enabling participants to contribute their own resources —especially upload bandwidth—to the system. For example, instead of downloading a 100 MB file from a single server, a user could download small chunks (e.g., 2 MB) from dozens of peers simultaneously, dramatically increasing throughput and scalability.

This concept led to the creation of Napster[9] in 1999, one of the first large-scale file-sharing systems. Although Napster used a central index server to locate files, the data transfer itself occurred directly between peers, marking a key milestone in the history of P2P networking. Following Napster, other peer-to-peer file-sharing systems emerged, such as Gnutella[10] and BitTorrent[11]. Gnutella is fully decentralised, as it does not rely on any central file index. Starting with version 0.6, it introduced the

[7] https://en.wikipedia.org/wiki/Silk_Road
[8] https://en.wikipedia.org/wiki/Treaty_of_Bern
[9] https://en.wikipedia.org/wiki/Napster
[10] https://en.wikipedia.org/wiki/Gnutella
[11] https://www.bittorrent.com/

concept of ultrapeers with the Gia protocol [30] — high-capacity nodes that help route queries and files across the network, improving scalability while preserving decentralisation. BitTorrent brought several notable innovations, including the tit-for-tat mechanism, which encourages fairness by balancing uploading and downloading among peers, and the use of the Kademlia Distributed Hash Table (DHT) for decentralised peer discovery [11] — eliminating the need for central trackers or hierarchical nodes such as ultrapeers. Another notable protocol is Tribler[12][13], which builds upon BitTorrent while introducing several key innovations, including a distributed search engine and an anonymisation layer. Uniquely, Tribler is an academic project [116] developed at Delft University of Technology (TU Delft) in the Netherlands, aiming to create a fully self-sustaining and censorship-resistant file-sharing network.

During the same period, another use case for decentralised networks emerged: anonymisation systems. The Internet Protocol itself does not provide any built-in mechanism for user anonymity or end-to-end encryption. To address this, anonymous overlay networks were developed on top of the Internet, designed to conceal both the content and the origin of communications. These systems typically rely on multi-hop routing and layered encryption, offering a high level of confidentiality at the cost of higher latency and complexity. The most notable examples are Freenet[14], I2P[15], and Tor[16]. Although Tor is not entirely peer-to-peer—since a small number of directory authorities coordinate the list of relays —it remains a decentralised system and the most widely used anonymisation network, with around 7,000 active nodes worldwide.

The success of these peer-to-peer protocols inspired other use cases for applications that were not fully decentralised but leveraged peer-to-peer technology to improve performance. Examples include Skype[17], which until 2014 relied on a peer-to-peer overlay network with supernodes for VoIP communications between users, and Streamroot[18], which used peer-to-peer technology during live football matches to reduce server load by sharing stream data among viewers. At the academic level, researchers have also explored hybrid architectures for online games [117], combining central servers with peer-to-peer mechanisms for data distribution and game state consistency.

Finally, we can mention distributed computing projects such as the Great Internet Mersenne Prime Search (GIMPS)[19], SETI@home[20], and Folding@home[21]. Although these systems are not peer-to-peer — since a central server distributes computation tasks to clients — they have demonstrated the feasibility and efficiency of large-scale volunteer computing. These early systems paved the way for later research on decentralised and federated computing models.

Interest in peer-to-peer networking peaked around 2004, with a surge of academic research and widespread adoption by end users. At that time, peer-to-peer applications accounted for roughly 60% of global Internet traffic (with BitTorrent alone representing about 35%) [118]. In subsequent years, the proportion of P2P traffic declined sharply[22], as server performance, bandwidth, and storage capacities increased, enabling efficient large-scale content delivery through centralized services such as streaming platforms and cloud-based distribution networks. Moreover, the association of P2P networks with copyright infringement and piracy—due to their lack of centralized control—discouraged mainstream

[12] https://www.tribler.org/
[13] I contributed very briefly to the development of Tribler in 2017, https://github.com/Tribler/tribler/issues/3240
[14] https://freenet.org/
[15] https://geti2p.net/en/
[16] https://www.torproject.org/
[17] https://en.wikipedia.org/wiki/Skype
[18] https://github.com/streamroot
[19] https://www.mersenne.org/
[20] https://setiathome.berkeley.edu/
[21] https://foldingathome.org/
[22] https://torrentfreak.com/bittorrent-is-no-longer-the-king-of-upstream-internet-traffic-240315/

users and pushed content providers toward centralized architectures. Nevertheless, BitTorrent remains actively used today for legitimate purposes, such as distributing Linux operating system images [23] and large open-source datasets, including machine learning model weights[24].

There was a resurgence of interest in peer-to-peer technology in the 2010s, following the emergence of Bitcoin [119], which introduced the first blockchain network. The key innovation of Bitcoin is that it enables a completely decentralised and secure payment system by combining peer-to-peer networking, cryptographic proofs and a consensus mechanism. Building on this innovation, and on later advances such as Ethereum's introduction of smart contracts [120], it became possible to create decentralised financial applications (DeFi)[25] and to assign ownership of digital assets such as NFTs[26].

Building on the principle of immutability introduced by blockchain systems—where every transaction is permanently recorded and verifiable—new approaches emerged to apply similar ideas to data storage and sharing. One of the most influential of these systems is the InterPlanetary File System (IPFS)[27]. Inspired by BitTorrent's peer-to-peer file distribution, IPFS generalises and extends it by introducing content addressing: every piece of data is identified by the cryptographic hash of its content, ensuring both integrity and permanence. In addition, IPFS structures data using a Merkle Directed Acyclic Graph (Merkle DAG), allowing efficient deduplication and versioning, much like Git but at the scale of a global network.

By combining the guarantees of blockchain immutability, the programmability of smart contracts, and the distributed storage capabilities of IPFS, new forms of decentralised cloud infrastructures have emerged. These systems aim to provide computing and storage services without relying on traditional centralised data centres. Notable examples include Golem[28], which offers a marketplace for distributed computing resources; Sia[29], which enables decentralised cloud storage through cryptographically secured contracts; and Filecoin[30], which builds directly on top of IPFS to incentivise data storage and retrieval through a native cryptocurrency. Together, these projects illustrate the ongoing shift towards a decentralised Internet infrastructure, where computation and storage are shared and coordinated through peer-to-peer and blockchain mechanisms rather than controlled by central entities.

A.3 Machine Learning

This appendix introduces the fundamental definitions and concepts of machine learning that underpin the decentralized learning protocols discussed in the main body of this thesis. The material presented here is drawn from the foundational works of Tom Mitchell [121] and the Deep Learning book [122]. We begin with a formal definition of learning, as introduced by Tom Mitchell, which serves as the backbone of all subsequent concepts.

[23] For instance, the Ubuntu 25.10 desktop ISO image can be obtained via BitTorrent: https://releases.ubuntu.com/25.10/ubuntu-25.10-desktop-amd64.iso.torrent

[24] For example, the Mixtral model was shared through a torrent link announced in a post on X in December 2023 by the company Mistral: https://x.com/MistralAI/status/1733150512395038967

[25] https://en.wikipedia.org/wiki/Decentralized_finance

[26] https://en.wikipedia.org/wiki/Non-fungible_token

[27] https://ipfs.tech/

[28] https://www.golem.network/

[29] https://sia.tech/

[30] https://filecoin.io/

Definition 1.3.1 (Machine Learning)

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Formally, we denote:
- E: the experience or data that the system uses to learn;
- T: the class of tasks the system is intended to perform;
- P: the performance measure used to evaluate success on tasks in T.

♣

This definition highlights that learning is the process by which a system improves its ability to perform a task through exposure to data or experience. Building on this notion of improvement through experience, machine learning algorithms can be broadly classified according to the nature of the experience they leverage. Machine learning algorithms are typically categorized into three main types: **supervised learning**, **unsupervised learning**, and **reinforcement learning**. Supervised learning involves learning a mapping from input data to known outputs, unsupervised learning aims to discover patterns or structure in data without labeled outputs, and reinforcement learning focuses on learning optimal decision-making policies through repeated interaction with an environment, guided by a reward signal that evaluates the agent's actions. In the context of this thesis, our primary focus is on **supervised learning**, as it provides the foundation for the federated and decentralized learning approaches.

Supervised learning involves observing several examples of a random vector x and an associated value or vector y, and learning to predict y from x, usually by estimating the conditional probability $p(y \mid x)$. All supervised learning algorithms share a common two-phase structure. During the **training phase**, the algorithm is exposed to labeled data and adjusts its internal parameters to minimize a measure of prediction error. Once training is complete, the resulting model is deployed during the **inference phase** to make predictions on previously unseen data, without further parameter updates.

Definition 1.3.2 (Supervised Learning)

Given a dataset of N examples:

$$D = \{(x_1, y_1), (x_2, y_2), ..., (x_N, y_N)\},$$

the goal of a supervised learning algorithm is to find a function $f_\theta(x)$ parameterized by θ that approximates the mapping from x to y.

♣

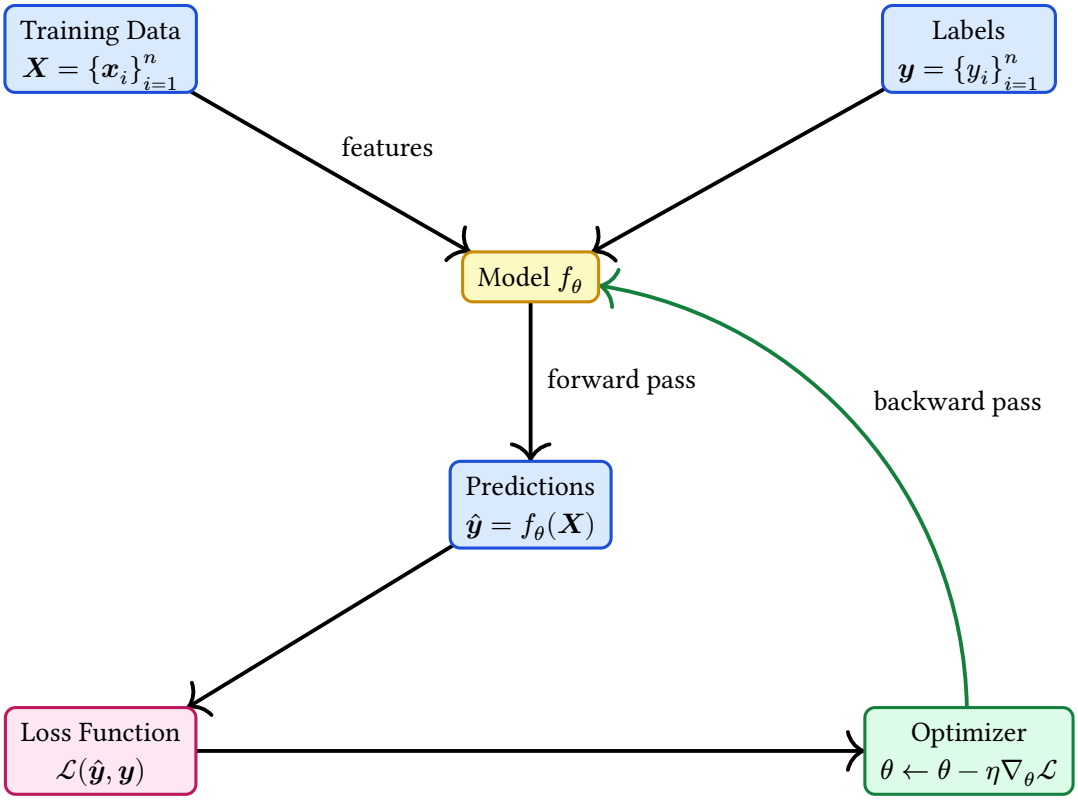

Figure 104: The supervised learning training loop.

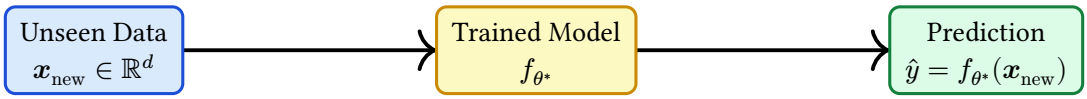

Figure 105: The supervised learning inference phase.

In practice, this mapping is typically found by minimizing a loss function $L(f_\theta(x), y)$ over the dataset D, which measures the discrepancy between the predicted outputs and the true labels, reflecting how well the model performs on a given example or dataset.

Definition 1.3.3 (Loss Function)

A loss function L takes a predicted output $f_\theta(x)$ and a true label y as inputs, and returns a non-negative real value measuring the discrepancy between the two:

$$L(f_\theta(x), y) \in \mathbb{R}_{\geq 0},$$

where smaller values indicate better predictions. Given a dataset $D = \{(x_1, y_1), ..., (x_N, y_N)\}$, the overall loss is computed as the average over all examples:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} L(f_\theta(x_i), y_i).$$

Beyond the loss function, which measures the discrepancy between predicted and true labels during training, it is useful to introduce a complementary performance metric that is more directly interpretable in classification settings. **Accuracy** measures the proportion of correctly classified examples, and provides an intuitive assessment of model quality on held-out data.

Definition 1.3.4 (Accuracy)

Let $D^{\text{test}} = \{(x_i, y_i)\}_{i=1}^{M}$ be a test dataset and let $\hat{y}_i = f_{\theta^*}(x_i)$ be the predicted label for input x_i. The **accuracy** of a model f_{θ^*} is defined as:

$$\text{Accuracy} = \frac{1}{M} \sum_{i=1}^{M} \mathbb{1}\{\hat{y}_i = y_i\},$$

where $\mathbb{1}\{\cdot\}$ is the indicator function, equal to 1 if the prediction is correct and 0 otherwise.

♣

Remark

Accuracy and loss measure complementary aspects of model performance. The loss quantifies the magnitude of prediction errors and drives optimization, while accuracy provides a threshold-based measure of correctness that is directly interpretable. In classification tasks, a model with low loss will typically achieve high accuracy, but the two metrics need not be perfectly aligned, particularly in the presence of class imbalance.

Common examples of loss functions include the **Mean Squared Error (MSE)**, $L(f_\theta(x), y) = \|f_\theta(x) - y\|^2$, widely used in regression tasks, and the **Cross-Entropy Loss**, $L(f_\theta(x), y) = -\sum_i y_i \log f_\theta(x)_i$, commonly used in classification tasks.

These two loss functions naturally reflect the two main types of prediction tasks encountered in supervised learning: **regression** and **classification**. Regression problems aim to predict a continuous value, while classification problems aim to assign an input to one of a discrete set of classes. In this thesis, we focus on **classification problems**, specifically **binary classification** (two possible classes) and **multinomial classification** (more than two classes).

Binary classification refers to the supervised learning task where each input x is associated with a label y that can take only two possible values, typically denoted $y \in \{0, 1\}$. The goal is to learn a function $f_\theta(x)$ that outputs a predicted label $\hat{y}$ that approximates the true label y.

Definition 1.3.5 (Binary Classification)

Given a dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$ where $x_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$, binary classification aims to find a function $f_\theta : \mathbb{R}^d \to [0, 1]$ that estimates the probability $p(y = 1 \mid x)$, by solving:

$$\theta^* = \text{argmin}_\theta \frac{1}{N} \sum_{i=1}^{N} L(f_\theta(x_i), y_i),$$

where L is a loss function measuring the discrepancy between predicted and true labels.

♣

Multinomial classification generalizes binary classification to the case where each label y can take one of $K > 2$ possible classes: $y \in \{1, 2, ..., K\}$. The goal is to learn a function $f_\theta(x)$ that outputs either a class label $\hat{y}$ or a probability distribution over the K classes.

Definition 1.3.6 (Multinomial Classification)

Given a dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$ where $x_i \in \mathbb{R}^d$ and $y_i \in \{1, ..., K\}$, multinomial classification aims to find a function $f_\theta : \mathbb{R}^d \to [0, 1]^K$ that estimates the probability distribution $p(y = k \mid x)$ over all K classes, by solving:

$$\theta^* = \text{argmin}_\theta \frac{1}{N} \sum_{i=1}^{N} L(f_\theta(x_i), y_i),$$

where L is a loss function measuring the discrepancy between predicted and true labels.

♣

In supervised learning, once a loss function $L\left(f_{\theta(x)}, y\right)$ has been defined, the next step is to find a way to minimize this loss. Minimizing the loss corresponds to improving the model's performance on the task, that is, making its predictions closer to the true labels. This is achieved using an **optimization algorithm**. While many optimization algorithms exist, the most widely used in practice is **gradient descent** [123] and its variants, due to its simplicity and efficiency in handling large datasets.

Gradient descent iteratively updates the parameters of the model in the direction of the negative gradient of the loss function. This procedure requires that the loss function be **differentiable**, so that the gradient exists, and ideally have a **continuous gradient** to ensure stable updates. If the function is **convex**, then gradient descent is guaranteed to converge to the global minimum. However, in most machine learning applications, the loss function is **non-convex**, meaning that gradient descent may only reach a local minimum. Despite the lack of theoretical guarantees for reaching the global minimum, in practice gradient descent and its variants often yield very good results.

Definition 1.3.7 (Gradient Descent)

Gradient descent is an iterative optimization algorithm used to minimize a differentiable function, such as a loss function in machine learning. The idea is to update the model parameters θ in the direction opposite to the gradient of the loss function with respect to these parameters:

$$\theta_{t+1} = \theta_t - \eta * \nabla_\theta L(\theta_t),$$

where:

- θ_t are the parameters at iteration t,
- $\eta > 0$ is the learning rate controlling the step size,
- $\nabla_\theta L(\theta_t)$ is the gradient of the loss function with respect to θ evaluated at iteration t.

By repeatedly applying this update rule, the algorithm moves towards a local minimum of the loss function, thereby improving the model's performance.

The algorithm is typically stopped when one of the following conditions is met:

(i) The absolute difference between successive loss values is smaller than a predefined threshold: $|L(\theta_{t+1}) - L(\theta_t)| < \varepsilon$, with $\varepsilon > 0$, in which case we consider that the algorithm has **converged**.

(ii) A maximum number of iterations $T_{\max}$ is reached. In this case, the algorithm stops and the loss value at the final iteration is used.

♣

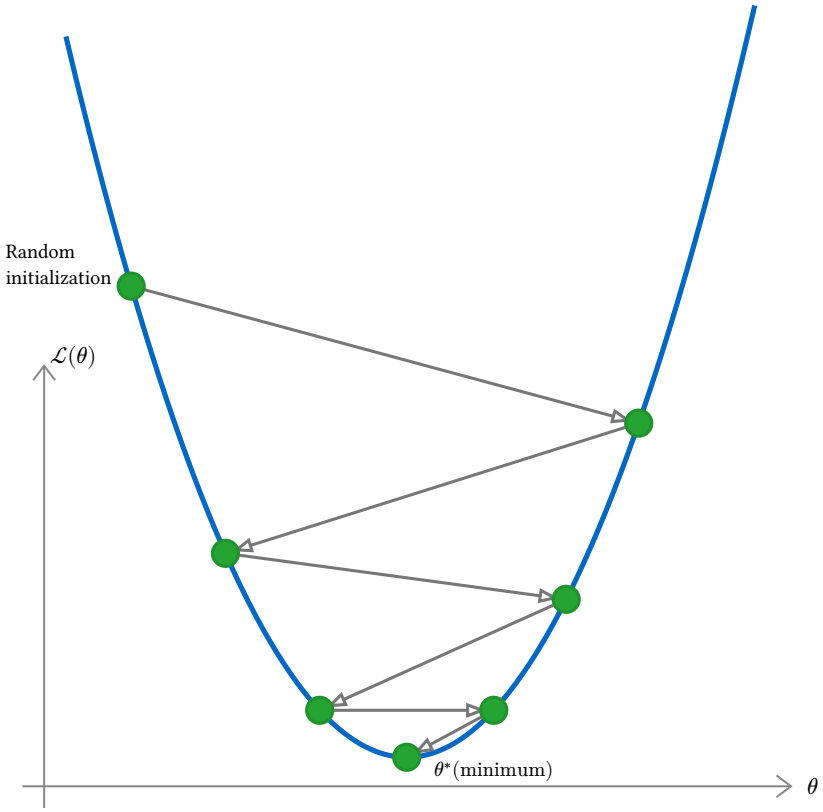

Figure 106: Gradient descent on a convex loss surface $\mathcal{L}(\theta)$: starting from a random initialization, the parameters θ are iteratively updated in the direction opposite to the gradient until convergence to the minimum θ^*.

In its basic form, gradient descent is applied to the entire dataset at once, meaning that the gradient is computed over all available samples before each parameter update.

In practice, the available dataset is divided into a **training set** and a **test set**. The model's parameters θ are updated via gradient descent to minimize the loss function $L(\theta)$ on the training set, while the test set is held out entirely and used only to evaluate the model's performance on unseen data, providing an estimate of its generalization ability.

This separation is essential to detect **overfitting**, a phenomenon that occurs when the model memorizes the training data rather than capturing general patterns. A key indicator of overfitting is a divergence between the two losses: while the training loss continues to decrease, the test loss starts to increase. By monitoring both losses throughout training, one can assess whether the model generalizes well to unseen data or merely fits the training set.

> **Remark**
>
> In the sense of Definition 1.3.1, the task T corresponds to binary or multinomial classification, the experience E to the labeled dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$ from which the model learns, and the performance measure P to the loss function $\mathcal{L}(\theta)$ that quantifies how well the model performs on this task.

A.3.1 Datasets

Previously, we introduced the notion of dataset in the supervised learning setting, defining it abstractly as a collection of labeled examples drawn from an unknown joint distribution. To ground this abstrac-

tion, we now present two concrete datasets drawn from the machine learning literature, covering binary and multiclass classification respectively. These examples illustrate what such a dataset looks like in practice.

Spambase

The Spambase dataset [93] is a binary classification benchmark originally compiled by Hewlett-Packard Labs and made publicly available through the UCI Machine Learning Repository. It consists of 4,601 email messages, each represented as a feature vector of 57 continuous attributes, grouped into three categories. The first 48 attributes, of type `word_freq_WORD`, measure the percentage of words in the email that match a given keyword, computed as $100 \times (\text{count of WORD}) / \text{total words}$. The following 6 attributes, of type `char_freq_CHAR`, measure the percentage of characters in the email matching a given special character, computed analogously over the full character sequence. The remaining 3 attributes capture the structure of capital letter sequences: `capital_run_length_average` is the average length of uninterrupted sequences of capital letters, `capital_run_length_longest` is the length of the longest such sequence, and `capital_run_length_total` is the total number of capital letters in the email. The task is to classify each email as either spam ($y = 1$) or legitimate ($y = 0$), making this a standard binary classification problem. The dataset is moderately imbalanced, with approximately 39% of instances labeled as spam. Its relatively small size and tabular structure make it well suited for evaluating lightweight models such as support vector machines and shallow neural networks in a distributed setting.

Table 24: Selected attributes of the Spambase dataset. The full feature set comprises 48 word frequency attributes, 6 character frequency attributes, and 3 capital run-length attributes.

Attribute	**Type**	**Description**
`word_freq_meeting`	Continuous $[0, 100]$	% of words matching "meeting"
`word_freq_original`	Continuous $[0, 100]$	% of words matching "original"
`word_freq_project`	Continuous $[0, 100]$	% of words matching "project"
⋮	⋮	⋮
`char_freq_;`	Continuous $[0, 100]$	% of characters matching ";"
`char_freq_(`	Continuous $[0, 100]$	% of characters matching "("
⋮	⋮	⋮
`capital_run_length_average`	Continuous $[1, +\infty[$	Average length of capital letter runs
`capital_run_length_longest`	Integer $[1, +\infty[$	Length of longest capital letter run
`capital_run_length_total`	Integer $[1, +\infty[$	Total number of capital letters
Class	Binary	Spam ($y = 1$) or legitimate ($y = 0$)

MNIST

The MNIST dataset [95] is a multiclass classification benchmark consisting of 70,000 grayscale images of handwritten digits, partitioned into 60,000 training samples and 10,000 test samples. Each image is of size 28×28 pixels, yielding a 784-dimensional input vector after flattening. The task is to assign each image to one of ten classes corresponding to the digits 0 through 9. MNIST is one of the most widely used benchmarks in the machine learning literature, serving as a standard testbed for evaluating classification models ranging from logistic regression to deep convolutional networks. It provides a more demanding evaluation setting than Spambase, due to its higher input dimensionality and the multiclass nature of the learning task.

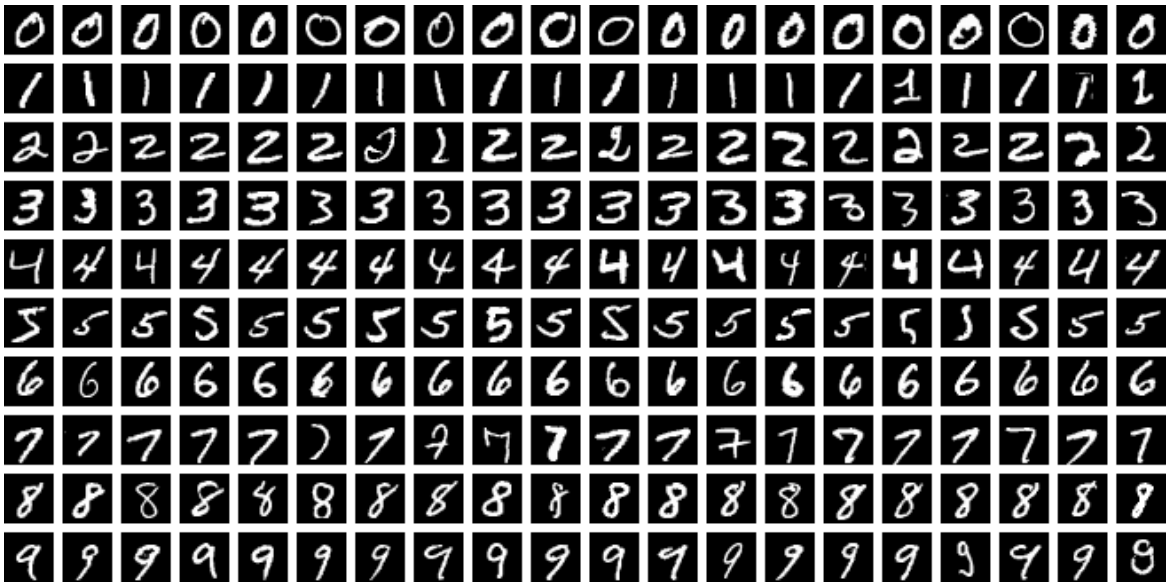

Figure 107: MNIST Dataset.

A.3.2 Machine Learning Models

Having introduced the key components of supervised learning, we now have all the ingredients to formally define a supervised learning model as a mathematical tool for solving the supervised learning problem.

Definition 1.3.8 (Machine Learning Model)

A supervised learning model is defined by:

- A **hypothesis class** $\mathcal{F} = \{f_\theta : \mathcal{X} \to \mathcal{Y} \mid \theta \in \Theta\}$, that is, a parametrized family of functions mapping inputs $x \in \mathcal{X}$ to outputs $y \in \mathcal{Y}$ (see Definition 1.3.2),
- A **parameter space** Θ, which is the set of all admissible values for the parameters θ,
- A **loss function** $L : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0}$, chosen to reflect the assumptions of the model and the nature of the task (see Definition 1.3.3).

Training the model consists in finding the optimal parameters θ^* via gradient descent (Definition 1.3.7):

$$\theta^* = \operatorname*{argmin}_{\theta \in \Theta} \mathcal{L}(\theta) = \operatorname*{argmin}_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^{N} L(f_\theta(x_i), y_i).$$

♣

In supervised learning, there is no single universal model: infinitely many hypothesis classes $\mathcal{F}$ can in principle be considered, each encoding different assumptions about the structure of the mapping f_θ. The choice of model is therefore guided by the nature of the task, the structure of the data, and practical constraints such as computational efficiency and interpretability. In what follows, we introduce three widely used supervised learning models that serve as building blocks for the federated and decentralized learning frameworks studied in this thesis: **linear regression**, suited to regression problems; **logistic regression**, suited to binary classification problems; and **multilayer perceptrons (MLPs)**, suited to tasks where the relationship between inputs and outputs is too complex to be captured by a linear model. These three models also represent increasing levels of complexity and expressiveness in the hypothesis class $\mathcal{F}$.

Linear Regression

Linear regression is one of the simplest and most widely used models in machine learning. It is a type of supervised learning model used to predict a continuous output variable y from one or more input features x. The model assumes a linear relationship between the input variables and the output, making it easy to interpret and efficient to train.

Linear regression is often used as a baseline model before trying more complex algorithms, and it also serves as a foundation for understanding more advanced models such as generalized linear models and neural networks.

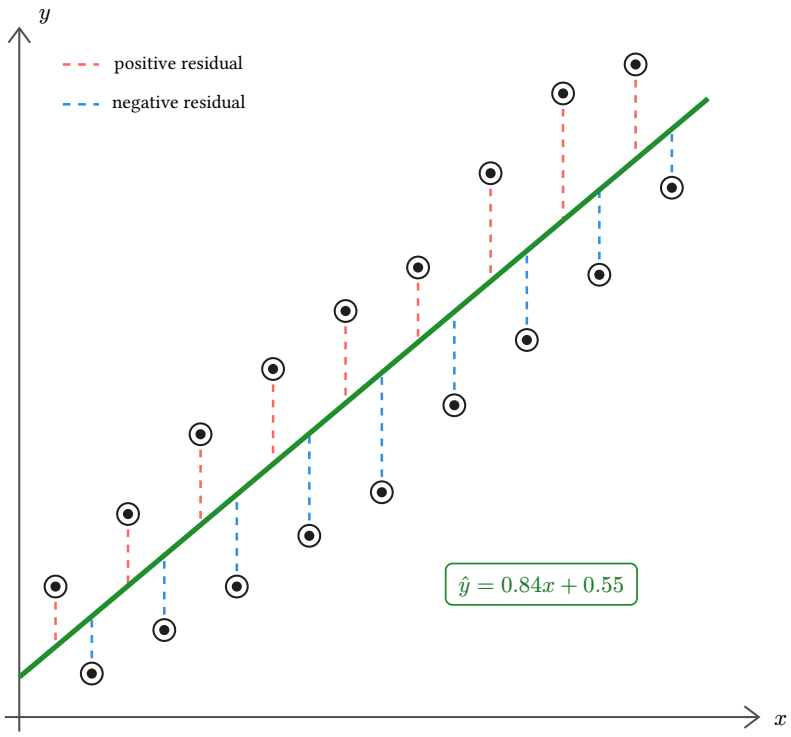

Figure 108: Linear regression: the green line minimizes the sum of squared residuals between predicted and observed values.

Definition 1.3.9 (Linear Regression)

A linear regression model predicts a continuous output $y \in \mathbb{R}$ from an input feature vector $x \in \mathbb{R}^d$ using an affine function:

$$\hat{y} = f_\theta(x) = w^T x + b,$$

where:

- $w \in \mathbb{R}^d$ is the weight vector,
- $b \in \mathbb{R}$ is the bias term,
- $\theta = (w, b)$ denotes the set of model parameters.

Given a training dataset

$$D = \{(x_1, y_1), ..., (x_N, y_N)\},$$

the parameters θ are learned by minimizing the mean squared error (MSE):

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} (y_n - f_\theta(x_n))^2.$$

Logistic Regression

Logistic regression is a supervised learning model used for classification tasks, rather than predicting continuous values. It is particularly suited for binary classification problems, where the goal is to predict whether an instance belongs to one of two classes. Unlike linear regression, logistic regression outputs a probability value between 0 and 1, which can then be thresholded to assign a class label.

The model is based on a linear combination of input features, transformed by the logistic (sigmoid) function, allowing it to model the probability of class membership.

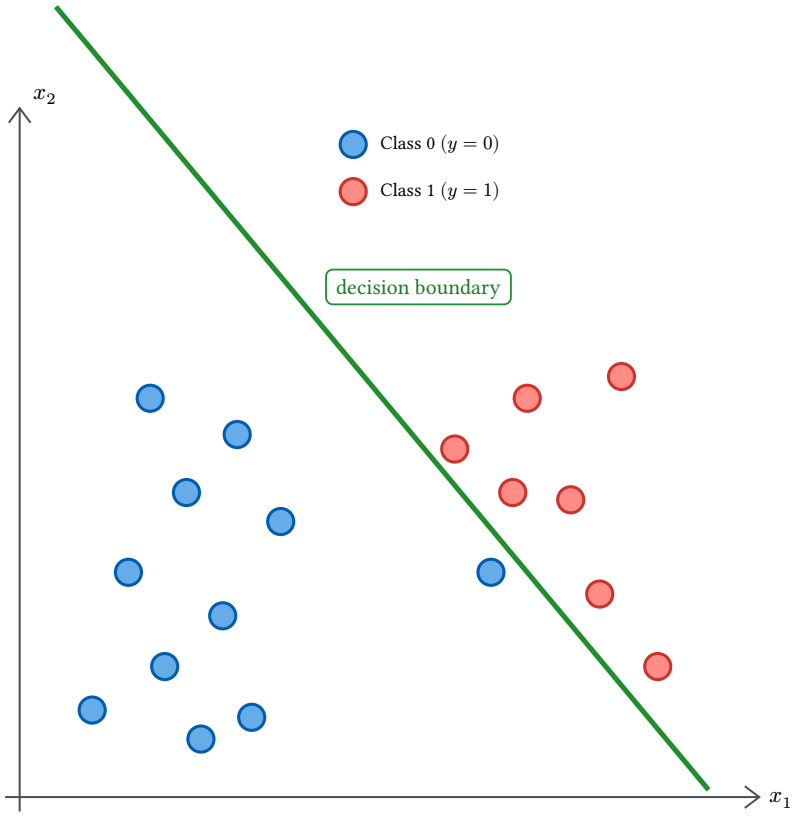

Figure 109: Logistic regression: a linear decision boundary separates two classes in the feature space (x_1, x_2).

Definition 1.3.10 (Logistic Regression)

Logistic regression is a supervised learning model for binary classification. It estimates the probability that an input $x \in \mathbb{R}^d$ belongs to the positive class:

$$p(y = 1 \mid x; \theta) = \sigma(w^T x + b),$$

where:

- $\sigma(z) = \frac{1}{1+\exp(-z)}$ is the sigmoid function,
- $\theta = (w, b)$ are the model parameters,
- $\hat{y}_n = \sigma(w^T x_n + b)$ denotes the predicted probability for the n-th sample.

The parameters are learned by minimizing the binary cross-entropy loss:

$$L(\theta) = -\frac{1}{N} \sum_{n=1}^{N} [y_n \log(\hat{y}_n) + (1 - y_n) \log(1 - \hat{y}_n)].$$

Multinomial Logistic Regression

While binary logistic regression predicts the probability of an instance belonging to one of two classes, multinomial logistic regression generalizes this approach to problems with $K > 2$ classes. In this case, the model outputs a probability distribution over all possible classes for each input instance. The probabilities are obtained using the softmax function, which ensures they sum to 1.

Multinomial logistic regression is widely used for multi-class classification tasks such as handwritten digit recognition, text categorization, or image labeling.

Definition 1.3.11 (Multinomial Logistic Regression)

For a classification problem with K classes, multinomial logistic regression models the conditional class probabilities using the softmax function. Let:

- $x \in \mathbb{R}^d$ be an input vector,
- $W \in \mathbb{R}^{K \times d}$ be the weight matrix,
- $b \in \mathbb{R}^K$ be the bias vector.

The probability of class k is given by:

$$p(y = k \mid x; \theta) = \frac{\exp((Wx + b)_k)}{\sum_{j=1}^{K} \exp\big((Wx + b)_j\big)},$$

where:

- $(Wx + b)_k$ denotes the k-th component of the score vector,
- $\theta = (W, b)$ are the model parameters.

The model is trained by minimizing the categorical cross-entropy loss:

$$L(\theta) = -\frac{1}{N} \sum_{n=1}^{N} \sum_{k=1}^{K} y_{nk} \log(p(y_n = k \mid x_n; \theta)),$$

where $y_{nk} \in \{0, 1\}$ indicates whether the n-th example belongs to class k, following a one-hot encoding of the true labels.

♣

Remark

For $K = 2$, this formulation reduces to binary logistic regression.

Neural Networks and Multi-Layer Perceptrons

The models introduced so far rely on a linear mapping of the form $W^T x + b$ applied to the input features. While these models are simple, efficient, and well understood, their expressive power is fundamentally limited: they can only represent linear decision boundaries in the input space.

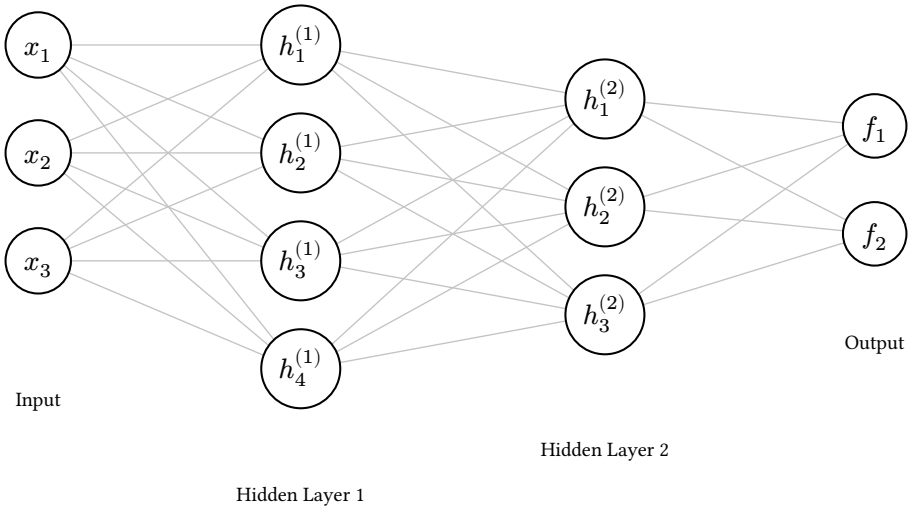

Figure 110: A fully connected neural network with two hidden layers ($L = 3$).

Artificial neural networks extend these models by composing multiple linear transformations with nonlinear activation functions. The simplest neural network, known as the **single-layer perceptron**, consists of a single linear unit followed by a nonlinear activation function. Although this model already allows for binary classification, it remains limited to linearly separable problems.

To overcome this limitation, neural networks introduce **hidden layers**, leading to the so-called **Multi-Layer Perceptrons (MLPs)**. An MLP is a feedforward neural network composed of several layers of perceptrons, where each layer applies an affine transformation followed by a nonlinear activation. By stacking multiple such layers, MLPs are able to learn complex, nonlinear mappings between inputs and outputs.

This layered structure allows neural networks to progressively transform the input representation into higher-level features, making them powerful models for a wide range of tasks, including classification, regression, and function approximation.

Definition 1.3.12 (Multi-Layer Perceptron (MLP))

A Multi-Layer Perceptron (MLP) is a feedforward neural network composed of a finite sequence of layers, where each layer applies an affine transformation followed by a nonlinear activation function. Let $x \in \mathbb{R}^{d_0}$ be an input vector. An MLP with L layers defines a sequence of hidden representations $(h^1, h^2, ..., h^L)$ as follows:

$$h^0 = x,$$

$$h^l = \varphi\big(W^l h^{l-1} + b^l\big), \quad l = 1, ..., L-1,$$

where:
- $W^l \in \mathbb{R}^{d_l \times d_{l-1}}$ is the weight matrix of layer l,
- $b^l \in \mathbb{R}^{d_l}$ is the bias vector of layer l,
- $\varphi : \mathbb{R} \to \mathbb{R}$ is a nonlinear activation function applied element-wise (e.g. ReLU, sigmoid),
- $h^l \in \mathbb{R}^{d_l}$ is the output of layer l.

The output layer applies a task-specific transformation:

$$f_\theta(x) = \varphi^L(W^L h^{L-1} + b^L),$$

where φ^L is chosen according to the task: the identity function for regression, the sigmoid for binary classification, or the softmax for multinomial classification.

The full set of trainable parameters is $\theta = \{W^1, b^1, ..., W^L, b^L\}$.

♣

A.4 Learning Paradigms

Machine learning models can be trained under very different assumptions regarding data availability, computational resources, and the organization of the learning process. These assumptions define what is called a **learning paradigm**, which specifies how data is accessed, how computation is distributed, and how the model is updated during training.

A.4.1 Centralized Learning

Centralized learning refers to a learning paradigm in which all training data are collected and stored at a single location, and the learning process is performed using the complete dataset.

Definition 1.4.1 (Centralized Learning)

Centralized learning is a learning paradigm in which a single learner has full access to a dataset $D = \{(x_1, y_1), ..., (x_N, y_N)\}$ and trains a model f_θ by solving:

$$\theta^* = \text{argmin}_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^{N} L(f_\theta(x_i), y_i),$$

where L is a loss function chosen according to the task, and Θ is the parameter space.

This setting assumes that all data are available to a single computing entity throughout training, which enables exact gradient computation over the full dataset at each iteration of gradient descent:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \frac{1}{N} \sum_{i=1}^{N} L(f_\theta(x_i), y_i).$$

♣

In practice, training is often carried out using mini-batches for computational efficiency, particularly to leverage GPU or accelerator architectures. However, this does not alter the fundamental assumption of centralized learning, which requires that all data be available to the learner, either in advance or on demand.

As a consequence, centralized learning typically relies on a single machine or a tightly coupled computing cluster with sufficient computational and memory resources to process the full dataset.

While computationally straightforward, this paradigm imposes strong assumptions: all data must be collected, stored, and processed at a single location, raising fundamental challenges in terms of scalability, data privacy, and data locality.

A.4.2 Online Learning

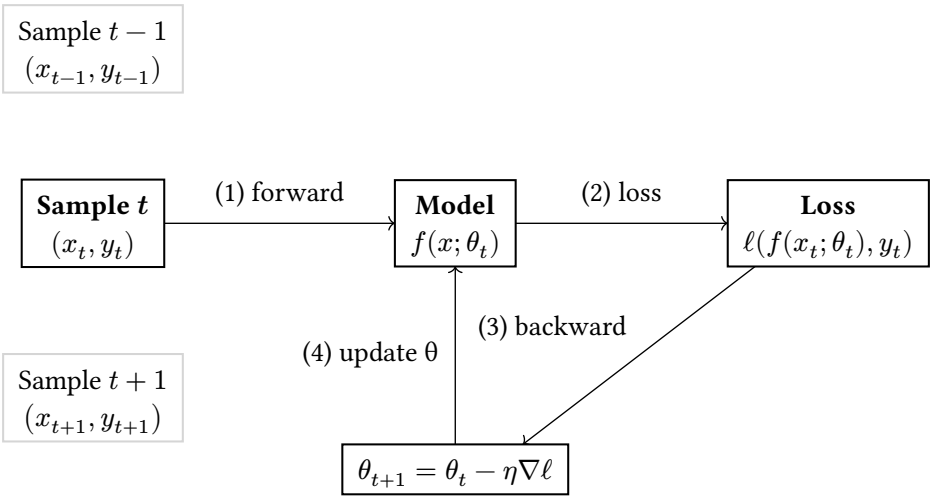

Figure 111: Online learning: the model receives one sample (x_t, y_t) at a time, computes the loss, and updates its parameters θ before processing the next sample.

Centralized learning, as introduced in Definition 1.4.1, assumes that the entire dataset D is available before training begins. However, in many real-world settings, data is generated sequentially over time, possibly in large volumes or under resource constraints, making this assumption impractical.

Online learning [124] addresses this limitation by allowing a model to be updated incrementally as new data becomes available. Instead of learning from a fixed dataset, the model continuously adapts to a stream of observations, enabling learning in dynamic, non-stationary, or distributed environments. This paradigm is particularly relevant in decentralized systems, which are central to the context of this thesis.

Definition 1.4.2 (Online Learning)

Online learning is a learning paradigm in which model parameters are updated sequentially as data arrives, rather than being trained once on a fixed dataset.

At each time step t, the learning algorithm receives an input x_t, produces a prediction $\hat{y}_t$, and then observes the true label y_t. Based on this feedback, the model parameters θ_t are updated using an online optimization rule, typically of the form:

$$\theta_{t+1} = \theta_t - \eta_t * \nabla_\theta L\big(f_{\theta_t}(x_t), y_t\big)$$

where:
- θ_t denotes the model parameters at time t,
- $\eta_t > 0$ is a possibly time-dependent learning rate,
- L is a loss function measuring the prediction error.

The objective of online learning is to minimize the cumulative loss over time, often expressed as:

$$\sum_{t=1}^{T} L\big(f_{\theta_t}(x_t), y_t\big)$$

while adapting efficiently to new data and potential changes in the data distribution.

♣

A.4.3 Ensemble Learning

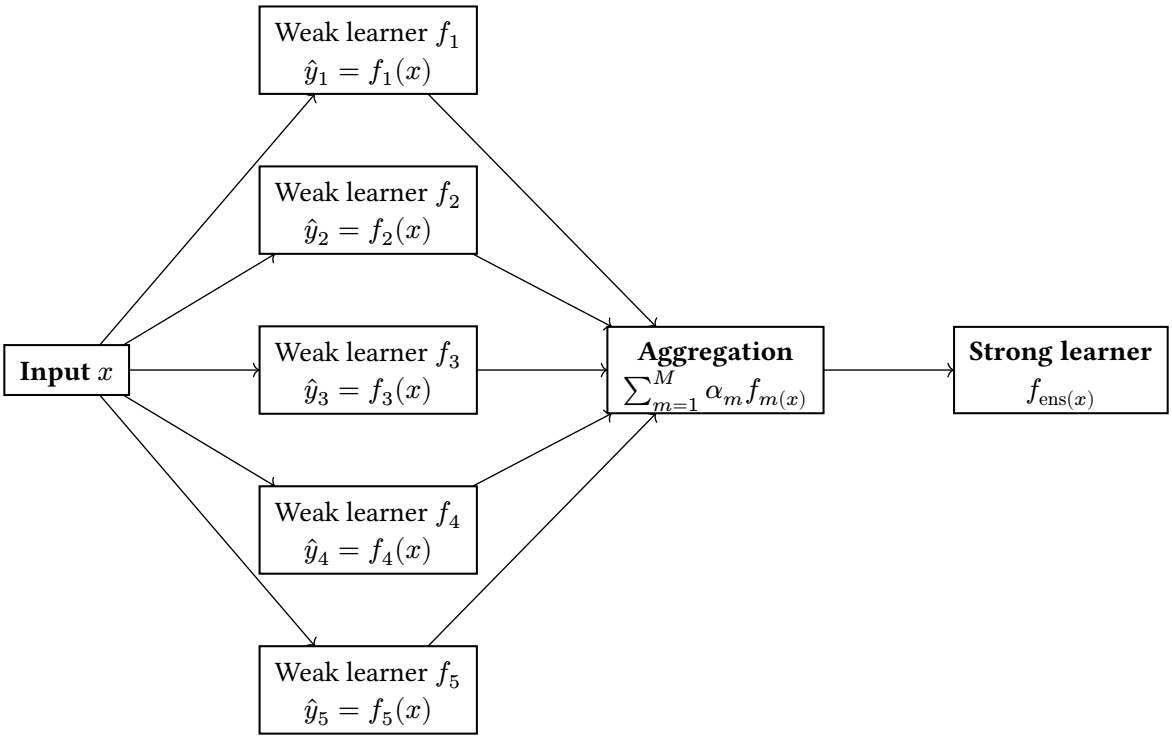

Figure 112: Ensemble learning: an input x is fed to M weak learners $f_1, ..., f_M$. An aggregation step combines their predictions via a weighted sum $\sum_{m=1}^{M} \alpha_m f_{m(x)}$

Beyond individual learning models, an important paradigm in machine learning consists in combining multiple models in order to improve predictive performance. This approach, known as ensemble learning [125], is based on the observation that multiple imperfect or weak models can collectively yield a more accurate and robust predictor than any single model alone.

Ensemble methods are particularly effective at reducing variance, improving generalization, and increasing robustness to noise or model misspecification. They play a central role in modern machine learning and are especially relevant in distributed and decentralized settings, where multiple models may be trained independently and later combined.

Definition 1.4.3 (Ensemble Learning)

Ensemble learning is a learning paradigm in which a set of models $\{f_1, f_2, \ldots, f_M\}$, often referred to as weak learners, are combined to form a single predictor f_{ens} with improved performance. ♣

A common form of ensemble model is a weighted aggregation of individual predictors:

$$f_{\text{ens}(x)} = \sum_{m=1}^{M} \alpha_m f_{m(x)},$$

where:
- f_m denotes the prediction of model m,
- $\alpha_m \in R$ is a weight associated with model m.

Remark

When all weights are equal, i.e. $\alpha_m = 1/M$ for all m, the weighted aggregation ensemble model reduces to a simple average:

$$f_{\text{ens}(x)} = \frac{1}{M} \sum_{m=1}^{M} f_{m(x)}.$$

In classification settings, this corresponds to a majority vote: each weak learner f_m casts a vote for a class, and $f_{\text{ens}(x)}$ returns the class receiving the most votes.

Remark

The weights α_m in the ensemble formulation should not be confused with the weight matrices W^l introduced in Definition 1.3.12. Here, $\alpha_m \in \mathbb{R}$ is a scalar coefficient assigned to each model f_m, reflecting its relative contribution to the ensemble prediction. It bears no relation to the learnable parameters internal to any individual model.

For linear models, such as linear or logistic regression, this aggregation is equivalent to a single model of the same class, with parameters equal to the weighted sum of the individual parameters. In this case, the aggregation is order-independent and preserves linearity.

For nonlinear models, such as neural networks, the aggregation generally does not admit an equivalent representation within the same hypothesis class. The interaction between nonlinear decision functions may lead to more complex behaviors, and the resulting ensemble cannot, in general, be reduced to a single model.

Despite the lack of general theoretical guarantees for nonlinear ensembles, ensemble learning has been shown empirically to significantly improve predictive performance in a wide range of applications.

A.4.4 Distributed Learning

While ensemble learning focuses on combining multiple models to improve predictive performance, it typically assumes that models are trained independently and that their aggregation is performed in a

centralized manner. More generally, most classical machine learning algorithms rely on a centralized learning paradigm, in which all data and computation are collected and processed at a single location.

However, the increasing scale of data, computational requirements, and the emergence of decentralized systems have motivated the development of distributed learning approaches. In distributed learning, data, computation, or decision-making are spread across multiple nodes, which collaboratively contribute to the training process while operating under communication, synchronization, and resource constraints.

Data parallelism

As deep neural networks have grown in size and complexity, training them on a single device has become increasingly time-consuming. Modern architectures may require days or even weeks of computation on a single processor, which makes reducing training time a central concern in distributed learning [126]. A natural response to this challenge is to exploit the parallel computing capabilities of modern hardware: GPUs expose hundreds to thousands of cores that can perform floating-point operations simultaneously, making them well-suited for the kind of matrix computations that dominate neural network training.

Data parallelism is one of the most common strategies for distributed training and leverages this hardware parallelism by distributing the training data across multiple workers. The training dataset is partitioned into disjoint subsets, each assigned to a different worker, while all workers maintain a replica of the same model. Neural networks are particularly amenable to this form of parallelism: because the gradient of the loss with respect to the parameters decomposes additively over individual samples, the full gradient can be approximated by aggregating local gradients computed independently on each worker. This property makes distributed SGD a natural fit for data parallelism [126].

During training, each worker computes gradients on its local data partition using mini-batch SGD. These gradients are then aggregated across workers — typically by averaging — to produce a global gradient estimate, which is used to update the shared model parameters. This process is repeated iteratively until a convergence criterion is met.

Data parallelism preserves the centralized learning objective, as the model is effectively trained on the full dataset, while distributing the computational load. In its synchronous form, it ensures that each parameter update is consistent with the global gradient. However, this approach still relies on frequent synchronization and communication between workers, and assumes a coordinated training process under a common optimization objective.

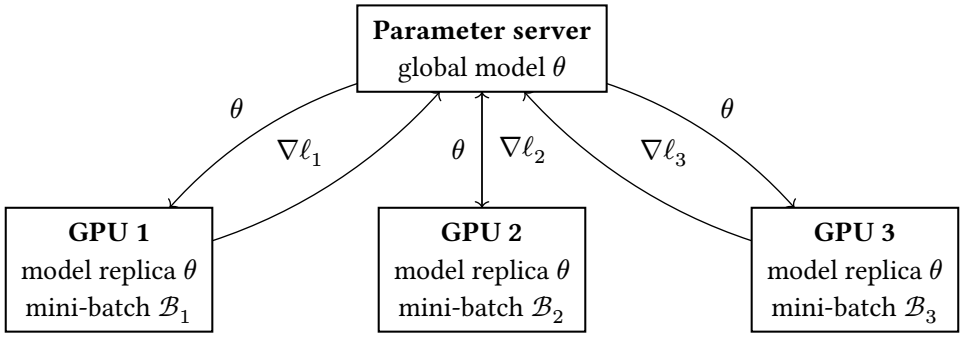

Figure 113: Data parallelism: each GPU holds a replica of the model and processes a distinct mini-batch.

Model parallelism

As neural networks have grown to billions of parameters, storing and training them on a single device has become infeasible: the model simply does not fit within the memory of a single CPU or GPU [126].

Model parallelism addresses this constraint by partitioning the model itself across multiple devices, rather than replicating it as in data parallelism.

The most straightforward form of model parallelism is **pipeline parallelism**, in which the layers of the network are divided into sequential stages, each assigned to a dedicated device. Neural networks are well suited to this form of parallelism: because computation flows naturally from one layer to the next, the model can be split along layer boundaries without altering the learning objective. During the forward pass, each device computes its stage and transmits the resulting activations to the next; during the backward pass, gradients flow in the reverse direction. A practical limitation of this approach is the **pipeline bubble**: when a device is waiting for the output of the preceding stage, it remains idle, reducing overall hardware utilization.

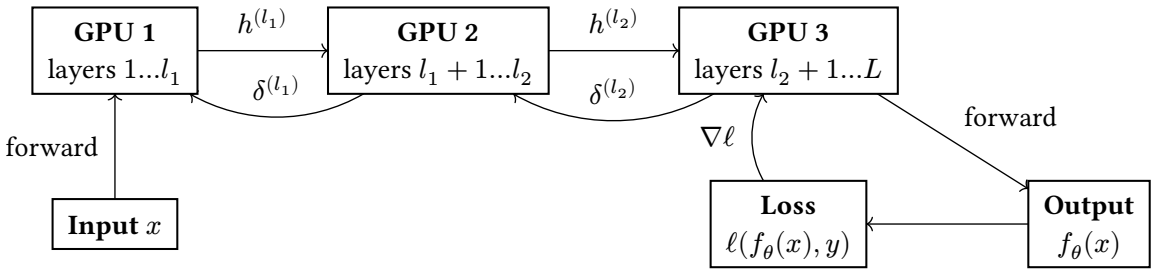

Figure 114: Pipeline parallelism: the layers of the network are partitioned into three stages, each assigned to a dedicated GPU.

A more advanced form is **tensor parallelism**, in which individual operations — such as matrix multiplications within a single layer — are themselves distributed across devices [127]. This approach can be more efficient than pipeline parallelism, as it reduces inter-stage dependencies and better utilises available compute. However, it requires the model architecture to be explicitly designed or adapted for distributed tensor operations, making it harder to implement in practice and not universally applicable.

At the hardware level, Tensor Processing Units (TPUs) embody a related philosophy: they are specialised accelerators built around a systolic array architecture optimised for large-scale matrix and tensor computations, and are designed to efficiently distribute such operations across a large number of processing elements. In this sense, tensor parallelism and TPU-based computation share the same foundational idea of exploiting the structure of tensor operations to achieve scalable parallelism.

While model parallelism enables the training of models that would be infeasible on a single device, it typically incurs higher communication overhead than data parallelism and is more sensitive to latency. In practice, large-scale systems often combine model parallelism and data parallelism to balance memory constraints, computational efficiency, and communication costs [126].

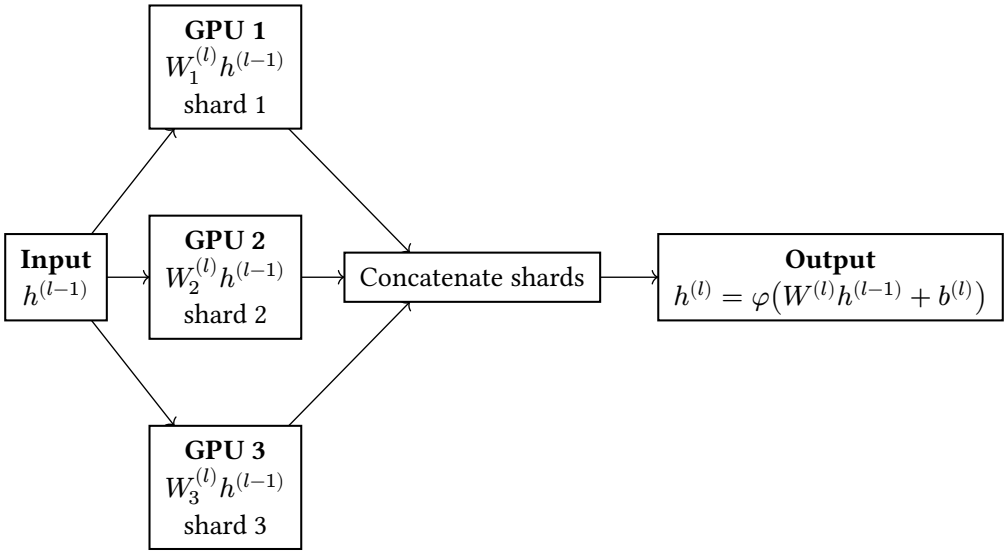

Figure 115: Tensor parallelism: the weight matrix $W^{(l)}$ of a single layer is partitioned into shards, each stored and computed on a dedicated GPU.

Multi-agent reinforcement learning

Multi-Agent Reinforcement Learning (MARL) extends the reinforcement learning framework to settings involving multiple agents that learn and act simultaneously within a shared environment [128]. Unlike supervised learning, which is driven by labeled datasets and a fixed optimization objective, MARL relies on interaction, exploration, and reward signals: each agent aims to learn a policy that maximises its expected cumulative reward, while the dynamics of the environment are jointly shaped by the actions of all agents.

Agents in MARL may be cooperative, competitive, or operate in mixed settings, depending on whether their reward structures are aligned or opposed [128]. In most formulations, agents observe either the same global state or partial views of that state, and must act without full knowledge of the other agents' policies or intentions.

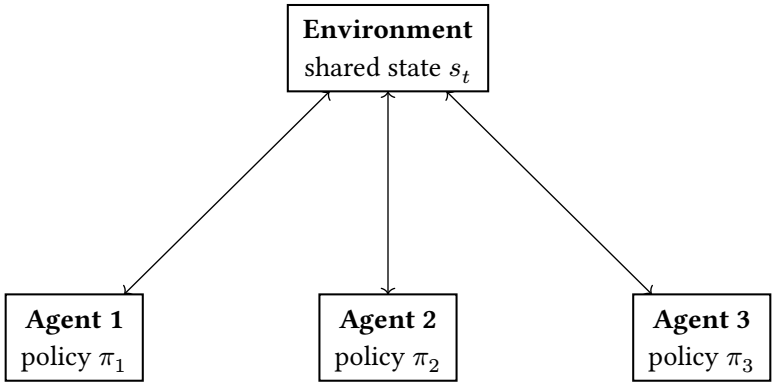

Figure 116: Multi-agent reinforcement learning: three agents interact simultaneously within a shared environment.

This thesis focuses on supervised learning and its distributed variants; MARL therefore falls outside its primary scope. Nevertheless, it is relevant to mention here because it shares structural similarities with decentralized learning: in both paradigms, multiple autonomous learners operate locally, without centralized coordination, and their individual behaviors collectively determine the outcome of the learning process. The key distinction is that MARL agents optimize reward signals through interaction

with an environment, whereas decentralized learning nodes optimize a supervised loss over local datasets.

Transfer learning and fine-tuning

Transfer learning refers to a learning paradigm in which knowledge acquired from one task or domain is reused to improve learning performance on a different, but related, task or domain [129]. A central motivation is the scarcity of labeled data: when a target task does not have sufficient training examples, leveraging representations learned on a larger source dataset can substantially reduce training cost and improve generalisation.

In a typical transfer learning setup, a model is first pretrained on a large source dataset to solve a source task, allowing it to acquire general-purpose representations. The pretrained model is then reused for a target task, which may involve a smaller, noisier, or differently distributed dataset. The source and target tasks may differ in label spaces or objectives, but are assumed to share some underlying structure [129]. A special case of this setting is domain adaptation, in which the task remains the same but the distribution of the data shifts between source and target.

This paradigm has become dominant in modern deep learning: large neural networks pretrained on massive datasets — such as language models or vision transformers — are routinely adapted to downstream tasks, often with limited additional data.

Fine-tuning is a specific instantiation of transfer learning in which the pretrained model is further trained on the target dataset by continuing the optimisation process, typically with a smaller learning rate. Depending on the application, fine-tuning may involve updating all model parameters or only a subset of them, such as the final layers of the network.

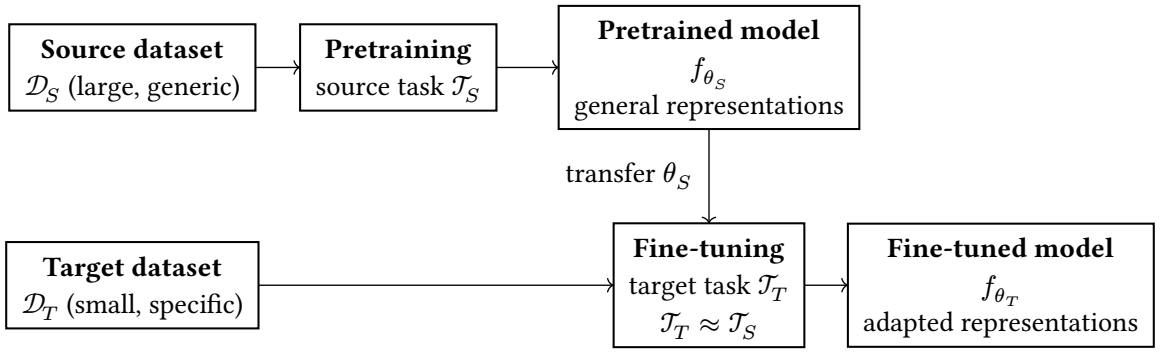

Figure 117: Transfer learning: a model f_{θ_S} is first pretrained on a large source dataset $\mathcal{D}_S$ for a source task $\mathcal{T}_S$. Its parameters θ_S are then transferred to a fine-tuning stage on a smaller target dataset $\mathcal{D}_T$, yielding an adapted model f_{θ_T} suited to the target task $\mathcal{T}_T$. Transfer is effective when $\mathcal{T}_S$ and $\mathcal{T}_T$ share underlying structure.

While transfer learning and fine-tuning are not distributed learning techniques per se, they are often complementary to federated and decentralised learning systems. A common pattern, known as personalised federated learning, is to train a global model in a federated manner and subsequently fine-tune it locally on each node using its private data, thereby adapting the shared representations to each node's local data distribution [130].

A.5 Simulators

Federated and decentralized learning protocols are inherently difficult to evaluate analytically: their behavior depends on the dynamic interplay between network topology, asynchronous message passing, heterogeneous data distributions, and fault injection — conditions that resist closed-form characterisation and demand empirical investigation at scale. Conducting such experiments on

physical infrastructure is costly, barely reproducible, and ill-suited to the systematic exploration of parameter spaces that comparative evaluation requires. Simulation is therefore not an auxiliary tool in this thesis: it is the primary experimental substrate on which the claims of our protocols rest.

This chapter documents a substantial body of work that remained largely invisible in the main chapters. Over the course of this thesis, significant effort was invested in adapting, instrumenting, and validating the simulation frameworks used to evaluate our protocols. The simulators were originally designed for network-level experiments in a different era of distributed systems research; making them suitable for modern federated learning workloads required non-trivial engineering work. This included porting and configuring them for execution on a compute cluster, integrating contemporary tooling for dependency management and job scheduling, implementing missing protocol components, designing reproducible fault-injection mechanisms, and ensuring that the simulation environment faithfully reflects the theoretical model. Without this foundational work, the experimental results presented in Chapter 4 and Chapter 6 would not have been obtainable.

Beyond implementation, this appendix serves a second purpose: reproducibility. Decentralized learning experiments involve many interacting moving parts — simulator configuration, cluster deployment, parameter sweeps, result collection, and post-processing — and the absence of documented procedures is a common obstacle to replication. The following sections therefore describe the full simulation workflow in sufficient detail to allow an independent researcher to reproduce the experimental conditions of our work from scratch: from cluster installation and job submission to result extraction and aggregation.

A.5.1 Peer-to-peer simulations

Simulating peer-to-peer protocols at scale requires a dedicated framework that can handle large, dynamic networks while remaining flexible enough to accommodate the specific requirements of each protocol under study. Several simulators were considered during the preparation of this thesis, including OMNeT++, NS-3, and Shadow, each of which offers high-fidelity network emulation at the cost of complexity, steep learning curves, and implementations primarily in C++. These tools are well-suited for transport-layer and network-layer research but impose unnecessary overhead for protocol-level experiments in which the details of packet scheduling and link simulation are not the primary concern.

PeerSim [54] was selected as the simulation platform for this thesis. The choice is motivated by several factors. PeerSim has an established presence in the peer-to-peer research community: it has been used to evaluate foundational gossip-based protocols such as gossip-based peer sampling [21], and continues to appear in recent decentralized systems publications such as SecureCyclon [51]. Its component model is explicitly modular — protocols, topologies, and observers are interchangeable building blocks — and the distribution ships with a large library of ready-to-use components covering overlay construction, aggregation, and topology management. In principle, the cycle-based engine scales to networks of hundreds of thousands of nodes, which covers the range of sizes evaluated in this thesis. PeerSim natively supports dynamic graphs: nodes may join, leave, or fail during a simulation, and the overlay topology evolves accordingly, making it suitable for the churn and fault scenarios of Chapter 6. Finally, PeerSim is written in Java, which is considerably more accessible than the C++ codebases of alternative simulators and facilitated the substantial extensions described in Appendix A.5.1.4.

PeerSim is open-source, developed at the University of Bologna, and designed specifically for large-scale peer-to-peer protocol research. Its guiding design principles are extreme scalability and support for dynamic network membership: nodes may join and leave continuously, and the simulator has been demonstrated to handle networks of millions of nodes. PeerSim is structured around pluggable, interchangeable components — protocols, initializers, and control objects — combined through a plain-

text configuration file, making it straightforward to swap overlay topologies, aggregation strategies, or fault models without modifying protocol code.

PeerSim provides two distinct simulation engines. The *cycle-based engine* operates under simplifying assumptions: there is no transport layer simulation and no concurrency. Instead, nodes are granted control sequentially at each discrete cycle, during which they may exchange state with neighbors and perform local computation. This model trades realism for performance, enabling scalable parameter sweeps over large networks. The *event-based engine* is more realistic — it models transport-layer behavior and supports asynchronous message passing — at the cost of higher computational overhead. Cycle-based protocols can additionally be executed under the event-based engine, providing a migration path when fidelity requirements increase. The experiments of this thesis rely on the cycle-based engine, whose simplifying assumptions are appropriate for the epidemic and gossip protocols studied.

Architecture

A PeerSim simulation is composed of four kinds of objects, each implementing a dedicated interface.

A **`Node`** is a container for protocols. It provides access to all protocol instances it holds and exposes a unique, fixed numeric identifier.

A **`CDProtocol`** (cycle-driven protocol) defines the behavior executed by each node at every cycle via a `nextCycle()` method. This is the primary extension point for implementing new distributed algorithms.

A **`Linkable`** defines the local neighborhood of a node — its partial view of the overlay. Rather than managing neighbor lists internally, protocols delegate topology access to a separate `Linkable` component, allowing any overlay (random graph, Newscast, T-Man, etc.) to be composed with any protocol without code changes.

A **`Control`** object is scheduled for execution at configurable points during the simulation. Controls serve two roles: *initializers* (prefixed `init` in the configuration file) set up the initial state of nodes and protocols before the first cycle; *observers* collect metrics and emit them to standard output, which may be redirected to a file for post-processing. The same interface covers both roles; the distinction is purely one of scheduling.

The simulation life-cycle proceeds as follows. PeerSim reads a plain-text configuration file specifying network size, protocol classes, initializer order, control schedules, and all component-level parameters. It then constructs the node array by cloning a single prototype instance of each protocol — a design that requires careful implementation of the `clone()` method in any custom protocol. After initialization controls have run, the cycle-driven engine repeatedly invokes all active components once per cycle, in a configurable order, until the target number of cycles is reached or a component signals termination. Metric output is written to standard output and is trivially redirectable to a file for collection.

Configuration

Experiments are fully described by a configuration file consisting of key–value pairs. Global parameters (network size, cycle count, random seed) are declared at the top level; component-specific parameters follow the pattern `<type>.<name>.<parameter>`. For example:

```
network.size        10000
simulation.cycles   100
random.seed         1234567890
protocol.lnk        IdleProtocol
protocol.avg        example.aggregation.AverageFunction
protocol.avg.linkable lnk
init.rnd            WireKOut
```

```
init.rnd.protocol   lnk
init.rnd.k          20
control.obs         example.aggregation.AverageObserver
control.obs.protocol avg
```

This configuration defines a simulation of 10,000 nodes running for 100 cycles. The random seed is fixed to ensure full reproducibility of all stochastic elements of the simulation. Two protocols are declared. The first, `IdleProtocol`, is a passive link container: it holds the neighbor list of each node but executes no logic of its own, modeling a static, non-adaptive overlay network. The second, `AverageFunction`, implements gossip-based aggregation: at each cycle, every node contacts one neighbor drawn from its local view and replaces its current value with the average of the two. The `linkable` parameter binds `AverageFunction` to `IdleProtocol`, specifying that peer selection must consult the static neighbor list. The initializer `WireKOut` wires the overlay before the first cycle by assigning exactly $k = 20$ outgoing links to each node chosen uniformly at random, producing a 20-regular random directed graph. Finally, `AverageObserver` is a control object scheduled at every cycle: it traverses all nodes, computes network-wide statistics over their current values (minimum, maximum, mean, and variance), and emits them to standard output, where they can be redirected to a file for post-processing.

The configuration format supports arithmetic expressions via the Java Expression Parser, allowing derived quantities (e.g. `SIZE 2^MAG`) to be defined once and referenced throughout the file, which reduces transcription errors in parameter sweeps.

Simulation protocol

Engine selection

Among the two simulation engines offered by PeerSim, the cycle-based engine was selected for all experiments in this thesis. The event-based engine is designed to approximate real deployment conditions as closely as possible — asynchronous message delivery, concurrent execution, and transport-layer simulation — which makes it well-suited for high-fidelity validation. However, Elevator had already been evaluated in a real distributed environment; a second round of realistic execution-level experiments was therefore not the primary objective here. The cycle-based engine, by contrast, executes node logic sequentially and deterministically within each round, which offers two practical advantages: parameter sweeps are fully reproducible given a fixed random seed, and the absence of concurrency eliminates a class of non-deterministic artifacts that would complicate the interpretation of aggregate metrics. These properties make it the appropriate choice for the comparative, large-scale evaluation carried out in Chapter 4 and Chapter 6.

Experimental workflow

Each simulation runs for 1,000 cycles. To reduce the influence of random initialization — topology wiring, initial model weights, failure sampling — every experimental condition is repeated 100 times with distinct seeds, and all reported metrics are averaged across these repetitions. The full workflow proceeds in five stages.

Stage 1 — Simulation execution. Each of the 100 runs for a given condition is launched independently, producing one output stream per run. A dedicated observer, scheduled at every cycle, serializes the current state of the overlay graph to a file in GML format at the end of each round. A run of 1,000 cycles therefore produces 1,000 GML snapshots, each capturing the full adjacency structure and per-node protocol state at that point in time.

Stage 2 — Result collection. Upon completion, the GML files from all 100 runs are collected and organized by condition, network size, and cycle index, forming a structured archive that constitutes the raw experimental record.

Stage 3 — Metric computation. A post-processing script reads the GML files, reconstructs the graph at each cycle, and computes the metrics of interest — convergence rate, model accuracy, connectivity properties, and protocol-specific indicators. For each metric and each cycle index, the 100 values obtained across repetitions are averaged, yielding a smooth, low-variance time series that reflects the expected behavior of the protocol rather than the outcome of any single run. The data is stored as CSV files.

Stage 4 — Visualization. The aggregated time series (CSV files) are passed to a plotting pipeline that produces the figures reported in Chapter 4. Each figure is generated programmatically from the metric files, ensuring that all visual results are directly traceable to the underlying data.

Stage 5 — Condition sweep. Stages 1 through 4 are repeated for every experimental condition in the evaluation matrix. Conditions vary along multiple axes: fault scenario (fault-free, node failures, churn, …), protocol variant, and network size (100, 200, 500, 1000, …). The total number of simulations executed across all conditions is therefore substantial.

Engineering contributions

The experimental infrastructure underlying this thesis required substantial software engineering work across three independent components: an extensively modified version of PeerSim, a cluster deployment and orchestration system, and two successive generations of metric computation and visualization tooling. Each is described in turn.

PeerSim extensions

The modifications made to PeerSim over the course of this thesis are extensive enough to constitute a distinct fork of the simulator. The original codebase was hosted on Subversion (SVN); the first step was migrating the project to Git to enable proper version control, branching, and collaboration. Dependency management was absent from the original project: third-party libraries were bundled as untracked JAR files. The build system was rewritten to use Gradle, which formalizes dependency declarations, reproducible builds, and installation. A Dockerfile was added to containerize the simulator, and a GitLab CI/CD pipeline was configured to enforce automated builds and tests on every commit. The code is available at https://gitlab.lip6.fr/legheraba/peersim.

The most significant technical contribution was parallelizing the cycle-based engine. The original implementation was entirely sequential: a single thread drove the simulation loop, visiting each node in turn. The core data structures were not thread-safe, and making the engine concurrent required modifying nearly every source file to replace shared mutable state with thread-safe equivalents. On top of this foundation, a simulation-level parallelism mode was implemented: multiple independent runs, each with a distinct seed, execute concurrently across available cores. Because the runs share no state, this constitutes an embarrassingly parallel workload, and throughput scales linearly with the number of cores allocated. General performance optimizations were carried out alongside this work to reduce per-cycle overhead.

On the protocol side, fault models were implemented from scratch: static node failures with graceful disconnection, churn (concurrent arrivals and departures), and node addition following a configurable random process. Byzantine fault tolerance was also integrated into PeerSim, implementing the attack models defined in Chapter 4. This required embedding Byzantine behavior at the core of the simulator, which was not designed with adversarial node models in mind. A dedicated class hierarchy was introduced, in which Byzantine node classes extend the standard node class and override its communication and aggregation behavior to inject the targeted attack. When the overlay state is serialized to GML at the end of each cycle, Byzantine nodes are annotated with a distinguishing label, allowing down-

stream metric computation tools to identify them and compute protocol-specific resilience metrics accordingly.

The following protocols were implemented in full, each derived from the pseudocode of the corresponding publication: Elevator (with multiple configuration variants, Chapter 4), Chord, Proofs, Phenix, and Fedlay. Finally, a machine learning layer was implemented in Java and integrated directly into PeerSim, with the aim of running decentralized federated learning experiments entirely within the simulator. Although this approach was ultimately superseded by the Gossipy-based evaluation reported in Chapter 6, it established the feasibility of in-simulator learning and informed the design of the final experimental setup.

Cluster deployment and orchestration

PeerSim was deployed on a Slurm-managed compute cluster via Docker. Each experimental condition — a combination of protocol, network size, fault scenario, and repetition count — is described by a dedicated Slurm job script that invokes the containerized simulator with the appropriate configuration file and seed range. A top-level orchestration layer ties all job scripts together so that the entire experimental campaign can be (re)launched with a single command. Once triggered, the workflow requires no further human intervention: jobs are queued, executed, and their GML output files collected automatically. The result is a fully reproducible experimental pipeline in which the mapping from a configuration parameter to its corresponding raw output is unambiguous and auditable.

Metric computation and visualization

Two generations of tooling were developed to process the GML output files and produce the figures reported in this thesis.

The first tool, `compute-metrics`[31], was written in Python and Java. Graph-theoretic metrics were computed using JGraphT, chosen over pure Python because NetworkX proved too slow and difficult to parallelize at the scale of the experiments; Python handled aggregation and figure generation via Matplotlib. Like PeerSim itself, `compute-metrics` was version-controlled on Git, containerized with Docker, covered by a GitLab CI/CD pipeline, and executed on the cluster via Slurm. Metric data and generated figures were committed directly to the repository, providing a persistent, indexed record of all results.

After two years of thesis work, `compute-metrics` was rewritten from scratch in Julia[32] as a new tool named `graph-metrics`[33]. The rewrite was motivated by the desire for a single, uniform language covering both metric computation and visualization, with better performance and simpler parallelism than the Python–Java combination. `graph-metrics` is designed as a command-line interface: metrics and figures are requested by issuing CLI commands with explicit parameters, and the tool automatically resolves the corresponding GML files from a structured directory hierarchy organized by protocol, network size, and fault context. The same DevOps practices apply: Git versioning, GitLab CI/CD, Docker containerization, and co-location of data and figures in the repository.

A.5.2 Decentralized learning simulations

Candidate tools and their limitations

Unlike peer-to-peer protocol simulation, which benefits from two decades of dedicated tooling, the simulation of federated and decentralized learning is a recent and still maturing field. No established, general-purpose simulator exists for decentralized learning at the time of writing, and the evaluation

[31] https://gitlab.lip6.fr/legheraba/compute-metrics
[32] https://julialang.org/
[33] https://gitlab.lip6.fr/legheraba/graph-metrics

of a new protocol such as HEAL required navigating a fragmented landscape of partial solutions, each with significant limitations.

Several candidate platforms were evaluated before settling on the final approach. Flower [131] is a widely used federated learning framework that includes a simulation mode, but its architecture is centered on the canonical server–client topology of standard federated learning. Adapting it to a fully decentralized, serverless setting would have required deep modifications to its aggregation and communication abstractions, amounting to a near-total rewrite of its core coordination layer. This effort was judged disproportionate given that the resulting tool would remain tightly coupled to Flower's design assumptions.

Adding machine learning capabilities directly to OMNeT++ and NS-3 was also considered. Both are mature network simulators with rich protocol libraries, but they are designed for physical and link-layer network modeling: their primary abstractions are packets, interfaces, and routing tables. Overlay networks and model-averaging protocols fit poorly into this paradigm, and the integration of a machine learning layer would have required bridging fundamentally incompatible levels of abstraction.

The option of extending PeerSim with machine learning support was pursued more seriously, and is described in Appendix A.5.1.4. This direction had a precedent: the authors of gossip learning [74] implemented machine learning models directly in Java within PeerSim, without relying on any external library, by reimplementing the required model architectures by hand. While functional, this approach is inflexible: every new model architecture requires a full manual reimplementation, and keeping custom Java implementations consistent with modern deep learning frameworks is unsustainable as model complexity grows. An alternative approach — bridging PeerSim to PyTorch by serializing model parameters across the Java–Python boundary — was attempted but proved impractical: the serialization overhead and the complexity of managing two runtimes synchronously introduced more fragility than it resolved.

Gossipy [55], a Python-based gossip learning simulator, was also evaluated as a potential base. Gossipy natively supports machine learning workloads and provides a clean implementation of several gossip protocols, but its overlay model is not designed to accommodate custom peer-to-peer protocols such as Newscast or Elevator. Adapting it to support dynamic networks would have required restructuring its core communication layer.

A final candidate, decentralizepy [132], was developed by the authors of Epidemic Learning and is the closest existing tool to what this thesis requires: a simulator designed explicitly for decentralized machine learning over overlay networks. Decentralizepy was studied in depth over several months and partial experiments were conducted with it. However, the simulator was at an early stage of development at the time of evaluation: recurring crashes and instabilities under the experimental conditions of interest made it impossible to obtain reliable results, and the effort required to stabilize it for production use was judged excessive.

In light of these evaluations, a hybrid approach was adopted, combining PeerSim and Gossipy: PeerSim handles overlay topology construction and peer-to-peer protocol logic, while Gossipy provides the machine learning substrate.

Hybrid simulation protocol

The simulation of decentralized learning experiments in this thesis follows a two-phase hybrid approach that combines PeerSim and Gossipy, each handling the aspects of the problem for which it is best suited.

Phase 1: topology generation

The first phase produces the sequence of GML snapshots consumed by Gossipy in phase 2. Two distinct generation paths are used depending on whether the experiment involves a static or a dynamic topology.

For dynamic topologies — those involving node failures, churn, or protocol-driven overlay evolution — PeerSim is used as described in Appendix A.5.1.3. The simulator produces one GML snapshot per cycle, capturing node identities, active connections, and any structural changes induced by the fault scenario, yielding a complete, deterministic record of the overlay's evolution over the 1,000 cycles of the experiment.

For static topologies, PeerSim is not involved. Instead, a dedicated command-line tool, `topology_generator`[34], was developed to produce fixed GML graphs using NetworkX. The tool accepts parameters controlling the graph family, network size, and degree, and supports the following topology types: random graph, power-law, ring, star, multi-star, and Chord. Since the topology does not change over time, a single GML file is generated and reused identically at every cycle. This avoids running PeerSim for conditions in which no dynamic behavior is needed, and ensures that static baseline topologies are generated by a well-tested graph library rather than by the simulator's wiring routines. In both cases, the GML files produced by this phase constitute the sole topological input to phase 2.

Phase 2: decentralized learning with Gossipy

Gossipy was extended to consume the GML files produced by PeerSim. At the start of each cycle, the simulator reads the corresponding GML snapshot and reconstructs the overlay graph, determining which nodes are active and which connections are available for that round. Each active node then executes its machine learning algorithm for that cycle: local model update, peer selection according to the aggregation protocol, and model exchange with neighbors. At the end of each cycle, the per-node accuracy is evaluated and written to a CSV file. Once all cycles have been processed, Matplotlib is used to generate figures showing the evolution of accuracy over time, averaged across the 5 repetitions of the experiment.

The aggregation protocols evaluated in this thesis were each implemented within Gossipy. Gossip learning and Epidemic learning were already partially available but required adaptation to handle dynamic network membership. HEAL required a full implementation of its multi-phase protocol: local training, hub selection, intra-hub aggregation, inter-hub aggregation, and model broadcast, each executed in the correct sequence within the Gossipy cycle loop.

Fault handling

The hybrid architecture handles the three fault scenarios of Chapter 6 as follows. For static node failures, a node that is marked as down in the PeerSim GML at cycle t ceases all learning activity from that cycle onward: it neither performs local training nor participates in aggregation. The same policy applies to nodes that depart under churn. Nodes that join the network during churn are initialized with a randomly drawn model; they immediately begin participating in aggregation and progressively converge toward the rest of the network through repeated model exchanges. This mirrors the situation of a late-joining participant in a real decentralized system, which has no prior knowledge of the models already in circulation and must recover through the protocol's natural convergence mechanism.

Deployment and reproducibility

Gossipy was adapted for execution on the Slurm cluster following the same DevOps practices applied to PeerSim. Each experimental condition is described by a Slurm job script that configures a Conda virtual environment with all required dependencies, specifies the path to the relevant GML files, and

[34] https://gitlab.lip6.fr/legheraba/topology_generator

launches the Gossipy simulation with the appropriate parameters. A GitLab CI/CD pipeline validates the environment on every commit. As with the PeerSim pipeline, metric data and generated figures are committed to the repository, providing a versioned, indexed record of all experimental results.

Limitations

The hybrid approach rests on the assumption that topology and learning dynamics can be decoupled: PeerSim determines who is connected to whom at each cycle, and Gossipy determines what is exchanged. This separation is a simplification. In a real deployment, the outcome of a learning round may influence subsequent peer selection decisions, and the latency of model transfers would affect the effective topology seen by each node. The present protocol does not capture these feedback effects. It is nonetheless a principled compromise: it preserves the structural realism of the overlay dynamics, ensures full reproducibility, and allows the aggregation protocols to be evaluated under identical, controlled topological conditions — which is the primary requirement for the comparative analysis of Chapter 6.

FLAIR simulations on ns-3

The simulation of FLAIR, introduced in Chapter 6, required a different infrastructure from the PeerSim–Gossipy hybrid used for HEAL. FLAIR targets physical wireless networks, and its evaluation therefore demands a simulator that faithfully models radio propagation, interference, and the MAC-layer behavior of Wi-Fi links. ns-3 was selected for this purpose, as it provides mature and well-validated Wi-Fi network components that cover the physical and link layers required by the FLAIR evaluation scenarios.

The ns-3 distribution already includes the necessary network-level building blocks: Wi-Fi channel models, IEEE 802.11 MAC implementations, and node mobility support. What it does not provide is any machine learning substrate. Integrating a learning layer into ns-3 raised the same fundamental challenge encountered throughout the simulator evaluation process: bridging a network simulation framework with machine learning primitives.

Several integration strategies were considered. The ns-3 Python bindings offer a scripting interface to the simulator, which in principle allows Python-side machine learning code to be called from within a simulation. However, this interface imposes significant overhead at the boundary between the C++ simulation core and the Python interpreter, and its support for fine-grained per-node, per-cycle callbacks is limited. A second approach — connecting ns-3 to PyTorch through model serialization, analogous to the PeerSim–PyTorch bridge attempted earlier — was evaluated but discarded for the same reasons: the serialization overhead and the complexity of synchronizing two independent runtimes proved unmanageable at the scale of the experiments. A third option, a dedicated ns-3 machine learning package, was also examined but found to be insufficiently mature for the model architectures required by FLAIR.

The approach ultimately retained was to reimplement the required machine learning models directly in C++, within the ns-3 codebase. This decision carries a significant development cost: model architectures, training routines, and aggregation operators must all be written from scratch without the support of an automatic differentiation framework. It yields, however, substantial advantages in return. The simulation is entirely self-contained: there are no external runtime dependencies, no serialization boundaries, and no synchronization overhead. Launch procedures are identical to those of any standard ns-3 simulation. Performance is optimal, as the learning computations execute natively within the same process as the network simulation. The resulting setup allowed FLAIR to be evaluated under realistic wireless conditions with full control over the experimental parameters, at the cost of a longer initial development phase.

A.5.3 Conclusion

The simulation infrastructure described in this chapter was not built in a straight line. The exploration of candidate platforms, the dead ends, the partial implementations, and the successive tool rewrites consumed a significant share of the time and energy invested in this thesis. This cost was not without return. Each evaluated platform — whether ultimately adopted or abandoned — deepened the understanding of what a decentralized learning simulator must provide, and each implementation effort, including those that were discarded, sharpened the understanding of the protocols themselves: their failure modes, their sensitivity to topology, and the subtleties of their aggregation logic that only become apparent when one attempts to implement them faithfully.

The hybrid PeerSim–Gossipy architecture that emerged from this process is a pragmatic solution rather than an ideal one. Its limitations are acknowledged in Appendix A.5.2.2. It was, however, sufficient to produce the experimental results of Chapter 4 and Chapter 6, and the engineering discipline applied throughout — reproducible builds, containerization, automated pipelines, versioned data — means that those results are fully auditable and re-runnable.

Toward the end of this thesis, work began on a new simulator designed from scratch to address the shortcomings identified over the preceding years. The tool is written entirely in Python, structured as a modular, extensible library with a command-line interface designed to make experimental configuration and execution straightforward. The simulation engine is fully parallelized from the ground up. The machine learning layer is based on NumPy rather than PyTorch: a deliberate choice motivated by the observation that the model operations performed in decentralized learning — weighted averaging, gradient accumulation, parameter broadcasting — do not require the full apparatus of an automatic differentiation framework, and that a lighter dependency leads to faster simulation and simpler deployment. The simulator natively supports dynamic network conditions: node failures, churn, and Byzantine participants. It also handles heterogeneous data distributions and personalized learning scenarios, two dimensions that the current infrastructure addresses only partially. The tool is still under active development, but early results are encouraging.

Bibliography

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*, 2017, pp. 1273–1282.

[2] C. Cachin, R. Guerraoui, and L. Rodrigues, *Introduction to reliable and secure distributed programming*. Springer Science & Business Media, 2011.

[3] J. Poon and T. Dryja, "The bitcoin lightning network: Scalable off-chain instant payments." Seattle, WA, USA, 2016.

[4] R. Diestel, *Graph theory*. Springer Nature, 2016.

[5] D. Peleg, *Distributed computing: a locality-sensitive approach*. SIAM, 2000.

[6] D. Stutzbach and R. Rejaie, "Understanding churn in peer-to-peer networks," in *Proceedings of the 6th ACM SIGCOMM conference on Internet measurement*, 2006, pp. 189–202.

[7] P. Holme and J. Saramäki, "Temporal networks," *Physics reports*, vol. 519, no. 3, pp. 97–125, 2012.

[8] M. Raynal, *Fault-tolerant message-passing distributed systems: an algorithmic approach*. springer, 2018.

[9] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," *ACM SIGCOMM computer communication review*, vol. 31, no. 4, pp. 149–160, 2001.

[10] A. Rowstron and P. Druschel, "Pastry: Scalable, decentralized object location, and routing for large-scale peer-to-peer systems," in *IFIP/ACM International Conference on Distributed Systems Platforms and Open Distributed Processing*, 2001, pp. 329–350.

[11] P. Maymounkov and D. Mazieres, "Kademlia: A peer-to-peer information system based on the xor metric," in *International workshop on peer-to-peer systems*, 2002, pp. 53–65.

[12] T. Clouser, M. Nesterenko, and C. Scheideler, "Tiara: A self-stabilizing deterministic skip list and skip graph," *Theoretical Computer Science*, vol. 428, pp. 18–35, 2012.

[13] P. T. Eugster, R. Guerraoui, S. B. Handurukande, P. Kouznetsov, and A.-M. Kermarrec, "Lightweight probabilistic broadcast," *ACM Transactions on Computer Systems (TOCS)*, vol. 21, no. 4, pp. 341–374, 2003.

[14] A. Montresor, H. Meling, and Ö. Babaoğlu, "Toward self-organizing, self-repairing and resilient distributed systems," *Future Directions in Distributed Computing: Research and Position Papers*. Springer, pp. 119–123, 2003.

[15] R. Van Renesse, K. P. Birman, and W. Vogels, "Astrolabe: A robust and scalable technology for distributed system monitoring, management, and data mining," *ACM transactions on computer systems (TOCS)*, vol. 21, no. 2, pp. 164–206, 2003.

[16] A. J. Ganesh, A.-M. Kermarrec, and L. Massoulié, "Peer-to-peer membership management for gossip-based protocols," *IEEE transactions on computers*, vol. 52, no. 2, pp. 139–149, 2003.

[17] R. Melamed and I. Keidar, "Araneola: A scalable reliable multicast system for dynamic environments," in *Third IEEE International Symposium on Network Computing and Applications, 2004. (NCA 2004). Proceedings.*, 2004, pp. 5–14.

[18] A. Stavrou, D. Rubenstein, and S. Sahu, "A lightweight, robust P2P system to handle flash crowds," *IEEE Journal on Selected Areas in Communications*, vol. 22, no. 1, pp. 6–17, 2004.

[19] M. Jelasity, R. Guerraoui, A.-M. Kermarrec, and M. Van Steen, "The peer sampling service: Experimental evaluation of unstructured gossip-based implementations," in *ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing*, 2004, pp. 79–98.

[20] M. Jelasity, W. Kowalczyk, and M. Van Steen, "Newscast computing," 2003.

[21] M. Jelasity, S. Voulgaris, R. Guerraoui, A.-M. Kermarrec, and M. Van Steen, "Gossip-based peer sampling," *ACM Transactions on Computer Systems (TOCS)*, vol. 25, no. 3, p. 8–es, 2007.

[22] A. Allavena, A. Demers, and J. E. Hopcroft, "Correctness of a gossip based membership protocol," in *Proceedings of the twenty-fourth annual ACM symposium on Principles of distributed computing*, 2005, pp. 292–301.

[23] S. Voulgaris, D. Gavidia, and M. Van Steen, "Cyclon: Inexpensive membership management for unstructured p2p overlays," *Journal of Network and systems Management*, vol. 13, no. 2, pp. 197–217, 2005.

[24] L. Massoulié, E. Le Merrer, A.-M. Kermarrec, and A. Ganesh, "Peer counting and sampling in overlay networks: random walk methods," in *Proceedings of the twenty-fifth annual ACM symposium on Principles of distributed computing*, 2006, pp. 123–132.

[25] H. Terelius and K. H. Johansson, "Peer-to-peer gradient topologies in networks with churn," *IEEE Transactions on Control of Network Systems*, vol. 5, no. 4, pp. 2085–2095, 2018.

[26] J. Leitao, J. Pereira, and L. Rodrigues, "HyParView: A membership protocol for reliable gossip-based broadcast," in *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, 2007, pp. 419–429.

[27] C. S. Meiklejohn, H. Miller, and P. Alvaro, "{PARTISAN}: Scaling the Distributed Actor Runtime", in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 63–76.

[28] J. C. A. Leitao, J. P. d. S. F. Moura, J. O. R. N. Pereira, L. E. T. Rodrigues, and others, "X-bot: A protocol for resilient optimization of unstructured overlays," in *2009 28th IEEE International Symposium on Reliable Distributed Systems*, 2009, pp. 236–245.

[29] A. Montresor and others, "Gossip and epidemic protocols," *Wiley encyclopedia of electrical and electronics engineering*, vol. 1, p. 4, 2017.

[30] Y. Chawathe, S. Ratnasamy, L. Breslau, N. Lanham, and S. Shenker, "Making gnutella-like p2p systems scalable," in *Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications*, 2003, pp. 407–418.

[31] A. Montresor, "A robust protocol for building superpeer overlay topologies," in *Proceedings. Fourth International Conference on Peer-to-Peer Computing, 2004. Proceedings.*, 2004, pp. 202–209.

[32] R. H. Wouhaybi and A. T. Campbell, "Phenix: Supporting resilient low-diameter peer-to-peer topologies," in *IEEE INFOCOM 2004*, 2004.

[33] M. Sasabe, N. Wakamiya, and M. Murata, "LLR: A construction scheme of a low-diameter, location-aware, and resilient p2p network," in *2006 International Conference on Collaborative Computing: Networking, Applications and Worksharing*, 2006, pp. 1–8.

[34] V. Vishnumurthy and P. Francis, "On heterogeneous overlay construction and random node selection in unstructured p2p networks," in *Proceedings IEEE INFOCOM 2006. 25TH IEEE International Conference on Computer Communications*, 2006, pp. 1–12.

[35] A. Brocco, F. Frapolli, and B. Hirsbrunner, "Bounded diameter overlay construction: A self organized approach," in *2009 IEEE Swarm Intelligence Symposium*, 2009, pp. 114–121.

[36] H. Guclu and M. Yuksel, "Limited scale-free overlay topologies for unstructured peer-to-peer networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 20, no. 5, pp. 667–679, 2008.

[37] S. Eum, S. Arakawa, and M. Murata, "Self-organizing scale free topology for peer-to-peer networks," in *2009 IEEE Globecom Workshops*, 2009, pp. 1–6.

[38] M. Jelasity, A. Montresor, and O. Babaoglu, "T-man: Gossip-based fast overlay topology construction," *Computer networks*, vol. 53, no. 13, pp. 2321–2339, 2009.

[39] S. Voulgaris and M. Van Steen, "Vicinity: A pinch of randomness brings out the structure," in *ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing*, 2013, pp. 21–40.

[40] M. Ripeanu, A. Iamnitchi, I. Foster, and A. Rogers, "In search of simplicity: a self-organizing group communication overlay," *Concurrency and Computation: Practice and Experience*, vol. 22, no. 7, pp. 788–815, 2010.

[41] E. Bulut and B. K. Szymanski, "Constructing limited scale-free topologies over peer-to-peer networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 4, pp. 919–928, 2013.

[42] V. Marza, M. Dehghan, and B. Akbari, "A new Peer-to-Peer topology for video streaming based on complex network theory," *Journal of Systems Science and Complexity*, vol. 28, no. 1, pp. 16–29, 2015.

[43] J.-H. Park, "Distributed algorithm for making scale-free network by preferential rewiring without growth," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 6, pp. 3125–3132, 2018.

[44] C. T. Diggans, J. Fish, and E. M. Bollt, "Emergent Hierarchy Through Conductance-based Degree Constraints," *Northeast Journal of Complex Systems (NEJCS)*, vol. 3, no. 1, p. 4, 2021.

[45] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *science*, vol. 286, no. 5439, pp. 509–512, 1999.

[46] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Trans. Program. Lang. Syst.*, vol. 4, no. 3, pp. 382–401, July 1982, doi: 10.1145/357172.357176.

[47] E. Bortnikov, M. Gurevich, I. Keidar, G. Kliot, and A. Shraer, "Brahms: Byzantine resilient random membership sampling," in *Proceedings of the twenty-seventh ACM symposium on Principles of distributed computing*, 2008, pp. 145–154.

[48] E. Anceaume, Y. Busnel, and B. Sericola, "Byzantine-tolerant uniform node sampling service in large-scale networks," *International Journal of Parallel, Emergent and Distributed Systems*, vol. 36, no. 5, pp. 412–439, 2021.

[49] A. Auvolat, Y.-D. Bromberg, D. Frey, and F. Taïani, "BASALT: A Rock-Solid Foundation for Epidemic Consensus Algorithms in Very Large, Very Open Networks," in *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, ACM, 2021, pp. 231–240.

[50] G. P. Jesi, A. Montresor, and M. van Steen, "Secure peer sampling," *Computer Networks*, vol. 54, no. 12, pp. 2086–2098, 2010.

[51] A. Antonov and S. Voulgaris, "SecureCyclon: Dependable Peer Sampling," in *2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*, 2023, pp. 1–12.

[52] A. Mukam, J. Bruneau-Queyreix, and L. Réveillère, "AUPE: Collaborative Byzantine fault-tolerant peer-sampling," in *2024 22nd International Symposium on Network Computing and Applications (NCA)*, 2024, pp. 17–24.

[53] D. E. Knuth, *The Art of Computer Programming, Volume 3: Seminumerical Algorithms*, 2nd ed. Addison-Wesley, 1997.

[54] A. Montresor and M. Jelasity, "PeerSim: A Scalable P2P Simulator," in *Proc. of the 9th Int. Conference on Peer-to-Peer (P2P'09)*, Seattle, WA, Sept. 2009, pp. 99–100.

[55] R. Ormándi, I. Hegedűs, and M. Jelasity, "Gossip learning with linear models on fully distributed data," *Concurrency and Computation: Practice and Experience*, vol. 25, no. 4, pp. 556–571, 2013.

[56] Y.-B. Xie, T. Zhou, and B.-H. Wang, "Scale-free networks without growth," *Physica A: Statistical Mechanics and its Applications*, vol. 387, no. 7, pp. 1683–1688, 2008.

[57] C. W. Lynn, C. M. Holmes, and S. E. Palmer, "Emergent scale-free networks," *PNAS Nexus*, p. pgae236, 2024.

[58] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowledge-Based Systems*, vol. 216, p. 106775, 2021.

[59] N. Rodr\iguez-Barroso, D. Jiménez-López, M. V. Luzón, F. Herrera, and E. Mart\inez-Cámara, "Survey on federated learning threats: Concepts, taxonomy on attacks and defences, experimental study and challenges," *Information Fusion*, vol. 90, pp. 148–173, 2023.

[60] S. Biswas *et al.*, "Low-cost privacy-preserving decentralized learning," *arXiv preprint arXiv:2403.11795*, 2024.

[61] A. Pham, M. Potop-Butucaru, S. Tixeuil, and S. Fdida, "Data poisoning attacks in gossip learning," in *International Conference on Advanced Information Networking and Applications*, 2024, pp. 213–224.

[62] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *International conference on artificial intelligence and statistics*, 2020, pp. 2938–2948.

[63] S. V. Albrecht, F. Christianos, and L. Schäfer, *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. [Online]. Available: https://www.marl-book.com/

[64] M. Zinkevich, M. Weimer, L. Li, and A. Smola, "Parallelized stochastic gradient descent," *Advances in neural information processing systems*, vol. 23, 2010.

[65] X. Lian, C. Zhang, H. Zhang, C.-J. Hsieh, W. Zhang, and J. Liu, "Can decentralized algorithms outperform centralized algorithms? a case study for decentralized parallel stochastic gradient descent," *Advances in neural information processing systems*, vol. 30, 2017.

[66] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International conference on machine learning*, 2020, pp. 5132–5143.

[67] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," *Advances in neural information processing systems*, vol. 30, 2017.

[68] K. Hsieh *et al.*, "Gaia:{Geo-Distributed} machine learning approaching {LAN} speeds", in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 629–647.

[69] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *ICC 2020-2020 IEEE international conference on communications (ICC)*, 2020, pp. 1–6.

[70] T. An, S. Fdida, M. Potop-Butucaru, and S. Tixeuil, "Abd-hfl: byzantine-resistant decentralized hierarchical federated learning," in *Proceedings of the 37th ACM Symposium on Parallelism in Algorithms and Architectures*, 2025, pp. 62–74.

[71] Z. Wang, Q. Hu, M. Xu, Y. Zhuang, Y. Wang, and X. Cheng, "A systematic survey of blockchained federated learning," *arXiv preprint arXiv:2110.02182*, 2021.

[72] P. Ramanan and K. Nakayama, "Baffle: Blockchain based aggregator free federated learning," in *2020 IEEE international conference on blockchain (Blockchain)*, 2020, pp. 72–81.

[73] Y. Qu *et al.*, "Decentralized privacy using blockchain-enabled federated learning in fog computing," *IEEE Internet of Things Journal*, vol. 7, no. 6, pp. 5171–5183, 2020.

[74] I. Hegedűs, G. Danner, and M. Jelasity, "Decentralized learning works: An empirical comparison of gossip learning and federated learning," *Journal of Parallel and Distributed Computing*, vol. 148, pp. 109–124, 2021.

[75] G. Danner and M. Jelasity, "Token account algorithms: the best of the proactive and reactive worlds," in *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*, 2018, pp. 885–895.

[76] G. Danner, I. Hegedűs, and M. Jelasity, "Improving gossip learning via limited model merging," in *International Conference on Computational Collective Intelligence*, 2023, pp. 351–363.

[77] L. Giaretta and Š. Girdzijauskas, "Gossip learning: Off the beaten path," in *2019 IEEE International Conference on Big Data (Big Data)*, 2019, pp. 1117–1124.

[78] J. Wang, A. K. Sahu, Z. Yang, G. Joshi, and S. Kar, "MATCHA: Speeding up decentralized SGD via matching decomposition sampling," in *2019 Sixth Indian Control Conference (ICC)*, 2019, pp. 299–300.

[79] A. Koloskova, N. Loizou, S. Boreiri, M. Jaggi, and S. Stich, "A unified theory of decentralized SGD with changing topology and local updates," in *International conference on machine learning*, 2020, pp. 5381–5393.

[80] Á. Berta, I. Hegedűs, and R. Ormándi, "Lightning fast asynchronous distributed k-means clustering," 2014.

[81] C. Hu, J. Jiang, and Z. Wang, "Decentralized federated learning: A segmented gossip approach," *arXiv preprint arXiv:1908.07782*, 2019.

[82] N. Onoszko, G. Karlsson, O. Mogren, and E. L. Zec, "Decentralized federated learning of deep neural networks on non-iid data," *arXiv preprint arXiv:2107.08517*, 2021.

[83] Y. Belal, A. Bellet, S. B. Mokhtar, and V. Nitu, "Pepper: Empowering user-centric recommender systems over gossip learning," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 6, no. 3, pp. 1–27, 2022.

[84] M. De Vos, S. Farhadkhani, R. Guerraoui, A.-M. Kermarrec, R. Pires, and R. Sharma, "Epidemic learning: Boosting decentralized learning with randomized communication," *Advances in Neural Information Processing Systems*, vol. 36, pp. 36132–36164, 2023.

[85] S. Biswas *et al.*, "Noiseless privacy-preserving decentralized learning," *arXiv preprint arXiv:2404.09536*, 2024.

[86] M. De Vos *et al.*, "Energy-aware decentralized learning with intermittent model training," in *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2024, pp. 1172–1174.

[87] S. Biswas, A.-M. Kermarrec, R. Sharma, T. Trinca, and M. De Vos, "Fair decentralized learning," in *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2025, pp. 714–734.

[88] S. Biswas, A.-M. Kermarrec, A. Marouani, R. Pires, R. Sharma, and M. De Vos, "Boosting asynchronous decentralized learning with model fragmentation," in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 685–696.

[89] A. Dhasade, A.-M. Kermarrec, E. Lavoie, J. Pouwelse, R. Sharma, and M. De Vos, "Practical federated learning without a server," in *Proceedings of the 5th Workshop on Machine Learning and Systems*, 2025, pp. 1–11.

[90] S. Biswas *et al.*, "Mosaic Learning: A Framework for Decentralized Learning with Model Fragmentation," *arXiv preprint arXiv:2602.04352*, 2026.

[91] Y. Hua *et al.*, "Towards Practical Overlay Networks for Decentralized Federated Learning," *arXiv preprint arXiv:2409.05331*, 2024.

[92] D. W. Hosmer Jr, S. Lemeshow, and R. X. Sturdivant, *Applied logistic regression*. John Wiley & Sons, 2013.

[93] F. G. Hopkins Mark Reeber Erik and S. Jaap, "Spambase." 1999.

[94] Y. LeCun *et al.*, "Backpropagation applied to handwritten zip code recognition," *Neural computation*, vol. 1, no. 4, pp. 541–551, 1989.

[95] C. C. Yann LeCun and C. J. Burges, "MNIST Database of Handwritten Digits." [Online]. Available: https://yann.lecun.com/exdb/mnist/

[96] M. De Vos, S. Farhadkhani, R. Guerraoui, A.-M. Kermarrec, R. Pires, and R. Sharma, "Epidemic learning: Boosting decentralized learning with randomized communication," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[97] "Information technology – Open Systems Interconnection – Basic Reference Model: The Basic Model," no. ISO/IEC7498–1. 1994.

[98] N. Ketkar and J. Moolayil, "Introduction to pytorch," *Deep learning with python: learn best practices of deep learning models with PyTorch*. Springer, pp. 27–91, 2021.

[99] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[100] W. R. Heinzelman, A. Chandrakasan, and H. Balakrishnan, "Energy-efficient communication protocol for wireless microsensor networks," in *Proceedings of the 33rd annual Hawaii international conference on system sciences*, 2000, p. 10–pp.

[101] "IEEE Standard for Information Technology—Telecommunications and Information Exchange between Systems—Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications," technical report 802.11-2024, Apr. 2024. [Online]. Available: https://standards.ieee.org/ieee/802.11/10548/

[102] S. Micali, M. Rabin, and S. Vadhan, "Verifiable random functions," in *40th annual symposium on foundations of computer science (cat. No. 99CB37039)*, 1999, pp. 120–130.

[103] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," *Modeling and tools for network simulation*. Springer, pp. 15–34, 2010.

[104] S. Nelakurthi, "Dataset for Predicting Watering the Plants." 2021.

[105] F. Bai and A. Helmy, "A survey of mobility models," *Wireless Adhoc Networks. University of Southern California, USA*, vol. 206, p. 147, 2004.

[106] M. A. Legheraba, M. Potop-Butucaru, and S. Tixeuil, "Brief Announcement: A Self-* and Persistent Hub Sampling Service," in *International Symposium on Stabilizing, Safety, and Security of Distributed Systems*, 2024, pp. 461–465.

[107] M. A. Legheraba, M. Potop-Butucaru, S. Tixeuil, and S. Fdida, "Emergent Peer-to-Peer Multi-Hub Topology," in *2024 22nd International Symposium on Network Computing and Applications (NCA)*, 2024, pp. 219–226.

[108] M. A. Legheraba, N. Rachdi, M. Potop-Butucaru, and S. Tixeuil, "LIFT: Byzantine Resilient Hub-Sampling," in *2025 Thirteenth International Symposium on Computing and Networking (CANDAR)*, 2025, pp. 1–9.

[109] M. A. Legheraba, S. Galkiewicz, M. Potop-Butucaru, and S. Tixeuil, "HEAL: Resilient and Self-* Hub-based Learning," in *International Conference on Advanced Information Networking and Applications*, 2025, pp. 200–211.

[110] M. A. Legheraba, M. Potop-Butucaru, S. Tixeuil, and S. Fdida, "Des noeuds anonymes hissés au rang de stars," in *ALGOTEL 2025–27èmes Rencontres Francophones sur les Aspects Algorithmiques des Télécommunications*, 2025.

[111] M. A. Legheraba, S. Galkiewicz, M. Potop-Butucaru, and S. Tixeuil, "Les étoiles montantes de l'apprentissage distribué," in *ALGOTEL 2025–27èmes Rencontres Francophones sur les Aspects Algorithmiques des Télécommunications*, 2025.

[112] P. Erdős and A. Rényi, "On the evolution of random graphs," *A Magyar Tudományos Akadémia Matematikai Kutató Intézetének Közleményei*, vol. 5, no. 1–2, pp. 17–61, 1959.

[113] D. J. Watts, "Strogatz-small world network Nature," *Nature*, vol. 393, no. June, pp. 440–442, 1998.

[114] R. Cohen and S. Havlin, "Scale-free networks are ultrasmall," *Physical review letters*, vol. 90, no. 5, p. 58701, 2003.

[115] P. W. Holland, K. B. Laskey, and S. Leinhardt, "Stochastic blockmodels: First steps," *Social networks*, vol. 5, no. 2, pp. 109–137, 1983.

[116] J. A. Pouwelse *et al.*, "TRIBLER: a social-based peer-to-peer system," *Concurrency and computation: Practice and experience*, vol. 20, no. 2, pp. 127–138, 2008.

[117] E. Buyukkaya, M. Abdallah, and R. Cavagna, "VoroGame: A hybrid P2P architecture for massively multiplayer games," in *2009 6th IEEE Consumer Communications and Networking Conference*, 2009, pp. 1–5.

[118] Federal Trade Commission, "Peer-to-Peer File-Sharing Technology: Consumer Protection and Competition Issues," Washington, D.C., USA, technical report, June 2005. [Online]. Available: https://www.ftc.gov/sites/default/files/documents/reports/peer-peer-file-sharing-technology-consumer-protection-and-competition-issues/050623p2prpt.pdf

[119] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Available at SSRN 3440802*, 2008.

[120] V. Buterin and others, "Ethereum white paper," *GitHub repository*, vol. 1, no. 22–23, pp. 5–7, 2013.

[121] M. Learning, "Tom mitchell," *Publisher: McGraw Hill*, p. 31, 1997.

[122] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[123] A. Cauchy and others, "Méthode générale pour la résolution des systemes d'équations simultanées," *Comp. Rend. Sci. Paris*, vol. 25, no. 1847, pp. 536–538, 1847.

[124] S. Shalev-Shwartz, "Online learning and online convex optimization," *Foundations and Trends® in Machine Learning*, vol. 4, no. 2, pp. 107–194, 2025.

[125] T. G. Dietterich, "Ensemble methods in machine learning," in *International workshop on multiple classifier systems*, 2000, pp. 1–15.

[126] J. Dean *et al.*, "Large scale distributed deep networks," *Advances in neural information processing systems*, vol. 25, 2012.

[127] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[128] S. V. Albrecht, F. Christianos, and L. Schäfer, *Multi-agent reinforcement learning: Foundations and modern approaches*. MIT Press, 2024.

[129] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1345–1359, 2009.

[130] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," *Advances in neural information processing systems*, vol. 33, pp. 3557–3568, 2020.

[131] D. J. Beutel *et al.*, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[132] A. Dhasade, A.-M. Kermarrec, R. Pires, R. Sharma, and M. Vujasinovic, "Decentralized learning made easy with DecentralizePy," in *Proceedings of the 3rd Workshop on Machine Learning and Systems*, 2023, pp. 34–41.